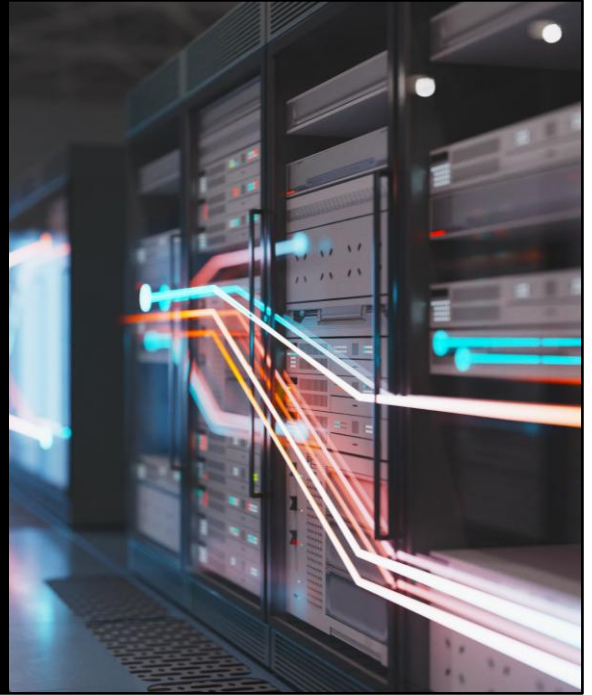
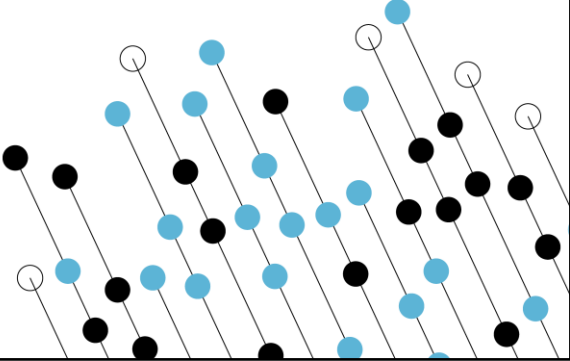


Intermediate Python

Course Introduction
QAINTPYFS



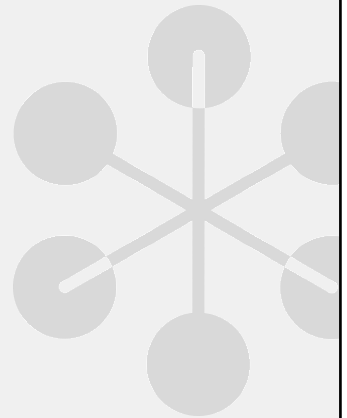
Prerequisites



Prerequisites

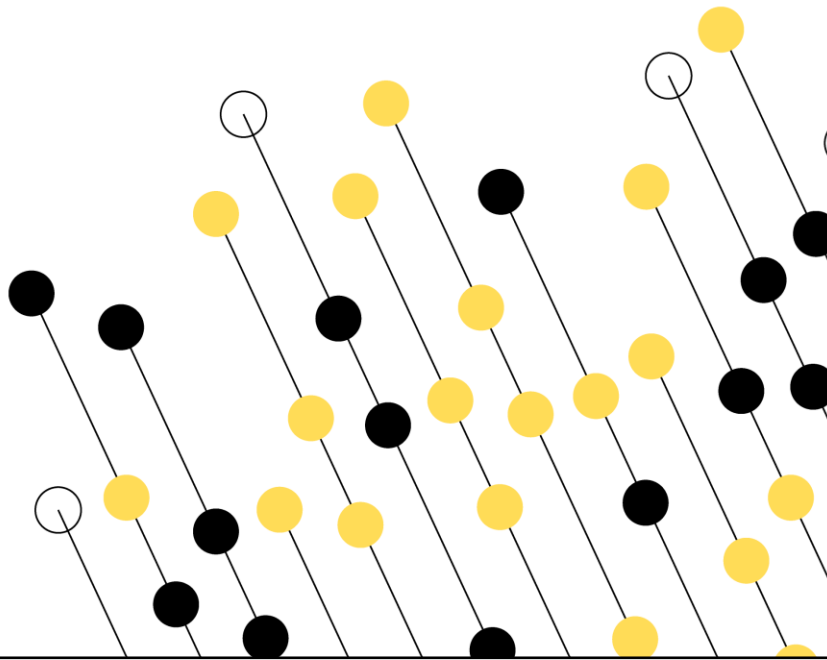
QA recommend for this course that learners should have the following prerequisite knowledge:

- Knowledge of Python to the level in the QA course Python 3 Programming (QAPYTH3) is assumed, including types, data structures, functions, and flow control (selection, iterations, etc.)
- Recent experience of using Python is suggested.
- For the 'Extending Python' chapter, a working knowledge of C is helpful (but not essential) and ideally should include an understanding of pointers and structs.



QA

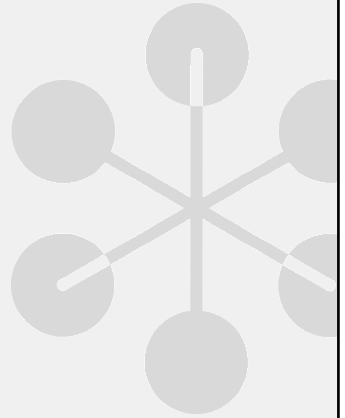
and Outcomes



Learning aim and outcomes

Learning aim

The learning aim of the Advanced Python course is for you to be able to successfully use advanced features and functions of the language to create complex scripts and powerful applications without needing to be overly complicated.



QA

5

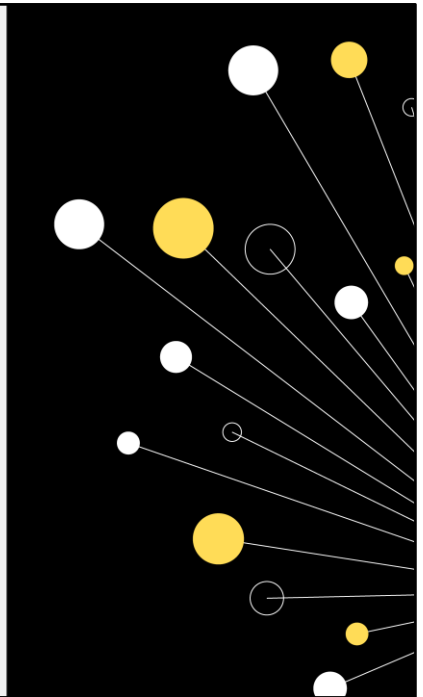
We will review progress on these at the end of each chapter and in the programme's summary.

Learning objectives

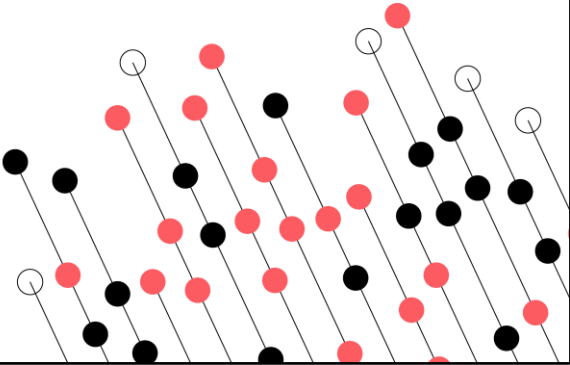
By the end of the programme, you should be able to:

- Control file input/output, including persistent data objects.
- Create well-organized and efficient code using functions and modules.
- Use a range of powerful object-oriented programming techniques in Python 3.
- Build robust applications with appropriate error trapping and reporting.
- Take advantage of multithreading and multiprocessing techniques to handle I/O- and CPU-bound processing more efficiently.
- Confidently handle JSON and XML data in Python.
- Write functional unit tests and find their coverage.
- Scaffold & run a production-ready FastAPI applications
- Run and control other programs from Python.
- Extend Python functionality using custom-build C modules.

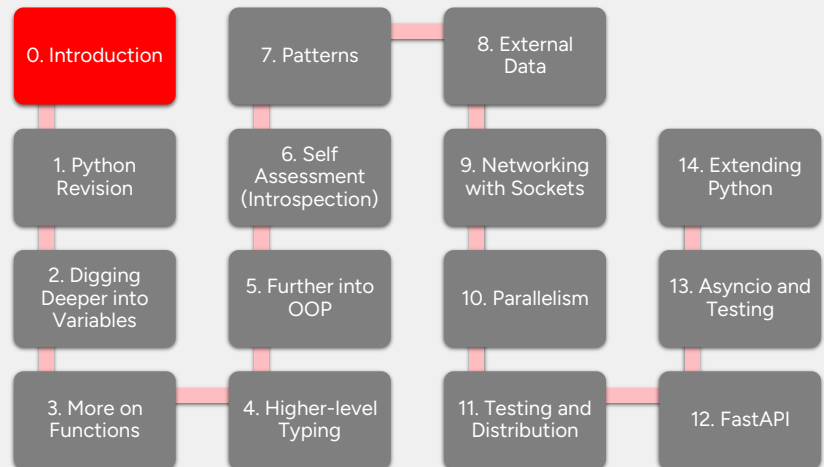
QA



Schedule



Module roadmap



QA

This is the *suggested* running order; we can adapt this (a little) based on timings, progress and learner preferences, i.e. in terms of content weightings between essential and nice-to-have. However, some topics are vertical in nature so can't be swapped around.

Each topic's slide deck represents the critical content; in any session there may be lesser or greater emphasis on certain aspects. You'll have access to all material on the first day to support your learning and help you plan any questions you want to ask on those trickier topics.

Many slides have additional notes which amplify their coverage; don't forget to check these too.

Advanced Python

Day 1

AM1	Introduction
AM2	Python Revision
MID	Digging Deeper into Variables
PM1	More on Functions
PM2	Higher-Level Typing

QA

Advanced Python

Day 2

AM1	Further Into OOP
AM2	Further Into OOP
MID	Introspection
PM1	Patterns
PM2	External Data

QA

Advanced Python

Day 3

AM1	Networking with Sockets
AM2	Networking with Sockets
PM1	Parallelisim
PM2	Parallelisim

QA

Advanced Python

Day 4

AM1	Testing and Distribution
AM2	Testing and Distribution
PM1	FastAPI
PM2	FastAPI

QA

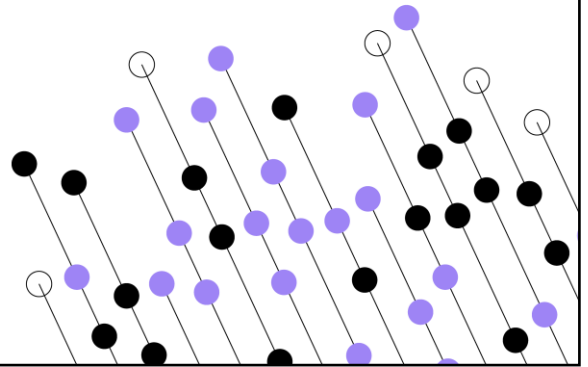
Advanced Python

Day 5

AM1	Asyncio and Testing
AM2	Asyncio and Testing
PM1	Extending Python
PM2	Extending Python
PM3	Review and course close

QA

Housekeeping



Housekeeping



QA

15

Housekeeping slide

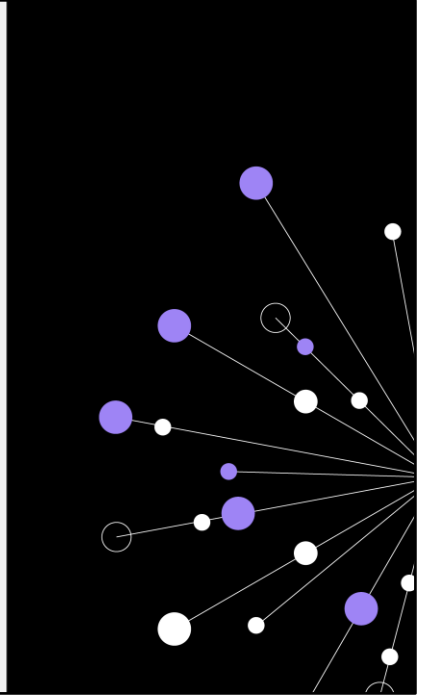
Please remind learners of centre rules re: Health and Safety and course organisation, e.g. scheduled breaks, fire drill (as appropriate) etc.

Registration (at reception each morning) please per QA duty of care.

Tell us a little bit about yourself...

Background, technical interests, programming experience, etc.

QA

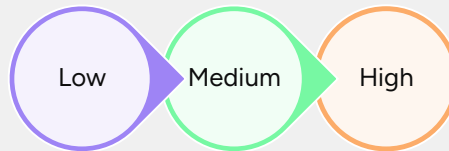


Knowing learner needs is important to us; it's why we're here.

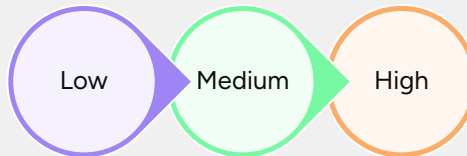
Having an idea of your previous experience, your expectations of the programme, your preferred learning styles etc. will help us to tailor the course more accurately for you and your peers.

Competence and Confidence Check

Confidence



Competence



QA

17

Context: Python Programming

Where would you place yourself? Principally we're interested from a Python Programming and Software Engineering perspective...

We'll revisit this, at the programme's end, to measure distance travelled for each aspect.

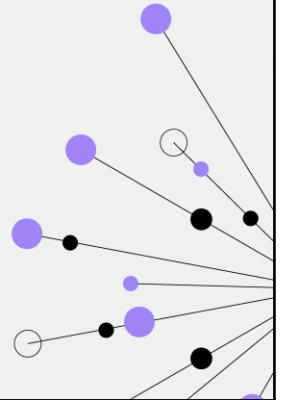


"The beginning of wisdom is found in doubting;
by doubting we come to the question, and by
seeking we may come upon the truth."

Pierre Abelard

Let's begin!

QA



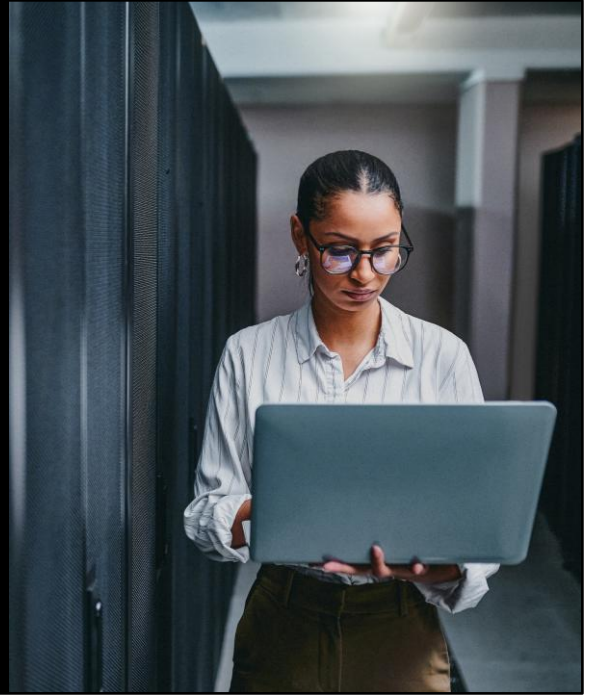
Questions are healthy! And there are no silly questions, all are valuable. ☺

Typically, there are three types:

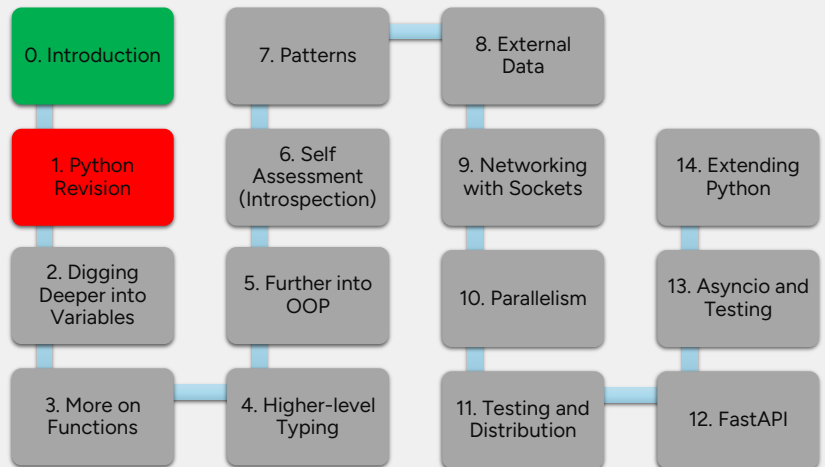
1. Ones that can be answered in-flow as they are directly in line of current delivery
2. Ones that may be answered in next slide, upcoming topic etc.
3. Ones that may constitute a bit of a rabbit-hole (these especially occur in Advanced Python) and may side-track unnecessarily – perhaps better for 1:1?

Intermediate Python

Python Revision



Module Roadmap



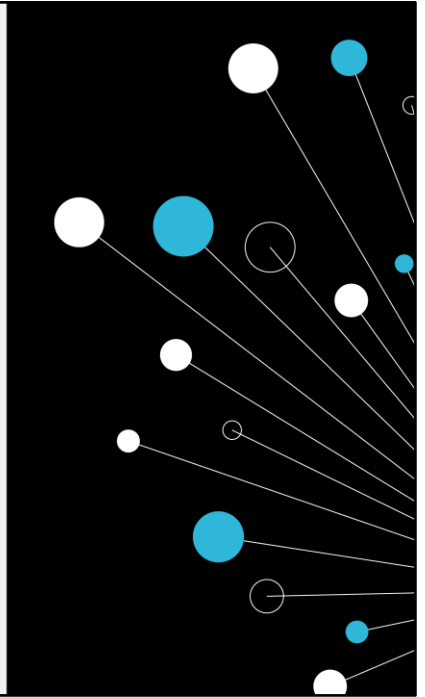
QA

Learning Objectives

By the end of this session, you will be able to:

- Refresh core Python concepts: objects, variables, and built-in types.
- Use list, set, and dictionary comprehensions to create containers efficiently.
- Build containers from multiple iterables using zip.
- Apply Pythonic idioms and style conventions in real-world code.
- Distinguish between **LBYL** and **EAFP** approaches and use them appropriately.
- Follow **PEP 8** guidelines for clean, maintainable Python code.

QA



Python Objects Overview

Everything in Python Is an Object

- Python programs work by creating objects and defining how those objects interact.

Common object types:

- Integers
- Strings
- Lists
- Functions
- Modules etc.

First-Class Functions

- Functions in Python are first-class objects, meaning they can be:
 - Assigned to variables
 - Stored in collections (e.g., dictionaries)
 - Passed as arguments to other functions
 - Returned from functions

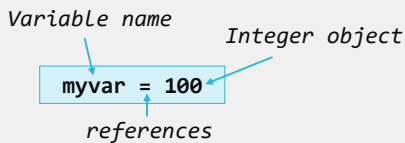
QA

22

Not all Python objects behave the same. Unlike some other programming languages, certain Python objects do not provide attributes or methods, and some cannot be subclassed or extended.

Python Variables

Python variables are references to objects:



- Variables are created dynamically, and their types are inferred at runtime.
 - A variable name can be initialised to a special object known as `None`.
 - Variables can be removed using the `del` statement.
 - When an object is no longer referenced, its memory may be reclaimed and reused.
 - Variables are local to a function by default when defined inside that function.
-
- Use the built-in **type** function to discover a reference's data type
 - The built-in **id** function can be used to discover its memory address (CPython implementation); unique during its lifetime.

QA

23

Like many scripting languages, Python creates variables automatically, and their type is determined only when a value is assigned. In practice, variables act as references to objects—assigning a string to a variable simply makes that name point to a string object. Attempting to use a name that has not been defined results in a `NameError` or `AttributeError`. A name may also be explicitly set to reference the special object `None` (similar to `NULL`, `nil`, or `undef` in other languages).

Some even argue that Python doesn't have variables in the traditional sense, since names are merely labels bound to objects.

If an object is immutable, multiple names may refer to the same underlying value without conflict.

Unlike many other scripting languages, names created inside a function are local to that function by default and must be declared global if they are intended to refer to a global name.

You probably already knew all of this, but we have to start somewhere. Sometimes people come on the wrong course...

Python Variables – Reference Counting

Python keeps a *count* of references which point to an object:

```
import sys
my_list = []
my_list2 = my_list
```

How many references *are there* to the empty List object?

```
print(sys.getrefcount(my_list)) # 3
```

The count is 3 because we need one for each reference created...

```
print(id(my_list)) # 2224859643456
print(id(my_list2)) # 2224859643456
```

...and an additional one for the *argument* passed to the `getrefcount()` function!

QA

24

Using a `del` statement (for example, `del my_list2`) decreases an object's reference count by one.

When an object's reference count drops to zero, Python can reclaim the memory it occupies.

As we'll see later, the way Python handles references has important consequences for multithreading and for writing Python extensions.

Monkey Patching

Modifying Attributes at Runtime

- Python makes it easy to replace or add attributes (often methods) dynamically—typically through simple assignment.
- This can occur unintentionally.
- It can also be done deliberately by design.

```
len = 100  
if var > len(x):
```

**TypeError: 'int' object
is not callable**

Legitimate Uses of Monkey-Patching

- Useful in automated testing, such as client–server setups where one side is replaced with a mock component.
- Can be applied to correct bugs or add features to third-party libraries.
 - Be aware that doing so may make you responsible for maintaining the patched behaviour

Potential Risks

- Monkey-patching can introduce fragile, hard-to-diagnose issues.
- Code may no longer behave in line with documented or expected functionality.

```
import sys  
normal_out = sys.stdout  
err_log = open("error_log.log", "a")  
sys.stdout = err_log  
print("An error message", flush=True)  
sys.stdout = normal_out
```

QA

25

Monkey-patching refers to modifying existing classes or functions by replacing their attributes at runtime. Python makes this particularly straightforward—sometimes *too* straightforward. New Python users often unintentionally shadow built-in functions by choosing variable names such as `len`, `list`, or `str`.

While it can be tempting to use monkey-patching as a quick workaround, it comes with risks. Altering behaviour that users expect—especially behaviour defined in documentation—can lead to unpredictable or unsupported code. In many situations, using inheritance or composition is a safer and more maintainable alternative.

That said, there are valid scenarios where monkey-patching is useful, particularly in testing. For example, to capture what a program writes to `stdout`, you can temporarily redirect `stdout` to a file-like object and inspect the results.

Python Types

Immutable	Numbers 3.142, 200, 0b1001, 0x4e, 0o736, 2.4e3	Sequence
	Bytes b'Sent', b'Received'	
	Strings 'Bananas', "Bob's Burgers", r'C:\Temp'	
	Tuples (21, 'QA', 'Info', 56.3, True)	
Mutable	Lists ['Kamran', ['Sadia', 'Jessica'], 'Rodrigo']	
	Bytearrays bytearray(b'Info')	
	Dictionaries* {'first_name': 'Hans', 'last_name': 'Zimmer'} Sets {'Jazz', 'Classical', 'Rock', 'Pop'}	

* Note – Dictionaries retain their insertion order from CPython ^3.6

26

A *sequence* is a general term for an ordered collection of items. In Python, the most commonly used sequence types are **strings**, **lists**, and **tuples**.

Strings and tuples are **immutable**, meaning their contents cannot be changed once created.

Dictionaries can be iterated in insertion order, which can make them appear sequence-like. However, they are **not** sequence types. Internal optimisations in CPython happen to maintain insertion order, but this behaviour is an implementation detail rather than a language guarantee. If ordering is essential, consider using **OrderedDict** from the collections module.

Python also provides dedicated types for working with binary data. A **bytes** object is an immutable sequence of 8-bit values, while a **bytearray** is a mutable counterpart.

Python's string model is fully Unicode by default, so no special prefix is needed for Unicode text. Instead, the `b'...'` prefix is used to represent single-byte sequences.

List Comprehensions

A list comprehension returns a list

It consists of:

- An expression which identifies a list item
- A loop - typically a **for** loop
- An optional condition to *filter* items
 - *Pythonic* replacement of the **filter** built-in function

```
principal = 10_000
annual_rate = 0.06
years = 10
n = 12 # compounding periods per year

# Year-end balances using a list comprehension
year_end_balances = [
    principal * (1 + annual_rate / n) ** (n * t)
    for t in range(1, years + 1)
]

print(year_end_balances)
```

Pure Compound Interest
(No Contributions)

```
target = 50_000
# Find the earliest month index where balance >= target (if any)
hit_months = [i for i, bal in enumerate(balances, start=1) if bal >= target]
first_hit = hit_months[0] if hit_months else None
```

QA

Find the First Month You Hit a Target

27

List comprehensions provide a natural and expressive way to build lists. Inspired by set-builder notation and popularised in functional languages such as Haskell, they offer a concise syntax for transforming and filtering data.

A list comprehension has three key components:

- an expression applied to each item,
 - a **for** clause that iterates over a sequence, and
 - an optional condition that selects which items are included.
- Only items that satisfy the condition are returned.

List comprehensions integrate well with Python's growing use of iterators, enabling efficient, readable data processing.

Although **map** and **filter** provide similar functionality from a functional-programming perspective, list comprehensions are generally considered the more idiomatic Python approach.

List Comprehensions with the "Walrus" Operator :=

Comprehensions offer an opportunity to use the walrus operator

- Introduced in Python 3.8
- Uses Pascal-like := assignment operator
- Full details, see PEP 572

Note:

parentheses are needed due to operator precedence, note the difference:

- Be cautious, as this can introduce a semantic error that's hard to diagnose.

```
principal = 5000
rate = 0.05
years = 10
threshold = 6000
```

```
balances = [
    balance
    for year in range(1, years + 1)
    if (balance := principal * (1 + rate) ** year) > threshold
]
```

```
[(6077.531250000001, 4), (6381.4078125000015, 5), ...]
```

```
if num := len(balances) > 3:
    print(num)
```

True



```
if (num := len(balances)) > 3:
    print(num)
```

7



QA

28

The walrus operator allows the programmer to pass the assignment of a variable into an expression; typically reducing two separate statements into one or prevents duplication of a functions call. For example, useful opportunities to use the walrus operator include:

- if statements (4 lines):

```
account_types = ["current", "deposit", "mortgage", "Isa"]
qty = len(account_types )
if qty > 3:
    print(f"{qty} is too many items!")
```

Becomes (3 lines):

```
account_types = ["current", "deposit", "mortgage", "Isa"]
if (qty := len(account_types )) > 3:
    print(f"{qty} is too many items!")
```

Also useful in:

- Regular expression matches and extracts
 - While loops to test termination conditions
- Use sparingly to reduce code complexity.

Final note.

The walrus operator can be used in function calls, e.g.

```
result = my_func(my_var:= 30)
```

However, it can't be used in conjunction with keyword arguments:

```
result = my_func(param1 = my_var:= 30)
```

Unless parenthesis differentiates, again...

```
result = my_func(param1 = (my_var:= 30))
```

Set and Dictionary Comprehensions

Comprehensions can be carried out on sets...

```
gbp_prices = {10, 25, 40, 100}
exchange_rate = 1.17 # GBP → EUR
eur_prices = {round(price * exchange_rate, 2) for price in gbp_prices}
```

{11.7, 29.25, 46.8, 117.0}

... and dictionaries

```
Exchange_rates = {"EUR": 1.17, "USD": 1.32, "JPY": 189.50}

# Each client has a list of (currency_code, amount_in_GBP) tuples
client_requirements = {"client_123": ("EUR", 100), "client_456": ("USD", 250), ...}

# Convert each client's amount into the requested currency
converted = {
    client: (currency, round(amount * exchange_rates[currency], 2))
    for client, (currency, amount) in client_requirements.items()
}
```

Dictionary of tuples

{'client_123': ('EUR', 117.0), 'client_456': ('USD', 330.0), ...}

QA

29

In addition to list comprehensions, Python 3 also supports set and dictionary comprehensions.

A set comprehension takes a set and iterates through it to produce another set. The example, converts a set of prices in GBP into EUR using a fixed exchange rate.

Dictionary comprehensions are more complex, in that they need a *key*, *value* tuple to feed. In the example:

- { ... } Creates a new dictionary
- for client, (currency, amount) in client_requirements.items()
Iterates over each client and unpacks the requested conversion
- client: Sets the dictionary key
- (currency, round(amount * exchange_rates[currency], 2))
Computes and sets the dictionary value
- exchange_rates[currency] Looks up the correct FX rate

Constructing Containers Using zip

zip returns items from each list

- In Python 3 it returns a zip object
- any number of iteration objects can be zipped
- zipping stops on the shortest sequence/iteration
 - However, if *strict=True* keyword supplied the code will raise a ValueError

Create a dictionary from two lists using dict()

```
my_dict = dict(zip(keys, vals))
```

Create a dictionary from two lists using a dictionary comprehension

```
my_dict = {k: v for k, v in zip(keys, vals)}
```

Create a list of tuples, one item from each list

```
my_zlist = [*zip(keys, vals)]
```

Create a list of tuples, one item from each list (zip returns an iterator not a list)

```
my_zlist = list(zip(keys, vals))
```

QA

30

This is a trick that is useful for constructing containers, particularly dictionaries.

In the example, we have a list of keys and a list of values, corresponding to the required keys and values in a dictionary. There are several ways to do this.

Given the keys and vals lists as defined below:

```
keys = ["a", "b", "c"]
vals = ["apple", "banana", "carrot"]
```

The examples on the slide will produce the following output:

```
{'a': 'apple', 'b': 'banana', 'c': 'carrot'}
{'a': 'apple', 'b': 'banana', 'c': 'carrot'}
[('a', 'apple'), ('b', 'banana'), ('c', 'carrot')]
[('a', 'apple'), ('b', 'banana'), ('c', 'carrot')]
```


Best Practice Following Pythonic Style

What does “Pythonic” mean?

- Writing code that follows Python’s idioms, conventions, and overall design philosophy.
- Guided by the Zen of Python (import this, PEP 20).
- Should follow PEP 8, the official style guide for Python
 - <https://www.python.org/dev/peps/pep-0008/>
 - PEP 8 is widely supported and enforced by modern IDEs.
- Making effective use of the language’s built-in features
 - rather than reinventing the wheel with unnecessary complexity.

Common characteristics of Pythonic code

- Embraces functional-style tools:
 - iterators, generators, itertools, functools
- Uses comprehensions for clear, concise data transformation
- Makes appropriate use of operators like in, is, etc.
- Uses slicing effectively when working with sequences

QA

31

Being *Pythonic* can be subjective, and we still need to stay pragmatic. Writing Pythonic code means embracing Python’s idioms and taking advantage of the language’s strengths—often through iterators, comprehensions, and functional-style tools.

Python’s current design makes it straightforward to write code in this style. For example:

```
mapping = {"EUR": 1.17, "USD": 1.32, "JPY": 189.5}

for code, rate in mapping.items():
    print(f"{code} → {rate}")
```

Here, `mapping.items()` returns an iterator of (key, value) pairs, making it efficient and natural to use in loops.

At its core, being Pythonic is about producing code that is readable, consistent, and efficient—probably in that order. It’s simply good programming practice. Consider not just *what* you write, but *how* you write it. The recommendations for this are outlined in PEP 8.

Are you a Pythonist, a Pythonista, or a Pythoneer? Does it matter?

LBYL vs. EAFP – Best Practice

Python typically encourages an EAFP style of programming:

- EAFP – “Easier to ask forgiveness than permission”
- Assume the operation will work, go ahead and try it, and handle any exceptions if something goes wrong.

This contrasts with the more traditional LBYL approach found in other languages:

- LBYL – “Look before you leap”
- Check conditions in advance and avoid performing the action unless you’re certain it’s safe.

```
trade = {"trade_currency": "EUR", "settlement_currency": "USD", "fx_rate": 0.89, "amount": 100.00}
# LBYL
if "fx_rate" in trade and trade["fx_rate"] is not None:
    value_gbp = trade["amount"] * trade["fx_rate"]
else:
    value_gbp = None
```



```
# EAFP - considered best practice
try:
    value_gbp = trade["amount"] * trade["fx_rate"]
except (KeyError, TypeError):
    value_gbp = None
```



QA

32

This can feel counter-intuitive at times—especially for those with experience in other languages—but it is consistent with Python’s design philosophy.

Keep in mind that EAFP is intentionally explicit (and consistent with the principles laid out in the Zen of Python) and that Python implements exceptions as a control-flow feature. This means that exception handling is reasonably efficient in practice and has the comparable performance of a standard if statement.

To find out more, see: <https://devblogs.microsoft.com/python/idiomatic-python-eafp-versus-lbyl/>

Other Python Conventions

- When defining a named function, prefer **def** over **lambda**.
- Control module exports with **__all__** instead of exposing all globals*.
 - Still prefix private names with an underscore.
- Compare with **None** using **is** or **is not**, not **==** or **!=**.
- Favour **dictionaries** for structured mappings.
- Use **in** for membership tests.
 - It also works with strings, which act as containers.
- Prefer **string methods** over regular expressions when possible.
- When using **re**, choose the **re.VERBOSE (re.X)** flag to support comments and readability.
- Avoid manual counting loops.
 - Use **range()** or **enumerate()** instead.

* **__all__** is used to control what gets imported when someone uses `from module import *`.

QA

33

Have you ever come across overly complicated lambda expressions and wondered why they were used? Lambdas are meant for small, one-off functions—and they're limited. Regular functions defined with **def** have no such restrictions, so if the logic is non-trivial, a proper function is usually clearer.

Some argue that lambdas avoid adding names to the global namespace, but that's not a real limitation—functions are objects and can live in any scope.

Controlling what a module exports through **__all__** is far more intentional than exposing everything by default, and it encourages you to think about the module's public API.

Tools like **__dir__** can also help define how your objects present themselves.

Remember, there is only **one** **None** object, and everything that represents "no value" refers to it.

Python relies heavily on dictionaries, so avoid manually looping through lists, strings, or dict keys to search for a value—use **in** instead. It's simpler and usually faster.

Regular expressions are powerful, but not always necessary. If a built-in string method (such as `str.replace`) does the job, prefer it—your code

will be faster, cleaner, and less error-prone. A win all around.

If you miss patterns like:

```
for (i = 0; i < size; i++)
```

it's time to let them go. Python seldom needs explicit counting loops. But use **range()** or **enumerate()** when you do.

In short: the best way to write Python is to write Pythonic code—embrace the language rather than forcing patterns from elsewhere.

PEP8 Important Truths

EP 8 provides guidelines, not strict rules.

Consistency and readability matter more than perfect adherence.

Python code should be **implementation-agnostic**,

- because relying on internal behaviour can make optimisation harder.

Code Layout

- Use **4 spaces** for indentation.
- Avoid tabs entirely — they are often misunderstood (Guido van Rossum, 2013).
- Insert a **space after commas**, but **not** before parentheses.

Naming Conventions

- Follow consistent naming patterns throughout your codebase.

Exceptions

- Avoid using a bare except:.
- Exception classes should derive from **Exception**, not **BaseException**.

QA

34

PEP8 is like the Pirate's Code ([Pirate code – Wikipedia](#)) a set of guidelines rather than rules. Local coding standards should always take precedence.

A few notes:

- Avoid trapping bare except: there are two exceptions(!) allowed in PEP008 which are:
 - **Logging or printing the traceback:** Error is still visible; not silent. [\[dlemfh.github.io\]](#)
 - **Cleanup followed by raise:** Exception is not suppressed; program still fails appropriately.

PEP8 Naming Conventions

Avoid using the single-letter variables `l`, `O`, or `I`.

- They are visually ambiguous, which is precisely the problem.

Underscore-based meanings

- A name starting with **one underscore** indicates it is intended for internal use within a *module*.
- A name starting with **two underscores** is considered private to a *class*
 - This triggers *name mangling* (though it can still be reversed if needed).
- Names wrapped in **double underscores** are reserved for Python's special ("magic") behaviours, such as `__init__` or `__str__`.

Function and method names: use *lowercase* or *lower_case_with_underscores*.

Method parameters:

- Use **`self`** for instance methods.
- Use **`cls`** for class methods.

Classes

- Class names follow **UpperCamelCase** (also known as *CapsWords* or *PascalCase*).
- Custom exception classes should end with the word **Error**.

Modules, Packages, and Constants

- **Modules:** short, all lowercase; use underscores only if needed for readability.
- **Packages:** follow the same rule as modules, but typically *without* underscores.
- **Constants:** written in **UPPERCASE**, using underscores where helpful.

QA

35

When naming identifiers, avoid:

`l`: lower case 'el'

`O`: upper case 'oh'

`I`: upper case 'eye'

These can easily be muddled with numbers and other letter, especially so for a number of fonts.

Over the years, various proposals have tried to “modernize” PEP 8, but most of them essentially attempt to impose Java- or C#-style naming rules—and all have been rejected.

Write Python in a Pythonic style. But if you genuinely prefer to structure your Python code as though it were Java, there's no style-police force that will stop you.

Tools to Help Ensure Code Quality - 1

There are a range of tools can help improve overall code quality including:

Linters

- Linters can be run directly from most IDEs.
- They highlight and remove minor issues in source code, including:
 - **Logical checks**
 - Variables that are declared but never used
 - Faulty or incorrect logic
 - Code patterns that may be unsafe
 - **Stylistic checks**
 - Inconsistent formatting or structure
 - Non-standard or unconventional coding styles

Code Analysis Tools

- These tools focus on spotting deeper improvement opportunities, such as where refactoring might be beneficial.
- They often compute measurable characteristics of code, such as:
 - **McCabe (cyclomatic) complexity**, which reflects how complicated the control flow is
 - **Basic statistics** like source lines of code (SLoC), comment density, blank lines, and similar metrics

Tools to Help Ensure Code Quality - 2

Ruff and Flake8

- Ruff is a modern, all-in-one linter (and formatter) written in Rust (very fast). "New kid on the block".
- Flake8 is a wrapper tool that combines **pycodestyle**, **PyFlakes**, and the **McCabe** complexity checker into a single linting command.
- Both check **style violations**, **syntax issues**, **unused variables**, and **code complexity**, helping maintain clean and consistent Python code.

Feature	Ruff	Flake 8
Linting	Yes	Yes
Plugin ecosystem	Yes (built in equivalents)	Yes (Huge)
Auto-fix issues	Yes	No
Formatting	Yes (built in)	No (needs Black)
Import sorting	Yes (built in)	No (Needs isort)
Complexity Checks	Yes	Yes

QA

37

The tools mentioned on the two slides are well-know, trusted and do their jobs well.

Pylama

Pylama is a code-audit tool that aggregates multiple linters and analysis tools into a single unified interface.

It wraps several popular tools, including **pycodestyle**, **pydocstyle**, **PyFlakes**, **McCabe**, **Pylint**, **Radon**, **eradicate**, **Mypy**, and **Vulture**.

Supports **multiple configuration styles**, including INI files, TOML configuration, modelines, and per-file settings.

Provides a **single consolidated output**, merging results from all checkers into a unified report for easier review.

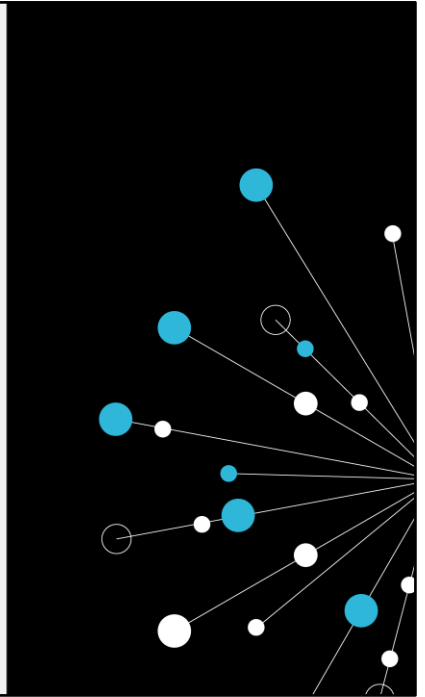
Exercise EG01: Python fundamentals

Activity instructions:

On your own or with a programming partner, implement a solution that takes a Pythonic approach and uses recognised best-practice techniques.

Compare and contrast your solution with your peers.

QA



Suggested time: 30-40 mins.

Activity includes differentiated tasks.

Summary

- In Python, **all things are objects**.
 - But Python's object model differs in important ways from those in many other languages.
- A Python variable **simply holds a reference** to an object.
 - Python tracks how many references an object has—this detail matters for memory management.
 - Even within a version, type behaviour can evolve over time (for example, changes to dict).
- **List, set, and dictionary comprehensions** are considered idiomatic Python.
 - They quickly become intuitive with practice.
- Understand the distinction between **LBYL (Look Before You Leap)** and **EAFP (Easier to Ask Forgiveness than Permission)**
 - both are used in Python but recognise EAFP is preferred
- The `zip()` function enables some clever patterns—just use it thoughtfully.
- **PEP 8** provides guidance on writing clean, readable Python code.
 - These are recommendations, not strict rules.
- A range of tools can support code quality:
 - **Linters**
 - **Code analysis tools**
- Aim to **write Python in a Pythonic style**—approach problems with Python's idioms and mindset.
 - As the saying goes: *"Some programmers can write Fortran in any language."*

Appendices



Python Operators

Operator	Definition
or	logical OR
and	logical AND
not	logical NOT
< <= > >=	comparison operators
== !=	equality operators
is	object identity test
in	object membership test
^	bitwise OR, XOR
&	bitwise AND
<< >>	bitwise shift
- +	subtract, add
* / // %	multiply, divide, integer-divide(floor), modulo
~ **	complement, exponentiation
:=	walrus operator; in-line assignment

QA

The operators listed in reverse order of precedence (**or** is the lowest precedence)

The difference between a single forward slash (/) and a double forward slash (//) is explained by the following example:

```
a = 6
b = a/7    # 0.857142857

# 0 - integer-divide floor
# // - rounds down to nearest whole integer
b = a//7   # 0 - integer-divide floor
```

Python reserved words

The following is a list of Python's reserved words (keywords). As such you cannot use them as variable or function names:

False	None	True	and	as*	assert
async!	await!	break	class	continue	def
del	elif	else	except	finally	for
from	global	if	import	in	is
lambda	nonlocal+	not	or	pass	raise
return	try	while	with*	yield	

* version 2.5 and later
+ version 3.0
! version 3.5

You can programmatically test for potential keywords or retrieve a list of them:

```
import keyword

keyword.iskeyword("for")
print(keyword.kwlist)
```

Note. `exec` and `print` were *keywords* prior to 3.0, now they are *built-in functions* and therefore not keywords.

QA

42

Here's a brief list of keywords and their use:

and, not, or
assert, raise
async, await
function), then calls it
break
class
continue
loop
def
del
except
try block
finally
try block
for
from
names
global
if, else, elif
import

logical operators
trigger an exception
declares function as an asynchronous

exit the current loop
create a class object
do the next iteration of the current

create a function object
delete an item from a list
indicates an exception handler for a

statements always executed after a

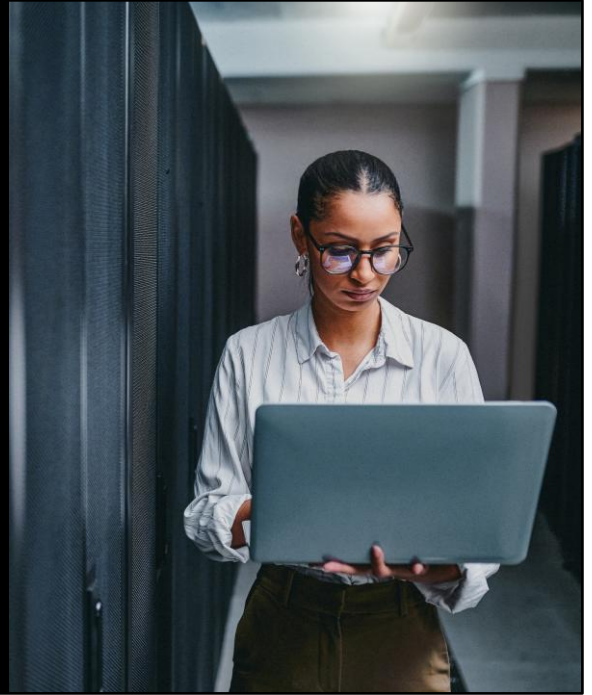
sequence iteration loops
used with 'import' to specify imported

declare variable as global
conditional clauses
find and load a module

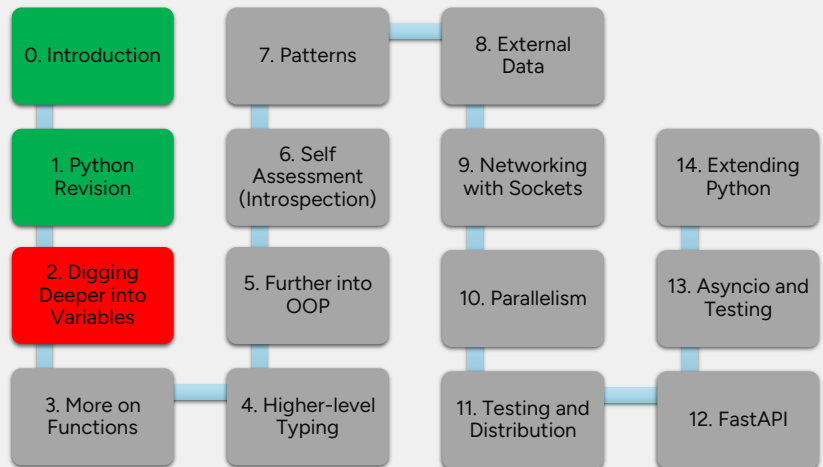
in	tests sequence membership
is	identity test
lambda	create an anonymous function
pass	empty statement (no-op)
placeholder	
return	return a value from a function
try	catches exceptions
while	conditional loop statement

Intermediate Python

Digging Deeper into Variables



Module Roadmap



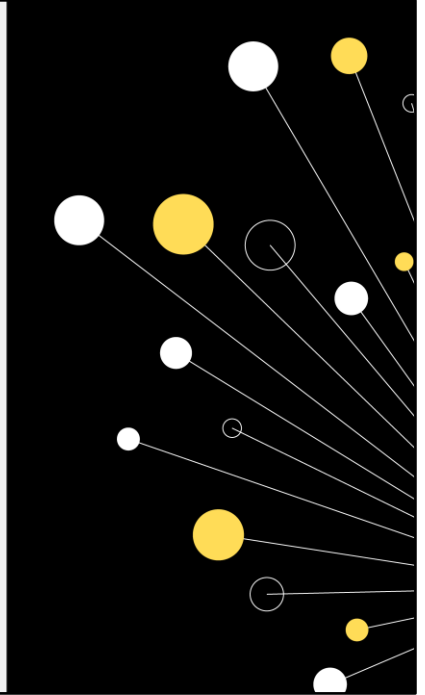
QA

Learning Objectives

By the end of this session, learners will be able to:

- Use the `types` module and standard type behaviours to work effectively with Python variables.
- Apply advanced techniques for handling strings, lists, tuples, and dictionaries.
- Work with generators, co-routines, and iterators to process data efficiently.
- Use key tools from the `collections` module—`namedtuple`, `deque`, `Counter`, and `defaultdict` - to structure and manage data.
- Optimise variable usage for performance and clarity in Python code.

QA



What Type of Type is This! - The Types Module

- The **types** module includes definitions of several **standard interpreter types**
 - Note, it does **not** include core types like **int**, **float**, and **str**
 - Useful for **isinstance** and **issubclass** checks:

```
import types

def myfunc(p):
    if isinstance(p, types.FunctionType):
        p() # invokes the passed in function
    else:
        print(p * 0.2) # assume variable
```

```
def greeting():
    print("Hi there!")
```

```
myfunc(5)
myfunc(greeting)
```

```
1.0
Hi there!
```

- Can also be used for Dynamic Type Creation (^3.3)

QA

46

If your code needs to handle many kinds of objects, the **types** module is worth knowing. It's often used to examine the type of a parameter and choose the appropriate response.

Note, builtins, functions, lambdas, generators and instance methods all have a different type, examples include:

Builtins	<code>types.BuiltinFunctionType</code> , <code>types.BuiltinMethodType</code>
Functions	<code>types.FunctionType</code>
Lambda	<code>types.LambdaType</code>
Instance methods	<code>types.MethodType</code>
Generator	<code>types.GeneratorType</code>
Coroutine	<code>types.CoroutineType</code>
Asynchronous generator-iterator	<code>types.AsyncGeneratorType</code>

This does overlap in functionality with aspects of the **inspect** module, which we shall see later.

Python Built-In Types Internal Behaviour – Strings

Strings are (of course) **immutable**

So += **does not** "append", it creates a **new** string object

- Almost always true, they may be resized in some cases (but we have no control over this)
- It is best to assume its use will be a performance disaster:

```
result = ''
for item in iterable:
    result += item
```



A list with join will almost always be more **efficient**:

```
lresult = []
for item in iterable:
    lresult.append(item)

result = ''.join(lresult)
```



Or, more Pythonically, a list comprehension:

```
result = ''.join([item for item in iterable])
```

An append is usually constant time (see over).
A join is linear $O(n)$

QA

47

The advice to avoid repeatedly appending to strings inside a loop is widely known, and the same caution applies in languages such as Java and PHP. From a developer perspective, Python strings are 100% immutable, what may or may not happens behind the scenes is irrelevant. However, if you want to understand how Python's built-in string type works under the hood, you can explore the CPython source code. The key files live in the **Include** directory—particularly `unicodeobject.h`—and in the **Objects** directory, which contains the implementation in `unicodeobject.c`. These define the internal layout and behaviour of Python's Unicode str objects. In Python, **strings are always immutable**, and CPython does not resize or mutate the contents of a str in place. Since Python 3.3, the Unicode implementation (PEP 393) uses a flexible internal representation that stores character data in a compact, memory-efficient form. CPython may allocate different internal layouts depending on the range of characters used, and it may lazily populate auxiliary caches such as a UTF-8 representation—but none of these internal details alter the immutability guarantee seen at the Python level.

Because all string modification results in the creation of a new object, performance considerations still matter. Operations like repeated concatenation inside a loop can lead to many temporary string allocations.

This is why the PEP 8 style guide recommends avoiding repeated `a += b` patterns for building large strings, encouraging instead the use of `".join(...)"` or other efficient construction techniques:

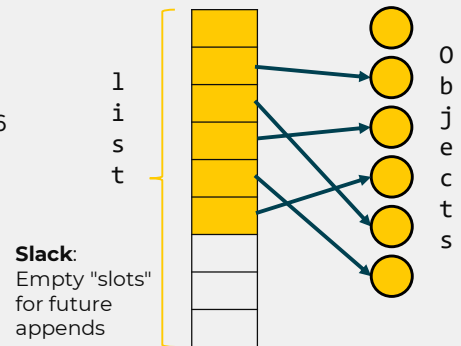
"...do not rely on CPython's efficient implementation of in-place string concatenation for statements such as `a += b` or `a = a + b`."

Although CPython includes internal optimisations, these should not be assumed or depended upon—your code should treat strings as fully immutable values whose concatenation always creates a fresh object.

Python Built-In Types Internal Behaviour – Lists

CPython lists are stored as arrays of object pointers

- Allocated on the heap with extra capacity (“slack space”) for growth
- List capacity increases in predefined steps (e.g., 0 → 4 → 8 → 16 → 25 → 35 → 46 → 58 → 72 → 88 ...)
- When expansion exceeds available slack, `realloc()` is used, which can be expensive
- Only the pointers move during reallocation—underlying objects are not copied
- The actual `PyObject` instances live separately and are dynamically allocated on the heap
- Python maintains its own heap-management layer on top of the C runtime library



QA

48

The slide above was created by examining the CPython source code (`Objects/listobject.c`). Naturally, other Python implementations may behave differently, and even CPython’s behaviour could change in future versions.

Python manages its own memory and heap structures in addition to what the C runtime library or operating system provides. As a result, while `realloc` is used, CPython tries to avoid calling it whenever possible.

The use of stepwise allocation, as illustrated, can make benchmarking unreliable. It’s easy to hit certain “sweet spots” that create misleading performance differences between allocation strategies.

If you’re building huge lists:

- Prefer **pre-sizing** where possible (e.g., build a list of the right size and fill by index, or use `list.__setitem__` where appropriate).
- If you’re repeatedly extending with **large chunks**, `list.extend()` is efficient (fewer growth operations).
- For appending millions of items one-by-one, the amortized behaviour remains good thanks to over-allocation; profiling is still prudent for

memory-sensitive workloads. (The growth factor trades RAM for fewer reallocations.)

Python on Windows uses `obmalloc` (`pymalloc`) for small-object memory allocation. But the Windows OS ultimately backs those allocations using `VirtualAlloc` via the Windows heap. For more information about Python's memory allocation, see `Objects/obmalloc.c`.

Python Built-In Types Internal Behaviour – Lists and Tuples

Tuples

- Tuples are represented differently from lists.
- Small tuples (up to 20 elements) are reused from a dedicated *free list*.
- This makes them efficient—no extra capacity is required and they never need resizing.

Working with Lists

- Using `append` is generally efficient, but performance can fluctuate when the list occasionally needs to grow.
- Resizing happens from time to time when the current capacity is exceeded.
- Any kind of insertion involves copying existing references.
- Only the pointers are copied—so insertion is cheaper than moving whole objects—and the list may not need to resize if spare capacity is available.
- Slicing tends to be slightly faster than calling `insert()` because it avoids an extra function call.
- Removing elements from the end of a list is an $O(1)$ operation.

Remember: All of these behaviours depend on the *current CPython implementation* and should not be treated as language-guaranteed features.

QA

49

Many built-in Python types make use of a *free list*—a pool of previously freed objects that can be reused without triggering a fresh memory allocation. For lists, this free list currently holds up to 80 entries.

Removing elements from a list may trigger a resize, and deleting items from anywhere other than the *end* can be slower. How noticeable this is largely depends on the overall size of the list.

Operations like `min()`, `max()`, and `sum()` run in linear time **$O(n)$** . Lists excel at constant-time random indexing, but if you frequently add or remove items at either end, the `collections.deque` type is a better fit.

Python Built-In Types Internal Behaviour – Dictionaries

Because dictionaries use hashing:

- Looking up any key typically takes the same amount of time.
- In some cases, a simple linear scan could be faster—it depends on where the item would fall during iteration.
- Key size can influence lookup cost, especially in older Python versions.

Dictionary growth behaviour:

- A new dictionary begins with **8 buckets**.
- As items are added, the table size **doubles** when needed.
- `dict.pop()` does not trigger a resize, and using `del` does not necessarily release memory back to the system.

Differences across Python versions:

- Different Python 3 releases may manage dictionary buckets and caching differently.
- Do not assume identical performance across versions—**even minor releases may behave differently**.

For deeper details:

- Consult the CPython source files:
`Objects/dictnotes.txt` and `Objects/dictobject.c`

QA

50

Python keys are not randomised completely, for common string keys they are optimised. This gives better performance (fewer collisions) for sequential ints or text.

Deleting a key does not automatically mean that the dictionary memory is resized, it will try to avoid resizing if possible. The main overhead is in creating the dictionary.

Within Python different minor releases cause dictionaries to behave differently:

Python 3.3 redesigned the dictionary layout

- Introduced **split tables** for instance dicts, allowing keys to be shared between objects.

Python 3.6 (CPython) changed hash table resizing and probing

- First appearance of the modern, compact, insertion-order-preserving layout (though not yet guaranteed).

Python 3.7 made order preservation a language guarantee

- The behaviour was no longer an implementation detail.

Python 3.8 adjusted key-sharing behaviour

- Improvements to memory layout for class dictionaries.

Python 3.10 & 3.11 include more changes

- Improved cache locality.
- Changed key/value storage layout again.
- Modified dictionary creation in comprehensions for speed.
- Internal changes to reduce pointer chasing.

Lazy Lists

Creating full lists in memory can be costly

- How large is the list you are generating?
- And what happens when the sequence has no natural endpoint?

Lazy sequences compute values only when needed

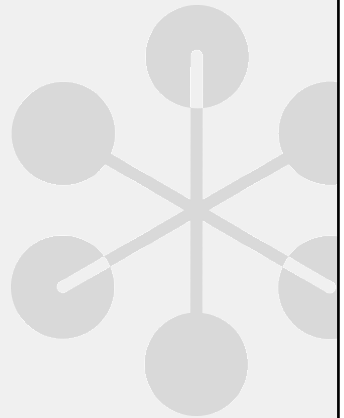
- They produce one item at a time, on demand.
- This makes them ideal when working with iterators.

Many Python 3 built-ins return iterators instead of lists

- Functions such as `range()`, `map()`, `filter()`, `reversed()`, `zip()`, and others all yield iterator objects.

Some dictionary methods return lightweight view objects

- `keys()`, `values()`, and `items()` provide dynamic views rather than materialised lists.



QA

51

In earlier versions of Python, many functions produced full in-memory lists for iteration. This worked well for small datasets but became inefficient as lists grew in size. For very large or unbounded sequences, the memory cost was simply too high—hence the move toward *lazy* approaches.

A *lazy list* produces each item only when it is requested, rather than generating the entire sequence up front. If iteration stops early, any remaining items are never created, saving both time and memory. Python implements this idea through **iterators**, which track only the current position instead of holding the entire sequence in memory.

Generators

A generator is a function which yields a **lazy list**

- A lazy list item is returned at the **yield** statement

```
def iter_paths(path, kind="files"):
    pattern = os.path.join(path, '*')

    for item in glob.iglob(pattern):
        if kind == "files" and os.path.isfile(item):
            yield item
        elif kind == "dirs" and os.path.isdir(item):
            yield item
```

- Generators can often replace list comprehensions
- Can be used anywhere an iterator is

```
for f in iter_paths("./", kind="files"):
    print("FILE:", f)
```

Print a list of files

```
dirs = list(iter_paths("./", kind="dirs"))
```

Get a list of directories

QA

52

Generators provide a way to evaluate sequences lazily, often serving as a more efficient alternative to list comprehensions.

When a generator function runs, it enters the function body only once. Each **yield** statement pauses execution, returns the next value, and preserves the function's state so it can resume exactly where it left off.

Using **return** inside a loop that contains a **yield** expression will raise a **SyntaxError**; if you need to stop early, use **break** instead.

This behaviour contrasts with filters and list comprehensions, which build a complete intermediate list in memory. With generators and other lazy sequences, only the value currently being processed exists at any given time, avoiding the overhead of constructing an entire list.

Note, the following imports have been omitted for clarity:

- `import os.path`
- `import glob`

Generator Objects and next

- A generator function produces a **generator object**, which can be useful when a standard **for** loop isn't the right tool.
- The built-in **next()** function retrieves the next value from a generator.

Using the generator function from the previous slide

```
gen = iter_paths("./", kind="files")
```

```
fname = next(gen, False)
```

```
while fname:
```

```
    print(fname)
```

```
    fname = next(gen, False)
```

```
.\file1.py  
.\file2.py  
.\file3.py
```

You don't need a loop!

```
folders = []
```

```
gen = iter_paths("./", kind="dirs")
```

```
folders.append(next(gen, False))
```

```
folders.append(next(gen, False))
```

```
folders.append(next(gen, False))
```

The second argument is a default value returned when the iterator is exhausted.

QA

53

A generator object is created whenever a generator function is called. Each call produces a *new* generator, and its iteration always begins from the start.

When using a generator outside of a loop, the built-in **next()** function retrieves the next value produced by **yield**.

(How **next()** is implemented on user-defined classes—and how that relates to generators—will be covered on a later slide.)

The **next()** function also accepts an optional second argument that specifies what value should be returned when the generator is exhausted. If this default is not provided, a **StopIteration** exception is raised when the generator has no more items.

Generator Expressions

A generator expression works much like a list comprehension, except that:

- it produces a **generator** rather than building a full list
- its syntax mirrors a list comprehension but uses **parentheses** instead of square brackets

```
principal = 1000
rate = 0.05
years = 10

# Generator expression: compute value per year using compound-interest formula:
balances = (principal * (1 + rate) ** year for year in range(1, years + 1))

print("Yearly balances:")
for amount in balances:
    print(f"{amount:,.2f}", end="")
```

Generator expressions are frequently paired with reduction functions like `max()`, `min()`, and `sum()`.

List Comprehensions as Generators

The generator expression may be used inside a generator

- Enclose the comprehension in () instead of []
- Use a return instead of a yield

Rewritten using a generator expression:

- Function returns a generator object, as before:

```
def iter_paths(path, kind="files"):
    pattern = os.path.join(path, '*')
    if kind == "files":
        return (item for item in glob.iglob(pattern) if os.path.isfile(item))
    elif kind == "dirs":
        return (item for item in glob.iglob(pattern) if os.path.isdir(item))
```

```
# The original
def iter_paths(path, kind="files"):
    pattern = os.path.join(path, '*')
    if kind == "files":
        for item in glob.iglob(pattern):
            if os.path.isfile(item):
                yield item
    elif kind == "dirs":
        for item in glob.iglob(pattern):
            if os.path.isdir(item):
                yield item
```

QA

55

Python provides another way to create generator-style behaviour by using a *generator expression* instead of explicitly writing a **yield** statement.

The only syntactic difference is that generator expressions use **parentheses** rather than square brackets. This matters because using **[]** would create a full list in memory, which defeats the purpose of lazy evaluation.

The slide shows an earlier example written with **yield**, followed by its equivalent rewritten as a generator expression.

Both versions behave the same way in that they each produce a generator object. However, a key limitation of generator expressions is that they cannot receive values from the caller using **.send()**, something that is possible with a generator function that uses **yield**. We will look more into this when we investigate coroutines later.

Generator delegation – Python ^3.3

Generator delegation makes it possible to break a large, complex generator into smaller, focused sub-generators

- much like splitting an oversized function into simpler helper functions.

Notation:

- `yield from iterable`

```
import os
import glob

def _iter_dirs(path):
    #Sub-generator: yields directory paths and count.
    pattern = os.path.join(path, '*')
    count = 0
    for item in glob.iglob(pattern):
        if os.path.isdir(item):
            count += 1
            yield item
    return count

def list_dirs(path):
    """Delegating generator: wraps _iter_dirs with a
    header/footer and captures its return value via the
    `yield from` expression."""
    yield f"Scanning {path}..."
    total = yield from _iter_dirs(path)
    yield f"Found {total} directories."

if __name__ == "__main__":
    for line in list_dirs('.'):
        print(line)
```

QA

56

Generators can become difficult to manage as they grow in size and complexity.

The **yield from** syntax, introduced in Python 3.3, allows a generator to delegate part of its work to another generator. This makes it easier to break complex logic into smaller, more maintainable pieces.

Generator delegation is especially useful when working with closures or when one generator needs to call another.

For more details and examples, see **PEP 380**.

Coroutines and the send() Method

Data can be **returned** to a generator using the **send** method

```
def compound_interest(initial_balance, annual_rate):
    # Coroutine tracks compound interest and accepts new deposits.
    balance = initial_balance
    rate = annual_rate
    # Prime: initial yield returns the starting balance
    value = yield balance
    while True:
        if isinstance(value, dict):
            # Allow dynamic updates
            balance += value.get("deposit", 0)
            rate = value.get("rate", rate)
            balance *= (1 + rate) # Apply one period of interest
            # Yield updated balance and wait for next instruction
            value = yield balance
```

`next()` and `c.send()`
both get the next
yielded value

```
# Start with £1000, interest rate 5%
c = compound_interest(1000, 0.05)
print("Start:", next(c))
print("After deposit:", c.send({"deposit": 200}))
print("Next period:", c.send({}))
print("After rate change:", c.send({"rate": 0.07}))
print("After another deposit:", c.send({"deposit": 500}))
```

```
Start: 1000
After deposit: £1,260.00
Next period: £1,323.00
After rate change: £1,415.61
After another deposit: £2,049.70
```

QA

57

The **send()** generator method can be used in place of **next()**, but with an important difference, **send()** not only resumes the generator it also passes a value back into it, which the generator can receive as the result of a **yield** expression. If **send()** is not used, a bare **yield** simply returns **None**.

This behaviour is known as coroutines. The concept is formalised in **PEP 342 – Coroutines via Enhanced Generators**.

The example on the slide, calculates compound interest, using **.send()** to feed new deposits or interest rates into a running calculation. This demonstrates:

- priming a coroutine and running repeated send/receive cycles.
- sending values in with **.send()** to dynamically adjust deposits or rates
- returning updated compound-interest totals
- maintaining state (balance and rate) internally in the coroutine

Standard Python Collections

Python's standard library hosts a `collections` module which includes:

CONTAINER	DESCRIPTION
<code>namedtuple</code>	Lightweight tuple subclasses with named fields
<code>deque</code>	Fast append/pop operations from both ends (double-ended queue)
<code>ChainMap</code>	Single view over multiple dicts
<code>Counter</code>	Counts hashable objects
<code>OrderedDict</code>	Dictionary that remembers insertion order
<code>defaultdict</code>	Dict with a default value for missing keys
<code>UserDict</code>	Wrapper around dictionary objects
<code>UserList</code>	Wrapper around list objects
<code>UserString</code>	Wrapper around string objects

QA

58

Python's built-in collection types already cover a wide range of needs, but the **`collections`** module provides additional, more specialised containers. Many of these are not entirely new data structures; instead, they extend the standard types (such as **`dict`**) with extra behaviour or convenient defaults.

The module also offers helper classes: **`UserDict`**, **`UserList`**, and **`UserString`** designed to make it easier to create custom container subclasses.

namedtuple

- Enables you to define a lightweight class with named fields
- namedtuple acts as a **class factory**, not an object factory
- More efficient than dictionaries for storing simple structured data
- Fields can be accessed either by name or by index, in any order

```
from collections import namedtuple
subclass = namedtuple("subclass name", "field1 field2 ...")
var = subclass(field values)
```

Fields can be specified as a space separated string or a list of strings.

QA

59

Named tuples are one of those features you don't realise you need until you try them—after that, it's hard to imagine working without them.

Note, **namedtuple** does **not** create a container directly. Instead, it generates a **new class**, and instances of that class behave like enhanced tuples. Field names can be provided as a single space-separated string or as a list of individual strings.

Field names must be unique and cannot clash with Python keywords; otherwise, a **ValueError** is raised. If conflicts are possible, the **rename=True** option will automatically replace problematic names—typically with underscore-prefixed index values such as `_1`, `_2`, and so on.

This is especially helpful when field names come from external sources such as database column names that may not follow Python's naming rules.

Several libraries return data in the form of named tuples. For example, the third-party **psutil** package uses named tuples when reporting information about open files or running processes.

namedtuple - Example

```
from collections import namedtuple

# Define the namedtuple type
account_fields = ["account_number", "customer_name", "balance"]
BankAccount = namedtuple("BankAccount", account_fields)

# Create account instances
acct1 = BankAccount(account_number="00123456", customer_name="Alice", balance=1500.00)
acct2 = BankAccount("00987654", "Bob", 320.75) # can be done positionally

# Accessing fields
print(f"{acct1.customer_name} has £{acct1.balance}")
print(f"Account {acct2.account_number} belongs to {acct2.customer_name}")

# Updating a balance using _replace (namedtuples are immutable)
acct1_updated = acct1._replace(balance=acct1.balance + 200)
print(f"After deposit: {acct1_updated.customer_name} now has £{acct1_updated.balance}")
```

```
Alice has £1500.0
Account 00987654 belongs to Bob
After deposit: Alice now has £1700.0
```

deque

A deque is like a double-ended queue or stack

- Supports adding or removing items from **either end** (`pop()` and `popleft()`)
- More efficient than a list for frequent insertions or removals
- Lists are slower when their length changes significantly
- Items can be accessed by index, but **slicing is not supported**

Deques can optionally have a fixed maximum size

- When full, adding new items automatically discards items from the opposite end

```
tail = deque(open('error.log'), maxlen=10)
while len(tail) > 0:
    print(tail.popleft())
```

QA

61

A **deque** is a common data structure that allows fast access and modification at **both ends**. If you only use one end, it behaves just like a stack.

Although you *can* use a regular Python list for similar behaviour, **`collections.deque`** is specifically optimised for rapid appends and pops on either end, making it much more efficient for these operations.

The example demonstrates how a deque can be created from **any iterable** object, even a file object. This allows us to implement a simple version of the UNIX **`tail`** command, keeping only the last *N* lines of a file by using the **`maxlen`** argument.

deque - Example

```
from collections import deque

transactions = deque([{"id": 1, "type": "payment", "amount": 200}, {"id": 2, "type": "withdrawal", "amount": 5000}, {"id": 3, "type": "deposit", "amount": 150},])

processed = []
retry_limit = 2
while transactions:
    txn = transactions.popleft()
    # Simulate rules
    if txn["amount"] > 3000 and not txn.get("reviewed"):
        print(f"Transaction {txn['id']} flagged for review")
        txn["reviewed"] = True
        transactions.appendleft(txn) # PRIORITY: re-process soon
        continue
    if txn.get("attempts", 0) < retry_limit:
        txn["attempts"] = txn.get("attempts", 0) + 1
        if txn["type"] == "payment":
            print(f"Temporary failure on txn {txn['id']}, retrying later")
            transactions.append(txn) # push to END for retry
            continue
    print(f"Processed txn {txn['id']}")
    processed.append(txn)
```

QA

62

The example demonstrates a heavily simplified transaction retry / fraud review system where some transactions fail temporarily maybe because of a network error. Rather than discarding them, the code retries the transaction later by adding it to the end of the deque. Some other transactions may need be paused to wait for a review check to complete (e.g. for transaction that exceed a threshold). Once validated they are given an escalated priority by adding them to the beginning of the deque.

In short, the deque allows us to:

- `popleft()` → normal processing
- `append()` → retry later (end of queue)
- `appendleft()` → urgent reprocessing (front of queue)

It should be noted, the example crudely demonstrates the retry features in that payments always fail twice before succeeding.

Counter

A dictionary type class

- Keys are the data elements (must be hashable)
- Values are the count
- Missing keys give a count of zero, not a `KeyError`

Counts can be set, or implied

```
from collections import Counter

lst = ["current", "deposit", "current", "mortgage"]
c1 = Counter(lst)
print(c1)
print("No. of currents:", c1['current'])

c12 = Counter(current=44, deposit=293, mortgage=73)
print("Accounts:", sum(c12.values()))

Counter({'current': 2, 'deposit': 1, 'mortgage': 1})
No. of currents: 2
Accounts: 410
```

Can also be used on a string...

```
text = "Alas poor Yorrick, I knew him, Horatio"
print(Counter(text))

Counter({' ': 6, 'o': 5, 'r': 4, 'i': 3, 'a': 2, 'k': 2, ',': 2, 'A': 1, 'l': 1, 's': 1, 'p': 1, 'Y': 1, 'c': 1, 'I': 1, 'n': 1, 'e': 1, 'w': 1, 'h': 1, 'm': 1, 'H': 1, 't': 1})
```

QA

63

Counter objects map each unique key to an integer count. These counts can be supplied when the **Counter** is created, accumulated later using the **update()** method, or automatically derived from repeated occurrences of keys in an iterable.

A particularly convenient feature of **Counter** is that missing keys default to a count of zero rather than raising a **KeyError**. This makes Counters ideal for tallying duplicates or frequencies—tasks that tend to be much more cumbersome when using ordinary dictionaries.

defaultdict

Provides a way to automatically supply a default value

- Accessing a missing key creates it on the fly
- The value is generated using the **default factory** function

```
from collections import defaultdict
# Incoming transactions across many accounts
transactions = [("ACC1001", 500.00), ("ACC1002", 125.00), ("ACC1001", -75.00),
                ("ACC1003", 300.00), ("ACC1002", -25.00), ("ACC1001", 50.00),]
# Start each unseen account at 0.0 automatically
balances = defaultdict(float)

for account_id, amount in transactions:
    balances[account_id] += amount

# Accessing a missing account safely returns 0.0 (and creates the key)
print("ACC9999 balance:", balances["ACC9999"])

# Report balances
for acc, bal in balances.items():
    print(f"{acc}: £{bal:.2f}")
```

```
ACC1001: £475.00
ACC1002: £100.00
ACC1003: £300.00
ACC9999: £0.00
```

QA

64

The awkwardness of handling default values in regular dictionaries is solved by using **defaultdict**.

The first argument to **defaultdict**, the *default factory*, can be any callable, including a lambda, which is used to generate the default value whenever a missing key is accessed.

```
dd = defaultdict(lambda: f"val-{len(dd)}")
print(dd['key1'], dd['key2'], dd['key3'], dd['key4'])
```


Helpers

- Many collection types provide helpful utility attributes and methods
- These helpers often use a single leading underscore (they're not private—see notes)
- Helpers may be implemented as methods or as properties
- For example, **namedtuple** offers two particularly useful helpers:
 - **_fields** returns the field names
 - **_asdict()** converts the instance to an ordered dictionary

```
from collections import namedtuple
# Define the namedtuple type
account_fields = ["account_number", "customer_name", "balance"]
BankAccount = namedtuple("BankAccount", account_fields)

# Create account instances
acct1 = BankAccount(account_number="00123456",
customer_name="Alice", balance=1500.00)
acct2 = BankAccount("00987654", "Bob", 320.75)

# see the schema (field names) of the namedtuple
print("BankAccount fields:", BankAccount._fields)

# turn a list of accounts into a list of plain dicts
accounts = [acct1, acct2]
as_dicts = [a._asdict() for a in accounts]
print("All accounts as dicts:", as_dicts)
```

```
BankAccount fields: ('account_number', 'customer_name', 'balance')
All accounts as dicts: [{'account_number': '00123456', 'customer_name': 'Alice', 'balance':
1500.0}, {'account_number': '00987654', 'customer_name': 'Bob', 'balance': 320.75}]
```

QA

65

_fields is useful when creating dynamic (or introspective) code – you'll note that it returns a tuple, of course!

Other useful helpers for **namedtuple** include:

_make creates a new instances from an iterable, e.g. using a list

```
#creating new instance from a list (or iterable)
my_account_data = ["0000234567", "Kamran", 1250.00]
my_account = BankAccount._make(my_account_data)
print(my_account)
```

```
BankAccount(account_number='0000234567', customer_name='Kamran',
balance=1250.0)
```

_replace creates a modifiable "shallow" copy, e.g.

```
#replacing value
my_new_account = my_account._replace(balance=2500.00)
print(my_new_account)
```

```
BankAccount(account_number='0000234567', customer_name='Kamran',
balance=2500.0)
```

Notes.

A bit of unusual Python inconsistency: Although helpers are typically named using the single underscore prefix which normally indicates they are

private and not part of the `class`' public interface, this is not the case with a `namedtuple`; the single underscores were added to *avoid* potential *name clashes* with user-defined fields.

Further Standard library modules...

Standard library modules

- `Array` Efficient arrays of numeric values
- `bisect` Array bisection algorithm
- `copy` Shallow and deep copy operations
- `collections` Container data types
- `decimal` Decimal fixed point and floating-point arithmetic
- `fractions` Rational numbers
- `heapq` Heap queue algorithm
- `itertools` Functions creating iterators for efficient looping
- `math` Mathematical functions
- `random` Generate pseudo-random numbers
- `struct` Interpret bytes as packed binary data
- `types` Dynamic type creation
- `weakref` Weak references

QA

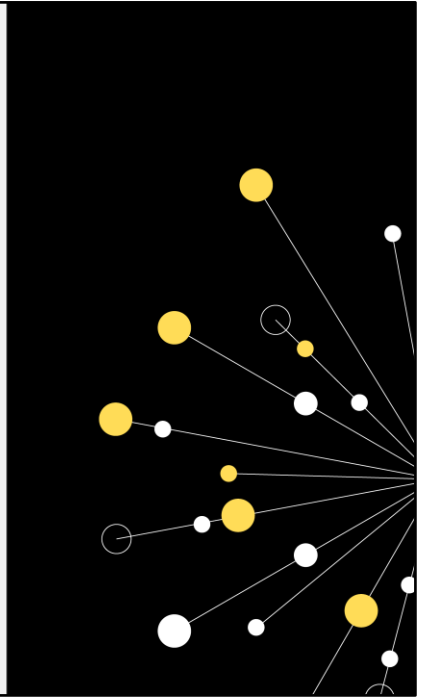
Exercise EG02: Advanced Variables

Activity instructions:

On your own or with a programming partner, implement a solution that takes a Pythonic approach and uses recognised best-practice techniques.

Compare and contrast your solution with your peers.

QA



Suggested time: 30-45 mins.

Activity includes differentiated tasks.

Summary

- Core type definitions are provided by the **types** module
- Implementation-specific behaviour of types can be used—but **with care**
- **Generators** produce values lazily, only as they are requested
- **Iterators** are created via an `__iter__()` method very often implemented internally using a generator
- The **collections** module adds several enhanced container types, including:
 - **namedtuple** – tuple subclasses with named fields
 - **deque** – list-like structure with fast appends and pops at both ends
 - **Counter** – dictionary subclass for counting hashable items
 - **defaultdict** – dictionary subclass that uses a factory function to provide default values
- Many classes in the **collections** module provide useful helper methods and attributes
- Most collection classes can be subclassed or extended to support additional custom behaviour

Appendices



View objects – examples

Can be used in:

- Iteration

```
acronyms = {"APR": "Annual Percentage Rate",
            "APY": "Annual Percentage Yield",
            "BACS": "Banker's Automated Clearing Services (UK)",
            "ROI": "Return On Investment"}

for kv in acronyms.items():
    print(kv)
```

```
('APR', 'Annual Percentage Rate')
('APY', 'Annual Percentage Yield')
('BACS', 'Banker's Automated Clearing Services (UK)')
('ROI', 'Return On Investment')
```

- To return keys into a list

```
lkeys = list(acronyms.keys())
print(lkeys)
```

```
['APR', 'APY', 'BACS', 'ROI']
```

- As part of set operations

```
new_acronyms = acronyms.keys() | {'ACH', 'BACS', 'SWIFT'}
print(new_acronyms)
```

```
{'SWIFT', 'BACS', 'ACH', 'APR', 'APY', 'ROI'}
```

QA

70

View objects returned by the `items()`, `keys()`, and `values()` methods can be iterated over and used to create lists.

The set operators (`&`, `|`, `^`, and `-`) are supported on `dict_keys` and `dict_items` view objects when used with a set, but they are not supported for `dict_values`.

ChainMap

A way to combine multiple dictionaries into a single view

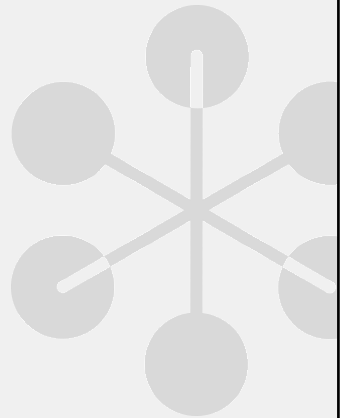
- More efficient than merging dictionaries with `dict.update()`
- Supports the usual dictionary methods on the chained result
- Changes to any underlying dictionary are immediately reflected, because they are stored by reference

Lookup behaviour

- Keys are searched **left to right** across the dictionaries
- This allows duplicate keys to coexist, with the leftmost one taking priority

Update behaviour

- All writes, updates, and deletions apply only to the **first** dictionary in the chain
- Additional methods allow you to insert a new dictionary or skip the current first one



QA

71

ChainMap is a relatively recent addition (3.3) to the collections module and is particularly useful in templating scenarios. Template systems often need to search both `locals()` and `globals()` together, and a **ChainMap** provides a simple way to treat them as a single combined mapping.

ChainMap can also offer performance benefits—looking up a key in a **ChainMap** requires only one logical lookup, whereas checking multiple dictionaries individually would require one lookup per dictionary. This can make a meaningful difference in operations executed millions of times.

However, from Python 3.9, the union (`|`) operator may be used with dictionaries, so removes the need for **ChainMap** in most cases

ChainMaps are flexible enough that they can even be used to mimic the behaviour of namespaces in C++.

ChainMap - Example

```
from collections import ChainMap
# Three separate directories
employees = {"Alice Green": "020 7000 1234", "John Smith": "020 7000 5678",}
customers = {"Mary Brown": "01632 960111", "Tom Harris": "01632 960222",}
suppliers = {"ACME Supplies": "0141 555 2020", "PaperCo": "0141 555 3030",}

# Combine them into a single searchable directory
phone_directory = ChainMap(employees, customers, suppliers)

# Look up entries
print(phone_directory["John Smith"]) # From employees
print(phone_directory["Mary Brown"]) # From customers
print(phone_directory["PaperCo"]) # From suppliers

# Adding a new employee (affects ONLY the first map)
employees["Chris Miles"] = "020 7000 9999"
print(phone_directory["Chris Miles"])

# Adding a new Layer – e.g., temporary contractors
contractors = {"Zara Khan": "07700 900111"}
phone_directory = phone_directory.new_child(contractors)

print(phone_directory["Zara Khan"]) # Found in the new top layer # Fails for union of
dictionaries
print(phone_directory.parents["Alice Green"]) # Underlying maps unaffected
```

```
020 7000 5678
01632 960111
0141 555 3030
020 7000 9999
07700 900111
020 7000 1234
```

OrderedDict

- A dictionary implementation that preserves the order in which keys were inserted Supports all standard dictionary operations, plus additional methods such as:
 - **popitem()** – returns a key/value pair
 - **move_to_end()** – relocates a key/value pair to either end of the dictionary
 - **reverse()** – iterates over the items in reverse order
- OrderedDict objects can be directly compared with regular dictionaries

```
from collections import OrderedDict
d = dict.fromkeys(['a', 'b', 'c', 'd'])
print(d)
od = OrderedDict.fromkeys(['a', 'b', 'c', 'd'])
print(od)
```

```
{'a': None, 'b': None, 'c': None, 'd': None}
OrderedDict([('a', None), ('b', None), ('c', None), ('d', None)])
```

QA

73

An **OrderedDict** is a specialised dictionary type that preserves the order in which items are inserted—something that was historically requested by many Python users.

Traditional dictionaries were originally **unordered**, meaning the insertion sequence was not guaranteed. As a result, developers often needed to sort the data before displaying it. **OrderedDict** provided a reliable way to maintain order explicitly.

However, unless you specifically need its additional features, it's generally best to avoid using **OrderedDict**. Its creator, Raymond Hettinger, notes that it is significantly slower—around five times slower—than standard dictionaries.

Note:

Since Python 3.6, built-in dictionaries have maintained insertion order due to changes in their internal implementation. Because of this, many standard library functions now return a regular dictionary—one that already preserves order—rather than an **OrderedDict**.

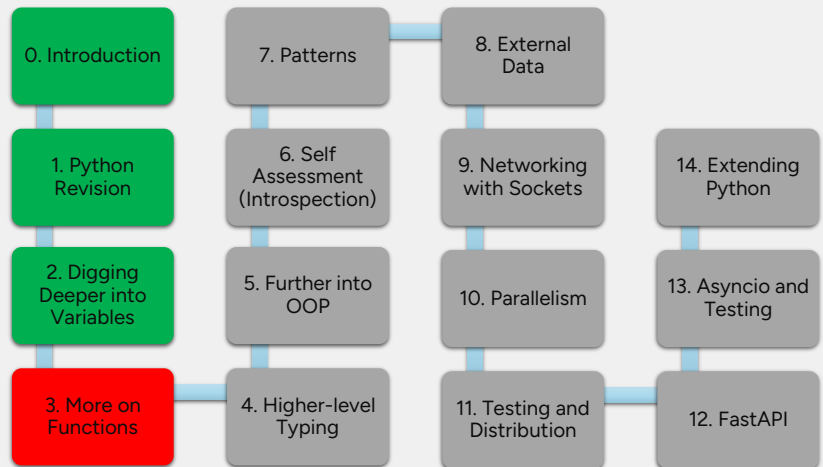
As a result, the practical need for **OrderedDict** has decreased, although it remains useful for backwards compatibility and for codebases that must support older Python versions.

Intermediate Python

More on Functions



Module Roadmap



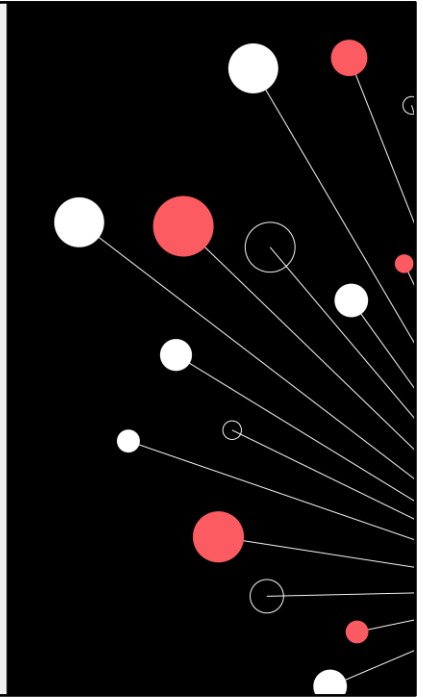
QA

Learning Objectives

By the end of this session, you will be able to:

- Use positional, keyword, and enforced parameter-passing techniques effectively.
- Avoid common pitfalls such as the mutable default-parameter trap.
- Apply recursion and build dispatch tables for cleaner control flow.
- Use tools like `operator.itemgetter` to simplify data extraction.
- Create nested functions, closures, and decorators to structure and extend functionality.
- Apply `functools.partial` and `functools.wraps` to build flexible and well-behaved functions.

QA



Passing Parameters – Review

The function:

```
def calculate_savings_balance(principal, int_rate, monthly_deposit=12):  
    print(f'principal: {principal}, rate: {int_rate}, deposit: {monthly_deposit}')
```

Passing by position:

```
calculate_savings_balance(1000, 0.03, 50)
```

```
principal: 1000, rate: 0.03, deposit: 50
```

Passing by default:

```
calculate_savings_balance(1000, 0.03)
```

```
principal: 1000, rate: 0.03, deposit: 12
```

Or passing by name:

```
calculate_savings_balance(principal=1000, monthly_deposit=50, int_rate=0.03)
```

```
principal: 1000, rate: 0.03, deposit: 50
```

QA

77

Before examining parameter passing in more detail, it is useful to review the mechanisms that you should already be familiar with.

Note: When defining keyword parameters and providing arguments in a function call, no spaces are used around the = sign, unlike in a typical assignment. This helps make it clear that the expression represents a single argument when reading the code.

Refer to the appendix of this chapter for additional notes about the *default parameter trap*.

Passing Parameters – Unpacking and Variadic Functions

Functions mostly have a fixed number of parameters

- *Unpacking (using the * prefix) allows the elements of a sequence to be passed as an individual argument:*

```
def calc_product(x, y, z):  
    print(f"product: {x * y * z}")  
  
numbers_tuple = 2, 4, 6  
calc_product(*numbers_tuple)
```

48

Variadic functions accept a variable number of arguments into a single parameter

- Declared as a variable (of type tuple) with a * prefix

```
def calc_products(a, *nums):  
    res = ""  
    for n in nums:  
        res += f"{str(a * n)} "  
    print(res)  
  
calc_products(5, 1, 2, 3, 4)
```

5 10 15 20

QA

78

A **variadic function** is one in which a collection of parameters is packed into a single tuple allowing the function to accept an indefinite number of parameters ranging from zero to pretty much any number you like.

In this context, the named parameter (*nums* in the example above) represents the **tuple itself**. Keep in mind that while the tuple's structure is **immutable**, it may contain references to objects that are **mutable**. Because a tuple has an unknown length, it must always be declared as the **final** (right-most) parameter.

Enforcing Named Parameters

Using a bare `*` forces users to name arguments explicitly for all parameters declared to its right

```
def calculate_savings_balance(*, principal, int_rate, monthly_deposit=12):  
    print(f'principal: {principal}, rate: {int_rate}, deposit: {monthly_deposit}')
```

```
calculate_savings_balance(principal=1000, int_rate=0.03, monthly_deposit=50)  
calculate_savings_balance(principal=1000, int_rate=0.03)
```

```
principal: 1000, rate: 0.03, deposit: 50  
principal: 1000, rate: 0.03, deposit: 12
```

Attempting to pass positional parameters will fail:

```
calculate_savings_balance(1000, 0.05)
```

```
TypeError: calculate_savings_balance() takes exactly 0 positional arguments (2 given)
```

QA

79

The example demonstrates a method of requiring users to provide arguments by name instead of by position. However, invoking the function without a third (`monthly_deposit`) argument is still valid because, in this case, it has a default value.

Any parameters defined after (to the right of) the `*` must be passed as named arguments, while those before it can still be provided positionally.

Enforcing Positional Only Parameters With "/"

You can use a bare / to specifying positional-only arguments in a function definition

- Introduced in Python 3.8
- All arguments *before* the / **must** be specified by position:

```
def calculate_savings_balance(principal, int_rate,/, monthly_deposit=12):  
    print(f'principal: {principal}, rate: {int_rate}, deposit: {monthly_deposit}')
```

```
calculate_savings_balance(1000, 0.03, monthly_deposit=50)
```

```
calculate_savings_balance(1000, 0.03)
```

```
principal: 1000, rate: 0.03, deposit: 50  
principal: 1000, rate: 0.03, deposit: 12
```

Attempting to pass named parameters will fail:

```
calculate_savings_balance(principal=1000, int_rate=0.03)
```

```
calculate_savings_balance() got some positional-only arguments passed as keyword  
arguments: 'principal, int_rate'
```

QA

80

Use a **forward slash (/)** to enforce **positional-only** arguments.

Parameters **left** of the / must be positional; those to the **right** can be named.

Default values remain functional if no argument is provided.

Keyword Parameters (kwargs)

Appear the same as dictionary key–value pairs (because they are!)

Use `**` before a parameter to indicate a dictionary

- Must appear at the end of the parameter list
- `**` can be used to unpack arguments from a dictionary when calling a function

```
def open_account(name, initial_deposit, **options):
    print(f"Opening account for {name}")
    print(f"Initial deposit: £{initial_deposit}")

    # kwargs let you handle optional settings
    for key, value in options.items():
        print(f"{key}: {value}")

open_account("Alice", 500, account_type="Savings",
             overdraft_limit=1000, notifications=True)

args_as_dict = { "account_type": "Mortgage",
                 "type": "2 year Fixed_Rate", "notifications": True }

open_account("Kamran", 50000, **args_as_dict)
```

```
Opening account for Alice
Initial deposit: £500
account_type: Savings
overdraft_limit: 1000
notifications: True
Opening account for Kamran
Initial deposit: £50000
account_type: Mortgage
type: 2 year Fixed_Rate
notifications: True
```

QA

81

Keyword parameters (commonly referred to as **kwargs**) allow arguments to be passed to a function as a dictionary.

To unpack named arguments, two asterisks (`**`) are used as a prefix. The arguments are collected into a dictionary, which is unordered, so the caller does not need to provide them in a specific order. This is especially helpful when a function has many parameters.

A parameter prefixed with `**` must appear at the end of the parameter list; placing it anywhere else will result in a syntax error.

Lambda Functions

- **Anonymous functions:** Lambdas act as concise, unnamed shorthand.
- **Expression-based:** Limited to a single expression; no **branches** or **loops**.
- **Conditional support:** Supports **conditional expressions** (ternary logic).
- **No statements:** Cannot include **explicit returns** or **variable assignments**.
- **Implicit return:** The evaluation of the final expression is the **automatic return value**

```
calculate_interest = lambda balance, rate: balance * rate
```

```
interest = calculate_interest(2000, 0.03)  
print("Interest earned:", interest)
```

Parameters

Expression
evaluates to implied return

Interest earned: 60.0

Commonly used in-line with `map()` and `filter()`

- Applies an specific function to every element in a list or iterable

```
balances = [1200, 3400, 560, 890]  
# Apply 2% monthly interest to each balance  
updated_balances = list(map(lambda b: b * 1.02, balances))  
print(updated_balances)
```

[1224.0, 3468.0, 571.2, 907.8000000000001]

QA

82

The term **lambda** originates from Lisp and can seem a little intimidating. Because it is a Greek letter and sounds mathematical, it may give the impression of complexity, but in Python lambda functions are actually quite simple. You can think of *lambda* as meaning an **inline function**.

Lambda functions cannot perform any tasks that regular functions defined with **def** cannot do; they are simply shorter and quicker to write. They are particularly useful for small, inline operations that do not require a function name. A lambda expression returns a reference to a function, which can be stored in a variable and then used just like a named function.

Many Python built-in functions accept a **callable object** as an argument. This could be the name of a function, a variable referencing a function, or a lambda expression used directly without assigning it a name. For example, the `map()` function can use a lambda to add one to every item in a list. This will be explored further in the following slides.

Lambda as a Sort Key and Alternatives

Takes the element to be compared

- Returns the key
- Sort each account by the second field, balance

```
accounts = [  
    ("Alice", 3200),  
    ("Bob", 1500),  
    ("Charlie", 5400),  
    ("Diana", 2750)  
]  
  
# Sort accounts by balance  
sorted_accounts = sorted(accounts, key=lambda account: account[1])  
  
print(sorted_accounts)  
[('Bob', 1500), ('Diana', 2750), ('Alice', 3200), ('Charlie', 5400)]
```

QA

83

Lambdas are frequently passed directly into built-in functions. Their use replaces the need for a separate, named comparison function. They take a single argument to extract the key field for sorting or comparison.

operator.itemgetter

Returns a callable that retrieves a specific element from an object

- Most widely used function in the operator module
- Frequently paired with sorting operations
- Lambda approach:
 - `sorted(data, key=lambda x: x[0])` – sorts by name
- `itemgetter` approach:
 - `sorted(data, key=itemgetter(2))` – sorts by currency
- Supports dictionary keys and multiple indices simultaneously

```
import operator
```

```
countries = [
    ["United Kingdom", 44, "Pounds"],
    ["Canada", 1, "Dollar"],
    ["Japan", 7, "Yen"],
    ["France", 34, "Euro"]
]
```

```
# Sort by name using a Lambda
```

```
c = sorted(countries, key=lambda c: c[0])
print(c)
```

```
# Sort by currency using itemgetter
```

```
c = sorted(countries, key=operator.itemgetter(2))
print(c)
```

```
[[ 'Canada', 1, 'Dollar'], [ 'France', 34, 'Euro'], [ 'Japan', 7, 'Yen'], [ 'United Kingdom', 44, 'Pounds']]
[[ 'Canada', 1, 'Dollar'], [ 'France', 34, 'Euro'], [ 'United Kingdom', 44, 'Pounds'], [ 'Japan', 7, 'Yen']]
```

QA

84

The `operator` module provides functional alternatives to **special methods** (e.g., `operator.add` vs. `__add__`). `itemgetter` is a high-performance tool for retrieving specific indices or keys from objects.

Frequently used as a **key function** in `sort()` and `sorted()` for objects implementing `__getitem__`.

Lambda Functions – Other Uses

Sometimes Python utilities like `map()` need to apply a function that requires extra arguments, not just the list elements themselves.

- The following code has issues because calling `apply_fee` with a parameter triggers a Type exception...

```
def apply_fee(amount, fee):  
    return amount - fee
```

```
transactions = [120, 85, 40]
```

```
# This fails
```

```
fees_applied = list(map(apply_fee(2), transactions))
```

```
print(fees_applied)
```

```
TypeError: apply_fee() missing 1 required positional argument: 'fee'
```

- The issue can be solved by using a lambda:

```
fees_applied = list(map(lambda t: apply_fee(t, 2), transactions))
```

```
print(fees_applied)
```

```
[118, 83, 38]
```

Note: This example would be better solved using a list comprehension:

```
fees_applied = [t - 2 for t in transactions]
```

QA

85

`functools.partial` could also be used to solve the problem when needing to pass arguments to function objects:

```
from functools import partial
```

```
def apply_fee(amount, fee):  
    return amount - fee
```

```
transactions = [120, 85, 40]
```

```
fee = 2
```

```
# Create a new one-argument version of apply_fee
```

```
apply_standard_fee = partial(apply_fee, fee=fee)
```

```
# Now map works perfectly, because apply_standard_fee(amount) expects  
only 1 argument
```

```
fees_applied = list(map(apply_standard_fee, transactions))
```

```
print(fees_applied)
```

Recursion

Recursion occurs when a function calls itself.

- A terminating (base) condition is required; without it, recursion continues indefinitely.
- Python imposes a maximum recursion depth to prevent infinite recursion.

RuntimeError: maximum recursion depth exceeded ...

- You can check the current limit using `sys.getrecursionlimit()`.
 - The default recursion limit is about 1000.
- The limit can be modified using `sys.setrecursionlimit()`.
 - **Caution:** Setting the limit too high can cause problems.
 - It may exceed the stack size limit enforced by the C compiler, especially on Microsoft Windows.
- Recursion limits are often reached due to bugs or inefficient program design.
 - The recursion limit exists for safety and stability reasons.
 - Recursion is computationally expensive in Python, so iteration is often a better alternative.

QA

86

Recursion provides a clean, logical solution for specific algorithms. Each recursive call adds a **stack frame**, consuming significant memory. Exceeding the limit typically indicates an **infinite loop** or a **design flaw**. Heavy recursion risks **stack overflow** or memory exhaustion.

Dispatch Tables

An alternative to using if...elif...else

```
bal = 100
bal = process_trans(bal, "deposit", 50)
bal = process_trans(bal, "withdraw", 20)
bal = process_trans(bal, "check_balance")
print(bal)
```

```
def deposit(balance, amount):
    return balance + amount

def withdraw(balance, amount):
    return balance - amount

def check_balance(balance, *_):
    return balance

# Dispatch table mapping operation names to functions
operations = {
    "deposit": deposit,
    "withdraw": withdraw,
    "check_balance": check_balance
}

def process_trans(balance, op, amount=0):
    try:
        action = operations[op]
    except KeyError:
        raise ValueError(f"Unknown operation: {op}")

    return action(balance, amount)
```

QA

87

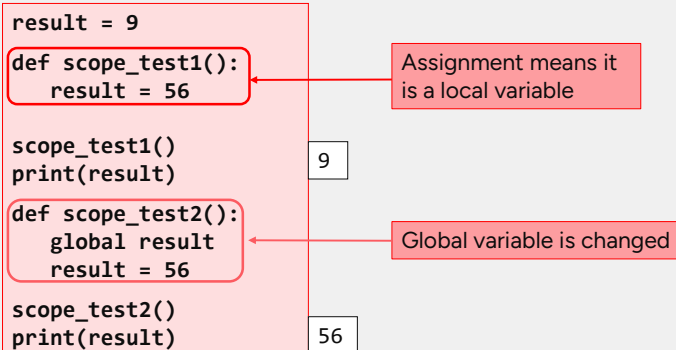
Python functions, including lambdas, are first-class objects that can be stored directly in dictionaries. This allows us to create a **dispatch tables**, which serve as an efficient alternative to **if...elif** blocks or **switch** statements found in other languages. It is a common way to link user inputs, like menu options, to specific logic.

When using this pattern, you must verify a **key exists** before attempting to call the associated function. While these functions should generally share the same **parameter count**, you can use **default values** or **variadic arguments** to maintain flexibility. Ultimately, this approach mirrors Python's own internal mechanism for looking up names and methods.

Variables in Functions

Variables in a function can be local or global:

- Variables are defined as **local** if a value is **assigned**, otherwise global.
- Global variables can be referenced using the global keyword.
 - But are local to the current module, or namespace.



QA

88

In Python, the **scope rules** determine whether a variable is treated as **local** or **global**. If a variable is **used** in a function without being assigned a value, Python assumes it refers to a **global variable**. However, if a variable is **assigned** within the function, it is considered **local** - unless explicitly declared as **global**.

Variables created inside a function exist only within that function's **local scope** and are discarded when the function exits. For example, two variables named `result`, one inside a function and one outside, are entirely independent. Assigning a new value to the local `result` does not affect the global one.

If you need to modify a global variable from within a function, you must declare it with the **global** keyword **before** using it. Although Python may allow the declaration later in the function, doing so can raise syntax warnings.

In practice, **global variables should be avoided** wherever possible. They can make code harder to maintain and are especially problematic in multi-threaded applications, where concurrent access can lead to unpredictable

results.

Nested functions

The **def** statement defines a function object

- This has the same scope as any other object

```
def outer_func():  
    num = 56  
    def inner_func():  
        print(num, "in inner func")  
    inner_func()  
    print(num, "in outer func")  
  
outer_func()  
inner_func()
```

Outer function hides inner function

Inner function call is in scope

```
56 in inner func  
56 in outer func  
NameError: name 'inner_func' is not defined
```

Nested functions can be used to encapsulate helper code

- Can be returned as *closures*

QA

89

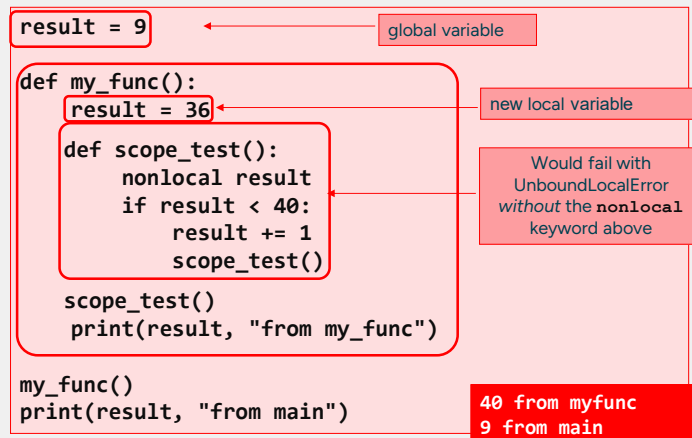
Defining a **function within another function** is a feature not supported by all programming languages, but in Python it can be very useful. It is commonly used for **simple helper functions**, for creating **closures**.

In the example, the inner function can access the variable `num` even though it is not declared as global - this works because `num` remains **in scope** within the enclosing function. However, if the inner function were to **assign** a new value to `num`, Python would treat it as a **new local variable**, visible only within that inner function.

Note: A **closure** in Python is a function that **remembers the values** from its **enclosing scope**, even after that scope has finished executing. In other words, the inner function "closes over" the variables it needs from the outer function, allowing it to access and use them later. Closures are often used to **preserve state**, create function factories, or implement encapsulation without using classes.

Variables in nested functions

Since scopes may be nested, we need to indicate that



QA

90

Python's clean syntax and dynamic typing mean there is no explicit way to declare new variables. This can sometimes cause confusion over when a new variable is created and when an existing one is reused. We have already seen how the `global` statement identifies a variable as belonging to the global scope.

But what about *enclosing* scopes? Python is unusual in allowing local or *nested* functions. To indicate that a variable belongs to an enclosing scope rather than the local one, we use the `nonlocal` keyword.

In the example on the slide, if `nonlocal` were omitted, the `if` statement would test a variable before it was defined, since `result` is not declared as `global` (nor should it be). Declaring it as `nonlocal` reuses the `result` variable from the outer function, allowing the recursion to work correctly.

If the inner function (`scope_test`) is returned and later called externally, it still retains access to the `nonlocal` variable:

```
def my_func():
    return scope_test

rfunc = my_func()
```

`rfunc()`

This behaviour is known as a **closure**.

Closures

A closure happens when an inner function “remembers” values from its enclosing scope

- Even after that outer function has finished running.

```
def make_interest_calculator(rate):  
    "Creates a function that applies a fixed interest rate."  
    def apply_interest(balance):  
        return balance + (balance * rate)  
  
    # This inner function closes over 'rate'  
    return apply_interest  
  
# Create two different calculators with different interest rates  
savings_interest = make_interest_calculator(0.03) # 3%  
loan_interest = make_interest_calculator(0.07) # 7%  
  
# Use them  
print(savings_interest(1000))  
print(loan_interest(1000))
```

1030.0
1070.0

QA

91

A closure is formed when a function is returned while still referencing variables from the surrounding scope. Each closure keeps its own private copy of those variables, giving every instance an independent state. Once created, closures behave just like regular functions.

They're especially handy for building functors—functions that carry their own data and perform work using that internal state. A closure can wrap a simple function call, an object method, or a class method, and it can capture static or default parameters, all hidden from the user.

Closures also play an important role in decorators, which we'll explore later. Another example:

```
def get_data(orig, tipe):  
    temp = orig.split(" ")  
  
    def get_tuple():  
        return tuple(temp)  
  
    def get_list():  
        return temp  
  
    if tipe == 1:  
        return get_tuple  
  
    elif tipe == 2:  
        return get_list
```

```
data = "Phil Frank Ron"
result1 = get_data(data, 1)
result2 = get_data(data, 2)

print(result1) # prints <function get_data.<locals>.get_tuple at 0x00...>
print(result2) # prints <function get_data.<locals>.get_list at 0x00...>

print(result1()) # ('Phil', 'Frank', 'Ron')
print(result2()) # ['Phil', 'Frank', 'Ron']
```


Closure Attributes

Closures maintain attributes of the original:

```
1030.0
Closure function __name__ :
  apply_interest
Closure function __doc__ :
  Apply the captured interest
  rate to a balance.
Has closure? : True
Captured value(s) : [0.03]
Factory __name__ :
  make_interest_calculator
Factory __doc__ :
  Creates a function that
  applies a fixed interest
  rate.
```

```
def make_interest_calculator(rate):
    """Creates a function that applies a fixed interest rate."""
    def apply_interest(balance):
        """Apply the captured interest rate to a balance."""
        return balance + (balance * rate)

    return apply_interest # This inner function closes over 'rate'

# Create two different calculators with different interest rates
savings_interest = make_interest_calculator(0.03) # 3%

# Use them
print(savings_interest(1000)) # 1030.0

# --- Demonstrate closure attributes ---
print("Closure function __name__ :", savings_interest.__name__)
print("Closure function __doc__ :", savings_interest.__doc__)

# Show what was captured in the closure
cells = savings_interest.__closure__
print("Has closure? :", cells is not None)
if cells:
    # There may be multiple cells; here we expect one (the 'rate')
    print("Captured value(s) :", [cell.cell_contents for cell in cells])

# Also show the outer factory's attributes for contrast
print("Factory __name__ :", make_interest_calculator.__name__)
print("Factory __doc__ :", make_interest_calculator.__doc__)
```

QA

92

Decorators

Decorators: Key Ideas

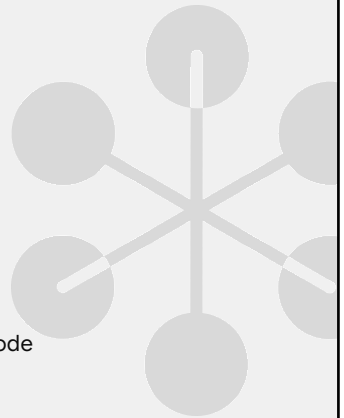
- Functions that accept another function as input
- Wrap existing functions to extend or modify their behaviour
- Often return a new function, though this isn't required

Decorator Characteristics

- Integrated directly into Python's function syntax
- A core technique in metaprogramming

Why Use Decorators

- Add optional, reusable functionality across many functions
- Support the Do Not Repeat Yourself (DRY) principle by eliminating repeated code
- Improve readability and structure
- Allow behaviour to be updated independently of the functions that use them



QA

93

Decorators were introduced in Python 2.4 and are a core part of function syntax designed to simplify metaprogramming. The technique allows code to modify itself at runtime, providing a way to wrap or enhance functions dynamically.

A decorator can be applied to any function that accepts another function as an argument. While they were originally added to clean up the syntax for class methods, they are now frequently used with various built-ins.

According to **PEP 318**, the primary justification for decorators is readability. Before their introduction, using tools like `property()` was clunky; programmers had to define functions with placeholder names and manually bind them to attributes later. Decorators solve this problem by clearly linking the enhancement to the function definition, making the logical connection immediate and obvious.

Decorator Syntax (1)

Higher-Order Function + Monkey Patch Decorator

This demonstrates:

- A decorator implemented as a higher-order function.
- Applying it via monkey patching (manually replacing the function with the decorated version at runtime).

```
def log_action(func):
    """A decorator that logs the function call."""
    def wrapper(*args, **kwargs):
        print(f"[LOG] Calling {func.__name__} with arguments {args} {kwargs}")
        return func(*args, **kwargs)
    return wrapper

# Core banking function (un-decorated)
def deposit(balance, amount):
    return balance + amount

# --- Monkey patching the function with its decorated version ---
deposit = log_action(deposit)  # ← monkey patch

# Use it
balance = 100
balance = deposit(balance, 50)
print("New balance:", balance)
```

```
[LOG] Calling deposit with arguments (100, 50) {}
New balance: 150
```

QA

94

This example relies on the fact that Python supports higher order functions – a function that accepts other functions as arguments.

When the function is called, the decorator is executed with the original function passed as an argument. Inside the decorator, optional actions can be performed *before and/or after* a call to the original function. In the example a "Log" message is printed to the console.

`log_action()` is a **higher-order function** that returns a new function. `deposit = log_action(deposit)` *replaces* the original function at runtime → **monkey patching**.

This approach has largely been usurped by "pie" syntax style of decorators (see next slide).

Decorator Syntax (2)

Pie Syntax Decorator

This demonstrates:

- the modern, clean approach to decorators

```
# Pie Syntax Decorator:
def limit_withdrawals(func):
    """Decorator enforcing a max withdrawal amount per transaction."""
    def wrapper(balance, amount):
        if amount > 300:
            raise ValueError("Withdrawal exceeds daily limit of £300.")
        print(f"[SECURITY] Withdrawal validated: £{amount}")
        return func(balance, amount)
    return wrapper

@limit_withdrawals # ← pie syntax decorating the function
def withdraw(balance, amount):
    return balance - amount

# Use it
balance = 500
balance = withdraw(balance, 100)
print("New balance:", balance)

balance = withdraw(balance, 400)
print("New balance:", balance)
```

```
[SECURITY] Withdrawal validated: £100
New balance: 400
ValueError: Withdrawal exceeds daily limit of £300.
```

QA

95

This second version of the Decorator pattern uses "pie" syntax (@ symbol) to "decorate" the function with the selected wrapper; it's a neater and syntactically clearer solution and is used, these days, in preference to the higher-order approach..

In both cases the same wrapper could be applied to multiple functions, affecting any of them, but without needing to make the same changes to each function.

Practical Decorator Example - a Function Timer

```
import time
import functools

def my_timer(fn):
    """Display execution time of decorated function"""
    def timer_wrapper_func(*args, **kwargs):
        # Start the timer
        start_t = time.perf_counter()
        # Call the function
        result = fn(*args, **kwargs)
        # End timer and work out elapsed time
        end_t = time.perf_counter()
        duration = end_t - start_t
        print(f"{fn.__name__} !r} execution time {duration:.5f} s")
        return result
    return timer_wrapper_func

@my_timer
def calc_cubes(qty):
    """Return sum of cubes from 1 to n inclusive"""
    value = sum([num ** 3 for num in range(1, qty)])
    return value

# Calling with decoration is completely transparent
print(calc_cubes(5))
```

```
'calc_cubes' execution
time 0.00001 s
100
```

QA

96

This example shows two different techniques for creating a practical decorator which adds timing behaviour to any pre-written function.

This concept relies on the fact that Python supports higher order functions – a function that accept other functions as arguments ("functions are first-class citizens").

This version uses "pie" syntax (@ symbol) to "decorate" the function with the selected wrapper; it's a little neater, syntactically than the older "monkey patch" solution.

When the function is called, the decorator is executed with the original function passed as an argument. Inside the decorator, timing actions are performed *before and after* a call to the original function.

In either case the same wrapper could be applied to multiple functions, affecting any of them, but without needing to make the same changes to each function.

Partial Application

`functools.partial()` function

Enables us to:

- Duplicate existing functions (even built-ins) with *some parameters already set*

Resulting in:

- A new "implementation" of the function which is:
 - functionally *similar* but *not identical*
 - well-documented
 - easily maintainable

```
# Fixed currency FX conversion function
# Great for risk engines or trade-capture components.
from functools import partial

def fx_convert(amount, rate, fees):
    return amount * rate * (1 - fees)

# Create a GBP->USD converter with fixed spread + fee
gbp_usd = partial(fx_convert, rate=1.2610, fees=0.0005)

print(gbp_usd(100_000)) # convert 100k GBP to USD
```

126036.95

QA

97

In this example we create a simple function "fx_convert" which converts currency using an amount, rate and fees. By passing values to the function it calculates the amount a customer would get when buying foreign currency.

The `partial()` method is then used to pre-fill the constant parameters (the rate and fees). `partial()` creates a new function (called `gdp_usd`) where the rate is permanently set to 1.2610 and the fees to 0.0005. This means the new function only needs the amount (100000).

This approach creates a domain-specific function, so instead of calling: `fx_convert(100_000, rate=1.2610, fees=0.0005)` you now call: `gbp_usd(100_000)`.

This makes for a cleaner and safer environment for a trading system where:

- the rate is agreed
- the spread/fee is fixed per product
- only the amount changes trade-to-trade

Normally function arguments are evaluated for left to right before the function call. With `functools.partial` only some of the arguments are evaluated, and others are supplied later.

Partial objects have no `__name__` or `__doc__` attributes, so attempting to read them will result in an `AttributeError`. This can be alleviated using `functools.update_wrapper()`. Other attributes can also be added using this function. Inside a class, partial objects are not bound to that class.

Customising Decorators with Parameters

To pass arguments to a decorator, you need an additional outer layer to "catch" those arguments before the actual decoration happens:

- **Outer Function:** Receives the custom parameters (e.g., transaction type).
- **Decorator Function:** Receives the original function being decorated.
- **Wrapper Function:** Performs the actual work (logging, validation) around the original function.

```
def transaction_logger(label): # 1. Outer: Catch the parameter
    def decorator(func):      # 2. Middle: Catch the function
        def wrapper(self, amount):
            print(f"[LOG] Starting {label} for {self.owner}...")
            result = func(self, amount)
            print(f"[LOG] {label} finished. Balance: ${self.balance}")
            return result
        return wrapper
    return decorator

class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    @transaction_logger("DEPOSIT") # Parameter "DEPOSIT" passed here
    def deposit(self, amount):
        self.balance += amount

account = BankAccount("Ted", 1000)
account.deposit(100)
```

```
[LOG] Starting DEPOSIT for Ted...
[LOG] DEPOSIT finished. Balance: $1100
```

98

Passing arguments to decorators is not quite as simple as it seems, because of the calling convention. To make it happen we need to write an outer function to handle the arguments, but we must pass the arguments by keyword. It can be helpful to think of it as a "factory" that creates the actual decorator. In the banking system example on the slide, the technique is being used to create a single logging decorator that can be customized for different transaction types like "DEPOSIT"

Why Use This?

- **Code Reusability:** One decorator can handle multiple scenarios just by changing the input parameter.
- **Separation of Concerns:** Keep your core banking logic (adding/subtracting money) separate from your logging or security logic.
- **Dynamic Behaviour:** You can change how a function is "wrapped" at runtime based on the argument provided.

Preserving Metadata with functools.wraps

The Problem: The "Identity Thief"

When you decorate a function in Python, the original function (e.g., `deposit`) is replaced by the wrapper. This causes the original function to "lose" its identity:

- `deposit.__name__` becomes "wrapper"
- `deposit.__doc__` (the docstring) disappears.
- This breaks debugging tools and automated documentation (like Swagger)

The Solution: `functools.wraps`

By using `@wraps(func)` inside your decorator, Python copies the metadata from the original function onto the wrapper.

```
from functools import wraps

def security_audit(func):
    @wraps(func) # Copies name and docst
    def wrapper(self, *args, **kwargs):
        # Internal audit wrapper logic.
        print(f"Securing {func.__name__}...")
        return func(self, *args, **kwargs)
    return wrapper

class BankAccount:
    @security_audit
    def deposit(self, amount):
        """ increases funds after identity verification. """
        self.balance += amount

# Without @wraps: account.deposit.__name__ => "wrapper"
# With @wraps:   account.deposit.__name__ is "deposit"
print(f"Function Name: {BankAccount.deposit.__name__}")
print(f"Docstring: {BankAccount.deposit.__doc__}")
```

We could add more functionality in here!

Ensures arguments can be passed by position or keyword

```
Function Name: deposit
Docstring: increases funds after identity verification.
```

99

`functools.wraps` is the same as:

```
functools.partial(functools.update_wrapper,
                  wrapped=wrapped, assigned=assigned, updated=updated)
```

Key Benefits of `Functools.wraps`

- **Debugging:** Tracebacks show the actual function name, not just "wrapper".
- **Documentation:** Keeps your API documentation accurate by preserving docstrings.
- **Introspection:** Allows other tools to "see" the original function's signature and attributes.

See also...

Standard library modules

- `functools` Higher-order functions and operations
- `itertools` Functions creating iterators for efficient looping
- `operator` Standard operators as functions



QA

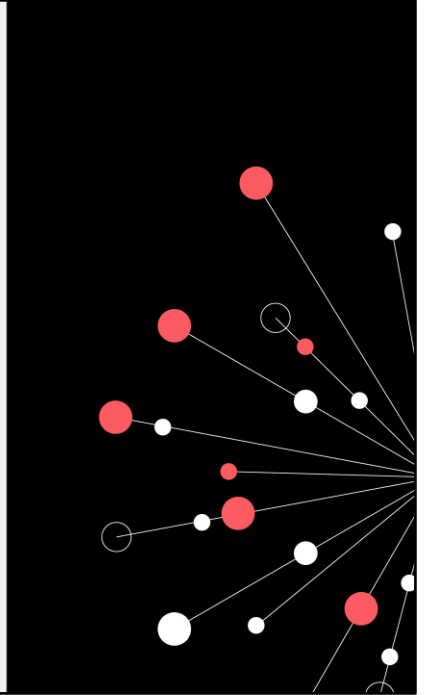
Exercise EG03: Advanced Functions

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA

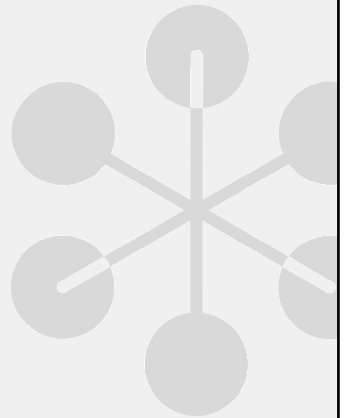


Suggested time: 30-45 mins.

Activity includes differentiated tasks.

Summary

- A function is a defined object
- And a "first-class" citizen in Python!
 - Be careful of mutable defaults
 - Calling a function imposes an overhead
 - Use bound functions where possible
 - Avoid deep recursions
 - Nested functions can be returned - closures
- They retain their `globals()` and `locals()`
 - Decorators can modify function behaviour
 - `functools partial` allows us to decouple names
 - Metadata can be set using `functools wraps`
- These can also be used to pass parameters to decorators



Appendices



The "Mutable Default Parameter" Trap

- **Functions are Objects:** In Python, defining a function is like creating any other object; it happens once at **definition time**.
- **Top-Down Execution:** Python evaluates default arguments only once—when the `def` statement is first encountered.
- **The Common Scenario:**
 - A function requires a parameter to be a list (or another **mutable** object).
 - The developer sets the default value to an empty list `[]`.
- **The Reality:**
 - Only **one** list object is created at definition.
 - The parameter reference becomes **static**, persisting and accumulating data between multiple calls.

```
def add_to_new_list(name, new_list=[]):  
    new_list.append(name)  
    return new_list
```

```
add_to_new_list("Hamza")  
add_to_new_list("Sadia")  
add_to_new_list("Kate")
```

```
['Hamza']  
['Hamza', 'Sadia']  
['Hamza', 'Sadia', 'Kate']
```

QA

104

This is an easy trap to fall into for beginners.

The problem is that most people assume default parameters are like function body assignments - that they are actioned when the function is called. Not so!

The default is executed when the function is defined - just once. In the example, the empty list is created when the function is defined. Only one list object is created. After that, each subsequent call to the function uses that same list object, repeatedly appending to it.

A similar problem exists for any mutable object given as a default.

Note: A disassembly view:

```
>>>import dis  
>>> m=[]  
>>> dis.dis("m.append('Jess')")  
1          0 LOAD_NAME           0 (m)  
          2 LOAD_METHOD        1 (append)  
          4 LOAD_CONST          0 ('Jess')  
          6 CALL_METHOD         1  
          8 RETURN_VALUE  
  
>>> f=m.append
```

```
>>> dis.dis("f('Frank')")
1          0 LOAD_NAME          0 (f)
           2 LOAD_CONST        0 ('Frank')
           4 CALL_FUNCTION      1
           6 RETURN_VALUE
```

The "Mutable Default Parameter" – A Solution

Best Practice: Never use mutable objects as default arguments

- Use None and initialize inside the function instead.

```
def add_to_new_list(name, new_list=None):  
    if new_list is None:  
        new_list = []  
    new_list.append(name)  
    return new_list  
  
add_to_new_list("Hamza")  
add_to_new_list("Sadia")  
add_to_new_list("Kate")
```

`['Hamza']`
`['Sadia']`
`['Kate']`

Now a new empty list is created at each runtime call

- Assuming the parameter is defaulted

QA

105

The Pythonic Solution is to use None. This should be considered best practice because default parameters should use **None** (or another immutable object like an empty string) instead of a mutable list along with a run time check that uses a conditional test (e.g., if param is None:) inside the function body to initialize the empty list.

This works because the function body executes only at runtime (when it is called), unlike the function definition on the previous slide. This creates a fresh, unique object for every single execution which eliminates "shared state" bugs and ensures each function call starts with a clean slate.

Recursion – Testing Stack Limit

The following code will **test** your stack limit:

```
def call_me_again_from_within(n):  
    n += 1  
    print(n)  
    call_me_again_from_within(n)  
  
call_me_again_from_within(0)
```

It has **no terminating condition** so will recurse *until* the stack limit is exceeded.

Any value above **990** is quite common!

```
Traceback (most recent call last):  
  File "<pyshell#14>", line 14, in <module>  
    call_me_again_from_within(1)  
  File "<pyshell#13>", line 12, in call_me_again_from_within  
    call_me_again_from_within(n)  
  File "<pyshell#13>", line 12, in call_me_again_from_within  
    call_me_again_from_within(n)  
  File "<pyshell#13>", line 12, in call_me_again_from_within  
    call_me_again_from_within(n)  
### [Previous line repeated 996 more times] ###  
  File "<pyshell#13>", line 12, in call_me_again_from_within  
    call_me_again_from_within(n)  
RecursionError: maximum recursion depth exceeded
```

QA

106

Recursion is a common tool for algorithms like tree traversal, Fibonacci sequences, or Newton's square root because it offers an elegant, clean solution.

The main drawback is memory usage. Every recursive call creates a new stack frame that must be stored until the function returns. If your recursion goes too deep, you will hit the recursion limit. When this happens, it usually signals one of two things: a bug in your logic (like an infinite loop without an exit condition) or a design flaw in the algorithm itself.

Recursion – Tracing a "Money Trail" Example

Using recursion to trace the origin of a transaction in an audit trail, where one transaction may be a "child" of a previous one :

```
def get_transaction_source(tx_id, ledger):
    # Recursively find the original transaction ID
    parent_id = ledger.get(tx_id)
    if parent_id is None:
        return tx_id
    # Recursive Case: Move up the chain to the parent
    return get_transaction_source(parent_id, ledger)

# Audit Ledger: {Transaction: Parent_Transaction}
audit_trail = {
    "TX-104": "TX-103", "TX-103": "TX-102",
    "TX-102": "TX-101", "TX-101": None # Original deposit
}

original_source = get_transaction_source("TX-104", audit_trail)
print(f"The original source of TX-104 is: {original_source}")
```

The original source of TX-104 is: TX-101

QA

107

Deciding when to use recursion is more of an art than a science. While recursion can solve many problems, it is rarely a strict requirement. In fact, recursive solutions are often less efficient than standard loops, consuming more memory and processing power.

Additionally, recursion makes testing and debugging significantly harder. Tracing the logic through multiple nested calls is far more complex than following a straightforward iterative process.

Bound Methods

Using `quick_deposit()` is a *local* lookup, `account.append()` is a *global* lookup

- Local lookups are faster!

Performance improvements are most noticeable when calling many global names within a loop

- Includes builtins, modules, and global variables

```
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount

def process_batch_transactions(acc, transactions):
    # Perf Optimization: "Bind" method to local name
    # Avoid o/h of looking up 'deposit' on 'acc' many times.
    quick_deposit = acc.deposit
    for amount in transactions:
        quick_deposit(amount) # Faster execution
        #account.deposit(amount)

# Simulating 1 million automated deposits
bulk_data = [10.50] * 1000000
my_account = BankAccount()
process_batch_transactions(my_account, bulk_data)
print(my_account.balance)
```

1050000000.0

108

QA

If you do this for just a single method then you almost certainly won't notice any difference but, done inside a loop doing many calls and lookups then it can become significant. This is because, without the bound method, a new stack-frame is created on each call. With a bound method only one stack frame is created.

functools.partial or Lambda?

A lambda could also achieve the same as a partial:

Partial objects have several advantages:

- partial objects can be pickled (saved to disk or sent over a network), whereas lambdas cannot.
- Lambdas have no `__name__` or `__doc__` either
 - Partial objects can retain metadata using `functools.wraps`.
- Partial objects are slightly faster: a new stack-frame is not created

```
def fx_convert(amount, rate, fees):  
    return amount * rate * (1 - fees)  
  
#gbp_usd = partial(fx_convert, rate=1.2610, fees=0.0005)  
# Create a GBP→USD converter using a Lambda instead of partial  
# We hardcode the rate and fees, leaving 'amount' as the argument  
gbp_usd = lambda amt: fx_convert(amt, rate=1.2610, fees=0.0005)  
  
print(gbp_usd(100_000)) # convert 100k GBP to USD
```

QA

109

You might could say you could use a lambda rather than `functools.partial` for certain tasks. Whilst this can be true there are considerable limitations on lambdas, not least because they are anonymous.

Function Attributes

<code>__annotations__</code>	Parameter comments
<code>__closure__</code>	A tuple containing bindings for the function's free variables.
<code>__code__</code>	Code object representing the compiled function body.
<code>__defaults__</code>	A tuple containing default argument values for those arguments that have default values.
<code>__dict__</code>	The namespace supporting arbitrary function attributes.
<code>__doc__</code>	The <i>docstring</i> defined in the function's source code.
<code>__globals__</code>	Reference to the global namespace of the module in which the function was defined.
<code>__kwdefaults__</code>	Dictionary containing defaults for keyword-only parameters
<code>__module__</code>	The name of the module the function came from
<code>__name__</code>	The function's name
<code>__qualname__</code>	The function's qualified name (where it was defined) 3.3

QA

Use function attributes in the following way:

```
def myfunc():  
    print (myfunc.__name__)
```

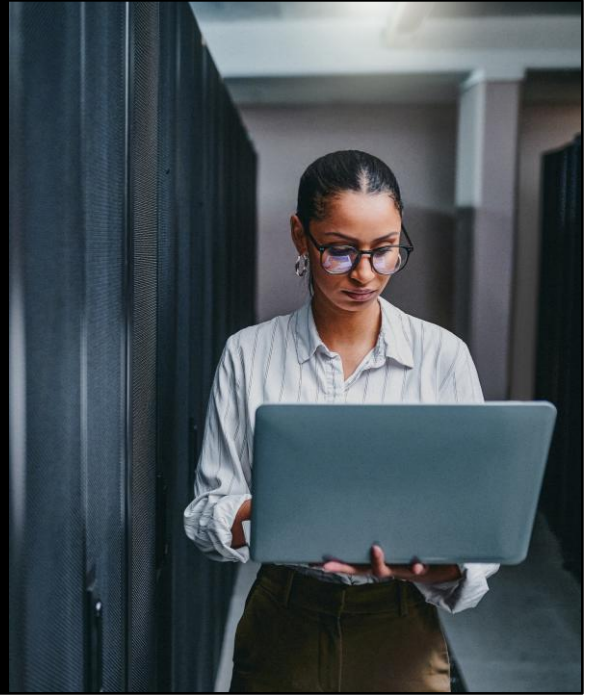
You might think there is little point in getting the name if you already know it, but it is used more with references to functions. To give a simple example:

```
fref = myfunc  
...  
print (fref.__name__)
```

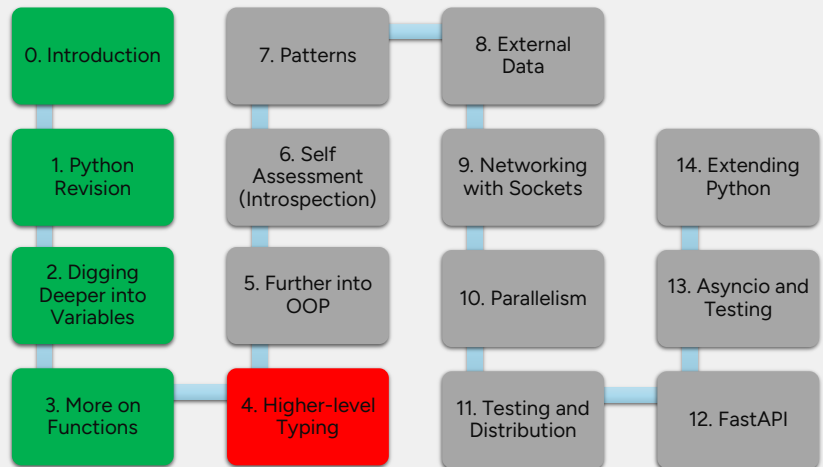
The attribute `__qualname__` (introduced at 3.3) will give the original name and where it was defined.

Intermediate Python

Higher-Level Typing



Module roadmap



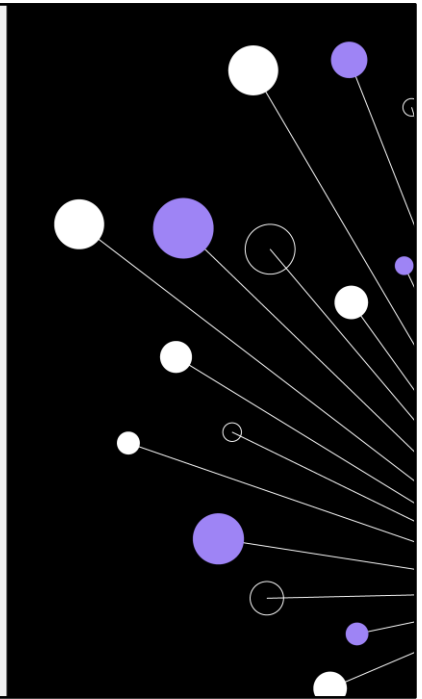
QA

Learning objectives

By the end of this session, you will be able to:

- Use comments and annotations to express type intent clearly.
- Compare Python typing approaches and add type hints to variables and functions.
- Create type aliases and custom types for cleaner, more expressive code.
- Work with unions, type variables, and Literals to model complex data.
- Apply static type checkers to validate code quality.
- Use tools like MonkeyType and typing-inspect to analyse and generate type information.

QA



Recap: Function Documentation

Comments have limited use

- Useful for maintainers, but not designed for users (they won't/can't see them)

Python supports *docstrings*

- Used for `help()` and for automated testing
- Define a bare string at the start of the function
- A triple quoted string, not an inline `#` comment.
- Or explicitly set the attribute `__doc__`

```
def print_greeting():  
    """This function takes no parameters and prints 'Hello'."""  
    print("Hello")
```

Use triple quotes
over several lines

```
>>> help(print_greeting)  
Help on function myfunc1 in module __main__:  
  
Print_greeting()  
    This function takes no parameters and prints 'Hello'.
```

QA

114

Python's comprehensive online help is a core feature of the language, built-in from the start rather than added as an afterthought. To document a function or module, simply place a string at the very beginning of the definition. While most developers use triple-quoted multi-line strings for clarity, any string format works. This string must immediately follow the `def` statement, with only whitespace or comments allowed in between. Alternatively, documentation can be defined by manually assigning a string to the special `__doc__` attribute.

Recap: Function Annotations (1)

Introduced in PEP 3107 for Python 3

An *optional* feature to add arbitrary comments to parameters and return types

Annotating *parameters*:

```
def my_func(dir:'str', files:'list'=[]):
```

Annotating *tuple and dictionary parameters*:

```
def print_vat(**kwargs:'VAT, gross and message'):
```

Annotating *return types*:

```
def calc_vat(gross:'float', vatpc:'float') -> 'list':
```

Accessible in special attribute `__annotations__`

```
print(calc_vat.__annotations__)
```

QA

115

Function annotations were introduced in Python 3 to add arbitrary metadata to arguments and return values. They are completely optional but can be useful for explaining the use of a parameter and return type by using an expression, e.g. 'str' or 'VAR, gross and message'. These expressions are evaluated at compile time and have no life or use in the runtime environment of the program.

Although annotations may resemble type declarations in other languages, they do not enforce type checking, Python remains a dynamically typed language.

Even in Python 3, annotations are effectively ignored and only really useful with 3rd party libraries. For instance, one library might use annotations to perform type checking or another might use them for improved help messages.

The annotations can be accessed using the special attribute: `function_name.__annotations__`.

Recap: Function Annotations (2)

Similar to inline comments

- Not supported for lambda functions

```
def print_vat (*,  
    gross: "Gross amount (including VAT)"=0,  
    vatpc: "VAT in percentage terms"=17.5,  
    message: "Free text"='Summary:') \  
    -> "No usable return value":  
    pass
```

Might appear as
"Unexpected token at
end of expression"
errors in VSC

Function attribute `__annotations__` gives details

```
for kv in print_vat.__annotations__.items():  
    print(kv)  
  
(  
    ('gross', 'Gross amount (including VAT)'),  
    ('message', 'Free text'),  
    ('vatpc', 'VAT in percentage terms'),  
    ('return', 'No usable return value')  
)
```

QA

116

Annotations allow meta-data to be attached to parameters and return values. Python itself ignores them, as it would comments, but they are available for use by third-party libraries. Aside from the basic syntax, there are no standard data types or formats - these are left flexible for third-party libraries to exploit.

The reason why VSC may complain about the annotations with "Unexpected token at end of expression" messages is whilst there is technically nothing wrong with the code it is considered better practice to move the descriptions into docstrings or to use the typing module's Annotated class and reserve annotations for actual type information:

```
def print_vat(*,  
    gross: Annotated[float, "Gross amount (including VAT)"] = 0,  
    vatpc: Annotated[float, "VAT in percentage terms"] = 17.5,  
    message: Annotated[str, "Free text"] = 'Summary:') ->  
    Annotated[None, "No usable return value"]:  
    ...
```

Typing – A Quick Comparison

Most programming languages have some kind of **Typing strategy**.

The basic distinction is between **dynamic** and **static** typing

While Python has introduced Type Hints, this is not a shift toward changing the language's fundamental typing strategy. Python remains dynamically typed at its core

QA

Dynamically typed

No Formal Declaration: Variables are created upon assignment without pre-defining a type.

Runtime Checking: Type errors are only caught while the code is actually running.

Flexible References: A single variable name can point to different types over time.

e.g. Python, JavaScript, PHP etc.

Statically typed

Variables must be formally declared with a specific type

Compile-time Checking: The compiler validates types before the program ever executes.

Fixed Types: Once a variable is defined as a specific type, it cannot be changed.

e.g. C, C++, C#, Java etc

117

Python is Dynamically Typed language:

- **Runtime Checking:** Type validation happens during execution, which can introduce a slight performance overhead.
- **Fluid References:** A variable's type can change throughout its lifecycle as it points to different objects.

Static Comparison (C, C++, C#, Java):

- **Pre-run Validation:** Types are checked during compilation, before the code ever runs.
- **Fixed Types:** Variables generally cannot change types, though they can be converted via **casting**.

The Future of Python:

- Python will remain **dynamically typed** at its core.
- **Duck Typing:** The focus remains on what a class *does* (its methods) rather than its formal name.

Hinting vs. Enforcing: There is a critical distinction between providing **Type Hints** for developers/IDEs and **Enforcing** types at the language level.

A Note on Terminology: Do not confuse these concepts with **"Strong" vs. "Weak" typing**; those refer to how strictly a language handles operations between different types (e.g., adding a string to an integer).

Typing – A Brief History

Types in Python have their roots in three basic PEPs:

- **PEP 3107 – Function Annotations**

Introduced December 2, 2006 by Collin Winter and Tony Lownds (Python 3.0)

This PEP established the foundation for attaching annotations to function parameters and return values.

<<https://www.python.org/dev/peps/pep-3107/>>

- **PEP 484 – Type Hints**

Introduced September 29, 2014 by Guido van Rossum, Jukka Lehtosalo, and Lukasz Langa (Python 3.5)

This PEP formalized the use of type hints in Python, laying the groundwork for static type checking.

<<https://www.python.org/dev/peps/pep-0484/>>

- **PEP 526 – Syntax for Variable Annotations**

Introduced August 9, 2016 by Ryan Gonzales, Philip House, Ivan Levkivskyi, Lisa Roach, and Guido van Rossum (Python 3.6)

This PEP added dedicated syntax for annotating variables directly, expanding Python's type annotation capabilities.

<<https://www.python.org/dev/peps/pep-0526/>>

Full pep reference: <https://www.python.org/dev/peps/>

Why Use Typing in Python?

- Improves code readability and serves as built-in documentation
- Docstring conventions (PEP 257) don't define strict standards—types help fill the gap
- Enhances collaboration within development teams
- Essential for large codebases where searching for definitions becomes difficult
- Enables linters and IDEs to provide smarter assistance
- Offers better argument suggestions and autocompletion
- Lowers the risk of runtime errors by catching type issues early
- Works seamlessly with static type checkers in CI pipelines

Potential Drawbacks

- Inconsistent support in Python versions prior to 3.6
- Requires developers to consistently add type hints, which adds overhead—especially for short, disposable scripts
- Importing the typing module can slow down application startup

QA

119

Python is a dynamically typed language, and this flexibility is often cited as one of the reasons for its widespread popularity. Compared with statically typed languages like C, where every variable must be declared with an explicit type, Python allows developers to write code quickly and iterate rapidly.

While this dynamic nature can slow Python down—and, combined with interpretation, makes deep optimization challenging—the convenience and ease of development frequently outweigh these drawbacks.

Because of this, some developers find the idea of adding type annotations counterintuitive. Yet the benefits of using typing in Python are significant and becoming increasingly difficult to ignore.

One important clarification: despite the growing use of type hints, there is no intention or movement to transform Python into a statically typed language. The goal is to enhance clarity and tooling—not to change Python's fundamental nature.

Adding Type Hints to Assignments

Type hints can be added to assignments

- Introduced in PEP 526
- It's relatively easy for **atomic** types:

```
user_name: str = "Sadia"  
is_valid: bool = True  
average_score: float = 15.5
```

- Hints can be added to *composite types*, e.g. collections:

```
user_names: list = ["Sadia", "Kamran", "Kate"]  
results: tuple = True, False, True  
limits: dict = {"max": 100, "min": 0}
```

- But the following is considered *much* better practice (*because it's more useful for static type checking*):

```
from typing import List, Tuple, Dict  
  
user_names: List[str] = ["Sadia", "Kamran", "Kate"]  
results: Tuple[bool] = True, False, True  
limits: Dict[str, int] = {"max": 100, "min": 0}
```

QA

120

Type hints require the use of the **typing module** and its specific types, for example they tend to use Pascal case to define the type and camel case for the declaration.

```
user_names: List[str] = list()
```

Note.

There is *no* need to initialise a variable when using type hints. For example:

```
is_available: bool  
quantity: int
```

Best Type for Financial Data in Python: Decimal

For anything involving money, currency calculations, interest, pricing, or balances, the `decimal.Decimal` type is the safest and most widely recommended.

- **Exact precision** — avoids the rounding errors that are common with `float`.
- **Designed for financial and accounting calculations.**
- **Configurable context** (rounding modes, precision rules, etc.).
- Widely recognized in fintech, banking, accounting, e-commerce, and audit-sensitive systems.

```
# The trouble with floats  
print(0.1 + 0.2)  # -> 0.30000000000000004
```

```
from decimal import Decimal  
  
price: Decimal = Decimal("101.2") # initial value will be 101.2 not a string
```


Why Do You Have To Write `Decimal("101.2")`?

`Decimal` must receive a **string** to avoid floating-point errors

- Writing `Decimal(101.2)` accidentally introduces a float first — which already contains a rounding error — and defeats the whole purpose of using `Decimal`.
- Remember type hints do not affect runtime execution price: `Decimal` is only for type checkers.
 - It does not automatically convert or initialise anything, so you must still call the constructor explicitly.

Alternatives for Financial Data

- Alternative cleaner ways (still using `Decimal`):

```
# Use local alias for Decimal
from decimal import Decimal as D

price: D = D("101.2")
tax_rate: D = D("0.20")
```

```
# Use factory function
from decimal import Decimal
def money(value: str) -> Decimal:
    return Decimal(value)

price = money("101.2")
```

- **Use `int` or `Annotated[int]` for the smallest currency unit**
 - Some systems store amounts as cents/pence
 - Avoids floating-point and `Decimal` overhead but requires discipline.

```
from typing import Annotated

Pence = Annotated[int, "amount in pence"]
balance: Pence = 1299 # £12.99
```

Type Hinting a Function

Adding type hints to a function declaration is very useful

- Adds useful documentation as you code by explicitly defining expected data types.
- Boosts productivity with IDE prompts that assist with function calls.
- Bugs caught earlier using static type checkers to validate logic.
- Helps to prevent common runtime type mismatches.

The standard format for type hinting in a function is:

```
def function(argument: input_type) -> return_type:  
    pass
```

Allowing for specified type hinting such as:

```
def myfunc(argument: str) -> int:  
    pass
```

Defaulted argument values are still possible...

```
def myfunc(argument: float=0.0) -> str:  
    pass
```

QA

124

Type Aliasing

Python makes it possible to create a *Type Alias*

- Alias should be descriptive (and meaningful), e.g

```
import matplotlib.pyplot as plt
from typing import List, Tuple

# Type Aliasing:
Price = float
Timestamp = str
StockData = List[Tuple[Timestamp, Price]]

def plot_market_trend(data: StockData, ticker: str) -> None:
    # Uses StockData alias to handle historical pricing.
    times, prices = zip(*data)

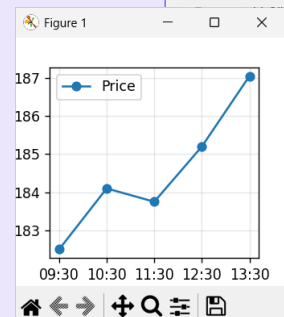
    plt.figure(figsize=(10, 5))
    plt.plot(times, prices, marker='o', linestyle='-', label='Price')

    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.show()

# Sample dataset using our defined types
history: StockData = [("09:30", 182.50), ("10:30", 184.10), ("11:30", 183.75),
                      ("12:30", 185.20), ("13:30", 187.05)]

plot_market_trend(history, "NVDA")
```

- Ideal for simplifying complex code, e.g., function arguments
- Note – aliasing doesn't define a new type (it's simply a shorthand)



QA

125

The code demonstrates how to alias a list of price/timestamp tuples as **StockData**, ensuring the function signature clearly communicates its intent to other developers or IDEs.

Instead of writing a messy type hint like `List[Tuple[str, float]]` every time, we create **StockData**. This tells anyone reading the code exactly what the list contains: a series of times and their corresponding prices. It transforms a generic data structure into a domain-specific one.

By using **data: StockData**, the function signature becomes self-documenting. If you hover over `data` in a modern code editor (like VS Code or PyCharm), it will show you exactly what the tuple structure should look like, reducing the chance of passing the wrong data format.

The `zip(*data)` trick is a Python "unzipping" idiom. It takes our list of pairs—`[("09:30", 182.50), ("10:30", 184.10)]`—and separates them into two independent lists: one for the **X-axis** (times) and one for the **Y-axis** (prices).

The rest of the code uses standard **matplotlib** commands:

- **plt.plot()**: Draws the line and dots.
- **plt.grid()**: Adds a faint background grid for better readability of price levels.
- **plt.show()**: Renders the final window.

Creating Custom Types

The `NewType()` method is used to create distinctive data types.

The creation a new type follows a different process to the creation of an alias.

```
from typing import NewType

# Both are floats, but the type checker now treats them as unique
USD = NewType('USD', float)
EUR = NewType('EUR', float)

def calculate_total_usd(balance: USD, deposit: USD) -> USD:
    return USD(balance + deposit)

# Valid usage
account_balance = USD(1000.00)
monthly_bonus = USD(500.0)
total = calculate_total_usd(account_balance, monthly_bonus)

# POTENTIAL ERROR: This would pass with a Type Alias,
# but a static type checker (Like MyPy) will flag this as an error:
travel_cash = EUR(450.0)
total_error = calculate_total_usd(account_balance, travel_cash)
```

```
mypy creating_custom_types_with_newtype.py
creating_custom_types_with_newtype.py:21: error: Argument 2 to "calculate_total_usd" has
incompatible type "EUR"; expected "USD" [arg-type]
Found 1 error in 1 file (checked 1 source file)
```

QA

126

New type checks are only enforced by the static type checker; there are no runtime checks.

The new types (in this case "USD" and "EUR") are in fact a *functions* that return whatever value you pass them; as such there is *no creation* of a new class nor any of the associated overhead of doing so.

Unfortunately, this means that it is **not possible** to use it to "cast" to a different type at run-time, e.g.

```
sum = USD(input("Enter amount: "))
```

Nor is it possible to subclass it.

However, it is possible to create a `NewType()` based on your previous one, e.g.

```
USD = NewType('USD', float)
EUR = NewType('EUR', USD)
```

Static type checking will work as expected on this new type.

Consequently, you should use aliases when you want to simplify a more complex type expression.

Whereas new types may be used when you want to "flag" to developers that a "special" type of an existing type is required, e.g. **USD** is a specific implementation of a **float**, has particular data needs and must be used correctly. Of course, there is no run-time checking of this.

Static Checker: mypy

mypy is an optional static type checker for Python

- Core development team is employed by Dropbox¹
- Homepage <http://mypy-lang.org>
- Documentation <https://mypy.readthedocs.io/en/stable/>
- Source code <https://github.com/python/mypy>
- Developer blog <https://mypy-lang.blogspot.com/>
- Social media @mypyproject

Installed via Python Package Index (PyPI.org):

`pip install mypy`

- **Supports both manual execution and programmatic integration.**
 - Embeds into CI pipelines to automatically validate developer commits.
- **Delivers high-speed performance with extensive feature options.**
 - Configures via command-line flags or dedicated configuration files.
- **Runs continuously in the background as a server daemon**

QA

127

mypy is a static analyzer tool which can be used with Python 3.

It is used to examine any type annotations that have been added to standard Python code and can use these to hunt down any type-related bugs, e.g. passing a string to a function which expects an int.

1

<https://dropbox.tech/application/our-journey-to-type-checking-4-million-lines-of-python>

Accessed 24/04/2020

Python's Union Type — What & Why

- **Purpose:** Indicates a value can be **one of several types** (e.g., `str` or `int`).
- **Syntax:**
 - Python 3.10+: `A | B` (pipe syntax)
 - Python 3.9 and earlier: `Union[A, B]` from typing
- **Common use cases:** API responses, optional returns, polymorphic IDs, and feature flags.
- **Optional[T] is shorthand for T | None** (e.g., `Decimal | None`).
- **Tooling:** Improves IDE hints, static checks (e.g., `mypy`, `pyright`), and code readability.
- **Good practice:** Keep unions **narrow and meaningful**; for complex behavior consider `@overload` or dedicated types instead of very broad unions

Union (1)

The typing module provides the Union type

- It lets us specify that a value may belong to more than one type

```
from typing import Union
```

```
my_union = Union[bool, int]  
print(my_union)  
print(type(my_union))
```

```
typing.Union[bool, int]
```

```
<class 'typing._UnionGenericAlias'>
```

Example: a variable named *result* could be either a Boolean or an Integer

- This union type can then be used directly in a function signature

```
def is_prime_number(number: int) -> my_union:  
    pass
```

```
>>> help(is_prime_number)
```

```
Help on function is_prime_number in module __main__:
```

```
is_prime_number(number: int) -> Union[bool, int]
```

QA

129

Union (2)

Union is also useful when a collection can contain mixed types

- Example: a parameter to a financial processing module whose elements could be either floats or strings or a dictionary :

```
from typing import Union, Dict
# Define a type alias for readability
DepositInput = Union[float, str, Dict[str, float]]
# OR:
# DepositInput = float | str | dict[str, float]

def process_deposit(account_id: str, deposit: DepositInput) -> float:
    #Processes different types of deposits and returns the final amount credited.
    if isinstance(deposit, float):
        print(f"Cash deposit received: £{deposit}")
        return deposit
    elif isinstance(deposit, str):
        # Simulate lookup for cheque/reference
        print(f"Looking up deposit reference: {deposit}")
        resolved_amount = 250.00 # mock lookup result
        return resolved_amount
    elif isinstance(deposit, dict):
        # Expecting something like {"amount": 100.0}
        amount = deposit.get("amount", 0.0)
        print(f"Bank transfer received: £{amount}")
        return amount
    else:
        raise ValueError("Unsupported deposit type")

balance = 1000.0
balance += process_deposit("ACC123", 200.0)
balance += process_deposit("ACC123", "CHK-987654")
balance += process_deposit("ACC123", {"amount": 150.0})
print(f"Updated balance: £{balance}")
```

QA

130

The use of Union in the example on the slide is helpful because:

- It lets you accept multiple input types safely while still getting static type checking from tools like mypy.
- You can model real-world flexibility (cash, cheque, transfer) without losing clarity.
- Static checkers will warn you if you try unsupported types.

Union (3)

Note: Union cannot be used to create a `_GenericAlias` and then assign items to it

```
from typing import List, Union
valid_data = Union[str, float]

data = List[valid_data] = ["123", 2.50, "hello", 105.2]
```



Doing so results in:

TypeError: '_GenericAlias' object does not support item assignment

This limitation can be addressed by using a **TypeVar** instead (see next page)

Type Variables - 1

Type Variables are flexible identifiers that adapt to any data type (defined via **TypeVar**).

- **Sequences:** Lists and Tuples maintain order but can store mixed types.

```
("Sadia", "Lotte", "Brigitte")  
[22, 43, 61, 00]  
[23, "Mohandas", 78]
```

- **Dynamic Returns:** Functions picking random items from sequences may return different types.

Type Variables - 2

A Type Variable (**TypeVar**) can be used to tighten this for static checking...

```
1  from decimal import Decimal
2  from typing import TypeVar
3
4  Permitted = TypeVar("Permitted", int, float, Decimal, str)
5  def normalise_trans_amnt(amount: Permitted) -> float:
6      if isinstance(amount, (int, float, Decimal)):
7          return float(amount)
8
9      if isinstance(amount, str):
10         # Handle formatting like "£1,234.56" or "1,234.56"
11         cleaned = amount.replace("£", "").replace(",", "")
12         return float(cleaned)
13
14  transactions = [125, 89.30, Decimal("12.99"), "£1,045.90",
15                 {"value": "250.00", "currency": "GBP"}]
```

```
16 normalised = [normalise_trans_amnt(t) for t in transactions]

mypy type_variables.py
Type_Variables.py:16: error: Value of type variable "Permitted" of
"normalise_transaction_amount" cannot be "object"
Found 1 error in 1 file (checked 1 source file)
```

QA

133

In this example, **TypeVar** has been used to limit the acceptable types for the **Permitted** to **int**, **float**, **Decimal** or **str**.

The transaction list contains a mixture of valid types, but also a dictionary which isn't acceptable and something which will be detected by a tool such as **mypy**.

It is possible to write code that can dynamically check for this using the **TypeVar** object which we will look at shortly.

Type Variables - 3

- **The Any Type** is used for loose type matching when strictness isn't required:

```
from typing import Any

def normalise_transaction_amount(amount: Any) -> float:
    # Accepts amounts in various formats (string, int, float, dicts) and returns a clean float.
    if isinstance(amount, (int, float)):
        return float(amount)
    if isinstance(amount, str):
        cleaned = amount.replace("£", "").replace(",", "")
        return float(cleaned)
    if isinstance(amount, dict) and "value" in amount:
        return float(amount["value"])

    raise ValueError(f"Unsupported amount type: {type(amount)}")

transactions = [125, 89.30, "£1,045.90", {"value": "250.00", "currency": "GBP"}]

normalised = [normalise_transaction_amount(t) for t in transactions]

print(normalised) # [125.0, 89.3, 1045.9, 250.0]
```

QA

134

This example simulates processing a bank transaction where some fields may legitimately come in with different types depending on the data source (e.g., API vs. CSV vs. legacy system). The use of **Any** allows a function to accept *anything* without forcing callers to conform to a strict type. The function contains the logic to coerce or reject unexpected formats.

Type Variables - 4

We can access the `TypeVar` object at run-time to check dynamic data.

```
from decimal import Decimal
from typing import TypeVar
Permitted = TypeVar("Permitted", int, float, Decimal, str)

def normalise_transaction_amount(amount: object) -> float:
    # Validate the incoming value dynamically using TypeVar's __constraints__ list.
    allowed_types = Permitted.__constraints__

    if isinstance(amount, allowed_types):
        if isinstance(amount, str):
            cleaned = amount.replace("£", "").replace(",", "")
            return float(cleaned)
        else:
            return float(amount)
    raise TypeError(
        f"Unsupported type '{type(amount).__name__}'. "
        f"Allowed types: {[t.__name__ for t in allowed_types]}"
    )

transactions = [125, 89.30, Decimal("12.99"), "£1,045.90",
                {"value": "250.00", "currency": "GBP"}]
normalised = []

for t in transactions:
    try:
        normalised.append(normalise_transaction_amount(t))
    except TypeError as e:
        print(f"Skipping invalid transaction: {t} -> {e}")

print(normalised)
```

QA

135

There are various techniques available to dynamically check data using the `TypeVar` object. The example on the slide uses the `__constraints__` **property**, which lists the constrained types and demonstrates how to:

1. Define a constrained `TypeVar`
2. Inspect `Permitted.__constraints__` inside the function
3. Validate data against those constraints dynamically
4. Apply it to normalising banking transaction amounts

Sample output from this test run:

```
Skipping invalid transaction: {'value': '250.00', 'currency':
'GBP'} -> Unsupported type 'dict'. Allowed types: ['int',
'float', 'Decimal', 'str']
[125.0, 89.3, 12.99, 1045.9]
```


Functions Implicit Return Type - None

The "None" Default: Understand that all Python functions return None unless specified (per PEP 484)

```
def print_greeting(name: str):  
    print(name)
```

Standardise Your Code: Transition from implicit returns to explicit def func() -> None:

```
def print_greeting(name: str) -> None:  
    print(name)  
  
print_greeting("Ada")
```

Error Prevention: Use **mypy** to catch accidental data returns in "void" functions.

```
def print_greeting(name: str) -> None:  
    print(name)  
    return 0  
  
mypy --strict greeting_printer.py  
greeting_printer.py :3: error: No return value  
expected  
Found 1 error in 1 file (checked 1 source file)
```

QA

136

Note:

It makes sense being strict.

The first coded example would be flagged by a static code checker such as mypy if a "strict" option is requested, e.g.

mypy --strict greeting_printer.py

myrepeater.py:1: error: Function is missing a return type annotation

Found 1 error in 1 file (checked 1 source file)

In mypy, the use of the "strict" option will actually enable a number of different options:

- warn-unused-configs
- disallow-any-generics,
- disallow-subclassing-any
- disallow-untyped-calls
- disallow-untyped-defs
- disallow-incomplete-defs
- check-untyped-defs
- disallow-untyped-decorators
- no-implicit-optional

- warn-redundant-casts
- warn-unused-ignores
- warn-return-any
- no-implicit-reexport

These can be used in a more granular fashion, if preferred.

Using TypeVar for Generic Classes

We can use TypeVar and Generic to create new generic classes

e.g., a FIFO (First In, First Out) Queue

```
from typing import TypeVar, Generic, List
# Define a Type Variable for generic support
T = TypeVar('T')
class Queue(Generic[T]):
    def __init__(self) -> None:
        # Internal storage using a List
        self._items: List[T] = []
    def is_empty(self) -> bool:
        """Return True if the queue contains no items."""
        return len(self._items) == 0
    def enqueue(self, item: T) -> None:
        """Add an item to the end of the queue."""
        self._items.append(item)
    def dequeue(self) -> T:
        """Remove and return the item from the front of the queue."""
        if self.is_empty():
            raise IndexError("Cannot dequeue from an empty queue")
        return self._items.pop(0)
```

QA

137

We can use this code to safely construct a queue of strings, ints, floats, Decimals etc.

The following code – instantiates a stack with "Decimal" used to set the type hint.

Example: A queue specifically for Decimals

```
from decimal import Decimal
```

```
decimal_queue = Queue[Decimal]()
```

```
decimal_queue.enqueue(Decimal("12.99"))
```

```
print(f"Is empty? {decimal_queue.is_empty()}") # Output: False
```

```
print(f"Dequeued: {decimal_queue.dequeue()}") # Output: 12.99
```

Also notice that the Generic TypeVar "T" represents the set type for the List (in the constructor), function argument types and return types.

Note, the code could create and enqueue data of any valid type, simply by changing the hint used in the instantiation.

And our static checker (mypy) would identify any attempt to enqueue anything other than a Decimal, e.g.

```
decimal_queue.enqueue(250.00)
```

Would result in:

```
mypy generic_queue.py
```

```
generic_queue.py:28: error: Argument 1 to "enqueue" of
```

```
"Queue" has incompatible type "float"; expected "Decimal"  
[arg-type]  
Found 1 error in 1 file (checked 1 source file)
```

Using Literals

The **Literal** type restricts a variable or function argument to a specific, predefined set of exact values (such as specific strings or integers) rather than just a general data type like `str` or `int`.

```
from typing import Literal, Union, TypedDict
# Define different structures for account types
class Savings(TypedDict):
    id: str
    bal: float
    rate: float

class Investment(TypedDict):
    id: str
    bal: float
    risk: Literal["Low", "Medium", "High"]
```

```
from typing import Literal, Union, TypedDict

# Define a Union for the possible return types
AccountInfo = Union[Savings, Investment]

# A function that takes a Literal and returns a Union
def get_acc_details(
    id: str,
    type: Literal["SAVINGS", "INVESTMENT"]
) -> Union[AccountInfo, TypeError]:
    # Fetches account data based on the Literal type provided.
    if type == "SAVINGS":
        return {"id": id, "bal": 1500.50, "rate": 0.02}
    else:
        return {"id": id, "bal": 50000.00, "risk": "Medium"}

# Example usage
try:
    print(get_acc_details("SA-123", "SAVINGS"))
    print(get_acc_details("IA-456", "INVESTMENT"))
    print(get_acc_details("XX-000", "CHECKING")) # mypy error
except TypeError as e:
    print(f"Skipping invalid checking: {e}")
```

QA

138

Literals allow developers to specify something is equivalent to a specific primitive value, e.g.

- A string, **"credit"**
- A bool, **True**
- An integer, **1**
- etc

In this example the first two calls to the `get_acc_details()` function will work as expected:

```
{'id': 'SA-123', 'bal': 1500.5, 'rate': 0.02}
{'id': 'IA-456', 'bal': 50000.0, 'risk': 'Medium'}
```

However, the third call, with `"CHECKING"` being passed as the second parameter will raise a `TypeError` (the explicitly-set default return value for the function) as it's outside range.

If we had checked this with a static checker such as `mypy`, the problem would have been immediately flagged:

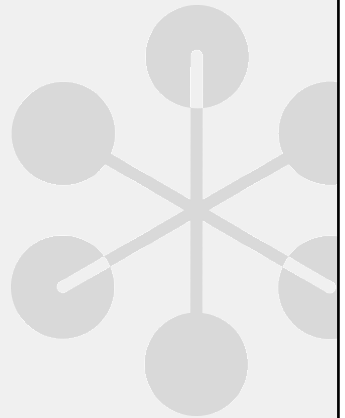
mypy literals.py

literals.py:34: error: Argument 2 to "get_acc_details" has incompatible type "Literal['CHECKING']"; expected "Literal['SAVINGS', 'INVESTMENT']" [arg-type]
Found 1 error in 1 file (checked 1 source file)

MonkeyType

A **MonkeyType** is a time-saving tool that automatically generates type annotations from existing code.

- During runtime, a MonkeyType can derive types from actual data processed during execution.
- Compatible with Python 3.6+ and installed via `pip install MonkeyType`.
- **Core Capabilities:**
 - Generates stub files with inferred parameter and return types.
 - Auto-annotates source files (ensure files are closed before applying).



MonkeyType Example Workflow - 1

```
# banking.py
def process_deposit(account_id, amount):
    """Update balance and return the new total."""
    # Simulated database lookup
    current_balance = 1000.0
    new_balance = current_balance + amount
    return new_balance

def get_transaction_type(code):
    """Convert a numeric code to a string label."""
    mapping = {1: "CREDIT", 2: "DEBIT"}
    return mapping.get(code, "UNKNOWN")
```

The "Before" Code (banking.py)

This is a standard, untyped module. Without type hints, it is unclear if amount should be an integer, a float, or a Decimal object.

```
# run_bank.py
from banking import process_deposit, get_transaction_type

# MonkeyType sees: str, float -> float
process_deposit("ACC-123", 250.50)

# MonkeyType sees: int -> str
get_transaction_type(1)
```

The Execution (The "Bridge")

You run a script (e.g., run_bank.py) that actually uses these functions with real data. MonkeyType "watches" this happen.

QA

140

In the example on the slide, the actual call of the function is analysed dynamically at run-time to identify the types of the functions' parameters (a string and a float for the `process_deposit` function and an integer for the `get_transaction` function). The data type returned by the function can also be identified by the analyser which simply looks at the return statement (another float for the first function and a string for the second).

MonkeyType Example Workflow - 2

```
monkeytype run run_bank.py

monkeytype stub banking
def get_transaction_type(code: int) -> str: ...
def process_deposit(account_id: str, amount: float) -> float: ...
monkeytype apply banking

# banking.py after running monkeytype apply banking
def process_deposit(account_id: str, amount: float) -> float:
    """Update balance and return the new total."""
    # Simulated database Lookup
    current_balance = 1000.0
    new_balance = current_balance + amount
    return new_balance

def get_transaction_type(code: int) -> str:
    """Convert a numeric code to a string label."""
    mapping = {1: "CREDIT", 2: "DEBIT"}
    return mapping.get(code, "UNKNOWN")
```

anotated
banking.py

QA

141

Using monkeytype's "run" option will cause the tool to dump all call traces into an SQLite database in a file located in the current working directory called monkeytype.sqlite3.

It is then possible to use MonkeyType's "stub" command to generate a stub file for a module, or use "apply" to patch the identified type annotations directly to the code.

See also...

Here's a list of type-related tools and modules

- mypy popular static type checker
- PyChecker A well-feature linter
- Pyre-check Facebook's Python type checker
- pytype Google's Python type checker
- Pylint Microsoft's Python type checker
- MonkeyType Autocreation of type annotations from code
- Typing Support for type hints
- Typing-inspect An experimental API for runtime inspection of types defined in typing module

And many others!

QA

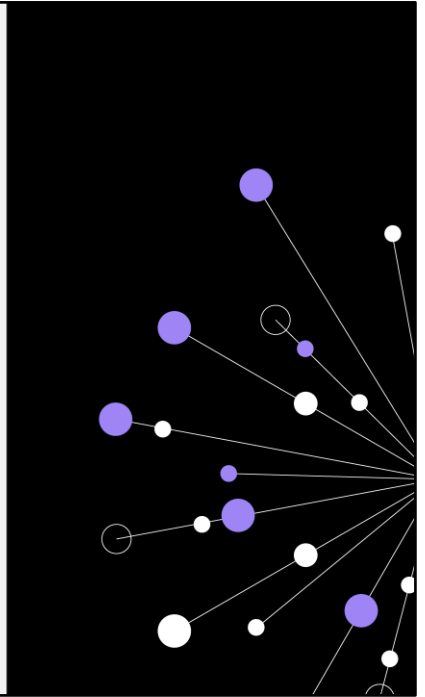
Exercise EG04: Typing

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA

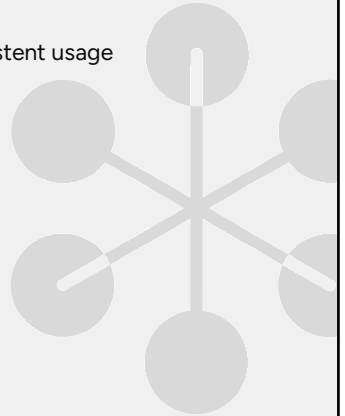


Suggested time: 20-30 mins.

Activity includes differentiated tasks.

Summary

- Comments within functions can improve code readability
- Function annotations have existed for quite some time
 - However, there has historically been little standardization, leading to inconsistent usage
- Python is a dynamically typed language
 - This offers advantages, such as flexibility and ease of use
 - But it also introduces challenges, particularly around formal type checking
- Typing provides a solution
 - It introduces a standardized approach
 - It offers a rich, evolving, and well-documented syntax
 - It enables linting tools and static type checkers such as mypy
 - It enhances readability and developer productivity
 - It can help reduce semantic errors caused by incorrect data types
- However...
 - Typing is not mandatory and, in Python, it almost certainly never will be



Appendices



Gradual typing

Gradual typing is a type system developed in 2006

- Developed by Jeremy Siek and Walid Taha
- Allows parts of a codebase to be *statically* typed...
- leaves other parts *dynamically* typed
- Developer therefore *has control* over which parts have annotation
- Particularly useful in large codebases, i.e., *allows phased introduction* of static typing; many advantages e.g.:
 - reduces refactoring costs
 - keeps low barrier to entry for software engineers
 - retains compatibility with legacy code
 - Less disruption

Remember – static type checks can typically catch errors earlier, reducing the cost of fixing!

QA

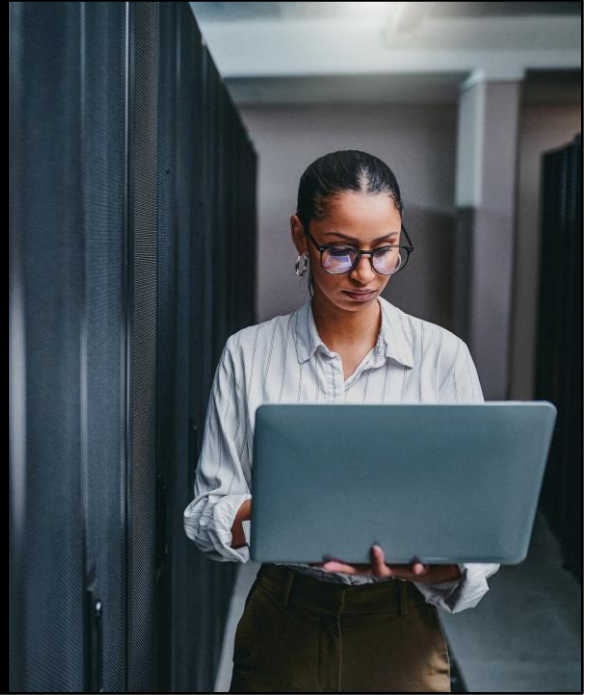
146

Notes

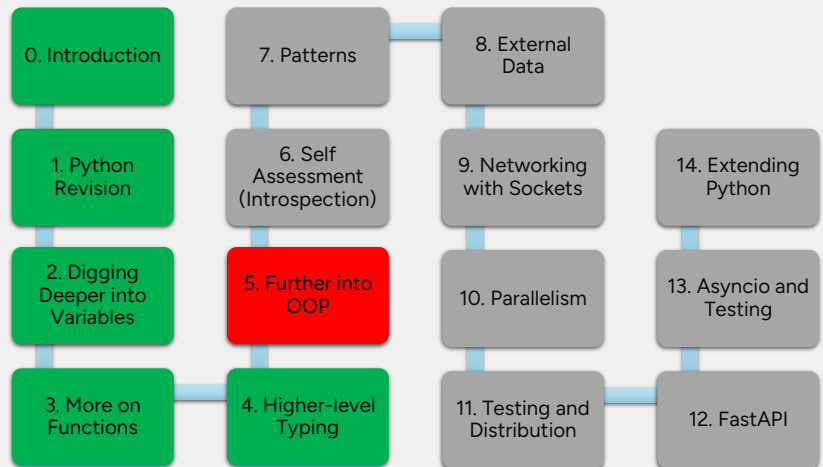
For further background reading, we suggest: [Gradual typing in Python – GeeksforGeeks](#) the following video is also interesting: [BOB - Python Gradual Typing: The Good, The Bad and the Ugly](#)

Intermediate Python

Further into OOP



Module roadmap



QA

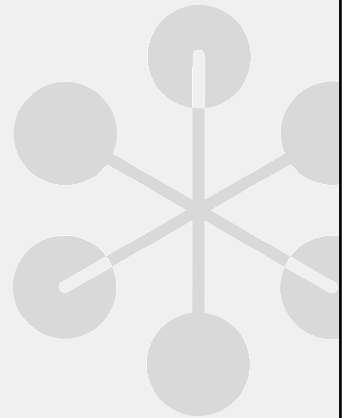
Further object oriented programming

Learning Outcomes:

- Use Object Oriented programming techniques in Python 3

Prerequisites:

Delegates are required to have knowledge and skills relating to basic Python programming, including data structures, functions, loops and flow control.



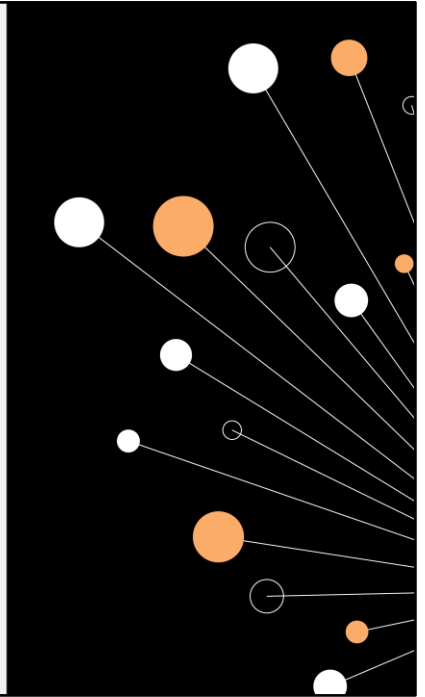
Appendices include details on simple database access, XML and reading binary files created in other programming languages (e.g. C)

Learning objectives

By the end of this session, you will be able to:

- **Master Object Internals:** Apply name mangling, memory optimization, and dynamic properties.
- **Control Attribute Lifecycles:** Implement custom lookup, modification, and deletion via `__getattr__` and `__setattr__`.
- **Architect Class Hierarchies:** Extend standard libraries, deploy reusable Mixins, and introspect inheritance paths.
- **Enforce System Interfaces:** Design Abstract Base Classes (ABCs), build Metaclasses, and leverage duck-typing.
- **Refine Object Behaviours:** Implement total ordering, create custom iterable containers, and use delegation patterns.
- **Utilize Advanced Mechanics:** Construct custom context managers and inspect code signatures programmatically.

QA



Revision: Python name mangling

Underscore naming conventions — a quick refresher

- Names prefixed with a single underscore are intended to be private to a module
 - Example: `_name`
- Names prefixed with double underscores (“dunder”) are intended to be private to a class
 - Example: `__name`
- Names surrounded by double underscores have special meanings in Python (often called “magic methods”)
 - Example: `__str__`
- Double underscore prefixes trigger a mechanism known as name mangling
- Python internally rewrites the variable name by prefixing it with the class name
 - For example, `__var` inside class `MyClass` becomes `_MyClass__var`

QA

151

Note: Name mangling is exposed during introspection:

If your class looks like this:

```
class MyClass:
    def __init__(self):
        self.__private_var = "hidden"
```

`print(MyClass.__dict__)` will display a dictionary similar to this:

```
{
    '__module__': '__main__',
    '__init__': <function MyClass.__init__ at 0x...>,
    '__dict__': <attribute '__dict__' of 'MyClass' objects>,
    '__doc__': None
}
```

Revision – method types

Three core method types are commonly used in practice:

- Belongs to the class as one of its attributes
- Can be called on the object (except when marked as "private")
- Has direct access to the object's data
- No decorator needed
- Takes "self" as its first parameter

instance



- Belongs to the class as one of its attributes
- Can be called directly on the class
- Cannot access object-level properties
- Has access to class-level properties
- Requires the @classmethod decorator
- Takes "cls" as its first parameter

class



- Belongs to the class as one of its attributes
- Can be called directly on the class
- Can also be called on the method itself
- Cannot access object-level properties
- Has access to class-level properties
- Requires the @staticmethod decorator

static



Your choice of method(s) depends on your specific programming requirements and the role your class is designed to fulfil.

The following slide illustrates all three types with examples.

QA

152

The `staticmethod` decorator is mainly useful for code organization. It allows functions that logically belong to a class to be placed within it, rather than existing as standalone or "orphaned" functions in the codebase. This makes it easier for developers to locate and understand the function, since it lives in a class that provides relevant context.

Revision – method types: example

```
class BankAccount:
    bank_name = "Global Bank"
    total_accounts = 0

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
        BankAccount.total_accounts += 1

    # Instance method: works with a specific account
    def deposit(self, amount):
        if BankAccount.is_valid_amount(amount):
            self.balance += amount
            return f"{self.owner} new balance: {self.balance}"

    # Class method: works with class-level data
    @classmethod
    def get_total_accounts(cls):
        return f"Total accounts in {cls.bank_name}: {cls.total_accounts}"

    # Static method: utility logic related to accounts
    @staticmethod
    def is_valid_amount(amount):
        return amount > 0
```

```
acc1 = BankAccount("Alice", 100)
acc2 = BankAccount("Bob", 50)

# Instance method
print(acc1.deposit(50)) # uses that account's balance

# Class method
print(BankAccount.get_total_accounts()) # uses shared class data

# Static method
print(BankAccount.is_valid_amount(25)) # simple utility check
```

```
Alice new balance: 150
Total accounts in Global Bank: 2
True
```

QA

153

In this example we can create a class which keeps track of some bank account processing. We also create a quick "is_valid_amount" static method which can be invoked without having to create a full object. This same static method is accessed via the instance-based deposit method.

Also note that a class method "get_total_accounts" returns the number of accounts that have been created which are tallied in the class initialiser. This can be accessed from any instance and only exists once.

If we check each method, it is possible to see how Python views them in terms of typing:

```
print(type(acc1.__init__))
<class 'method'>

print(type(acc1.deposit))
<class 'method'>

print(type(acc1.get_total_accounts))
<class 'method'>
```

```
print(type(acc1.is_valid_amount))  
<class 'function'>
```

Essentially, all are identified as class methods *apart from* the `staticmethod`, which is seen as a function. All should be **callable**.

Revision – overloading

Python supports extensive operator and special function overloading

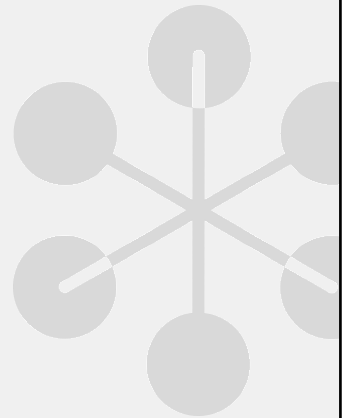
- Through special "magic" __functions__, e.g. `__str__`, `__add__` etc.
- available as a function interface in the `operator` module
- See also `@functools.total_ordering`

Examples for container classes:

```
__getitem__(self, index)
    [ ] element getter ( overloads obj[index] )
__setitem__(self, index, value)
    [ ] element setter ( overloads obj[index] )
__delitem__(self, index)
    del on an element ( del obj[index] )
__contains__(self, item)
    implements 'in' ( item in obj & item not in obj )
```

If building your own iterator:

```
__iter__(self) ( iter(obj) & for x in obj: )
__next__(self) ( next(iterator) )
```



QA

154

Almost everything can be overridden - see the operator module documentation for details.

The above can be confusing but note methods like `__str__` are defined in the ultimate super class (object) and on declaring them in a class you are overriding them (assuming their signatures remain the same). However, having declared a "dunder" method Python maps built-in operations to special methods, for example `str(x)` calls `x.__str__()` which overloads the behaviour of a built-in operation.

Overloading (in Python's common terminology) means defining special methods so that built-in operators or language constructs behave differently for your type.

PEP008 says that if you overload some of the comparison functions, like `__eq__` and `__lt__`, then you should also supply all the others. That tedious job is done for us by the decorator `@functools.total_ordering`.

Revision – an iterable container

Classes can be used to create iterable containers:

The example creates a supply of transaction values to a BankAccount class

As you will have already seen, a **generator** (a function which *yields* values as a *lazy* list) performs a similar task.

A container can supply more complex objects than this, of course!

```
# --- Usage ---
account = BankAccount("Alice")
account.add_transaction(100)
account.add_transaction(-20)
account.add_transaction(50)
```

```
for t in account:
    print(t)
```

```
100
-20
50
```

```
class BankAccount:
    def __init__(self, owner):
        self.owner = owner
        self.transactions = []

    def add_transaction(self, amount):
        self.transactions.append(amount)

    # Make the account iterable
    def __iter__(self):
        return TransactionIterator(self.transactions)

class TransactionIterator:
    def __init__(self, transactions):
        self.transactions = transactions
        self._index = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self._index >= len(self._transactions):
            raise StopIteration
        value = self._transactions[self._index]
        self._index += 1
        return value
```

155

QA

Let's try further testing of the iterable object...

#access via next() builtin function

```
account = BankAccount("Bob")
account.add_transaction(70)
account.add_transaction(80)
account.add_transaction(90)
account_iter = iter(account)
print(next(account_iter))
print(next(account_iter))
print(next(account_iter))
print(next(account_iter)) #this will raise an exception;
```

Will generate:

70

80

90

Traceback (most recent call last):

```
File "09_iterable_container.py", line 45, in <module>
    print(next(account_iter)) #this will raise an exception;
```



```
object is exhausted
      ~~~^^^^^^^^^^^^^^^^^^
File "09_iterable_container.py", line 23, in __next__
    raise StopIteration
StopIteration
```

Total ordering

The `functools.total_ordering` decorator

- Allows us to:
 - Automatically generate full “rich” comparison behaviour for data objects
 - Add comparison functionality to custom classes through decoration
- Typical requirements include:
 - Defining `__eq__` to test equality
 - Implementing at least one comparison method, such as `__lt__` (“less than”)
- Supported comparison methods include:

<code>__eq__()</code>	Equal to	<code>==</code>
<code>__lt__()</code>	Less than	<code><</code>
<code>__gt__()</code>	Greater than	<code>></code>
<code>__le__()</code>	Less than or equal to	<code><=</code>
<code>__ge__()</code>	Greater than or equal to	<code>>=</code>

Total ordering – example

total_ordering example:

In this example
functools.total_ordering helps
define ordering (comparisons) for
account objects based on balance....

```
acc1 = BankAccount("Alice", 1000)
acc2 = BankAccount("Bob", 500)
acc3 = BankAccount("Charlie", 1000)
```

```
print(acc1 > acc2)      # True
print(acc1 == acc3)     # True
print(acc2 <= acc1)     # True
```

Sorting accounts

```
accounts = [acc1, acc2, acc3]
for acc in sorted(accounts):
    print(acc)
```

```
Bob: £500
Alice: £1000
Charlie: £1000
```

```
from functools import total_ordering

@total_ordering
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def __eq__(self, other):
        if not isinstance(other, BankAccount):
            return NotImplemented
        return self.balance == other.balance

    def __lt__(self, other):
        if not isinstance(other, BankAccount):
            return NotImplemented
        return self.balance < other.balance

    def __str__(self):
        return f"{self.owner}: £{self.balance}"
```

157

Although only overloading behaviour has been defined for the == and < operators, the use of the total_ordering decorator found in the functools modules will "polyfill" the operator gaps, providing coverage for all the missing relational operators.

This means that adding the following line of code:

```
print(acc1 > acc2)
```

Will work just fine, even though __ge__ hasn't been explicitly defined.

However, removal of the total_ordering decorator will stop the "polyfill" behaviour and the comparison will generate an error.

Traceback (most recent call last):

```
File "11_total_ordering.py", line 29, in <module>
    print(acc2 <= acc1)      # True
    ^^^^^^^^^^^^^^^
```

TypeError: '<=' not supported between instances of
'BankAccount' and 'BankAccount'

Property()

`property()` is a built-in function

- It creates and returns a property object
- It ties methods to attribute-like access via "getter" accessors and "setter" mutators

Operation	Method called
<code>account.balance</code>	<code>get_balance()</code>
<code>account.balance = x</code>	<code>set_balance(x)</code>
<code>del account.balance</code>	<code>del_balance()</code>

```
property(fget=None, fset=None, fdel=None, doc=None)
```

```
class Account:
    def __init__(self):
        self.__balance = ""

    def get_balance(self):
        return self.__balance

    def set_balance(self, new_balance):
        if new_balance < 0:
            raise ValueError("Balance must be positive")
        self.__balance = new_balance

balance = property(get_balance, set_balance)

account = Account()
account.balance = 101.56
print(account.balance)
```

QA

158

Object Oriented developers are usually familiar with the need to create "public" "getter" and "setter" methods which access "private" attributes.

Python provides a clever device that removes the need for explicitly calling the "getter" and "setter" methods, but that still hides and protects the private attributes.

This is achieved with the `property()` builtin function.

The named arguments in the signature of the `property()` function are:

<code>fget</code>	gets the value of the property (accessor/getter)
<code>fset</code>	sets the value of the property (mutator/setter)
<code>fdel</code>	deletes a property
<code>doc</code>	a documentation string

Programmers usually focus on the first two (getter and setter) arguments using them positionally (as in the above example).

When the attribute (which is a *property object*) is used to retrieve the

underlying value (e.g. on the right-hand side of an assignment) the "getter" method (first parameter) is used. When used to update the underlying value (e.g. on the left-side of an assignment) the "setter" is used.

This single line of code sets the "fget" (accessor) and "fset" (mutator) arguments:

```
balance = property(get_balance, set_balance)
```

Note: they could be rewritten as two separate lines:

```
balance = balance.getter(get_balance)
balance = balance.setter(set_balance)
```

In both cases this effectively creates a new property called "balance" which provides a safe interface to the private attribute "__balance".

An alternative and more Pythonic technique (principally because it avoids creating additional functions), is to use the @property decorator – we will look at this next.

@property

@property is a decorator

- Builtin decorator for the `property()` function
- Provides a *pythonic* way to tie in the "getter" accessors and "setter" mutators
- Removes the need for additional function names and reduces unnecessary namespace pollution

```
class Account:
    def __init__(self):
        self.__balance = ""

    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, new_balance):
        if new_balance < 0:
            raise ValueError("Balance must be +ve")
        self.__balance = new_balance

account = Account()
account.balance = 101.56
print(account.balance)
```

QA

159

This implementation is generally recognised as the preferred technique and you will almost certainly see this property decorator in use whilst investigating different codebases.

For completeness, the following could be used to provide safe deletion of the `__balance` attribute in this example:

```
@balance.deleter
def balance(self):
    del self.__balance
```

A class is more than just a type!

Don't ask what type an object is, only ask what the object can do

```
hasattr(object, name)
getattr(object, name[, default])
```



This is known as *duck-typing*

"If it walks like a duck, swims like a duck, and quacks like a duck ..."

```
if hasattr(x, '__str__'):
    val = str(x)
```

In the example, we don't care *what* class *x* belongs to, only that it *supports* representation as a string

Based on the original concepts of Object Orientation

- Send/receive messages to/from an object
- Signature-based polymorphism

QA

160

A foundation concept of Object Orientation is that messages are sent to objects requesting actions. We should not need to check the class of the object, only that it can carry out the action requested. That principle has been lost with many OO languages, particularly static ones. Even those that fully support polymorphism, the practice is often to test the class rather than the behaviour.

Python, and most dynamic languages, allows the programmer to check if behaviour *exists*. This is called *Duck Typing*: we don't care what kind of thing it is, if it *quacks* that's good enough. It could be some kind of duck mimic, who cares?

setattr

Dynamically set the value of an attribute

```
setattr(object, name, value)
```

Why bother, when we can do routine assignment?

Because the attribute name is *dynamic*

- A string which can be generated at run-time
- Cannot be used for attributes hidden by `__` prefix
 - But a new attribute prefixed `__` can be added and is visible!

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

# --- Usage ---
account = BankAccount("Alice", 1000)

# Dynamically update attributes
setattr(account, "balance", 1200)
setattr(account, "account_type", "Savings") #
attribute didn't exist before

print(account.owner)           # Alice
print(account.balance)         # 1200
print(account.account_type)    # Savings
```

QA

161

Just as we can query an attribute whose name is stored in a string, so we can set a value as well: `setattr` is the set equivalent of `getattr`.

A property of `setattr` is that the user can add a new attribute to the object, even one prefixed with `__`. This can be dangerous, because the class will be unaware that user code is doing this, and might use the same attribute name in a future release – it somewhat negates the advantages of encapsulation.

Example setattr and getattr

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

# --- Usage ---
account = BankAccount("Alice", 1000)

updates = {
    "balance": 1500,
    "overdraft_limit": 200
} # Could come from file or API stream

for key, value in updates.items():
    current_value = getattr(account, key, None) # get existing value (if any)
    print(f"Updating '{key}': {current_value} -> {value}")

    setattr(account, key, value)
```

```
Updating 'balance': 1200 -> 1500
Updating 'overdraft_limit': None -> 200
```

QA

162

The following uses `getattr` to retrieve a set of attributes, some part of the original declaration, some dynamically added (via `setattr`) which may or may not be present.

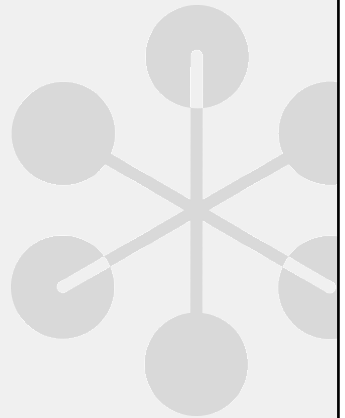
```
attributes = {
    "owner",
    "balance",
    "overdraft_limit",
    "account_type"
}

for key in attributes:
    current_value = getattr(account, key, None) # get existing
value (if any)
    print(f"Current attributes: '{key}': {current_value}")
```

Signature and parameter objects

A Signature object represents the parameter list of a function

- Available from Python 3.3 onwards
- Acts as a container for Parameter objects
- Can produce a list of parameter names
- **Signature and Parameter classes are imported from the inspect module**
 - Supports listing and creating signatures for functions and methods
- **Signatures are built from a collection of Parameter objects**
 - Parameters are defined using a string name and a type
 - Example type: `POSITIONAL_OR_KEYWORD`
- **A Signature can be bound to a function using:**
`Signature.bind(*args, **kwargs)`
- Refer to the chapter appendix for further details and examples



QA

163

Signatures are relatively new, and they represent the call interface of a function. Other languages might call this a prototype or a template, Ruby has a `parameters` attribute, and signature objects are part of Perl 6.

In Python, Signature and Parameter objects are created by the `inspect` module. A Signature is immutable, but you can create a new one using `Signature.replace()`.

A Signature has a `parameters` attribute, which is a container (mapping) of Parameter objects.

Signatures are described in PEP 362.

Revision: Inheritance

- **Leverage Parent Code:** Reuse attributes and methods from base classes automatically.
- **Core OOP Principle:** Inheriting functionality is central to object-oriented design.
 - **Flexible Inheritance:** Python allows multiple parent classes, though it is rarely required.
- **Automatic Fallbacks:** Missing attributes in a child class default to the parent class.
- **Extending Python Built-ins:** Regularly subclass native modules like threading or exceptions.

```
class MyDerivedClass(base_class):  
  
    def __init__(self, arguments):  
        super().__init__(self, arguments)  
  
    # Other methods...
```

QA

164

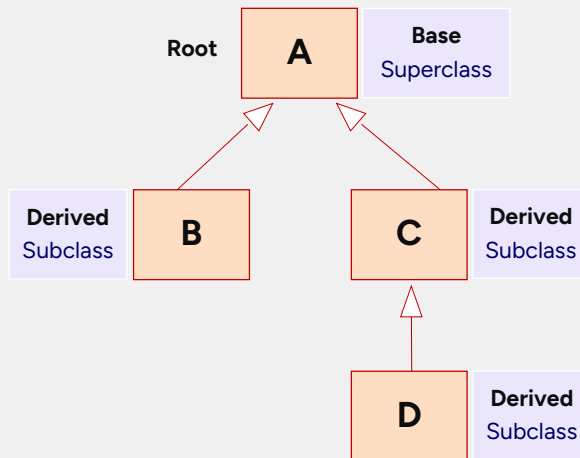
In object-oriented programming, inheritance establishes an "is a" relationship between classes, where a derived class automatically gains all the attributes and methods of its base class. The derived class can also introduce its own data members and methods to extend or specialise its behaviour. When a base class method doesn't fully suit the derived class's needs, the derived class can override it — a concept known as polymorphism.

Inheritance mirrors how we naturally think in everyday life. From an early age, we categorise objects by their shared characteristics. Take cars, for instance: the moment we say "account," a set of common properties is implied — `account_number`, `account_name`, `balance`, and so on — without needing to specify whether we mean a current, savings or ISA account. The same logic applies in object-oriented design, where inheritance lets developers group related classes under a common umbrella, allowing them to be treated either collectively in general terms or individually as distinct, specialised types.

Leveraging the proven functionality of existing classes leads to faster development and greater confidence in the final product. Inheritance also

promotes code efficiency, as a single base class method can serve multiple derived classes, reducing redundancy. In Python especially, it is standard practice to build custom classes by deriving from existing ones.

Revision: Python's Inheritance terminology



QA

165

There are several different methodologies available for expressing object-oriented designs, and many have an accompanying notation for representing object-oriented designs graphically. We shall adopt the simple notation shown to represent a hierarchy of derived classes, using arrows to indicate which is the base class.

For example, class C is derived from class A and class D is derived from class C. So, D inherits all the members of C, which in turn inherits all the members of A.

Inheritance introspection

The use of standard library builtins are the recommended technique for introspection:

isinstance(object, classinfo)

- Returns **True** if *object* is of class *classinfo*
- Overridden in `abc.ABCMeta` by `__instancecheck__`

issubclass(class, classinfo)

- Returns **True** if *class* is a derived class of *classinfo*
- Overridden in `abc.ABCMeta` by `__subclasscheck__`

```
...
acc = CurrentAccount(
    "Sadia Patel", '79927398718', 200.00, 500.00)

if isinstance(acc, CurrentAccount):
    print(acc, "is a CurrentAccount!")

if isinstance(acc, Account):
    print(acc, "is an Account!")

if issubclass(CurrentAccount, Account):
    print("CurrentAccount is a subclass of Account")
```

All these conditions
return True
(based on the inheritance
example)

QA

166

This is the preferred way of finding if an object or class belongs to a particular class, rather than using lower levels of introspection. In both cases, *classinfo* can be a tuple of classes.

The previously mentioned PEP3119 gives the specification and justification for these functions.

`abc.ABCMeta` is a built-in **metaclass** in Python used to create **Abstract Base Classes (ABCs)**. It provides the underlying machinery that allows you to define abstract interfaces and enforce that subclasses implement specific methods.

`abc.ABCMeta` handles two primary jobs behind the scenes:

1.Prevents Direct Instantiation: It stops you from creating an instance of a class if it has missing abstract methods.

2.Overrides Type Checking: It overrides `__instancecheck__` and `__subclasscheck__`. This allows you to register completely unrelated classes as "virtual subclasses" so they pass `isinstance()` and `issubclass()` tests.

The methods `class.__instancecheck__` and `class.__subclasscheck__` are looked up on the type (metaclass) of a

class. They cannot be defined as class methods in the actual class. Remember that special methods are called on instances, only in this case the instance is itself a class.

Inheriting from standard library classes

Existing classes can be extended, especially ones that are created by class factories such as "namedtuple".

Efficient Customisation:

Extending native classes adds bespoke functionality without the overhead of building new reference classes.

Inherent Data Security:

Leverages the immutable nature of tuples to make subclass data read-only by default.

```
from collections import namedtuple
# Base namedtuple
BankAccount = namedtuple("BankAccount", ["owner", "balance"])

# Extend namedtuple with methods
class SavingsAccount(BankAccount):
    def deposit(self, amount):
        return self._replace(balance=self.balance + amount)
    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        return self._replace(balance=self.balance - amount)

account = SavingsAccount("Alice", 1000)
print("Original:", account)
# Deposit money
account = account.deposit(500)
print("After deposit:", account)
# Withdraw money
account = account.withdraw(300)
print("After withdrawal:", account)
```

```
Original: SavingsAccount(owner='Alice', balance=1000)
After deposit: SavingsAccount(owner='Alice', balance=1500)
After withdrawal: SavingsAccount(owner='Alice', balance=1200)
```

QA

167

Extending a namedtuple can be useful

- adds new functionality via subclass
- can be quicker than creating single method classes

Note: `self._replace()` is a method of the namedtuple instance that returns a new copy with the specified fields changed.

Mixins

- Useful when you want to optionally extend a class with additional functionality, or apply the same functionality across multiple classes

```
class Sub1(Mix1, Mix2):
```

Mix1 is searched first

VS

```
class Sub2(Mix2, Mix1):
```

Mix2 is searched first

- Achieved in Python through multiple inheritance
- Mixin classes are not typically instantiated directly
- Not always the ideal approach — composition is worth considering as an alternative

Method Resolution Order (MRO)

- MRO can play a critical role when working with mixins
- In a simple inheritance hierarchy, methods are resolved from left to right

QA

168

These are sometimes called mix-ins.
The MRO can be determined using `inspect.getmro()`.

Mixins - Example

```
class TransactionLoggingMixin:  
    def log_transaction(self, message):  
        print(f"[LOG] {message}")
```

```
class FeeMixin:  
    def apply_fee(self, amount):  
        fee = amount * 0.02  
        self.balance -= fee  
        self.log_transaction(f"Fee applied: £{fee}")
```

```
class BankAccount:  
    def __init__(self, owner, balance=0):  
        self.owner = owner  
        self.balance = balance  
  
    def deposit(self, amount):  
        self.balance += amount
```

```
class PremiumAccount(TransactionLoggingMixin, FeeMixin, BankAccount):  
    def withdraw(self, amount):  
        if amount > self.balance:  
            self.log_transaction(f"{self.owner} failed withdrawal")  
        else:  
            self.balance -= amount  
            self.log_transaction(f"{self.owner} withdrew £{amount}")  
            self.apply_fee(amount)
```

```
acct = PremiumAccount("Peter", 100)
```

```
acct.deposit(50)  
acct.withdraw(100)
```

```
[LOG] Peter withdrew £100  
[LOG] Fee applied: £2.0
```

Multiple inheritance

What is a metaclass?

Metaclasses

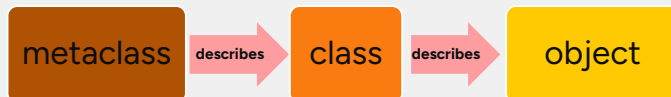
- A metaclass is an object responsible for creating class objects — essentially, it is the class of a class
- Just as a class defines the behaviour of its instances, a metaclass defines the behaviour of its classes

Metaclasses vs. Class Decorators

- Both are similar in concept, but differ in scope: a class decorator applies only to a single class, whereas a metaclass applies to an entire hierarchy and propagates downward through it

Common Use Cases

- Most frequently found in frameworks, where they are used to govern the behaviour of user-defined classes
- Typically involve reimplementing `__init__()` or `__new__()`
- May not modify anything at all — sometimes their sole purpose is to gather information, often by inspecting the class dictionary



QA

170

Every time you define a new class in Python, a metaclass is quietly at work behind the scenes — because classes themselves are objects, and it is a metaclass that brings them into existence. Python handles this transparently by default, but you are free to define your own when needed. Metaclasses are most commonly found in frameworks and GUI debuggers, where they serve a practical purpose such as tracking and managing classes throughout an application. It is worth noting that in a class hierarchy, the superclass can itself be thought of as playing the role of a metaclass.

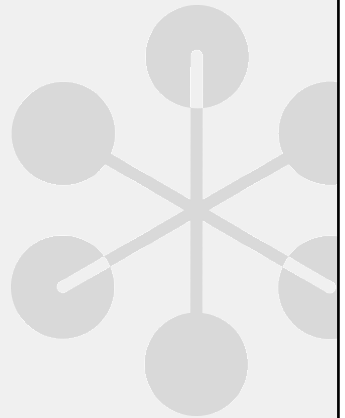
Metaclass hooks

Metaclass Hook

- A metaclass hook is what connects a user-defined class to its metaclass
- In Python 3, this is done by specifying the metaclass as a keyword argument in the base class tuple:
`class Shape(metaclass=abc.ABCMeta):`

Fallback Behaviour

- If no metaclass hook is defined, Python falls back to the first element of the base class tuple
- If no base class is specified at all, `type()` is used by default, since all classes implicitly inherit from `object`



The metaclass hook is just a mechanism to indicate that the class should be created by a user supplied metaclass, rather than the system default or the super class.

The example on the next slide uses `ABCMeta`, for Abstract Base Classes.

Abstract Base Classes (ABCs)

- An **Abstract Base Class (ABC)** cannot be instantiated directly, but it can be subclassed.
- Any **abstract methods** defined in the ABC must be implemented by subclasses; otherwise, those subclasses cannot be instantiated either.
- There are multiple ways to define an abstract method:
 - Provide a method implementation in the base class but:
 - raise a `NotImplementedError`, or
 - Use `abc.abstractmethod`, typically as a decorator, for example:

```
from abc import *
```

```
class Account(ABC):
```

```
    @abstractmethod
```

```
    def getType(self):
```

```
        pass
```

```
class CurrentAccount(Account):
```

```
    def getType(self):
```

```
        print("CurrentAccount is an Account")
```

```
>>> acc = CurrentAccount()
>>> acc.getType()
Account is a CurrentAccount
```

```
>>> acc = Account()
```

```
Can't instantiate abstract class Account...
```

QA

172

`ABCMeta` is exported by the `ABC` module. With meta-classes we might want to provide a `__new__` function, since the actual class used for the object creation could be different to the current class.

See PEP 3119 Introducing Abstract Base Classes for further details.

`abc-class.register(subclass)` would register `subclass` as a subclass of `abc-class`. This will affect the result of `isinstance()` and `issubclass()`.

ABC worked example (1)

```
from abc import ABC, abstractmethod

class Account(ABC):
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
    @abstractmethod
    def withdraw(self, amount):
        """Withdraw money from the account"""
        pass

class CurrentAccount(Account):
    def __init__(self, owner, balance=0, overdraft_limit=0):
        super().__init__(owner, balance)
        self.overdraft_limit = overdraft_limit
    def withdraw(self, amount):
        if self.balance - amount < -self.overdraft_limit:
            print(f"{self.owner}: Overdraft limit exceeded")
        else:
            self.balance -= amount
            print(f"{self.owner}: Withdrew £{amount}")
```

QA

173

In this example, we can create two different account objects (current and high interest savings) based on a more formless, abstract class called "Account".

While account can have an owner and a balance and has a concept of a withdraw method, it cannot practically *carry out a withdrawal* it because it has no sense of identity. Because of this limitation, the withdraw() method in the base Account class is marked as abstract using a suitable decorator.

This method then *has* to be implemented by any class (CurrentAccount, HighInterestAccount etc) which inherit from the Account ABC.

ABC worked example (2)

```
class HighInterestAccount(Account):
    MIN_BALANCE = 10000.0
    def __init__(self, owner, balance=10000.0, interest_rate=0.05):
        super().__init__(owner, balance)
        self.interest_rate = interest_rate
    def withdraw(self, amount):
        if self.balance - amount < self.MIN_BALANCE:
            print(f"{self.owner}: Cannot go below
£{self.MIN_BALANCE}")
        else:
            self.balance -= amount
            print(f"{self.owner}: Withdrew £{amount}")
    def apply_interest(self):
        interest = self.balance * self.interest_rate
        self.balance += interest
        print(f"{self.owner}: Interest added £{interest:.2f}")

current = CurrentAccount("Peter", 100, overdraft_limit=50)
high_interest = HighInterestAccount("Alice", 15000,
interest_rate=0.05)
current.withdraw(130)      # allowed (overdraft)
current.withdraw(50)      # exceeds overdraft
high_interest.withdraw(3000) # blocked if it drops below 10,000
high_interest.apply_interest()
```

Peter: Withdrew £130
Peter: Overdraft limit exceeded
Alice: Withdrew £3000
Alice: Interest added £600.00

QA

174

As you can see here, both the `CurrentAccount` and `HighInterestAccount` derived classes implement their own "concrete" versions of `withdraw()`, both specific to their differing calculations.

Also of note is the use of `super()` to pass the initialisation values of "owner" and "balance" to the parent's own initialiser through a simple (and common) "chaining constructors" mechanism.

One of advantage of this type of technique is that construction of related objects can be homogenized with methods (such as "`withdraw()`") being called generically – and – therefore predictably.

In other words, it doesn't matter what type of account we're dealing with – the interface is common in both instantiation (provide owner, balance then class specific values (`overdraft_limit/interest_rate`) and processing; simply call "`withdraw()`", it's all you really need to know.

Hiding (or at least name-mangling) the private data behind some decorated properties also makes accessing the `withdraw`, `owner` and `balance` from the parent relatively easy.

Custom context managers

Context managers run setup and cleanup code automatically

- Use the special methods `__enter__` and `__exit__`
- `__exit__` can handle exceptions and release resources
- Typically used with the `with` statement

```
with context_object as variable:  
    code
```

File objects are built-in context managers

- Eliminates the need for explicit `finally` blocks

```
with open('data.txt', 'r') as my_file:  
    for line in my_file:  
        print(line, end='')
```

```
print(my_file)
```

```
<_io.TextIOWrapper name='data.txt' encoding='UTF-8'>
```

- Custom context managers can be created using object-oriented programming techniques

QA

175

Context managers let us create and clean up objects within a defined scope. While a destructor may not execute immediately, the `__exit__` method is guaranteed to run when leaving the context, even if an exception occurs.

Custom context manager

A sample custom context manager which opens and deserializes JSON:

```
import json

# Our JSON handling file-based context manager! requires magic methods
class MyJsonFile():
    ...
    def __init__(self, filename, access_mode):
        """initialisation"""
        self.__filename = filename
        self.__access_mode = access_mode
    ...
    def __enter__(self):
        """open file and deserialize"""
        self.__open_file = open(self.__filename, self.__access_mode)
        self.__deserialize()
        return self.__myJsonData
    ...
    def __deserialize(self):
        """deserialize the json file into an object"""
        self.__myJsonData = json.load(self.__open_file)
    ...
    def __exit__(self, *args):
        """ensure file is safely closed"""
        self.__open_file.close()

with MyJsonFile('mydata.json', 'r') as mydata:
    """what do we have?"""
    print(mydata)
```

QA

176

A context manager can be used to abstract levels of processing.

In the example shown above, all the JSON processing is completed transparently.

Given the file contents of "mydata.json":

```
{"username": "auser",      "email": "a.user@qa.com",
"projects": ["email", "python", "regex"]}
```

This code will produce the following:

```
{'username': 'auser', 'email': 'a.user@qa.com', 'projects':
['email', 'python', 'regex']}
```

An alternative approach is to use the contextmanager decorator which can be found in the contextlib module:

```
from contextlib import contextmanager
```

```
@contextmanager
```

```
def my_json_file(filename, access_mode):
    open_file = open(filename, access_mode)
    yield json.load(open_file)
```

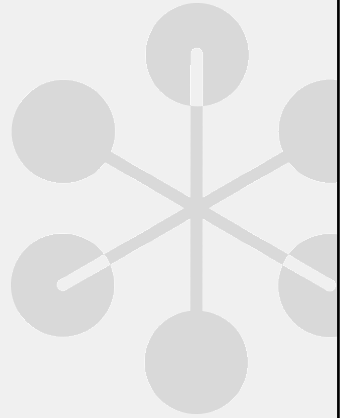
```
open_file.close()

with my_json_file('mydata.json', 'r') as mydata:
    """what do we have?"""
    print(mydata)
```

See also...

Standard library modules

- `contextlib` Context manager utilities



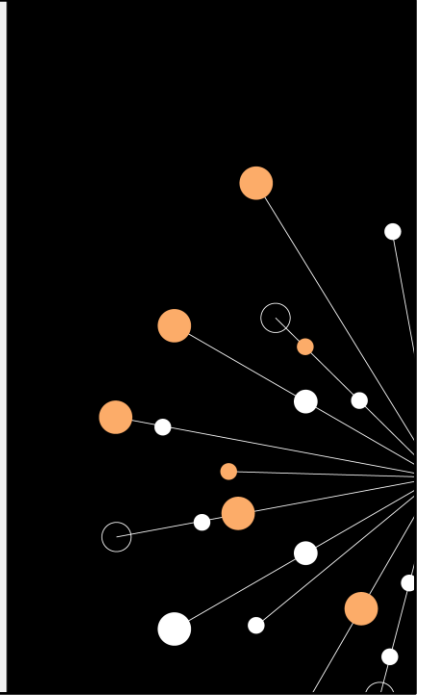
Exercise EG05: Further OOP

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA

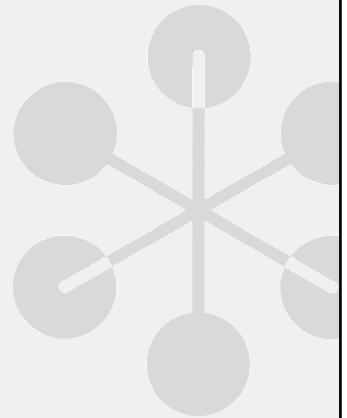


Suggested time: 30-45 mins.

Activity includes differentiated tasks.

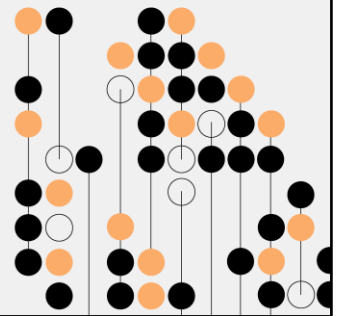
Summary

- Mixins are helper classes designed to be inherited by other classes
 - They are not intended to be instantiated directly
- `getattr()` and `hasattr()` allow attribute-based access without relying on object type
- `setattr()` enables attributes to be added or modified dynamically
 - Behaviour can be customised using special methods
- Determine an object's type using:
 - `isinstance()`
 - `issubclass()`
- Context managers provide clean and reliable resource handling
 - Often preferable to using finally blocks or `__del__` methods



Any questions?

QA



Appendices



Custom context manager – alternate technique

`__del__` may be unreliable – use a context manager instead

```
def add_ctxmgr(cls):
    def enterf(self):
        return self

    def exitf(self, exc_type, exc_value, tb):
        if tb:
            print("Error from line:", tb.tb_lineno,
                  file=sys.stderr)
            return True
        else:
            return False

    cls.__enter__ = enterf
    cls.__exit__ = exitf

    return cls

@add_ctxmgr
class SomeClass:
    ...
```

Return the object used as the target (var in the example)

Prevent the exception from propagating

with SomeClass(...) as var:
 print(var)

QA

182

The calling of `__del__` is not guaranteed (it's a request) and depends to some extent on the whims of the garbage collector (and this can vary between different implementations; CPython is generally fine due to its use of reference counting). Fortunately, we don't need a destructor as often as in, say, C++. An alternative is to use a context manager to explicitly handle the destruction of unwanted objects.

The `__enter__` method should return the object explicitly, the constructor has already been called before `__enter__`. The simple example above just passes the object through, but we could alter it or return a different object.

The `__exit__` method gets four arguments, `self`, followed by an exception type, value, and a traceback object – the same as returned by `sys.exc_info()`. If there has not been an exception, then the three exception parameters are all `None`.

If `__exit__` returns `True` this indicates that the exception has been handled and is not propagated. A return value of `False` indicates there has not been an exception.

The destructor, `__del__`, is called after `__exit__` (if at all).

Slots

A class variable, listing the *legal* attribute names

```
class Person:
    __slots__ = ('__name', '__gender')
```

An optimisation feature, designed to save memory

- Has hidden dangers, don't use unless you really need to
- Classes derived from a base class using `__slots__` should also use `__slots__`
- Have to be added in `__new__`

`__slots__` can break code which uses *introspection*

QA

183

The `__slots__` property looks like a neat data hiding trick on the face of it, but it was never designed as a safety feature. It is an optimisation feature which does away with the `__dict__` dictionary.

It has a number of disadvantages. First, it has to be constructed and maintained 'manually'. Second, inherited classes must use it as well, otherwise they might be *slower*. Thirdly, it breaks some introspection code.

We show it here not as an encouragement to use `__slots__`.

Descriptors

Descriptors are a generalisation of properties

- Descriptors can apply to any class attributes
- Use the special methods `__get__`, `__set__`, `__delete__`
- Uses the class's `__dict__`, so does not work with `__slots__`

```
class TraceUse(object):
    def __init__(self, pname, pos):
        self.tpos = f"{pname} line: {pos}"

    def __get__(self, inst, cls):
        return self.tpos

    def __set__(self, inst, value):
        self.tpos = str(value)

    def __delete__(self, inst):
        pass

class Person(object):
    t1 = TraceUse('Mark A',
                 inspect.currentframe().f_lineno)
```

TraceUse is a
descriptor class

Person is an
owner class

QA

184

Whereas properties define set/get/del/doc functions by attribute, descriptors can apply to any attributes of a class. Descriptors can only be used in an owning class - you cannot have a descriptor class look at its own attributes. When the attribute is accessed, one of the special functions is called.

`__get__(self, instance, owner)`

From the example, *self* will be a `TraceUse` object, *instance* will be a `Person` object, and *owner* will be the `Person` class object. `__get__` should return the attribute value.

`__set__(self, instance, value)`

self and *instance* are as `__get__`, *value* is the value to set.

`__delete__(self, instance)`

self and *instance* are as `__get__`

A fuller implementation of using `TraceUse` in `Person` might be:

```
class Person(object):
    t1 = TraceUse('Initial',
                 inspect.currentframe().f_lineno)

    def __init__(self, name):
        self.__name = str(name)
        # This gives the line number of the construction call
```

```
    Person.t1 = "%s line: %d" % ('__init__',
                                  inspect.currentframe().f_lineno)

def __str__(self):
    x = Person.t1
    return "%s %s" % (self.__name, x)
```

`__getattr__`, `__setattr__` and `__delattr__`

Special object methods called when an attribute is used

`__getattr__(self, name)`

- Must return a value or raise an `AttributeError`
- `__getattr__` is called if:
 - The attribute does not exist and `__getattribute__` is not supplied
 - `__getattribute__` raises an `AttributeError`

Explicitly called by the user	Automatically called	When called
<code>getattr</code>	<code>__getattr__</code>	See above
<code>setattr</code>	<code>__setattr__</code>	On any attribute assignment
<code>delattr</code>	<code>__delattr__</code>	On explicit deletion
<code>getattr</code>	<code>__getattribute__</code>	On any attribute get

QA

185

If the user calls the `getattr()` builtin then `__getattribute__` will be called if it exists.

`__getattr__` will only be called if `__getattribute__` does not exist or `__getattribute__` raises an `AttributeError`.

getattr and hasattr

Used when the attribute name is held in a *string*

- Can be useful to check for methods in a Mixin

Return the value of an attribute

```
getattr(object, name[,default])
```

```
print(getattr(me, 'telephone'))  
print(me.telephone)
```

Same*

Does an object have a named attribute?

```
hasattr(object, name)
```

Neither work for attributes hidden by `__` prefix unless you remember the name is munged

QA

186

These built-in functions are used when the attribute name is held in a string and avoids the dangerous use of `eval`. `getattr` will raise an `AttributeError` if the attribute does not exist, unless a default value is supplied. `hasattr` calls `getattr` and returns `False` if an exception is raised.

`getattr` should not be confused with Class methods `__getattr__` and `__getattribute__` that are used within a class to return attributes.

*Note

Why use `getattr()` rather than the more traditional `"."` (dot) member notation?

Well, the "name" (e.g. "telephone") string could itself *be* a variable, i.e. *dynamic*.

This type of indirection isn't available using the dot notation, i.e. having a variable "variable".

`__setattr__`, `__delattr__`

Dynamically set a new attribute

- Called when any attribute is assigned

```
__setattr__(self, name, value)
```

```
def __setattr__(self, name, value):  
    self.__dict__[name] = value
```

Delete an attribute

- Called when `del` is called on any attribute

```
__delattr__(self, name)
```

```
def __delattr__(self, name)  
  
    if name in self.__dict__:  
        del self.__dict__[name]
```

QA

187

Note that `__del__` is called when the whole object is deleted, not just an attribute.

`__delattr__` is *not* called automatically on object destruction, only on explicit attribute deletion.

Signature and Parameter objects – example (1)

```
from inspect import Signature, Parameter

class BankAccount:
    # Define allowed fields as a "schema"
    __signature__ = Signature([
        Parameter("owner", Parameter.POSITIONAL_OR_KEYWORD),
        Parameter("balance", Parameter.POSITIONAL_OR_KEYWORD),
        Parameter("overdraft_limit", Parameter.POSITIONAL_OR_KEYWORD, default=None),
    ])

    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

def apply_updates(obj, updates: dict, signature: Signature):
    allowed_fields = signature.parameters
    for key, value in updates.items():
        if key in allowed_fields:
            current_value = getattr(obj, key, None)
            print(f"Updating '{key}': {current_value} -> {value}")
            setattr(obj, key, value)
        else:
            print(f"Ignoring unknown field: {key}")
```

QA

188

The `setattr` example from earlier can be implemented using a Signature.

`inspect.Signature` and `inspect.Parameter` are mainly for **describing and validating callables**, not for dynamically setting attributes on instances. However, you *can* adapt your design so that updates go through a structured "parameter schema" instead of raw `setattr()` calls. A common pattern is to treat updates as a **validated contract** rather than ad-hoc attribute mutation.

Key idea

Signature acts like a **declared schema**

Parameter defines **allowed fields + defaults**

In the example `setattr()` is still used, but now it is:

- validated
- constrained
- structured

Signature and Parameter objects – example (2)

```
# --- Usage ---  
account = BankAccount("Alice", 1000)
```

```
updates = {  
    "balance": 1500,  
    "overdraft_limit": 200,  
    "unknown_field": "ignored"  
}
```

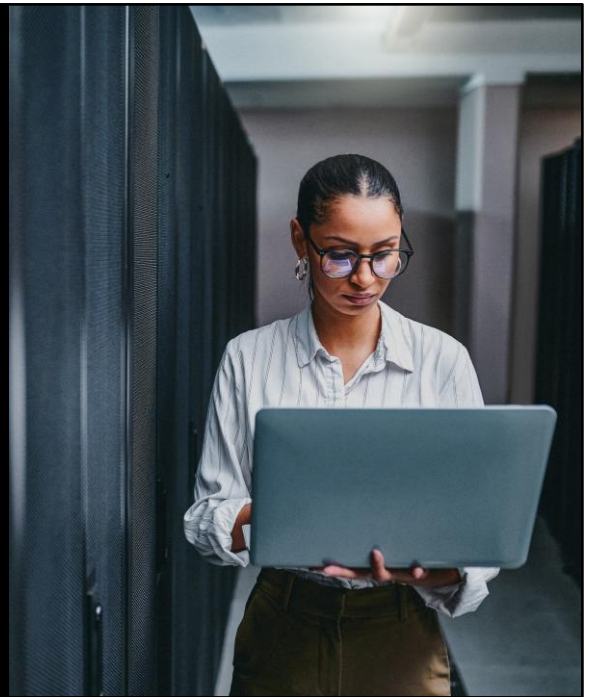
```
apply_updates(account, updates,  
BankAccount.__signature__)
```

```
# Inspect known attributes via signature  
for name in BankAccount.__signature__.parameters:  
    value = getattr(account, name, None)  
    print(f"{name}: {value}")
```

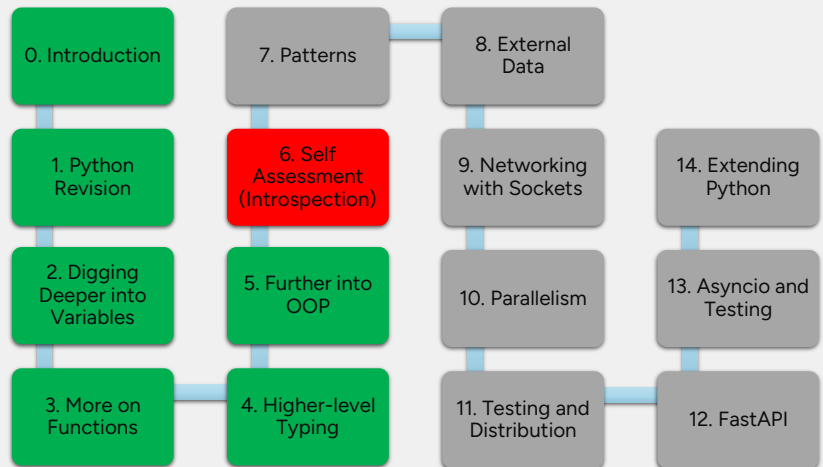
```
Updating 'balance': 1000 -> 1500  
Updating 'overdraft_limit': None -> 200  
Ignoring unknown field: unknown_field  
owner: Alice  
balance: 1500  
overdraft_limit: 200
```

Intermediate Python

Self Assessment (Introspection)



Module roadmap



QA

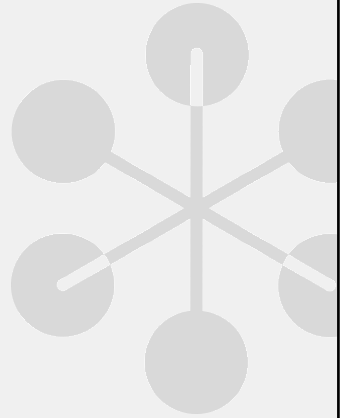
Self Assessment (Introspection)

Learning Outcomes:

- Use Introspective techniques to determine the type and features of an object at run-time

Prerequisites:

Delegates are required to have knowledge and skills relating to basic Python programming, including data structures, functions, loops and flow control.

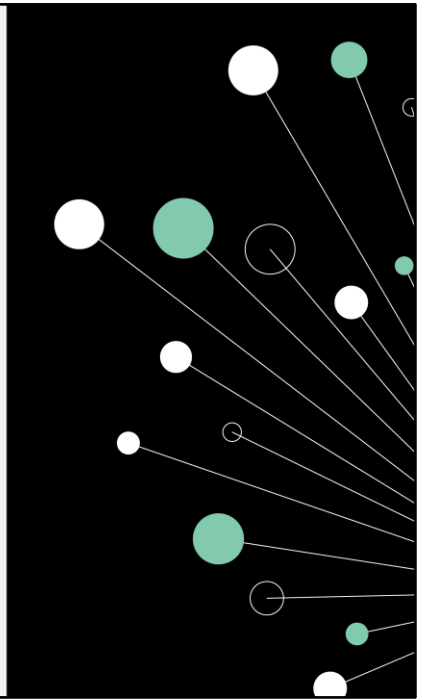


Learning objectives

By the end of this session, you will be able to:

- **Define Runtime Introspection:** Understand how to inspect and query active objects, types, and metadata during program execution.
- **Audit Namespaces & Scopes:** Map active identifiers using `locals()` and `globals()`, and examine object namespaces via `__dict__`.
- **Discover Object Attributes:** Utilize `dir()` to list valid attributes and `callable()` to identify executable routines.
- **Analyze Structural Metadata:** Evaluate object memory optimizations via `__slots__` and trace class hierarchies using inheritance introspection.
- **Audit Executing Code:** Leverage the `inspect` module to parse active frames and traceback to analyse the call stack.

QA



What is Introspection?

Introspection

- The ability of a program to examine both itself and its surrounding execution environment at runtime

What Can Be Examined

- The scope of introspection extends beyond the current code frame to include:
 - Variables, modules, and program code frames
 - The Python runtime itself
 - The underlying operating system
- Externally imported objects are also fair game

Introspection in Python

- In Python, introspection primarily involves examining objects — and it is worth remembering that modules, classes, and functions are all objects too
- Familiar built-in functions such as `dir()`, `id()`, `type()`, and `isinstance()` form the foundation, and are further complemented by a range of dedicated modules in the standard library

dir()

- **Native Directory Listing:** `dir()` is a built-in function displaying object or class attributes in alphabetical order.
- **Unified Treatment:** Lists both data fields and methods (callable attributes) indiscriminately.
- **Deep Visibility:** Exposes hidden/mangled names (`__`) alongside standard properties.
- **Customisable Behaviour:** Modifiable via the `__dir__` magic method to filter or alter output.
- **Scope Defaulting:** Discovers inherited attributes by default, or the local symbol table if no object is passed.
- **Dynamic Inspection:** Programmatically profiles objects at runtime to determine available features.

QA

195

We often encounter the built-in `dir()` function when working interactively in Python (for example in IDLE). It returns a list of an object's available attributes.

This behaviour can be customised by defining a `__dir__` method inside a class; otherwise, Python falls back to a default implementation.

By default, `dir()` behaves as follows:

- For a module: it returns the names of that module's attributes
- For a class or type: it returns the class attributes, including those inherited from base classes (recursively)
- For an instance: it returns attributes of the object itself, its class, and recursively its base classes

It is worth noting that `dir()` is primarily intended as an interactive exploration tool rather than something to rely on in production code, acting more as a convenience for inspection and documentation support.

locals() and globals()

Both functions return a reference to a dictionary

- Modifying this dictionary is not safe and should be avoided

locals() returns the current local symbol table

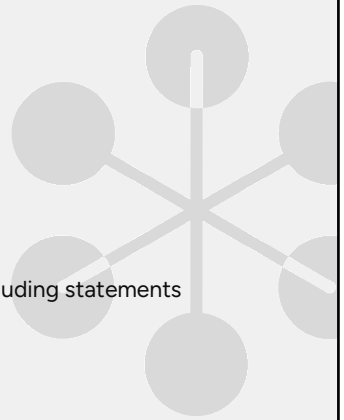
- It may also include free (non-local) variables

globals() returns the module-level symbol table

- This represents the state of the module from where globals() is called

Related built-in functions include eval() and exec()

- exec() executes dynamically compiled code from a string or code object, including statements
- eval() evaluates an expression and returns its result
 - only accepts expressions, not full statements



QA

196

locals() and globals() may appear simple at first glance, but they form the foundation of Python's introspection capabilities—especially since they can contain objects like functions and classes.

The distinction between eval and exec is significant, though it is often misunderstood by developers coming from languages such as Perl. The following examples highlight how they differ:

```
print(eval('37')) # 37
print(exec('37')) # None
```

```
print(exec('if True: print(37)')) # 37
print(eval('if True: print(37)')) # SyntaxError: invalid syntax
```

In essence, eval evaluates expressions and returns their result, whereas exec executes statements but does not return a value.

Finally, compile() serves as the underlying mechanism.

callable()

callable() is a built-in function that returns True if an object is callable

- Classes are "callable" (think constructors)
- An object should be callable if it has a `__call__` method

Given Python's EAFP idiomatic approach it's important to build in some error handling:

```
try:
    withdraw(100)
except NameError as err:
    print("'withdraw' function is not available")
```

```
class Account():
    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

def print_account(account):
    print(account.balance)

x = Account(1200)
if hasattr(x, "deposit") and callable(x.deposit):
    print("x.deposit exists and is callable")
if hasattr(x, "myfunc") and callable(x.myfunc):
    print("x.myfunc exists and is callable")
```

```
x.deposit exists and is callable
local function 'print_account' exists and is callable
'myfunc' function is not available
```

QA

197

EFAP - Easier to Ask for Forgiveness than Permission.

This built-in has had an interesting history in Python 3, having initially been removed before later being restored.

Its functionality overlaps somewhat with parts of the inspect module, particularly `inspect.ismethod()` and `inspect.isfunction()`.

However, `callable()` is more general, returning True for any object that can be invoked, including classes. Although classes may not immediately seem callable, creating an object actually involves calling the class with the arguments required by its constructor.

`callable()` also returns True for objects whose class defines a `__call__` method. This special method allows an instance to behave like a function.

Object self-analysis: A worked example (1)

Extensibility in Python

- Extensibility is at the heart of productive Python development — rather than trying to anticipate every future requirement, the goal is to write code that is as adaptable as possible
- This way, whoever maintains the code next — which may not be you — is well-equipped to handle evolving needs

A Practical Example: Password Validation

- The following object-oriented example demonstrates this principle through a password validator
- It works by introspectively building a list of required checks at runtime, discovering them automatically through a consistent naming convention
- The initial implementation includes two basic checks:
 - The password must be at least 8 characters long
 - The password must contain at least one uppercase letter

Designed for the Future

- Additional checks may well be needed down the line
- Thanks to the dynamic nature of the implementation, a new check can simply be added as a method to the class and it will be picked up and applied automatically — no further changes required

QA

198

Object analysis: A worked example (2)

```
from pprint import pprint

class Validator:
    MIN_SIZE = 8

    def __init__(self, new_pwd):
        """Constructor"""
        self.__password = new_pwd
        self.__methods = self.get_checks()

    def check_len(self):
        """Checks minimum length of password"""
        return len(self.__password) >= Validator.MIN_SIZE

    def check_uppercase(self):
        """At least one uppercase letter"""
        return any(char.isupper() for char in self.__password)

    def get_checks(self):
        """Dynamically assemble the check"""
        self.__methods = [method for attrib in dir(self)
                           if attrib.startswith("check_") and
                           callable(method := getattr(self, attrib))]
        return self.__methods
```

QA

199

Key Points to Note

- The solution is object-oriented by design, which makes it straightforward to search a defined scope for validation checks within a self-contained object
 - The constructor invokes a core method, `get_checks()`, which dynamically locates all methods prefixed with `check_` and stores references to them in a list
 - Each individual check adheres to the Single Responsibility Principle (SRP) — every method has one clearly defined job and does it well
- The following slide explores how these checks are invoked and applied

Object analysis: A worked example (3)

```
@property
def checks(self):
    _descriptions = [method.__doc__ for method in self.__methods]
    return list(zip(_descriptions, self.__results))

@property
def is_valid(self):
    self.__results = [func() for func in self.__methods]
    return all(self.__results)

password = input("Enter password: ")
new_validator = Validator(password)

if new_validator.is_valid is True:
    print(f"{password} is valid")
else:
    print(f"{password} is invalid")
    pprint(new_validator.checks)
```

```
Enter password:Testit
Testit is invalid
[('Checks minimum length of password', False),
 ('At least one uppercase letter', True)]
Enter password:testGood
testGood is valid
Enter password:testgood
testgood is invalid
[('Checks minimum length of password', True),
 ('At least one uppercase letter', False)]
```

Object self-analysis: A worked example (4)

Adding a New Rule

- To introduce an additional check — for example, verifying that the password contains at least one digit — all that is required is adding a correctly named method to the class

```
def check_isdigit(self):  
    """At least one digit"""  
    return any(char.isdigit() for char in self.__password)
```

- Because the code relies on introspection, the new method is discovered and incorporated into the validation process automatically

The Result

- No further modifications are necessary
 - it simply works

```
Enter password:Testit2  
Testit2 is invalid  
[('At least one digit', True),  
 ('Checks minimum length of password', False),  
 ('At least one uppercase letter', True)]  
Enter password:12testme  
12testme is invalid  
[('At least one digit', True),  
 ('Checks minimum length of password', True),  
 ('At least one uppercase letter', False)]  
Enter password:12Testme  
12Testme is valid
```



QA

201

This auto-discovery approach closely resembles the technique employed by many unit testing frameworks where new test cases are automatically recognised simply by virtue of their naming prefix, `test_`, as soon as they are added to the module.

__dict__

- **User-Defined __dict__:** Created automatically for custom modules and classes.
- **Built-in Omissions:** Absent from native data types like str, list, dict, file and set.
- **Slotted Omissions:** Omitted in optimized classes that explicitly define __slots__.
- **Global Exposure:** Includes __builtins__ when inspected at the module level.

```
import account
import pprint

pprint.pprint(account.__dict__)

{'__doc__': None,
 '__file__': 'C:\\qa\\python\\account.py',
 '__name__': 'account',
 '__package__': '',
 '_uhidden': <function _uhidden at 0x022757B0>,
 'uinterface': <function uinterface at 0x022757F0>}
```

Module functions,
including those
'hidden'

QA

202

Most modules expose their attributes through a __dict__ dictionary, which can be a handy tool for debugging — pprint() is particularly useful for presenting its contents in a readable format. That said, it does not always paint the full picture, and there are notable cases where its output can be incomplete or misleading.

As a general rule, manipulating __dict__ programmatically is considered poor practice and is best avoided.

__dict__ on objects and classes

Reveals attributes hidden using the __ prefix

- Demonstrates Python's name mangling behaviour

On a class:

- Identifies attributes created with property()
- Attributes inherited from parent classes are not specially labelled

```
mappingproxy({'__doc__': None, '__firstlineno__': 2,
              '__init__': <function CurrentAccount.__init__ at 0x000001D5480C9440>,
              '__module__': 'account2',
              '__static_attributes__': ('_overdraft_limit',),
              'withdraw': <function CurrentAccount.withdraw at 0x000001D5480C9620>})
```

On an object:

- Does not show attributes generated through property()
- Includes attributes inherited from superclasses

```
{'_Account__balance': 2301.48, '_balance': 2301.48,
 '_overdraft_limit': 500.0, 'id': '98127634',
 'name': 'Tina Smith'}
```

QA

203

Most objects and classes have a `__dict__` dictionary containing the attributes of the object or class.

Note: this breaks encapsulation and that's what introspection is all about. Any programmer using such techniques must take responsibility for their own code and remember that class implementations change.

Maybe there is another way...

Slots

Object attributes are stored in a dictionary called `__dict__`

- Places no fixed limit on the number of attributes an object can have
- Attributes can be added *dynamically* after the `__dict__` has been defined
 - This is good for flexibility but has a negative impact on readability and can reduce IDE and type-checking support.

Introducing `__slots__`

- To prevent attributes from being added dynamically, the `__slots__` attribute can be used — simply define it as a list within the class containing the names of the permitted attributes

Why Use `__slots__`?

- Optimises memory usage at instantiation, since the object's memory requirements are known upfront
- Acts as a safeguard against the unintended creation of dynamic attributes

Slots – some protection

Here's an example of preventing the creation of dynamic attributes:

```
class Account:
    __slots__ = ['__name', '__id', '__balance']

    def __init__(self, name="Anon", id="XXXXXXX", initial_balance=0):
        self.__balance = initial_balance
        self.__name = name
        self.__id = id

acc = Account("Brian May", "12345678", 1000.00)
# Attempt to create dynamic attribute
acc.email = "bmay.badger@stb.com"
```

The attempt to create and assign the new attribute will error:

```
Traceback (most recent call last):
  File "C:\QA\Introspection\account2.py", line 15, in <module>
    acc.email = "bmay.badger@stb.com"
    ^^^^^^^^^^^
AttributeError: 'Account' object has no attribute 'email' and no __dict__ for setting new attributes
```

Slots – take care!

Classes that define `__slots__` do not include a `__dict__`.

- This results in a memory optimisation.
- However, it can make introspection more difficult.
- That said, `__slots__` attributes can still be accessed and listed, similar to a `__dict__`.

```
>>> print(acc.__slots__)
['_name', '_id', '_balance']
```

How can you determine whether a class uses `__slots__`?

- The class object itself still retains its own `__dict__`.

```
if "__slots__" in Account.__dict__:
    print("slots:", acc.__slots__)
else:
    print("dict:", acc.__dict__)
```

slots: ['_name', '_id', '_balance']

dict: {'_Account__balance': 2301.48, '_Account__name': 'Tina Smith', '_Account__id': '98127634'}

This approach supports both styles of class construction when performing introspection.

QA

206

`__slots__` may initially appear to provide a form of data hiding, but it was never intended as a mechanism for encapsulation or safety. Its real purpose is optimisation, reducing memory usage by removing the need for an instance `__dict__`.

However, `__slots__` comes with several drawbacks. Unlike the dynamic nature of `__dict__`, slot definitions must be created and maintained manually. Inherited classes must also define `__slots__` correctly, otherwise some of the intended performance benefits may be lost. In addition, and particularly relevant to introspection, `__slots__` can break code that assumes objects provide a `__dict__` attribute.

Consequently, slots are presented here not as a recommendation, but as a reminder to be cautious when introspecting object attributes.

Exceptions and traceback objects

`sys.exc_info()`

- Returns a *tuple* of type, value, and a **traceback** object
- The traceback object includes attributes such as the line number where the exception occurred, and the stack trace

```
import sys

try:
    open("some file name")
except IOError as err:
    type, val, tb = sys.exc_info()
    print("Exception lineno:", tb.tb_lineno)
```

The traceback module

- Includes utilities to trace back through an exception cascade
- `traceback.print_exc()` is short for `traceback.print_exception(*sys.exc_info())`
- Also includes `tb_frame`, the current code frame ...

QA

207

Exceptions can occur deep within **nested function calls**, and one may trigger another. To inspect these errors, the **sys** and **traceback** modules provide useful tools.

`sys.exc_info()` returns a tuple containing the exception **type**, **value**, **and traceback**. While the traceback can be accessed directly, it's usually handled via the **traceback** module, which provides several utilities for examining exceptions.

The most common is `traceback.print_exc()`, which prints the stack trace in the same format as the interpreter. For example:

```
class Exc1(Exception):
    pass

def func1():
    try:
        open("some file name")
    except EnvironmentError as err:
        raise Exc1(err)

import sys, import traceback
try:
```

```
    func1()  
except:  
    traceback.print_exc()
```

By default, this prints to the console, but `traceback.format_exc()` returns the same information as a string for logging or custom error handling.

Frame objects

Executable Python code is stored in frame objects

- frame objects occur in **Traceback** objects
- Can be accessed by **inspect** module
- Can also be retrieved using: `sys._getframe(depth)`
- May also be passed to registered **trace functions**
- Sample code **frame attributes** include:

f_back	Previous stack frame (to the caller)	Read only
f_locals	Dictionary of local variables	Read only
f_code	Code being executed in the current frame	Read only
f_globals	Dictionary of Global variables	Read only
f_builtins	All built-in (intrinsic) names	Read only
f_lasti	Index of the Python bytecode of the code object (	Read only
f_trace*	Unless set to None, can be written to specify callable function used by a debugger during code execution	Writeable
f_lineno*	Current line number of the frame; debugger can alter	Writeable

QA

* may be altered, typically by a debugger, see `sys.settrace()`

208

Note: `sys._getframe()` is not guaranteed to be available on all non-CPython implementations of Python.

Several other functions also expose frame objects. One particularly useful example is `sys._current_frames()`, which returns a dictionary mapping thread identifiers to their current stack frames. This can be helpful when diagnosing issues such as deadlocks.

Frame objects can also be accessed via the `inspect` module, which we will look at next.

The inspect module – getting the full story

inspect is a module in the standard library that allows us to test complex "live" object types

- `inspect.ismodule()`, `inspect.isclass()`
- `inspect.isfunction()`, `inspect.ismethod()`
- `inspect.isgenerator()`, `inspect.isbuiltin()`, etc.

`inspect.getmembers()` inherits from `dir`

- Gives details of each member, not just their names

For example, getting details of our current code frame:

```
lineno = inspect.currentframe().f_lineno
code = inspect.currentframe().f_code
print(f"An error occurred on line: {lineno}", file=sys.stderr)
print(f"Code object: {code}", file=sys.stderr)
print(f"Raw compiled bytecode: {code.co_code}", file=sys.stderr)
```

QA

209

The inspect module provides a comprehensive set of tools for Python introspection. While much of the same information can be accessed through other means, the inspect module generally offers a more convenient and readable approach.

Further reading:

<https://docs.python.org/3/library/inspect.html>

Inspecting a function: arguments only

You can dynamically analyse a function to identify its:

- parameter names
- variable positional parameters (e.g., *args)
- keyword parameters (e.g., **kwargs)
- default argument values

This can be easily achieved using the inspect module.

```
import inspect

def myfunc(char, qty=10):
    return char * qty

print(inspect.getfullargspec(myfunc))
```

```
FullArgSpec(args=['char', 'qty'], varargs=None,
varkw=None, defaults=(10,), kwonlyargs=[],
kwonlydefaults=None, annotations={})
```


Inspecting a function: signature and parameters

By simply using the inspect module it is possible to dynamically profile an annotated function to discover its:

- Signature object
- Parameter objects

```
import inspect

def myfunc(text: str, qty: int = 5) -> str:
    return text * qty

args = inspect.signature(myfunc)
print(args.__slots__)
print(args._return_annotation)
print(args._parameters)
print(args._parameters['text'])
print(args._parameters['text']._name)
print(args._parameters['text']._annotation)
print(args._parameters['qty']._default)
```

```
('return_annotation', '_parameters')
<class 'str'>
OrderedDict({'text': <Parameter "text: str">, 'qty': <Parameter "qty: int = 5">})
text: str
<class 'str'>
5
```

QA

211

Python has a signature object for a function which contains `_annotation` and `_parameter` objects (located via `__slots__`).

The `_parameter` object is a form of `OrderedDict` which can be interrogated to access the name, type and, if applicable, the default value.

Inspecting modules – example 1

`inspect.ismodule()` can be used to identify and locate imported modules in the current frame:

```
import inspect, re
import pprint as pp
import currentAccount as ca

for name, val in list(locals().items()):
    if inspect.ismodule(val):
        fullname = str(val)
        if not '(built-in)' in fullname:
            match = re.search(r"'(.+)'.*'(.+)'", fullname)
            module, path = match.groups()
            print(f"{name:12s} maps to {path:s}")
```

```
inspect    maps to C:\\Python313\\Lib\\inspect.py
re         maps to C:\\Python313\\Lib\\re\\__init__.py
pp         maps to C:\\Python313\\Lib\\pprint.py
ca         maps to C:\\QA\\Files\\currentAccount.py
```

QA

212

The result of casting a module as a string might look something like this:

```
import re
print(re)
```

```
<module 're' from 'C:\\Python313\\Lib\\re\\__init__.py'>
```

Another module inspection tool is **sys.modules**, a dictionary of loaded modules. It can be manipulated to fool Python into reloading a module.

Inspecting modules – example 2

Which module aliases do you have?

```
import inspect, sys, re
```

spection.py

```
def look():
    for name, val in list(sys._getframe(1).f_locals.items()):
        if inspect.ismodule(val):
            fullname = str(val)
            if not '(built-in)' in fullname \
                and not '__name__' in fullname:
                match = re.search(r"'(.+)'.*'(.+)'", fullname)
                module, path = match.groups()
                print(f"{name:12s} maps to {path:s}")
```

```
import spection
import re
import pprint
spection.look()
```

```
re         maps to C:\\Python313\\Lib\\re\\__init__.py
pprint     maps to C:\\Python313\\Lib\\pprint.py
```

QA

213

The previous slide showed the modules loaded by the current code frame. The code on this slide is *almost* the same, but we are inspecting the previous code frame - the caller's. The `_getframe(1)` indicates that we wish to go back one level in the code frame stack. We can try to go back further, but of course there may be nothing there!

What is dis?

A traceback object can be disassembled

- Into Python bytecode
- Using the `dis` module
- `-->` marks the spot

Python source

```
a = 1
b = 0
try:
    x = a/b
except:
    import dis, sys
    tb = sys.exc_info()[2]
    dis.dis(tb)
```

compiled bytecode

```
C:\Python313\python.exe "C:\QAL\Files\25_what_is_dis.py"
0      RESUME                               0
1      LOAD_CONST                           0 (1)
      STORE_NAME                           0 (a)
2      LOAD_CONST                           1 (0)
      STORE_NAME                           1 (b)
3      NOP
4      L1:  LOAD_NAME                        0 (a)
      LOAD_NAME                        1 (b)
      --> BINARY_OP                      11 (/)
      STORE_NAME                        2 (x)
      L2:  RETURN_CONST                     2 (None)
--      L3:  PUSH_EXC_INFO
```

interpret

QA

214

Python code is compiled into bytecode regardless of whether it comes from a script or is entered interactively in the Python shell.

This bytecode is then executed by the Python virtual machine (VM).

When an exception is raised and successfully handled, the bytecode for the current frame remains available, as shown in the example above.

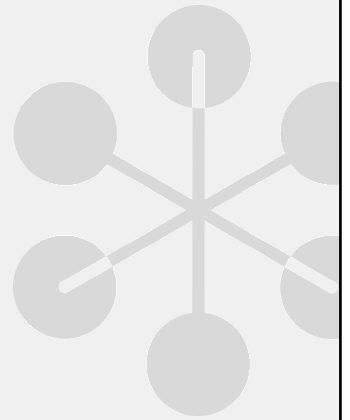
To explore this in more detail, you can inspect the generated bytecode with the `dis` module from Python's standard library. The output conveniently marks the point where the exception occurred with a `-->` indicator.

The `dis` module also provides several other useful functions, including `dis.dis()`, which can disassemble executable objects such as functions or classes. Examining the quantity and types of bytecode instructions can help identify potential performance bottlenecks.

See also...

Standard library modules

- **cgitb** Traceback manager for CGI scripts
- **dis** Disassembler for Python bytecode
- **inspect** Inspect live objects
- **pdb** The Python Debugger
- **pyclbr** Python class browser support
- **trace** Trace or track Python statement execution
- **traceback** Print or retrieve a stack traceback



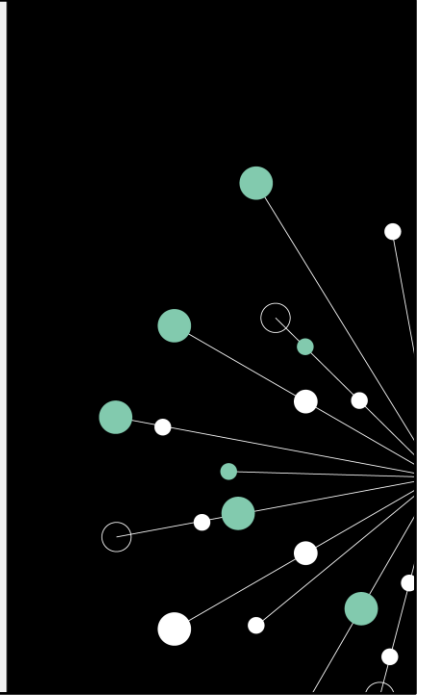
Exercise EG06: Introspection

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA

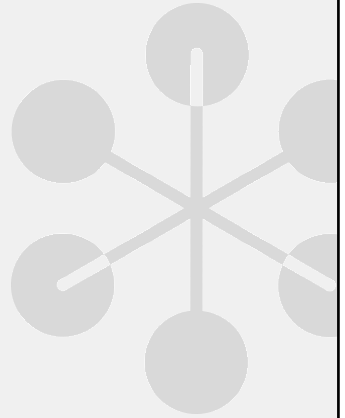


Suggested time: 30-45 mins.

Activity includes differentiated tasks.

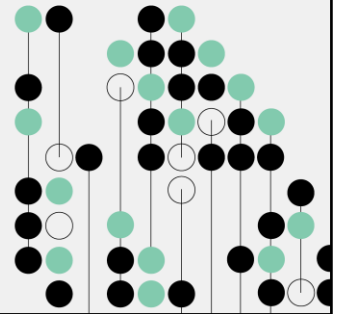
Summary

- Several built-in functions support introspection, including `locals()`, `globals()`, `dir()`, and `callable()`
- While explicit code is generally preferable, introspection enables more dynamic behaviour, which can help reduce ongoing maintenance
- Most Python objects provide a `__dict__` attribute for accessing their internal state
 - However, many built-in types do not expose this
- Some classes use `__slots__`, which changes how attributes are stored and limits the presence of `__dict__`
- The Python standard library offers extensive tools for introspection
- Many of these tools operate on code frame objects
- The `traceback` module is especially useful for analysing exceptions
- The `inspect` module provides a particularly rich set of introspection capabilities



Any questions?

QA



Appendices

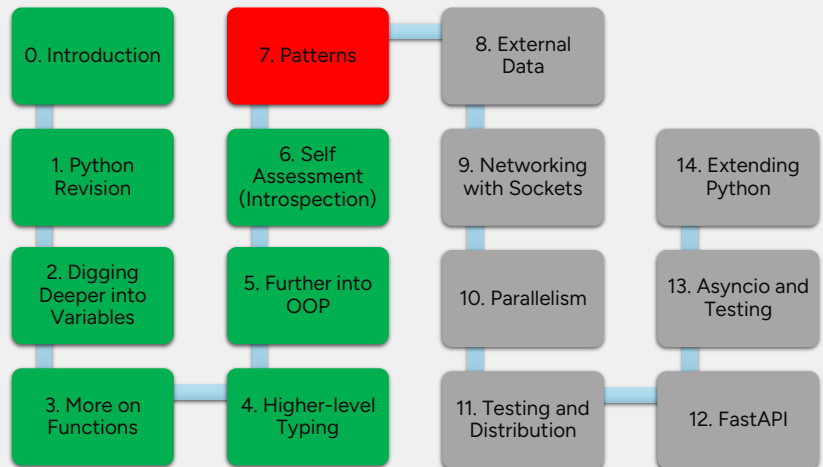


Intermediate Python

Patterns



Module roadmap



QA

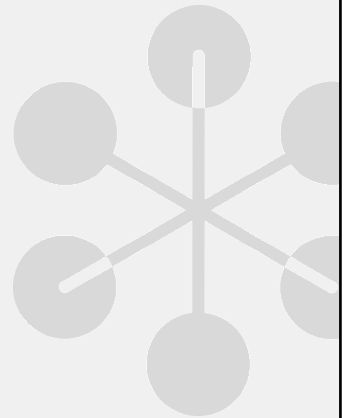
Object Oriented Programming - Patterns

Learning Outcomes:

- Use Object Oriented programming techniques in Python 3 to implement well-known design patterns that enhance problem-solving skills

Prerequisites:

Delegates are required to have knowledge and skills relating to basic Python programming, including data structures, functions, loops and flow control.



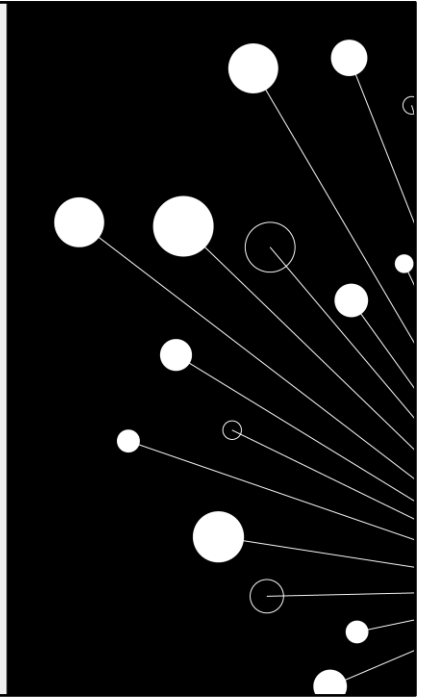
Appendices include details on suggested reading if you are interested in pursuing these concepts further.

Learning objectives

By the end of this session, you will be able to:

- **Define Core Patterns:** Understand what architectural design patterns are and the concrete steps required to implement them.
- **Master Creational Design:** Implement the Factory Method for decoupled object creation and the Singleton pattern for global state management.
- **Structure Flexible Objects:**
 - Deploy Delegation and Object Proxying to dynamically intercept, wrap, and forward method calls. Utilize the
 - Adapter pattern to seamlessly unify disparate class behaviours without changing underlying code.
- **Deconstruct Event Systems and Encapsulate Executable Actions :**
 - Engineer decoupled, reactive architectures by building custom Observer notification systems.
 - Convert operations into standalone, queuing-ready objects using the Command pattern.

QA



What is a design pattern?

What is a Design Pattern?

- A reusable, general-purpose solution to a recurring problem in software development
- It is not something that can simply be copied and pasted — it is a concept that must be interpreted and adapted to suit a specific context
- It is neither a function nor a library, and should not be thought of as a strict algorithm; rather, it is a blueprint that guides the structure of a solution
- By its nature, a design pattern is object-oriented

Categories

- Design patterns are broadly grouped into three widely recognised categories:

• Concerned with how objects are created
• Aim to make object creation flexible and reusable
• Examples include Singleton & Factory

Creational



• Concerned with how classes and objects are composed and organised
• Focus on building larger, complex structures from simpler components
• Examples include Adapter & Decorator,

Structural



• Concerned with how objects communicate and interact
• Focus on defining clear responsibilities and workflows between objects
• Examples include Observer and Command

Behavioural



QA

224

Python's dynamic nature, combined with its treatment of functions as first-class citizens, means that not all design patterns are strictly relevant or widely used in the language. The way patterns are implemented can also vary between programming languages, and in some cases multiple valid approaches exist for the same pattern.

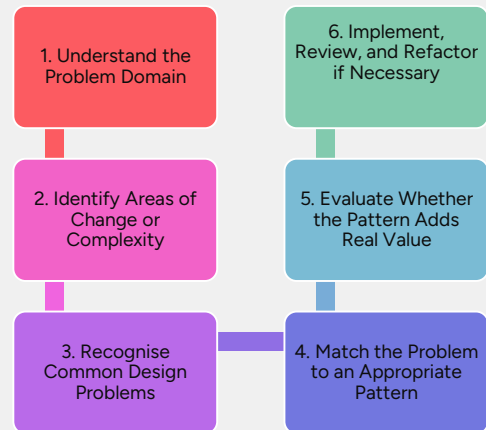
That said, the underlying concept — recognising recurring problems and applying well-established patterns to solve them — remains a valuable part of any Python developer's toolkit. In this chapter, we focus on those patterns most relevant to broadening and deepening your Python expertise.

How to use design patterns

- Using design patterns effectively comes down to understanding both which pattern to apply and, more importantly, when to apply it.
- This can be achieved by utilising the following steps shown to the right

Keep in mind:

- Design patterns are not suitable for every problem, particularly simple, one-off solutions where they may introduce unnecessary complexity and over-engineering.
- The real value of design patterns lies in familiarity and a team's shared understanding of them and the ability to communicate their use clearly with one another.



QA

225

Understand the Problem Domain: Clearly identify the problem you are trying to solve, including the system requirements, constraints, and expected behaviours. Focus on the relationships between objects and the responsibilities each part of the system should have.

Identify Areas of Change or Complexity: Look for parts of the design that are likely to change, grow, or become difficult to maintain. Design patterns are most valuable where flexibility, scalability, or repeated structure is needed.

Recognise Common Design Problems: Compare the problem against common categories addressed by design patterns, such as:

- Object creation problems
- Complex object interactions
- Reducing tight coupling between components

Match the Problem to an Appropriate Pattern: Select a pattern whose intent closely aligns with the problem. For example:

- Use **Factory** patterns for flexible object creation
- Use **Observer** for event-driven communication
- Use **Decorator** to add behaviour dynamically

Evaluate Whether the Pattern Adds Real Value: Ask whether the pattern improves maintainability, readability, flexibility, or team communication.

Avoid applying patterns to simple or short-lived solutions where they may introduce unnecessary complexity.

Implement, Review, and Refactor if Necessary: Apply the pattern carefully and review the resulting design. Ensure the solution remains understandable and that the pattern genuinely simplifies future development rather than complicating it.

Creational patterns

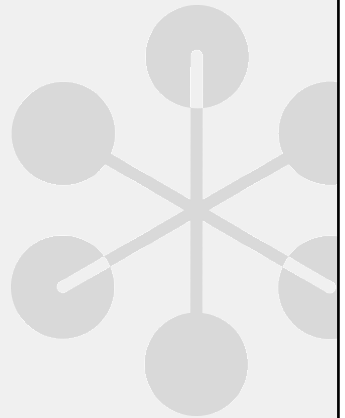
Common Creational Design Patterns

1. Factory Function (or Method)

- Responsible for creating objects from concrete classes
- Returns instances via a common abstract type or interface
- Often implemented using Abstract Base Classes (ABCs)
- Already common in Python (e.g. namedtuple acts as a class factory)

2. Singleton

- Ensures that only one instance of a class can exist
- The class controls and restricts its own instantiation
- Useful for managing shared state across an application (e.g. database connections)
- Naturally occurs in Python in some cases (e.g. modules, logging instances)
- Can be implemented in either a simple (“sloppy”) or thread-safe manner depending on requirements



QA

226

Creational patterns tend to be less useful in Python—primarily because of its dynamic nature.

In many cases, Python’s built-in object creation mechanisms are already flexible and expressive enough, so adding extra abstraction can feel unnecessary (though that doesn’t stop developers from doing it anyway!).

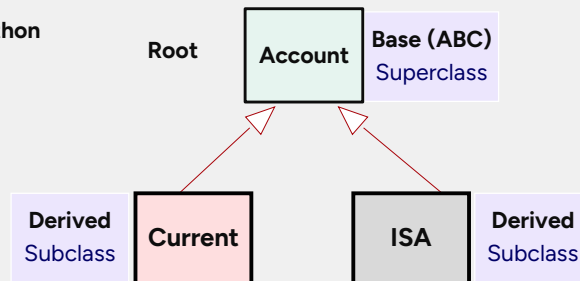
In traditional object-oriented languages, creational patterns often exist to abstract and manage use of the new operator.

However, Python doesn’t have a new operator in the same sense—object creation is already implicit and highly customizable.

That said, creational patterns are still useful in Python. In fact, some concepts—such as Singletons—naturally emerge in the language (for example, modules behave like singletons), which demonstrates that there is still a practical need for these patterns.

Creational patterns: Factory function – class hierarchy

1. Factory Method implementation in Python



We start by identifying a simple class hierarchy.

In this example, **Current** and **ISA** are both forms of **Account**, but they can (must) implement generic methods in their own way, e.g.:

Withdraw() will be a very different method for accounts that have overdraft limits and those that need advanced notice.

Creational patterns: Factory function – classes 1

```
Account ABC
from abc import ABC, abstractmethod

class Account(ABC):
    def __init__(self, account_number, balance=0.0):
        self.account_number = account_number
        self.balance = balance

    @abstractmethod
    def deposit(self, amount):
        pass

    @abstractmethod
    def withdraw(self, amount):
        pass

    def __str__(self):
        return f"Account {self.account_number},
Balance: {self.balance:.2f}"
```

QA

228

As previously seen, each derived class must implement (override) the methods which have been flagged as abstract in the ABC.

Creational patterns: Factory function – classes 2

CurrentAccount Class

```
from account import Account

class CurrentAccount(Account):
    def __init__(self, account_number, balance=0.0, overdraft_limit=500.0):
        super().__init__(account_number, balance)
        self.overdraft_limit = overdraft_limit

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited {amount:.2f}")
        else:
            print("Invalid deposit amount")

    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdrawal amount")
            return

        if self.balance - amount >= -self.overdraft_limit:
            self.balance -= amount
            print(f"Withdrew {amount:.2f}")
        else:
            print("Overdraft limit exceeded")
```

QA

229

As previously seen, each derived class must implement (override) the methods which have been flagged as abstract in the ABC.

Creational patterns: Factory function – classes 3

```
from account import Account
```

ISAAccount Class

```
class ISAAccount(Account):
    def __init__(self, account_number, balance=0.0, max_balance=20000.0):
        super().__init__(account_number, balance)
        self.max_balance = max_balance

    def deposit(self, amount):
        if amount <= 0:
            print("Invalid deposit amount")
            return
        if self.balance + amount <= self.max_balance:
            self.balance += amount
            print(f"Deposited {amount:.2f}")
        else:
            print("Deposit would exceed ISA limit")

    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdrawal amount")
            return
        if amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew {amount:.2f}")
        else:
            print("Insufficient funds")
```

QA

230

As previously seen, each derived class must implement (override) the methods which have been flagged as abstract in the ABC.

Creational patterns: Factory function - 1

Factory Class
<pre>from account import Account from currentaccount import CurrentAccount from isaaccount import ISAAccount import sys from importlib import import_module class Factory: @staticmethod def create(account_type, *args, **kwargs): if account_type == "CurrentAccount": return CurrentAccount(*args, **kwargs) elif account_type == "ISAAccount": return ISAAccount(*args, **kwargs) else: raise ValueError(f"Unknown account type: {account_type}")</pre>

QA

231

In this example, the "create" function is acting as our Factory Function.

It receives the type of account required (same as the class name, for convenience) and the account ID, balance and overdraft limit/maximum balance, then attempts to:

- Import its module (to access the derived class)

If all is good, the required object is instantiated (passing the required initialization arguments, e.g. the account ID) and a reference to it is returned from the function.

Creational patterns: Factory function - 2

Factory Class – Generic create method

```
from account import Account
import sys
from importlib import import_module

class Factory: # Continued from previous slide
    CLASS_MAP = {"currentaccount": "CurrentAccount", "isaaccount": "ISAAccount" }

    @staticmethod
    def create_generic(account_type, *args, **kwargs):
        try:
            module_name = account_type.lower()
            if module_name not in sys.modules:
                account_module = import_module(account_type)
            else:
                account_module = sys.modules[module_name]
            class_name = Factory.CLASS_MAP[module_name]
            account_class = getattr(account_module, class_name)
        except (AttributeError, ModuleNotFoundError):
            raise ImportError(f'{account_type} cannot be found')
        if not issubclass(account_class, Account):
            raise ImportError(f'{account_type} is not a type of account')
        return account_class(*args, **kwargs)
```

QA

232

In this example, the "create" function is acting as our Factory Function.

It receives the type of account required (same as the class name, for convenience) and the account ID, balance and overdraft limit/maximum balance, then attempts to:

- Import its module (to access the derived class)

A number of checks are performed here:

- We haven't imported the module before
- That the module can be found (a ModuleNotFoundError would be caught)
- That the class is actually derived from Account

If all is good, the required object is instantiated (passing the required initialization arguments, e.g. the account ID) and a reference to it is returned from the function.

Creational patterns: Factory function – Testing

```
from factory import Factory

if __name__ == "__main__":

    current = Factory.create("CurrentAccount", "CA123", 100, overdraft_limit=300)
    isa = Factory.create("ISAAccount", "ISA456", 500, max_balance=20000)

    current.withdraw(350)    # allowed within overdraft
    current.withdraw(100)   # exceeds overdraft

    isa.deposit(19000)      # may exceed limit
    isa.withdraw(100)
```

```
Withdrew 350.00
Overdraft limit exceeded
Deposited 19000.00
Withdrew 100.00
```

QA

233

In this simple testrunner, objects are created via our Factory Function.

Now there's no need to declare a reference with a normal Class and constructor argument – it's hidden/handled by the Factory Function; we just "create" things – that's how simple the interface has now become.

Once created we can test their methods.

Any attempt to instantiate a SavingsAccount will fail, of course – there's no module (or Class) written for this. However, in terms of future development, if we wanted to, it's just a question of adding that class and – voilà – the code will *just work* (-which of course is always the goal!) Do try it. Should be really painless.

Note.

If you attempt to create the "SavingsAccount" class but *intentionally don't inherit from Account* the running code will still fail due to the subclass check. We need to do this because we'd be manufacturing an object for which we don't know the interface, i.e. any superclass methods

which have not been overridden.

Creational patterns: Singleton (1)

"Sloppy" Singleton implementation in Python

```
_global_ledger = None # module-level variable. Not good!

class SloppyBankLedger:
    def __init__(self):
        self.transactions = []

    def add_transaction(self, account_id, amount):
        self.transactions.append({
            "account_id": account_id,
            "amount": amount
        })

    def get_transactions(self):
        return self.transactions

def get_ledger():
    global _global_ledger

    if _global_ledger is None:
        print("Creating sloppy global ledger...")
        _global_ledger = SloppyBankLedger()

    return _global_ledger
```

```
from sloppysingleton import SloppyBankLedger, get_ledger

ledger1 = get_ledger()
ledger2 = get_ledger()

ledger1.add_transaction("ACC123", 100)
ledger2.add_transaction("ACC999", -50)
print(ledger1.get_transactions())

# Issue: You can still bypass it
another = SloppyBankLedger() # new separate instance!
another.add_transaction("ACC999", 99.99)
```

QA

234

The classic pattern uses:

A close-as-possible-for-Python private constructor

- `__init__`

A static method which returns a reference to the object – it should be noted this is a generic approach and isn't particularly pythonic...

- `getInstance()`

Running this code will generate the following...

obj 43499776 database: ConnectionDB

obj 43499776 database: ConnectionDB

Traceback (most recent call last):

```
File "singletonclassictechnique.py", line 29, in <module>
    myobj3 = Singleton("PersonDB")
```

```
File "singletonclassictechnique.py", line 16, in __init__
    raise Exception("Can't instantiate again!")
```

Exception: Can't instantiate again!

However, this approach suffers from the following problems because it:

- Uses a **global variable**
- Doesn't prevent direct instantiation properly
- Is **not thread-safe**

- Can easily be broken

As you can see, subsequent attempts to create another object can succeed, and as noted this version isn't multi-threadsafe.

Many other implementations are possible, including the use of metaclasses.

Some developers will argue that having multiple objects isn't the issue – it's that they should share the same state; that's an interesting debate.

Creational patterns: Singleton (2)

A safer Singleton implementation in Python

```
class BankLedger:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            print("Creating the Bank Ledger instance...")
            cls._instance = super().__new__(cls)
            cls._instance.transactions = []
        return cls._instance

    def add_transaction(self, account_id, amount):
        self.transactions.append({
            "account_id": account_id,
            "amount": amount
        })

    def get_transactions(self):
        return self.transactions
```

```
from safersingleton import BankLedger

ledger1 = BankLedger()
ledger2 = BankLedger()

ledger1.add_transaction("ACC123", 500)
ledger2.add_transaction("ACC456", -200)

print(ledger1.get_transactions())
```

Creating the Bank Ledger instance...
[{'account_id': 'ACC123', 'amount': 500},
{ 'account_id': 'ACC456', 'amount': -200}]

QA

235

A more idiomatic approach this time, using the `__new__` magic method.

Notice:

- Only **one instance is created**
- Both `ledger1` and `ledger2` reference the same object
- Changes through one are visible in the other

Note: Banking systems are concurrent so you may need to lock access:
`import threading`

```
class SafeBankLedger:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        with cls._lock:
            if cls._instance is None:
                cls._instance = super().__new__(cls)
                cls._instance.transactions = []
            return cls._instance
```

Structural patterns

Common Structural Design Patterns include:

1.Delegation / Object Proxying

- Provides a stand-in or substitute for another object
- The proxy serves as an intermediary, managing access to the original object
- Enables additional processing or validation before or after requests reach the target object

2.Adapter

- Allows objects with incompatible interfaces to work together
- Commonly used when handling different data types or data formats
- Acts as a translation or bridging layer between components
- Can be implemented using either object-based or class-based approaches
- Particularly useful when existing library or third-party code cannot be modified

3.Decorator

- A pattern we have already explored extensively

Structural patterns – delegation ("Object proxying")

Objects that invoke methods on other objects

- Implemented in Python using the `__getattr__` method
- Forwards requests for missing attributes to another object
- Uses the built-in `getattr` function
- Retrieves an attribute by name from an object, with an optional default value

Commonly used to create wrapper or proxy classes

```
class proxy:
    def __init__(self, obj):
        self.wrapped = obj

    def __getattr__(self, aname):
        return getattr(self.wrapped, aname)
```

Returns the attribute
(method) of the
wrapped object

QA

237

Delegation — sometimes referred to as object proxying — is a well-established object-oriented design pattern. While it is occasionally positioned as an alternative to inheritance, the two are not mutually exclusive and frequently coexist within the same design.

At its core, delegation allows an object to dynamically invoke methods on other objects, with the decision of which method to call being made at runtime.

Unlike C#, Python has no dedicated `delegate` keyword. A notable distinction in Python, compared to statically typed languages, is that a method is treated as a named attribute — meaning `__getattr__` is invoked on every method call for every instance of that class.

Structural patterns – Proxy 1

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited £{amount}. New balance = £{self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds")
        else:
            self.balance -= amount
            print(f"Withdrew £{amount}. New balance = £{self.balance}")

    def get_balance(self):
        return self.balance
```

QA

238

In this example, the proxy is acting as a substitute for the real object. Because the proxy has the same interface as the normal object, it can be used interchangeably, particularly when you want to add some extra behaviour to an existing class but can't rewrite that code: use the proxy instead.

This will allow the proxy to filter access (e.g. in this case, check if a withdrawal needs to be approved) or do pre-processing before deciding to pass execution to same method in the *real* "service" object, i.e.

BankAccount.

Consider the following:

```
def handle_request(account: Account, amount: int) -> None:
    account.withdraw(amount)
```

Note, in the `handle_request()` wrapper function, launching a request becomes a generic action; it shouldn't care if dealing with real employees or a proxy – the call is identical, i.e. calling the `withdraw()` method on the received object reference. For example:

```
#Try a real BankAccount
bankaccount= BankAccount ()
handle_request(bankaccount, 3000)
#Try a proxy
```

```
proxy = Proxy(bankaccount)
handle_request(proxy, 4000)
```

The resulting output from this test could would be:

Insufficient funds

Proxy: Large withdrawal flagged! Approval required.

Structural patterns – Proxy 2

```
class BankAccountProxy:
    def __init__(self, account):
        self._account = account

    def withdraw(self, amount):
        # Add custom behaviour
        if amount > 1000:
            print("Proxy: Large withdrawal flagged! Approval required.")
        else:
            self._account.withdraw(amount)

    def __getattr__(self, name):
        """
        Delegate any missing attribute/method
        to the real account
        """
        print(f"Proxy forwarding '{name}'")
        return getattr(self._account, name)
```

QA

239

In this example, the proxy is acting as a substitute for the real object. Because the proxy has the same interface as the normal object, it can be used interchangeably, particularly when you want to add some extra behaviour to an existing class but can't rewrite that code: use the proxy instead.

This will allow the proxy to filter access (e.g. in this case, check if a withdrawal needs to be approved) or do pre-processing before deciding to pass execution to same method in the *real* "service" object, i.e. BankAccount.

Consider the following:

```
def handle_request(account: Account, amount: int) -> None:
    account.withdraw(amount)
```

Note, in the `handle_request()` wrapper function, launching a request becomes a generic action; it shouldn't care if dealing with real employees or a proxy – the call is identical, i.e. calling the `withdraw()` method on the received object reference. For example:

```
#Try a real BankAccount
bankaccount= BankAccount ()
handle_request(bankaccount, 3000)
#Try a proxy
```

```
proxy = Proxy(bankaccount)
handle_request(proxy, 4000)
```

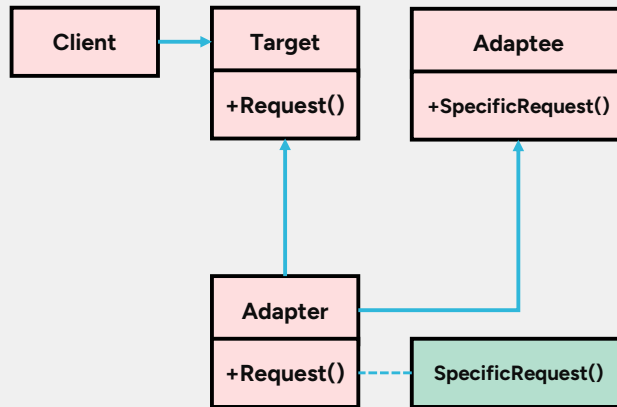
The resulting output from this test could would be:

Insufficient funds

Proxy: Large withdrawal flagged! Approval required.

Structural patterns – adapter (1)

Adapter pattern – a visual guide



Structural patterns – adapter (2)

```
# Target - what we normally use
class BankAccount:
    def __init__(self, owner, balance_gbp):
        self.owner = owner
        self.balance_gbp = balance_gbp

    def get_balance_gbp(self):
        return self.balance_gbp

# Adaptee - useful but interface is not compatible, can't be modified
from abc import abstractmethod
from existing_incompatible_bankaccount_class import BankAccount

class CurrencyAdapter:
    @staticmethod
    @abstractmethod
    def convert(amount: float) -> float:
        pass

class PoundsToDollars(CurrencyAdapter):
    @staticmethod
    def convert(amount: float) -> float:
        return amount * 1.34 # conversion rate 28/05/2026
```

Structural patterns – adapter (3)

```
# Build an adapter
class BankAccountAdapter:
def __init__(self, account: BankAccount, adaptee: CurrencyAdapter):
    self.account = account
    self.adaptee = adaptee

def get_converted_balance(self):
    gbp = self.account.get_balance_gbp()
    converted_amount = self.adaptee.convert(gbp)
    return converted_amount

# Client code - what we want to use whatever the data source
from existing_incompatible_bankaccount_class import BankAccount
from bankaccountadapter import BankAccountAdapter
from bankaccountadapter import PoundsToDollars

account = BankAccount("Peter", 1000.00)

# Wrap it with adapter
adapted_account = BankAccountAdapter(account, PoundsToDollars)

print(f"Balance in USD: ${adapted_account.get_converted_balance()}")
```

Behavioural patterns

Common Behavioural Patterns include:

1. Observer

- A subscription-based mechanism
- Allows objects to be notified whenever events occur in the publisher object they are monitoring
- Widely used in Graphical User Interface (GUI) frameworks

2. Command

- Based on the **Separation of Concerns (SoC)** principle, which organises applications into distinct layers
- Decouples the user interface from the underlying business logic by introducing encapsulated command objects that connect the two
- Enables the same commands to be triggered from multiple sources

3. Iterator

- A pattern we have already explored extensively

4. Template Method

- Another familiar pattern, often implemented using Abstract Base Classes (ABC) and abstract method overriding

Behavioural patterns – observer (1)

Subscribers

```
from abc import abstractmethod
class Observer:
    @abstractmethod
    def update(self, account):
        pass

class SMSAlert(Observer):
    def update(self, account):
        print(f"[SMS] Alert: Balance is now £{account.balance}")

class EmailAlert(Observer):
    def update(self, account):
        print(f"[Email] Alert: Balance updated to £{account.balance}")
```

subscribers
(observer)
classes

QA

244

There are many different ways to implement an observer pattern.

This version, whilst not the most basic solution possible, tries to keep the underlying connectivity clear and concise for readability's sake. The result is limited but hints at what's possible with further thought and refinement.

In this example we create two different subscriber (observer) types: `EmailAlert` and `SmsAlert`, both of which inherit from a class called `Observer` that defines an abstract method called `update()`.

The `BankAccount` class acts as our publisher or observee. Existing once, a publisher object must keep track of its subscribers (using a dictionary of subscriber objects and a reference to their handler method; a "callback") and minimally support an interface consisting of:

- `attach()` – a common method for adding an observer (and its preferred event "handler" method – although this may be defaulted).
- `notify()` – a method for taking a piece of data and calling each observee's handler, passing it on
- `detach()` – a method for removing an observer

Behavioural patterns – observer (2)

Observees

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance
        self._observers = []

    def attach(self, observer):
        self._observers.append(observer)

    def detach(self, observer):
        self._observers.remove(observer)

    def notify(self):
        for observer in self._observers:
            observer.update(self)

    def deposit(self, amount):
        self.balance += amount
        print(f'Deposited £{amount}. New balance = £{self.balance}')
        self.notify()
```

**Publisher
(observee)
class**

QA

245

In this example we create two different subscriber (observer) types: EmailAlert and SmsAlert, both of which inherit from a class called Observer that defines an abstract method called update().

The BankAccount class acts as our publisher or observee. Existing once, a publisher object must keep track of its subscribers (using a dictionary of subscriber objects and a reference to their handler method; a "callback") and minimally support an interface consisting of:

- attach() – a common method for adding an observer (and its preferred event "handler" method – although this may be defaulted).
- notify() – a method for taking a piece of data and calling each observee's handler, passing it on
- detach() – a method for removing an observer

Behavioural patterns – observer (3)

Client

```
from bankaccount import BankAccount
from concreteobservers import SMSAlert
from concreteobservers import EmailAlert

account = BankAccount("Peter", 1000)

sms_alert = SMSAlert()
email_alert = EmailAlert()
# Attach observers
account.attach(sms_alert)
account.attach(email_alert)
# Perform transactions
account.deposit(200)
account.withdraw(150)
# Detach observers
account.detach(sms_alert)
account.detach(email_alert)
# Perform transactions
account.deposit(400)
account.withdraw(250)
```

```
Deposited £200. New balance = £1200
[SMS] Alert: Balance is now £1200
[Email] Alert: Balance updated to £1200
Withdrew £150. New balance = £1050
[SMS] Alert: Balance is now £1050
[Email] Alert: Balance updated to £1050
Deposited £400. New balance = £1450
Withdrew £250. New balance = £1200
```

QA

246

Eventually the observer is deregistered (detached) and from that point onwards is unknown to the observee (publisher) and thus receives no more dispatched (generated) events.

It's a neat approach and one which can be expanded upon greatly with possibilities including the use of ABCs and adapters to create even more intelligent observers which can handle any type of event the publisher can dispatch.

Behavioural patterns – Command (1)

Receiver

```
import os
import shutil

class FileSystem:
    def create_folder(self, path):
        os.makedirs(path, exist_ok=True)
        print(f"Created folder: {path}")

    def create_file(self, filename):
        full_path = os.path.join(os.path.curdir, filename)
        # Create the file (and write content if provided)
        with open(full_path, "w") as f:
            f.write("# TRANSACTIONS")
        print(f"Created file: {full_path}")

    def move_file(self, src, dest):
        shutil.move(src, dest)
        print(f"Moved file from {src} to {dest}")
    ...
```

QA

247

In a bank, at the end of each day transaction files are generated (e.g. payments, transfers). These files are:

- Moved into processing folders
- Renamed with timestamps
- Archived after completion
- Organized into audit folders

These operations must be:

- **Logged**
- **Repeatable**
- **Undoable (sometimes)**
- **Executed in sequence**

This is a perfect fit for the **Command pattern** because each file system action can become a **command object**. In our example these are CreateFolderCommand, CreateFileCommand, MoveFileCommand, RenameFileCommand

The system can:

- Queue them

- Execute in order
- Retry on failure
- Audit everything

Behavioural patterns – Command (2)

Commands

```
from abc import abstractmethod
class Command:
    @abstractmethod
    def execute(self):
        pass

# Concrete Commands
class CreateFolderCommand(Command):
    def __init__(self, fs, path):
        self.fs = fs
        self.path = path
    def execute(self):
        self.fs.create_folder(self.path)

class MoveFileCommand(Command):
    def __init__(self, fs, src, dest):
        self.fs = fs
        self.src = src
        self.dest = dest
    def execute(self):
        self.fs.move_file(self.src, self.dest)
```

QA

248

The code demonstrates a couple of commands: CreateFolder and MoveFile though many other commands could be added e.g. CreateFile, RenameFile, etc.

Behavioural patterns – Command (3)

Invoker

```
class BatchProcessor:
    def __init__(self):
        self.commands = []

    def add_command(self, cmd):
        self.commands.append(cmd)

    def run(self):
        for cmd in self.commands:
            cmd.execute()
```

QA

249

The **Invoker** is responsible for:

Storing commands and executing them later — without knowing what they actually do.

It doesn't care whether it's creating folders, moving files or renaming files it just calls: `cmd.execute()` on all of the added commands.

The Invoker depends only on `cmd.execute()`. It doesn't need to know what type of command it is, what file operation it performs or how it works internally and the pattern thus promotes **loose coupling**.

Behavioural patterns – Command (4)

Client

```
# Daily Transaction Processing System
fs = FileSystem()
processor = BatchProcessor()

# 1. Create a daily processing folder
processor.add_command(CreateFolderCommand(fs,
"processing/2026-05-28"))

# 2. Move file into processing folder
processor.add_command(MoveFileCommand(
    fs,
    "transactions_2026-05-28.txt",
    "processing/2026-05-28/transactions_2026-05-28.txt"
))

# Execute all steps
processor.run()
```

QA

250

It can be helpful to think of the Invoker as a batch processing engine and is especially relevant to a banking scenario where, at the end of the day, the bank's systems might:

- Create processing folders
- Create transaction files
- Rename files with timestamps
- Move files for processing

Each step becomes a command and the Invoker is a job runner that executes them. Note, the invoker queues the commands but nothing happens until `run()` is called.

Extensions to the example given could (and should) include logging of the commands, the retrieval of failed ones and the ability to undo them.

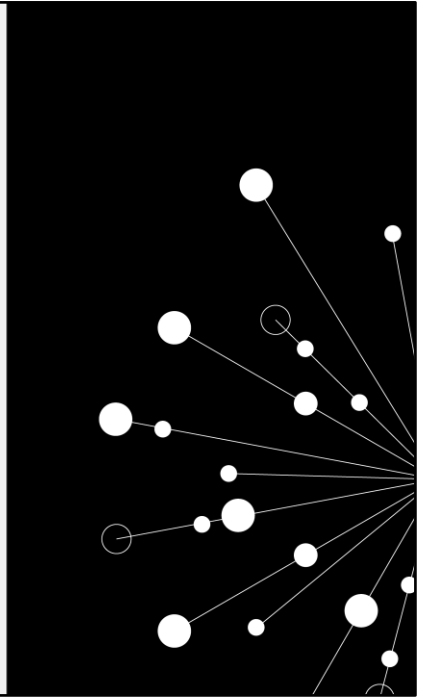
Exercise EG07: Deeper OO

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA



Suggested time: 40-60 mins.

Activity includes differentiated tasks.

Summary

Design Patterns in Python — A Balanced View

- Design patterns are a valuable tool, though they are not always the most natural fit for Python, given its dynamic nature, the absence of a new keyword, and its built-in use of factories
- That said, they can still offer elegant and innovative solutions to common problems

The Importance of Problem Analysis

- Breaking down and thoroughly analysing a problem is an essential developer skill — it helps surface recognisable patterns and informs better design decisions
- Design patterns do not solve every problem; applied inappropriately, they can introduce unnecessary complexity where a simpler approach would suffice

Patterns as a Communication Tool

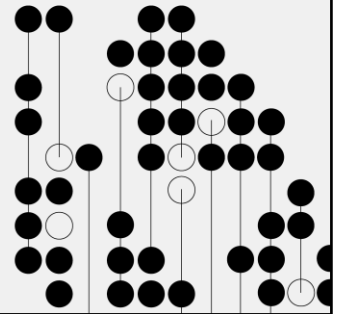
- One of their most practical benefits is as a shared vocabulary within development teams — a single well-chosen term can communicate a complex idea instantly
- For example, telling a colleague *"an Adapter would work well there"* conveys a great deal without further explanation

A Word of Caution — Maslow's Hammer

- *"I suppose it is tempting, if the only tool you have is a hammer, to treat everything as if it were a nail."*
- In other words, familiarity with design patterns should inform your thinking — not constrain it

Any questions?

QA



Appendices



Recommended reading

The seminal GoF (Gang of Four) book

Design Patterns: Elements of Reusable Object-Oriented Software (1994)

Addison-Wesley

ISBN10: 0-201-63361-2

Written by:

Erich Gamma,

Richard Helm,

Ralph Johnson and

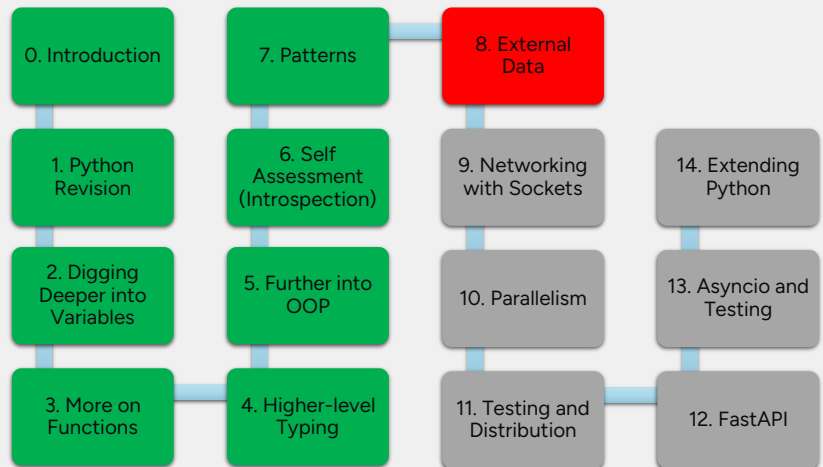
John Vlissides

Intermediate Python

External Data



Module roadmap



QA

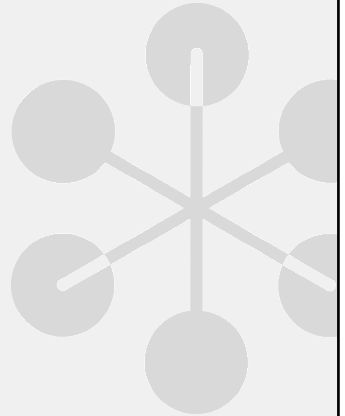
External data

Learning outcomes:

- Handle structured data formats such CSV, JSON XML and YAML in Python
- Read and write binary files
- Serialize data objects using Pickles
- Compress and decompress data files

Prerequisites:

Delegates are required to have knowledge and skills relating to basic Python programming, including data structures, functions, loops and flow control.



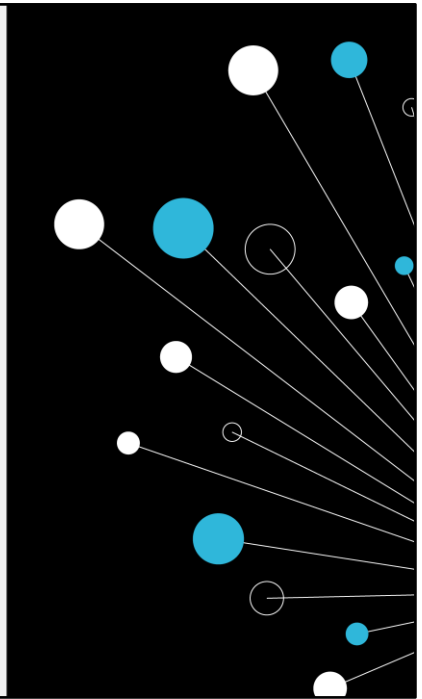
Appendices include details on simple database access, XML and reading binary files created in other programming languages (e.g. C)

Learning Objectives

By the end of this session, you should be able to:

- Work with structured text file formats, including CSV and the distinctions between JSON and XML
- Use Python's standard json module to read and write JSON data, and implement custom handlers where needed
- Apply Python's standard XML modules to parse and manipulate XML documents
- Evaluate the relative strengths and use cases of JSON, XML, and other serialisation formats
- Read from and write to binary files with confidence
- Use Python's pickle module to serialise and deserialise objects, with an understanding of the different pickle protocols available
- Apply compression techniques when working with file data

QA



Structured text file formats

There are four (three in reality because YAML is a superset of JSON) types of text file you are likely to encounter:

• Comma separated values • Oldest and most portable data format still in common use • Often the CSV format is not strictly standardised, e.g. files using <i>tab separation</i> may still carry the .csv extension! .csv 	• JavaScript Object Notation • Popular, lightweight and compact • Used In most RESTful APIs and many NoSQL databases, e.g. MongoDB .json 	• YAML Ain't Markup Language • A superset of JSON • Relies on indentation rather than brackets or braces • Widely used in tools like Docker, Kubernetes and Ansible .yaml 	• eXensible Mark up Language • Familiar to anyone who knows HTML • Popular format, used a lot by SOAP-based web APIs • Can be susceptible to security vulnerabilities .xml 
--	--	---	--

Python provides strong support for all four formats.

The format you ultimately work with will largely be dictated by the broader information pipeline you are operating within — both what you receive from other systems and what you are expected to produce. For that reason, it is well worth familiarising yourself with all four.

QA

260

We will take a look at practical examples of each data format.

Note: there are multiple libraries available to handle each data format, but we only have time to look into the most popular ones.

Structuring and formatting data

Comma Separated Values (CSV)

```
title,price,author
"Fly Fishing",12.99,"J R Hartley"
"War and Peace",19.99,"Leo Tolstoy"
```

JavaScript Object Notation (JSON)

```
[{"title": "Fly Fishing", "price": 12.99, "author": "J R Hartley"},
 {"title": "War and Peace", "price": 19.99, "author": "Leo Tolstoy"}]
```

YAML Ain't Markup Language (YAML)

```
- {author: J R Hartley, price: 12.99, title: Fly Fishing}
- {author: Leo Tolstoy, price: 19.99, title: War and Peace}
```

Extensible Markup Language (XML)

```
<books>
  <book title="Fly Fishing" price="12.99" author="J R Hartley"/>
  <book title="War and Peace" price="19.99" author="Leo Tolstoy"/>
</books>
```

QA

261

When extracting data from a system for transmission to another person or application, it is important to consider how that data will be formatted. Ideally, the format should be both machine-readable and human-readable, allowing software to process it efficiently while also making it easier for people to understand the meaning of each item. Data formatting is essentially the process of adding context, or *metadata*, to raw data in order to provide additional meaning and structure. Several common approaches are outlined below.

•CSV (Comma-Separated Values)

CSV stores data in comma-separated columns, with the first row typically containing headers that describe each column. Each record is separated using carriage return and line feed characters. The primary formatting characters used are commas (,) and quotation marks (").

•JSON (JavaScript Object Notation)

This module focuses mainly on JSON and YAML, while XML is covered in the appendices. JSON represents arrays using square brackets ([]) and objects using curly braces ({ }). Objects contain key-value pairs in the form "key": "value". JSON is also a subset of YAML version 1.2.

•YAML (YAML Ain't Markup Language)

YAML uses a structured, indentation-based format with key: headers and hyphens (-) to identify list elements. Unlike JSON, quotation marks are generally optional. YAML relies on indentation to define structure — similar to Python syntax. Two notable features of YAML are its support for relational tree structures and built-in

data typing. Because YAML is a superset of JSON, many JSON files can also be parsed as YAML.

- XML (eXtensible Markup Language)**

XML represents data using elements and attributes. This format will be explored in greater detail in the following slides.

Reading CSV files

Opening a CSV file for reading in Python is a straightforward process:

- Files contain an optional header (fieldname) row
- Each row is a separate record, values are usually separated by commas and non-numeric data is usually double-quoted
- DictReader creates a dictionary out of the values in each row and uses the header row labels as each entry's keys.

```
import csv

with open('accounts.csv','r') as csv_file:
    csv_reader = csv.DictReader(csv_file)
    for row in csv_reader:
        for field in row:
            print(f"{field}: {row[field]}")
```

```
account_name,id,balance
"John Smith","ACC100245",5230.75
"Emma Johnson","ACC100246",12890.40
```

```
account_name: John Smith
id: ACC100245
balance: 5230.75
account_name: Emma Johnson
id: ACC100246
balance: 12890.40
```

QA

262

In this example, a simple Python program is used to read, parse, and display data from a CSV file. In the past, developers often had to write relatively complex algorithms to handle this kind of processing. However, in modern languages such as Python, specialised libraries have made this task straightforward and largely trivial.

Understanding how to work with CSV files remains valuable due to their widespread use. Regardless of device, operating system, or application, CSV is often the default format for importing and exporting data.

Writing to CSV files

Creating a CSV file in Python is also straightforward:

- A new file is created for writing – potentially overwriting an existing one.
- CSV writer method can have delimiter, quote character and quoting strategies set.
- Header rows should ideally be written (this aids future processing)
- Data rows can be written from any source, in this case a simple list.

```
import csv

new_accounts = (('Kamran Liaquat', 'ACC100247', 104.67),
                 ('Sadia Saleem', 'ACC100248', 233.28))
# Open the file for writing
with open('accounts2.csv', 'w', newline='') as csvfile:
    # Set up the writer and the csv settings
    mywriter = csv.writer(csvfile, delimiter=',',
                          quotechar='"', quoting=csv.QUOTE_NONNUMERIC)

    # Write the csv headings
    mywriter.writerow(['account_name', 'id', 'balance'])

    for account in new_accounts:
        mywriter.writerow(account)

"account_name","id","balance"
"Kamran Liaquat","ACC100247",104.67
"Sadia Saleem","ACC100248",233.28
```

QA

263

In this example, a short Python program is being used to create a CSV data file and write account details to it. The `writerow()` method takes an iterable object which it converts and adds as a single "csv" row to the file. You could equally have written the following:

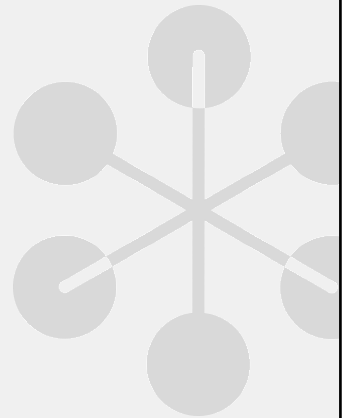
```
mywriter.writerow([account[0], account[1], account[2]])
```

But given the for loop is iterating around a tuple of tuples each account object is already iterable.

JavaScript Object Notation – JSON

JSON is a lightweight and widely adopted data interchange format

- Compared to XML, JSON is notably simpler in structure, making it easier for both humans to read and write, and for machines to parse and generate
- Not tied to any particular programming language, making it a versatile choice across different technology stacks
- Worth noting that JSON is a subset of JavaScript's object literal notation, and also a subset of YAML



While data transfer can often be a cumbersome process, relying on formats like XML introduces additional friction, requiring complex DOM parsers to read or write data that does not easily map into native programming objects. In contrast, JavaScript Object Notation (JSON) provides a lightweight data-interchange format that is uniquely simple for humans to read and write, while remaining highly efficient for machines to parse and generate. Because JSON exists strictly as a language-independent text format, it seamlessly adapts to the standard programming conventions that developers across almost all ecosystems already use.

JSON - Universal Data Structures

JSON is built on data structures that are supported by virtually all modern programming languages:

- A collection of name/value pairs
 - Typically represented as an object, associative array, hash, or dictionary depending on the language
- An ordered list of values
 - Typically represented as an array or list

JSON Objects

- A JSON object is an unordered set of name/value pairs
- Enclosed within curly braces { }
- Each name is followed by a colon : to separate it from its value
- Individual name/value pairs are separated by commas

```
{
  "account_holder": "Alice Johnson",
  "account_number": "12345678",
  "sort_code": "20-00-00",
  "account_type": "current",
  "balance": 2540.75,
  "overdraft_limit": 500.00,
  "address": {
    "street": "123 High Street",
    "city": "London",
    "postcode": "EC1A 1BB"
  },
  "transactions": [
    {
      "date": "2026-05-01",
      "description": "Salary",
      "amount": 3200.00,
      "type": "credit"
    },
    {
      "date": "2026-05-07",
      "description": "Grocery Shopping",
      "amount": -56.30,
      "type": "debit"
    }
  ]
}
```

Because JSON relies on structural patterns supported by virtually all modern programming languages, it is immediately intuitive and accessible to developers. At its core, the format mirrors a standard dictionary comprised of name-value pairs, which can also embed ordered lists of values known universally as arrays. This overall structure organizes an unordered collection of these name-value pairings inside familiar curly braces, separating each key from its value with a colon, and separating the pairs themselves with commas—a layout that maps perfectly to everyday programming collections.

JSON vs. XML – A Comparison

File Size

- XML files tend to be larger than their JSON equivalents, largely due to the overhead introduced by closing tags such as `</ending>`
- In practice, however, compression largely eliminates any meaningful difference in file size

Fundamentally Different by Design

- XML was originally designed as a markup language and lacks native data types, delegating that responsibility to schemas
- JSON, by contrast, was designed specifically as a serialisation format, with a structure that maps naturally to data types found across many programming languages

A Fair Comparison?

- Given these fundamental differences in purpose and design, it is worth asking whether directly comparing the two is entirely reasonable in the first place

The substantial visual footprint and increased file size of XML are frequently cited as disadvantages when compared to the leaner structure of JSON, although the highly repetitive markup patterns within XML mean it can actually be compressed and zipped with remarkable efficiency. Ultimately, executing a direct, adversarial comparison between the two formats is rarely meaningful, as they were originally engineered to solve entirely different architectural problems. While their functional capabilities undeniably overlap in modern data exchange—much like many other web technologies—they each serve distinct primary purposes rather than acting as simple substitutes for one another.

Reading JSON Files

Reading JSON files in Python is almost as straightforward as reading CSV files:

```
import json

with open('accounts.json') as accounts_file:
    accounts = json.load(accounts_file)
    for account in accounts['accounts']['account']:
        print('account_holder:', account['account_holder'])
        print('account_number:', account['account_number'])
        print('balance:', account['balance'])
```

```
{
  "accounts": {
    "account": [
      {
        "account_holder": "Alice Johnson",
        "account_number": "ACC45678",
        "balance": 2540.75
      },
      {
        "account_holder": "Sadia Saleem",
        "account_number": "ACC45679",
        "balance": 342.76
      }
    ]
  }
}
```

```
account_holder: Alice Johnson
account_number: ACC45678
balance: 2540.75
account_holder: Sadia Saleem
account_number: ACC45679
balance: 342.76
```

QA

267

You only need to write a few lines of Python code to efficiently read, parse, and display an external JSON data file. Much like its comprehensive treatment of CSV and XML, Python provides exceptionally rich, native support for JSON that aligns perfectly with its object-oriented data structures. Mastering this data serialization format has become an essential skill for modern developers, primarily because the vast majority of web services and modern application programming interfaces (APIs) now rely on JSON as their standard format for transmitting data responses.

Writing to JSON Files

Python standard library json module

- Easy to use interface - similar to pickle

```
import json

accounts = {"ACC45678": {
    "account_holder": "Alice Johnson",
    "balance": 2540.75},
    "ACC45679": {
    "account_holder": "Sadia Saleem",
    "balance": 342.76}
}

fo = open('accounts2.json', 'w')
json.dump(accounts, fo)
fo.close()
```

```
{"ACC45678": {
  "account_holder": "Alice Johnson",
  "balance": 2540.75},
 "ACC45679": {
  "account_holder": "Sadia Saleem",
  "balance": 342.76}}
```

accounts2.json

QA

268

Unlike pickles, successive calls to `json.dump()` do not write more objects to the file, it just corrupts it.

As a comparison, to write a Python dictionary as an XML file you would need something like the following:

```
from xml.etree.ElementTree import Element, tostring
```

```
# using same accounts variable as on the slide
```

```
def dict_to_xml(tag, d):
    elem = Element(tag)
    for key, val in d.items():
        child = Element(key)
        child.text = str(val)
        elem.append(child)
    return elem
```

```
e = dict_to_xml('accounts', accounts)
fo = open('accounts2.xml', 'wb')
fo.write(tostring(e))
fo.close()
```

The above code would produce the following:

```
<accounts>
```

```
<ACC45678>{'account_holder': 'Alice Johnson', 'balance':  
2540.75}</ACC45678>  
  <ACC45679>{'account_holder': 'Sadie Saleem', 'balance':  
342.76}</ACC45679>  
</accounts>"
```

JSON Custom Handlers (1)

- JSON natively supports only: dict, list, str, bool, and None
- To serialise custom class objects, use the `default` parameter with a conversion function

Custom Encoding

- The conversion function takes the object being encoded as its argument
- If defined as an object method, the object will be passed twice

Custom Decoding

- Use the `object_hook` parameter to deserialise custom objects
- The function is called automatically whenever a dictionary is decoded

```
import json
from datetime import datetime
from decimal import Decimal

def banking_json_handler(obj):
    if isinstance(obj, Decimal):
        return float(obj) # or str(obj) for exact precision
    if isinstance(obj, datetime):
        return obj.isoformat()
    raise TypeError(f"Type {type(obj)} not serializable")

transaction = {"account_id": "ACC123", "amount": Decimal("1025.75"),
               "currency": "GBP", "timestamp": datetime.now()}
# triggers TypeError: Object of type Decimal is not JSON serializable
json_data = json.dumps(transaction)
json_data = json.dumps(transaction, default=banking_json_handler)
print(json_data)
```

```
{"account_id": "ACC123", "amount": 1025.75, "currency": "GBP",
"timestamp": "2026-05-29T14:29:32.595074"}
```

QA

269

In JSON, dictionary keys must be string, int, float, bool. Non-standard key types can be ignored using:

```
json.dumps(data, skipkeys=True)
```

now those keys will not be dumped.

If we try to dump a non-standard object we get something like:

```
object at ... is not JSON serializable
```

But we can write conversion functions for `dump()` and `load()`. For dumping, you will probably want to use a member function to get at all the attributes. This means that the member function must actually take two arguments, *self*, and the object being dumped (which will be the same).

Since the `load()` hook only supports dictionary types, it is probably better to convert to that type. To implement a load hook use the `object_hook` argument, for example:

```
my_new_postcode = json.load(in_file,
                             object_hook=dict_to_postcode)
```

Python also supports `object_pairs_hook` for `collections.OrderedDict`

types.

The json module also supports encoder and decoder classes which work with the same basic functions as shown here.

JSON Custom Handlers (2)

An alternate solution, rather than place the conversion method inside the class, is to extend the json module's JSON Encoder class...

```
import json
from datetime import datetime
from decimal import Decimal

class BankingJSONEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, Decimal):
            return float(obj) # or str(obj) for full precision
        if isinstance(obj, datetime):
            return obj.isoformat()
        return super().default(obj)

transaction = {"account_id": "ACC123", "amount": Decimal("1025.75"),
               "currency": "GBP", "timestamp": datetime.now()}

json_data = json.dumps(transaction, cls=BankingJSONEncoder)
print(json_data)
```

Because the encoder has already generated the JSON string, it can simply be written to file using a standard write method.

QA

270

JSON Custom Handlers (3)

Returning a dictionary helps in preserving full information:

```
import json
from datetime import datetime
from decimal import Decimal

class BankingJSONEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, Decimal):
            return {"__type__": "Decimal", "value": str(obj)}
        if isinstance(obj, datetime):
            return {"__type__": "datetime", "value": obj.isoformat()}
        raise TypeError(f"Type {type(obj)} not serializable")

transaction = {"account_id": "ACC123", "amount": Decimal("1025.75"),
               "currency": "GBP", "timestamp": datetime.now()}

json_data = json.dumps(transaction, cls=BankingJSONEncoder)
print(json_data)
```

```
{"account_id": "ACC123", "amount": {"__type__": "Decimal", "value": "1025.75"}, "currency": "GBP", "timestamp": {"__type__": "datetime", "value": "2026-05-29T15:06:57.363436"}}
```

QA

271

Whilst not done in the previous examples returning a dictionary is sometimes important for reliable deserialisation. (especially in a banking scenario when converting a decimal to a float may lead to a loss of precision and rounding issues) because it preserves full information. Using a dictionary you preserve the exact value and the type information:

```
{
    "__type__": "Decimal",
    "value": "1025.75"
}
```

You can safely deserialize later:

```
def banking_decoder(dct):
    if "__type__" in dct:
        if dct["__type__"] == "Decimal":
            return Decimal(dct["value"])
        if dct["__type__"] == "datetime":
            return datetime.fromisoformat(dct["value"])
    return dct
```

There are negatives to doing this, namely more verbose JSON, extra complexity for consumers and it's not always necessary.

XML overview

'Well-formed' XML has the following structure:

```
<?xml version="1.0" encoding="UTF-8"?>
<accounts>
  <account account_id="ACC45678">
    <account_holder>Lotte Lenya</account_holder >
    <balance>&#163;19.99</balance>
  </account>
</accounts>
```

- The header declares the version and character encoding
- An optional validation entry may be included internal or external DTD
- All tags (elements) must be correctly nested
- Special characters — such as the £ sign — are represented using entity references

Well-formed XML requires correct syntax formatting and nesting, though a distinct validation phase is often necessary to ensure the document contains the specific elements and entities required for its intended purpose. While comprehensive XML validation falls outside the scope of this course, detailed resources regarding Document Type Definition (DTD) and Schema validations can be found by consulting the online documentation for the 1xml module. Additionally, depending on the specific character encoding utilized by a file, special characters may materialize as entity references, meaning standard symbols like the pound sign will often be converted into encoded strings.

Elements, attributes & entity references

Elements

- Each element opens with a `<tag>` and closes with a `</tag>`
- Elements may contain text, nested elements, or a combination of both
- Empty elements can be expressed using self-closing tags: `<tag/>`

Attributes

- Attributes are defined within the opening tag: `<book attrib="quoted_value">`
- An element may carry multiple attributes, each of which must be unique

Entity References

- Entity references are used to represent special characters or strings embedded within element text
- They may be specified as a character code, a named reference, or one of several predefined references such as `&`, `<`, and `>`

Specified	<code>&#163;</code>	£
Named	<code>&copy;</code>	©
Pre-defined (by XML)	<code>&amp;</code> <code>&lt;</code> <code>&gt;</code> <code>&apos;</code> <code>&quot;</code>	<code>&</code> <code><</code> <code>></code> ' "

QA

273

Element Tags

- Case-sensitive — the opening and closing tags must match exactly
- Must not begin with a digit, nor contain whitespace or the `@` character
- May contain text, nested elements, or a combination of both — known as mixed content
- Empty elements are typically written as `<tag/>`, though `<tag></tag>` is equally valid
- Must be correctly nested. Any inner element must be closed before its outer element, i.e tags are always closed in the reverse order to which they were opened

Attributes

- Defined as name/value pairs within the opening or empty tag of an element
- Multiple attributes may be defined for a single element, separated by whitespace within the tag

Entity Character References

- Special characters can be referenced using decimal format (e.g. `£`), hexadecimal format (e.g. `£`), or a predefined name (e.g. `©`)

Named Entity References

- Custom named entities can be defined in a validation file such as a DTD

— for example, `<!ENTITY fred "Fred Smith">` would cause `&fred;` to be replaced with *Fred Smith* wherever it appears in the document

Encoding

- For most use cases not involving characters required by XML syntax or non-European languages, UTF-8 encoding will handle the majority of special characters without the need for explicit entity references

Elemental relationships

- XML elements have hierarchical relationships with one another.
- XML processing APIs commonly use terminology that reflects these relationships.

In the example:

- All elements are descendants of the root element.
- The root element is the ancestor of every other element in the structure.
- A parent element is the immediate ancestor of its child elements, and it may also relate to sibling elements at the same level.
- A child element is typically a leaf node in this context, while a sibling element is another node at the same hierarchical level; both may contain no further nested elements, with the sibling element containing no additional content.

```
<root>
  <parent>
    <child>data</child>
    <sibling/>
  </parent>
  <sibling/>
</root>
```

QA

274

The document root serves as the outermost data element of the structure, strictly excluding technical headers such as the XML declaration, DTD definitions, and other markup-specific components. As application programming interfaces navigate this structure, they frequently interact with sibling and child elements, where siblings occupy the exact same structural level as the currently processed element, and child elements sit directly beneath it. This relational context continuously shifts as code traverses the hierarchical tree, eventually terminating at a leaf node, which represents a completely empty element containing no further nested data.

Python XML modules

Python's Standard XML Modules

- `expat` — a fast, low-level XML parser
- `SAX` — Simple API for XML
- `DOM` — Document Object Model
- `etree` — Element Tree

Security

- Python's documentation includes a useful summary of known XML vulnerabilities — worth reviewing before working with untrusted XML data
- Recent versions of Python include a natively patched `expat` core that blocks structural attacks by default

Third-Party Modules

- `lxml` (PyPI) — offers interfaces similar to the standard modules but is more flexible, and includes DTD and schema validation APIs

Further Reading

- Refer to the chapter appendix for guidance on working with the DOM

The standard `xml` module supplied natively as part of the core Python installation offers a partial implementation of both the SAX and DOM APIs. Reflecting this limited scope, the built-in DOM module is purposefully named `minidom` to signal to developers that it does not implement the full, robust specification.

For projects requiring a complete and highly advanced framework, developers can turn to the optional `lxml` third-party module, which is available via the Python Package Index (PyPI). This super-implementation significantly extends basic parsing capabilities by adding comprehensive support for XML validation against DTDs and Schemas. Because it is an external package, `lxml` must be explicitly built and installed into your platform-specific Python library before use. The `lxml` module is not covered on this course however, comprehensive setup instructions and guides can be reviewed directly at the official documentation site, www.lxml.de.

While the security vulnerabilities of XML—such as entity expansion bombs and external entity data leaks—are well-documented, modern Python environments neutralize these threats out of the box. Recent versions of Python include a natively patched `expat` core that blocks structural attacks by default, rendering older security wrappers like `defusedxml`,

`defusedexpat`, and `safedxml` largely redundant and obsolete. Today, secure parsing is achieved simply by keeping your Python environment up to date and, when using advanced third-party libraries like `lxml`, explicitly configuring the parser to disable entity resolution and block external network lookups.

Using ElementTree (etree)

- etree provides a straightforward way to create and manipulate XML in Python, working at three levels:
- The document as a whole, via `etree.ElementTree()` objects
- Individual elements, via `etree.Element()` objects
- Nested child elements, via `etree.SubElement()` objects

```
import xml.etree.ElementTree as ET
tree = ET.parse('accounts2.xml')
root = tree.getroot()

for account in root.findall('account'):
    for item in account:
        print(item.tag, ":", item.text)
```

```
account_holder :
Alice Johnson
balance : 2540.75
account_holder :
Sadia Saleem
balance : 342.76
```

Widely regarded as more Pythonic than DOM, making use of familiar Python constructs such as iterators, lists, and dictionaries

QA

276

The `xml.etree.ElementTree` approach offers a highly Pythonic implementation for processing XML documents. Rather than forcing developers to use rigid, language-agnostic specifications, it leverages native Python structures—such as iterators, lists, and dictionaries—to represent data hierarchies as an easily navigable tree of objects. The overarching `ElementTree` object encapsulates the entire XML document and provides optimized `iter()` and `find()` variations to traverse its contents. Navigating this structure occurs systematically from top to bottom, resolving the first child completely before moving on to sibling nodes in a standard depth-first search pattern.

At the heart of this hierarchy sits the document root, which manifests as an individual `Element` object. These individual nodes share similar traversal methods with the broader tree wrapper, but they also expose direct list-like characteristics to interact with nested tags. Furthermore, each `Element` provides immediate access to core data attributes: `tag` handles the node name, `text` and `tail` manage the inner text strings, and `attrib` surfaces the node's properties as a standard Python dictionary.

Modifying or expanding these data structures is equally straightforward. Developers can generate and attach new child nodes dynamically using the

`SubElement()` factory function provided natively by the module. For users seeking a deeper dive into these mechanics, the official Python documentation features a comprehensive, introductory tutorial available directly within the standard library references at docs.python.org/3/library.

Are you going to include a **visual code example** alongside this text to show exactly how `attrib` behaves like a dictionary and how `SubElement()` appends a node?

Pruning and grafting the tree

A document can be altered

- By adding or removing elements
- By altering the text of existing elements

```
import xml.etree.ElementTree as ET
tree = ET.parse('accounts3.xml')
root = tree.getroot()

account1 = root[0]
limit = ET.SubElement(account1, 'overdraft_limit', {})
limit.text = '£500.00'

for item in account1.iter():
    print(item.text)

account2 = root[1]
account2[1].text = '£12.07'
for item in account2.iter():
    print(item.text)
```

```
Alice Johnson
£2540.75
£500.00

Sadia Saleem
£12.07
```

QA

277

Because elements behave like lists, they can not only be iterated over but also accessed by index. The document's root element corresponds to the `<account>` node, meaning `root[0]` refers to the first `<account>` element and `root[1]` refers to the second. By extension, `account2[1]` gives us the balance `SubElement` of `account2`.

In this example, a new `SubElement` has been created as a child of the `account1` node, and the text of the balance `SubElement` belonging to `account2` has been updated.

Note that the empty dictionary `{ }` is used to represent the attributes of the new node — in this case, indicating that no attributes have been defined.

Creating XML output

Use the `ElementTree.write()` method to write to a file

- Or an open file object
- Default encoding is 'US-ASCII'
- Specify whether XML header is to be emitted

```
tree.write('newaccounts.xml',  
          encoding='UTF-8',  
          xml_declaration=True)
```

- Indentation may not be as expected if new elements have been added

```
<?xml version='1.0' encoding='UTF-8'?>  
<accounts>  
  <account>  
    <account_holder>Alice Johnson</account_holder>  
    <balance>£2540.75</balance>  
    <overdraft_limit>£500.00</overdraft_limit></account>  
  <account>  
    <account_holder>Sadia Saleem</account_holder>  
    <balance>£12.07</balance>  
  </account>  
</accounts>
```

QA

278

The `.write()` method of `ElementTree` objects is used to output the entire tree to a file, with control over encoding and the option to include or omit the XML header. The original whitespace layout from the parsed document is preserved in the output — however, any newly added nodes will not inherit this formatting, which can result in irregular indentation. This appears to be a quirk of the `etree` module, and if consistent formatting is required, it can be addressed manually by appending the appropriate whitespace to the `.tail` property of the affected node or nodes.

lxml and validating parsers

lxml supplies a validating parser

```
from lxml import etree
dtdparser = etree.XMLParser(dtd_validation=True)
tree = etree.parse('accounts.dtd.xml', dtdparser)
root = tree.getroot()
```

Can validate against an internal or external DTD

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<!DOCTYPE books [
  <!ELEMENT books (book)+>
  <!ELEMENT book (title, author, price)>
  <!-- ATTLIST book reference CDATA #REQUIRED -->
  <!ELEMENT title (#PCDATA)>
  <!ELEMENT author (#PCDATA)>
  <!ELEMENT price (#PCDATA)>
]>
```

QA

279

The lxml library, available from the Python Package Index with documentation at lxml.de, provides support for XML validation through a validating parser. To validate an XML document against an internal or external Document Type Definition (DTD) referenced within the document itself, you create a validating parser and pass it as the second argument to `etree.parse()`.

If validation fails, an `XMLSyntaxError` exception is raised.

Validation objects

Use dtd object to validate against unreferenced DTD

```
from lxml import etree
xml_file = "accounts4.xml"
dtd_file = "accounts4.dtd"
tree = etree.parse(xml_file)
root = tree.getroot()
dtd = etree.DTD(dtd_file)
if dtd.validate(root):
    print (f"XML in {xml_file} is valid")
else:
    # Display most recent DTD error from log
    print(dtd.error_log.last_error)
```

accounts.dtd

```
<!ELEMENT accounts (account)+>
<!ELEMENT account (account_holder, balance, overdraft_limit)>
<!ATTLIST account account_id CDATA #REQUIRED>
<!ELEMENT account_holder (#PCDATA)>
<!ELEMENT balance (#PCDATA)>
<!ELEMENT overdraft_limit (#PCDATA)>
```

QA

280

If the XML document undergoing validation doesn't include a reference to a DTD — whether internal or external — instantiate a DTD object by loading the necessary DTD file through `etree.DTD()` and invoking the `validate()` method on the XML document.

Any DTD-related errors will be written to a log file, which can be examined using available methods or the `.last_error` attribute.

The ElementTree package also supports additional validation approaches. These include RelaxNG, XMLSchema, and Schematron, all of which can be applied in much the same way as the DTD object method demonstrated in the slide. For further details, consult the documentation available at lxml.de/validation.

Binary files – struct.pack/unpack

Binary file formats do not have a direct mapping to Python variable types

- The exception being files that were originally written using Python, such as those created with pickle
- More commonly, these files are produced using languages like C or C++, or similar low-level languages

The pack and unpack functions are used to convert data to and from primitive types when working with such files

```
import struct

fOut = open("bindata.dat", "wb")
data = struct.pack("if20s", 100, 3.1412, b"To be or not to be...")
print(type(data))
fOut.write(data)
fOut.close()

fIn = open("bindata.dat", "rb")
data = fIn.read(32);
fIn.close()

clean = struct.unpack("if20s", data)
print(clean)
txt = clean[2].decode().rstrip('\x00') # str
print(txt)
```

```
(100, 3.141200065612793, b'To be or not to be..')
To be or not to be..
```

QA

281

Reading binary data requires knowing its format. Data from other languages like C is not directly compatible with Python types. This compatibility issue affects all high-level dynamic languages.

Consider a simple C struct written to a file. It contains two integers, one double-precision number, and an 80-character string. Python's built-in `struct.unpack()` function can convert this binary data. The function uses a format string, like `"if20s"`, to describe the data layout. Because C strings use single-byte characters, you must call `.decode()` to convert them to Python strings.

Watch out for compiler padding bytes. C compilers often insert extra bytes to align data on memory boundaries. This alignment changes between 32-bit and 64-bit systems. Developers can also change it manually using compiler directives. Finally, check for endianness issues, which are fully explained in Python's official struct documentation.

Creating binary files

In Python, bytes is an immutable object type

- It holds a sequence of values ranging from 0 to 255, representing 8-bit data
- byte objects support slicing via indexing, but their contents cannot be altered

A binary file serves as the means of transferring these bytes from RAM to a more permanent storage medium

Define binary format:

i = int (account_id)

50s = fixed-length string (name, 50 bytes)

f = float (balance)

f = float (overdraft_limit)

```
import struct

# Create account instances
accounts = [
    Account(45678, "Kamran Liaqat", 2540.75, 500.00),
    Account(45679, "Sadia Saleem", 342.76, 600.00)
]

FORMAT = "i50sff"

# Write to binary file
with open("accounts.dat", "wb") as f:
    for acc in accounts:
        # Encode name to bytes & pad/truncate to 50 bytes
        name_bytes = acc.name.encode("utf-8")
        name_bytes = name_bytes[:50].ljust(50, b"\x00")
        packed_data = struct.pack(
            FORMAT,
            acc.account_id,
            name_bytes,
            acc.balance,
            acc.overdraft_limit
        )
        f.write(packed_data)
```

QA

282

To open a file in binary write mode, the standard write access mode "w" must be combined with "b" to form "wb".

The write() function can be used to output bytes to the target file object. In this example, a few different approaches are demonstrated, one of which involves using the "b" string modifier to convert a Unicode string — Python 3's default string format — into bytes. Another, uses the struct module (which is good for fixed-format records) and its pack function to convert Python values into a sequence of bytes (binary data) according to a specified format.

Unlike text files, binary files treat all bytes equally; special characters such as "\0A" (\n or newline) carry no particular significance and are simply stored as raw byte values.

To inspect the contents of the resulting file on disk, a hexadecimal editor can be used. Windows users may find **HxD** a practical option, though it requires a separate download. On Linux, the hexdump utility is readily available and is commonly used with the canonical hex+ASCII display flag, as shown below:

```
hexdump -C data.bin
```

Reading binary files – worked example

```
import struct

FORMAT = "i50sff"

accounts_loaded = []

# Open the binary file for reading
with open("accounts.dat", "rb") as f:
    record_size = struct.calcsize(FORMAT) # records are fixed in size

    while True:
        chunk = f.read(record_size)
        if not chunk:
            break

        # Unpack the binary data converting bytes to Python values
        account_id, name_bytes, balance, overdraft = struct.unpack(FORMAT, chunk)

        # Decode and clean up the name
        name = name_bytes.decode("utf-8").rstrip("\x00")

        # Create Account object and add to list
        account = Account(account_id, name, balance, overdraft)
        accounts_loaded.append(account)

# Display loaded accounts
for acc in accounts_loaded:
    print(acc)
```

3

The Account class used in the code on the slide (and previous page) is as follows:

Define the Account class (same as before)

```
class Account:
```

```
    def __init__(self, account_id, name, balance,
overdraft_limit):
```

```
        self.account_id = account_id
```

```
        self.name = name
```

```
        self.balance = balance
```

```
        self.overdraft_limit = overdraft_limit
```

```
    def __str__(self):
```

```
        return f"{self.account_id} | {self.name} |
£{self.balance} | £{self.overdraft_limit}"
```

QA

Pickling

Pickling is the process of serializing Python objects into a byte stream

- The resulting byte stream is typically saved to a file or transmitted over a network
- It can be used to achieve program persistence, for example by preserving the processing state between separate executions
- Python also makes internal use of pickling, such as when sharing data between processes

```
import pickle

# Create a dictionary of accounts
accounts_dict = {
    account.account_id: {
        "name": account.name,
        "balance": account.balance,
        "overdraft_limit": account.overdraft_limit
    }
    for account in accounts
}

# Create the pickle!
pickle_filename = "accounts.pkl"
pickle_file = open(pickle_filename, "wb")
pickle.dump(accounts_dict, pickle_file)
pickle_file.close()
```

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	B0	04	95	7F	00	00	00	00	00	00	00	7D	94	28	4D	6E	["....."] (Mn
00000010	B2	7D	94	28	8C	04	6E	61	6D	65	94	8C	0D	4B	61	6D	");" (E.name"E.Kam
00000020	72	61	6E	20	4C	69	61	71	61	74	94	8C	07	62	61	6C	ran Liaqat"E.bal
00000030	61	6E	63	65	94	47	40	A3	D9	80	00	00	00	00	8C	0F	ance"G@f0E....E.
00000040	6F	76	65	72	64	72	61	66	74	5F	6C	69	6D	69	74	94	overdraft_limit"
00000050	47	40	7F	40	00	00	00	00	00	75	4D	6F	B2	7D	94	28	G@.0.....uMo");"
00000060	68	02	8C	0C	53	61	64	69	61	20	53	61	6C	65	65	6D	h.E.Sadia Saleem
00000070	94	68	04	47	40	75	6C	28	F5	C2	8F	5C	68	05	47	40	"h.G@ul(0A.h.G@
00000080	82	C0	00	00	00	00	00	75	75	2E							,A.....uu.

The resulting binary Pickle file contains data and the original Python structure (a dictionary).

284

QA

Python Pickling Overview

- **Definition:** The pickle module converts Python object hierarchies into byte streams (serialization/flattening).
- **File Requirement:** Always open storage files in **binary mode** ('wb' or 'rb'), not text mode.
- **What Can Be Pickled?**
 - **Primitives:** Booleans, integers, floats, complex numbers, strings, bytes, and bytearray.
 - **Collections:** Lists, tuples, sets, and dictionaries containing only picklable objects.
 - **Top-Level Entities:** Classes and function names defined at a module's top level.
 - **Note:** Actual executable code/logic cannot be pickled; use modules or the marshal module instead.

Key Limitations & Security

- **Security:** Data is **unencrypted** and **insecure**. Never unpickle untrusted data.
- **Compatibility:** Different protocol versions exist, creating potential compatibility issues.
- **Exclusions:** Cannot pickle defaultdicts, lambda functions, or generators. Use the dill module for these.

Pickle vs. JSON

- **JSON:** Better for portability, language-independent, and standardised.
- **Pickle:** Better for easily reconstructing complex, native Python object structures.

Performance & Compression

- Speed:** The C-implemented version (cPickle) runs roughly 1,000 times faster.
- Compression:** Custom byte streams can be compressed using modules like bz2 to save disk space.

```
import bz2, pickle
small_pickle_file = bz2.BZ2File('smallpickle.bzip', 'w')
pickle.dump(accounts_dict, small_pickle_file )
```

Unpickling

The process of unpickling a binary Pickle file is remarkably straightforward

- Simply specify the filename and invoke the load method to retrieve the stored data
- One limitation, however, is that the original name of the object cannot be automatically recovered during the process

See chapter appendix for more slides on pickle

```
import pickle

#Loading the pickle!
pickle_filename = "accounts.pkl"
pickle_file = open(pickle_filename, "rb")
accounts_dict = pickle.load(pickle_file)
pickle_file.close()

#checking the Pickle and its contents...
print(accounts_dict)
print(type(accounts_dict))
```

```
{45678: {'name': 'Kamran Liaqat', 'balance': 2540.75, 'overdraft_limit': 500.0}, 45679: {'name': 'Sadia Saleem', 'balance': 342.76, 'overdraft_limit': 600.0}}
<class 'dict'>
```

The previous pickled dictionary is reconstructed back to its original state.

Compression

Python supports a number of different compression and archiving formats including:

- **bz2** (bzip2)
- **gzip** (GNU zip files)
- **lzma** (algorithm)
- **tarfile** – TapeARchive (tar) access
- **zipfile** – ZIP archives
- **zlib** – GNU zlib compression



```
import gzip
import os

# Set the compressed archive filename
arc_filename = 'mytext.txt.gz'
arc_file = gzip.open(arc_filename, 'wb')
try:
    # Zip this content...
    arc_file.write(b'Hello, world!')
finally:
    arc_file.close()
    print(f"{arc_filename} contains ", end="")
    print(f"{os.stat(arc_filename).st_size} compressed bytes")
```

The **shutil** module also provides a number of high-level operations that include compression.

QA

286

This example demonstrates the **gzip** module being used to create a GNU zip file from a sequence of bytes.

The **zlib** module includes checksum functions (**adler32** and **crc32**) are intended to check data integrity and are not cryptographically secure. **bz2** supports a file-like class called **bz2.BZ2File**, which is used in a similar way to the **open()** built-in.

The **tarfile** module supports the usual expected mechanisms, in addition **getmembers()** returns all the metadata for files in an archive, not just the file name. The example shown was run on Windows, using a tar file with **-z** compression which came from Linux.

A higher level (simpler) generic interface is provided by the **shutil** module. This is a general module which includes archiving operations.

Compressing a CSV

Python's `shutil` module provides a number of high-level operations that include compression.

This example demonstrates `gzip` and `shutil` modules compressing an existing CSV file.

```
# One way of zipping your csv...
import gzip
import shutil

# Tested in Microsoft Windows
with open('accounts2.csv', 'rb') as infile:
    with gzip.open('mynewaccounts2.gz', 'wb') as
outfile:
    shutil.copyfileobj(infile, outfile)
```

All files must be read and written as Binary files – we are processing these byte-by-byte.



Compressing a pickle

The standard library includes `gzip`

- Open the file using the `gzip` method
- Same arguments as regular `open`
- Then call the usual methods on the file handle
- Often used with pickles, e.g.

Creating a compressed pickle:

```
import pickle, gzip
# creating a compressed pickle
outp = gzip.open('accounts.pgz', 'wb')
pickle.dump(accounts_dict, outp)
outp.close()
```

Unzipping and loading a compressed pickle:

```
import pickle, gzip
# unzipping and loading a compressed pickle
inp = gzip.open('accounts.pgz', 'rb')
new_accounts_dict = pickle.load(inp)
inp.close()

print(new_accounts_dict)
```

QA

288

For small amounts of data the file might not be any smaller (it might even be slightly larger). Notice that the zip files are open as binary, that means we get byte streams from zip files.

By the way, if you see a built-in function called `zip`, it has nothing to do with this. The `zip` built-in is for advanced combining of collections.

Tarring

- Tarring is the process of combining multiple files and directories into a single file
- The resulting file is called a **tarball** or a **tar file** (usually ending in .tar).
- **Note:** standard tarring only packages files together; it does **not** compress them

```
import tarfile
import os

# Folders for tar process (create it, copy sample files into the first one)
source_dir = "./tar_example_source_data"
destination_dir = "./copy_of_tar_example_source_data"

# Tar filename
newarchive = "newarchive.tar"

# Opens for gzip compressed writing; will auto close
with tarfile.open(newarchive, "w:gz") as tar:
    # Add all files for this folder
    tar.add(source_dir, arcname=os.path.basename(source_dir))

# Open existing tar file for reading; will auto close
with tarfile.open(newarchive, "r") as tar:
    # Iterate through the filenames (including directories)
    for filename in tar.getnames():
        print(filename)

    # Linux "ls"-style Listing
    tar.list()

    # Extract everything to a new folder (optional param - files to extract)
    tar.extractall(destination_dir)
```

QA

289

This example demonstrates how to create a new tar archive from a list of existing files. It also covers two different methods for listing archive contents and shows how to extract files to a destination directory.

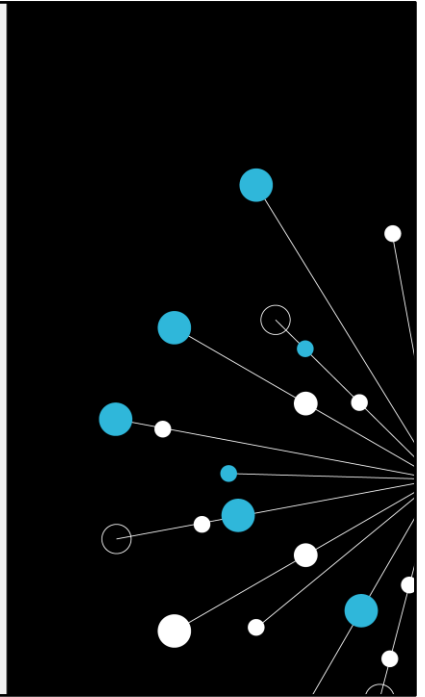
Exercise EG08: External Data

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA

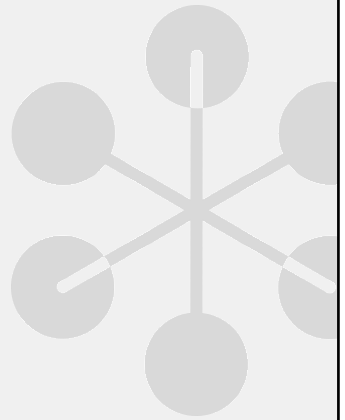


Suggested time: 40-60 mins.

Activity includes differentiated tasks.

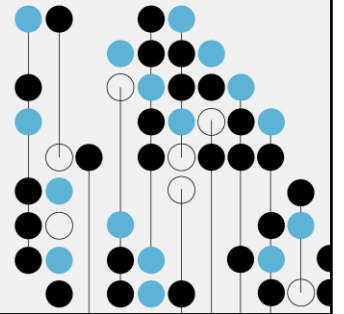
Summary

- CSV remains universally used and features excellent, long-standing native support in Python.
- JSON serves as a global standard serialization format, built directly into Python as a core module.
- Alternative formats like XML and YAML are also widely available for data serialization.
- Python offers multiple ways to handle XML, with ElementTree being the most popular choice.
- Binary files require byte-level reading and writing, and Python can parse binary data generated by C.
- Pickling preserves Python objects and their states, though it excludes types like defaultdicts and lambdas.
- Use the Dill module as a powerful alternative when pickling unsupported object types.
- Built-in tools allow files to be easily compressed using various algorithms and techniques



Any questions?

QA



Appendices



Parsing XML with expat

expat is a fast XML parser written in C

- Ideal for fast loading and checking for well-formedness

```
from xml.parsers import expat
import sys

Tags = 0
def count_tags(name, attr): global Tags; Tags += 1

ExParser = expat.ParserCreate()
ExParser.StartElementHandler = count_tags

try:
    ExParser.ParseFile(open('books.xml', 'rb'))
except expat.ExpatError:
    print("Error!", file=sys.stderr)
    exit(1)
else:
    print("XML is well-formed and has", Tags, "tags")
```

QA

294

expat is written in C for speed and efficiency. We need to create the parser object and then give it a handler for an event such as Start Element. In the example we have simply counted the number of tags, i.e. the number of starting elements. We could have displayed them, or even counted how many times each one occurred by creating a Python dictionary entry for each new tag and incrementing the count for subsequent instance of that tag.

Other **expat** events and data are catchable, including **EndElementHandler()**, **CharacterDataHandler()**, **CurrentByteIndex**, **CurrentColumnNumber**, **CurrentLineNumber**

Note that, in Python 3, the file must be opened in binary mode as the parser expects a bytes interface, rather than Unicode.

We are catching **ExpatError** exceptions. These have a number of attributes for diagnosing where in the XML the problem lies, such as line and column position and the XML error code. To get this to work correctly we should define the **ExpatError** exception object as an empty class, derived from **Exception**.

i.e.

```
class ExpatError(Exception):  
    pass
```

Using XML and the SAX

- **SAX is a stream-oriented API, similar to expat**
 - May call expat parser and then adds some functionality
- **SAX parses an XML file and detects events such as:**
 - `startElement` – called with element name and attributes
 - `endElement` – called with element name
 - `characters` – called with textual data between the tags
 - `startDocument` – called once at start of XML document
 - `endDocument` – called once at end of XML document
- **Only the currently-parsed element is in memory**
 - Not the entire XML document
 - Ideal for processing large or potentially large files.

QA

295

Simple API for XML works in a similar manner to expat, but the callbacks have different names and there may be more features. Depending on the implementation, SAX may call expat to parse the XML file.

These two APIs are useful for rapid parsing and reading of large XML documents as they do not load the entire structure into memory, they simply treat it as a stream of events. As each event is encountered the appropriate handler is called.

SAX events such as `startElement`, `endElement`, `characters` etc may have handlers assigned, as appropriate.

Unlike DOM, SAX does not load an entire XML document into memory. This makes it suitable for scanning and reading data from a large file, but does not lend itself to easy modification and re-writing of the data back to a file. Store any required information as normal Python objects, or build your own object class, as appropriate.

Defining a SAX handler object

Define a handler derived from ContentHandler

- This will be called by the XML parser object

```
import xml.sax.handler

class BookHandler(xml.sax.handler.ContentHandler):
    def __init__(self):
        self.text = ''

    def startElement(self, name, attributes):
        self.tag = name
        if attributes.items():
            print(attributes.items())

    def characters(self, data):
        if not data.isspace(): self.text += data

    def endElement(self, name):
        if self.text:
            print(self.tag, ': ', self.text)
            self.text = self.tag = ''
```

Then create a parser...

QA

296

The capture SAX events, define a new object derived from SAX's ContentHandler class. Inside this object, define methods to handle the required events.

The handler function will be called with the tag name and, possibly, attributes or textual data. These may be stored, or flags set to indicate a tag has been seen (important if processing sub-elements).

Character data includes formatting whitespace, such as tabs ('\t') and newlines ('\n') from the original XML layout. This may need to be detected and stripped-out during processing, or left in to reconstruct the original layout for later re-writing.

In the example we concatenate the character data as we receive it, then print it all out once the end of the element has been detected.

As is usual in Python, line-endings are converted to '\n' regardless of the platform (Windows/Linux/OS X/et al).

Element attributes (if present) are returned as an object with .keys(), .values() and .items() methods, allowing easy access to the names and values.

Create a SAX parser object

Create a parser and pass it your handler object

Call the parser to read the XML file

- Can parse a Python file object instead of a filename

```
<?xml version="1.0" encoding="UTF-8" ?>
<accounts>
  <account account_id="ACC45678">
    <account_holder>Alice Johnson</account_holder>
    <balance>£2540.75</balance>
    <overdraft_limit>£500.00</overdraft_limit>
  </account>
</accounts>
```

```
parser = sax.make_parser()
handler = AccountHandler() # class the implements xml.sax.handler
parser.setContentHandler(handler)
parser.parse("accounts4.xml")
```

```
{'account_id': 'ACC45678', 'account_holder': 'Alice Johnson',
'balance': 2540.75, 'overdraft_limit': 500.0}
{'account_id': 'ACC45679', 'account_holder': 'Sadia Saleem',
'balance': 342.76, 'overdraft_limit': 600.0}
```

QA

297

Notice that the attributes (if any) are returned as a list of tuples (name, value).

The complete books.xml file looks like this:

```
<?xml version="1.0" encoding="UTF-8" ?>
<accounts>
  <account account_id="ACC45678">
    <account_holder>Alice Johnson</account_holder>
    <balance>£2540.75</balance>
    <overdraft_limit>£500.00</overdraft_limit>
  </account>
  <account account_id="ACC45679">
    <account_holder>Sadia Saleem</account_holder>
    <balance>£342.76</balance>
    <overdraft_limit>£600.00</overdraft_limit>
  </account>
</accounts>
```

The second book was omitted for space reasons.

Note, owing to the UTF-8 encoding, '£' displays as expected, otherwise it would need to be encoded manually as '£'.

Defining more complex SAX handlers

SAX is a stream-based handler

- Set flags to maintain state information whilst handling descendant elements, unset flag on endElement event

```
def startElement(self, name, attributes):
    if name == 'account':
        inAccount= True
```

Create Python dictionaries from parsed data

- Keyed on tagname, set textual data as the value

```
...
self.elems = {}

def characters(self, data):
    if not data.isspace():
        self.text += data
def endElement(self, name):
    self.elems[name] = self.text
```

QA

298

Depending on the requirements, more complex event handlers may be created.

As SAX is stream-oriented you may need to set flags to indicate when you are 'inside' a node, parsing child elements of an outer node. If you find you are setting many flags and having problems keeping track of the structure, then DOM or etree may be better approaches to your problem.

When parsing, it is helpful to keep a record of tags and associated character data in a Python dictionary. Remember the character data may well be returned in different event calls, so may need to be concatenated into a single string prior to saving in the dictionary.

A complete SAX example:

```
from xml import sax
from xml.sax.handler import ContentHandler

inAccount = 0

class AccountHandler (sax.handler.ContentHandler):
    def __init__(self):
        self.text = ''
```

```

def startElement(self, name, attributes):
    global inAccount
    self.tag = name
    if attributes.items():
        print(attributes.items())
    if name == 'account':
        inAccount= True
        self.elems = {}

def characters(self, data):
    if not data.isspace():
        self.text += data

def endElement(self, name):
    global inAccount
    if inAccount and not name == 'account':
        self.elems[name] = self.text
    elif inAccount and name == 'account':
        inAccount = False
        for k,v in self.elems.items():
            print(k.ljust(15),":",v)
    if self.text:
        self.text = self.tag = ''

parser = sax.make_parser()
handler = AccountHandler() # class the implements
xml.sax.handler
parser.setContentHandler(handler)
parser.parse("accounts4.xml")

```

Using XML and the DOM

Document Object Model (DOM) loads the entire document

- Builds a linked, hierarchical structure in memory
- Exposes internal relationships via 'nodes' and node lists
 - **firstChild**
 - **lastChild**
 - **childNodes**
 - **nextSibling**
 - **previousSibling**
 - **Etc**
- Each node has (Python) attributes (DOM properties)
 - **nodeName, nodeType, nodeValue**
 - **getElementsByTagName(), appendChild(), removeChild()**
 - **etc**

QA

299

The DOM approach loads the entire XML document into memory as a tree of linked node objects.

The relationship between the nodes is provided by attributes that may be accessed or iterated over.

Each node type provides methods appropriate to handle the node's data and relationships.

Parsing with DOM

From a string

```
import xml.dom.minidom
text = "<account>...</account>"
doc = xml.dom.minidom.parseString(text)
```

From a file or file object

```
import xml.dom.minidom
doc = xml.dom.minidom.parse('accounts2.xml')
```

Creates a DOM object root node with child nodes:

```
print(doc.childNodes)
print(doc.firstChild.tagName)
```

```
[<DOM Element: accounts at 0x1fc58c44a70>]
accounts
```

QA

300

Minidom is a simple implementation of the full DOM specification.

Parsing an XML string or a file (or Python file object) using DOM will create a document root node object with child nodes created beneath, representing the XML layout with tags from the input document.

As usual, use 'dir(node_object)' to see the list of attributes and methods it supplies.

Processing DOM nodes

Loop through this node's `childNodes`:

```
for node in doc.childNodes:
    if node.nodeType == doc.ELEMENT_NODE:
        print(node.nodeName, "\n", node.childNodes)
```

```
accounts
[<DOM Text node "'\n    '",
 <DOM Element: account at 0x1fbd56f8830>,
 <DOM Text node "'\n    '",
 <DOM Element: account at 0x1fbd56f89e0>,
 <DOM Text node "'\n'>]
```

or use properties of the element nodes:

```
account1 = doc.firstChild.firstChild.nextSibling
print(account1.childNodes[1:-1:2])
```

```
[<DOM Element: account_holder at 0x1fbd56f88c0>,
 <DOM Element: balance at 0x1fbd56f8950>]
```

QA

301

In the first example, above, the first child of the document root (`doc`) is an element (`accounts`), containing several other nodes. These include text nodes containing the whitespace formatting of the original XML file, as well as some XML tag elements (`book` and `book`).

In the second example, we have assigned the name `book1` to the element referenced by `doc's firstChild's (accounts) firstChild's (text node) nextSibling (account)`. We only want the element nodes, not the whitespace text nodes, so we used a stepped slice to print only the odd-numbered child nodes.

In practice, we would use a combination of these two approaches to access the required elements by looping through the `childNodes` of nested elements, then stepping to the `nextSibling` to recurse through that set of nodes and so on.

Useful DOM node attributes

Use `dir (node)` to see a node's instance attributes

- Including the node's XML attributes attribute!
- DOM documentation refers to these as 'properties'

```
for account in doc.getElementsByTagName('account'):  
  
    print(account.getAttributeNode('account_id').nodeValue)  
  
    for child in account.childNodes:  
        if child.nodeType == account.ELEMENT_NODE:  
            for detail in child.childNodes:  
                print(detail.data)
```

Many others are available

```
ACC45678  
Alice Johnson  
£2540.75  
£500.00...
```

QA

302

Note that the constants `ELEMENT_NODE`, `TEXT_NODE`, `ATTRIBUTE_NODE` etc are defined as attributes of the node and can be used to select specific types of node by comparing with `.nodeType`.

The `xml.DOM` documentation refers to a tag's XML attributes as 'attributes' and to an object's Python attributes as 'properties' of the object.

Extracting all text from the DOM

Recurse all nodes and extract any textual data

```
import xml.dom.minidom

def getText(nodes):
    for child in nodes:
        if child.nodeType == child.TEXT_NODE:
            if not child.data.isspace():
                print(child.data)
        elif child.nodeType == child.ELEMENT_NODE:
            getText(child.childNodes)

doc = xml.dom.minidom.parse('accounts4.xml')
getText(doc.childNodes)
```

Whitespace-only nodes will also be extracted

- Detect and omit from output

QA

303

The example Python program uses the DOM interface to recursively extract all textual data from a loaded XML file. This will include the formatting text nodes containing only '\t' and '\n'. If you want to omit these, then use `str.isspace()` to check for whitespace-only nodes.

Changing the DOM

We can change the DOM structure using methods

- appendChild, insertBefore, replaceChild, removeChild
- appendData, deleteData, insertData, replaceData

```
for account in doc.getElementsByTagName('account'):  
    bal = account.removeChild(account.getElementsByTagName('balance')[0])  
    account.insertBefore(bal, account.lastChild)
```

And update the existing structure by direct reference:

```
balance = account.childNodes[5].firstChild  
balance.data = '£12.00'  
for child in account.childNodes:  
    if child.nodeType == account.ELEMENT_NODE:  
        print(f"{child.nodeName}: ", end="")  
        for detail in child.childNodes:  
            print(detail.data)
```

QA

304

In the first example, we are removing a child node (balance), saving the returned value (as bal), then re-inserting it at the same place in the book node, just before the last text node (lastChild).

In the second example above, the 6th node (index 5) is the balance text node. Note that the account.childNodes list looks like this:

```
[<DOM Text node ""\n">,  
 <DOM Element: account_holder at 0x277611d84d0>,  
 <DOM Text node ""\n">,  
 <DOM Text node ""\n">,  
 <DOM Element: overdraft_limit at 0x277611d8710>  
 <DOM Element: balance at 0x277611d8560>,  
 <DOM Text node ""\n">]
```

Updating the balance could have been done via the relationship map:

```
account.lastChild.previousSibling.firstChild.data = '£12.00'
```


Back to XML

Create an XML document from the DOM

- `node.toxml()` Generate XML
- `node.toprettyxml()` Generate spaced XML
- `node.writexml( WriteableFileObject )` Output to file

```
out = open('new.xml', 'w')
doc.writexml(out)           # Save it
out.close()

print(doc.toxml())          # Display it on stdout
```

Any level of the DOM may be emitted using these methods

- But usually used to write the full document from the root node

QA

305

`node.toxml()` generates XML from the DOM. Tags and text data are on the same line. e.g. `<title>New Ideas</title>`.

`node.toprettyxml()` generates well-spaced XML, such that the tags and the data are separated on different lines. e.g.

```
<title>
    New Ideas
</title>
```

`node.writexml()` writes to an opened, writeable, Python file object. Close the file object to ensure the data is actually written back to the disk.

Other serialization formats

YAML

- A superset of JSON (mostly), but easier to read (by humans)
- Portable - supported by several languages, including Python 2 and 3
- PyYAML – a popular YAML parser and emitter for Python
- ruamel.yaml is a YAML parser and emitter that supports comment preservation
 - Both available from PyPi

```
import yaml

accounts = [
    Account(45678, "Kamran Liaqat", 2540.75, 500.00),
    Account(45679, "Sadia Saleem", 342.76, 600.00)
]

# Create a dictionary of accounts
accounts_dict = {
    account.account_id: {
        "name": account.name,
        "balance": account.balance,
        "overdraft_limit": account.overdraft_limit
    }
    for account in accounts
}

fo = open('accounts.yaml', 'w')
yaml.dump(accounts_dict, fo)
fo.close()
```

accounts.yaml

```
45678:
  balance: 2540.75
  name: Kamran Liaqat
  overdraft_limit: 500.0
45679:
  balance: 342.76
  name: Sadia Saleem
  overdraft_limit: 600.0
```

YAML has support with the following languages: C, C++, Ruby, Python (2 & 3), Java, Perl, C#, PHP, OCaml, Javascript, Actionscript, and Haskell.

Pickle protocols

How would you like your data pickled?

- Currently 5 different protocols to choose from, `pickle.protocol=n`

Protocol	Meaning
0	ASCII, backwards compatible with earlier versions of Python
1	Binary format, also backwards compatible
2	Added in Python 2.3. Efficient pickling of classes
3	Added in Python 3.0. Not backward compatible
4	Added in Python 3.4. Very large objects and more kinds of objects (default)
5	Added in Python 3.8. Support for out-of-band data

The pickle module has protocol attributes.

- `HIGHEST_PROTOCOL` and `DEFAULT_PROTOCOL`

```
import pickle
outp = open('capitals.p', 'wb')
pickle.dump(caps, outp, pickle.DEFAULT_PROTOCOL)
outp.close()
```

Preferred protocol

- None of these protocols are secure.

QA

307

Pickle files can be written in several formats. When in doubt, use the default format, which is automatically selected based on your Python version.

If you choose the ASCII format, you don't need to open the file in binary mode, but be consistent and use the same mode for both reading and writing. ASCII pickles are easier to debug, while binary format is preferred for compatibility with older Python programs. In most cases, sticking with the default is the safest and most efficient option.

Pickle Protocol Versions:

- **Protocol 5** (Python 3.8+): Introduced support for out-of-band data buffers, allowing applications to transmit metadata across separate communication channels (e.g., paired sockets); also improves performance for in-band data.
- **Protocol 4** (Python 3.4+): Added support for very large and complex objects (e.g., data frames) and improved efficiency. Note that it is not backward compatible with earlier protocols.
- **Protocol 0–2** (Python 2): Early protocols used in Python 2.x. Compatibility issues arose when creating pickles in Python 3.0 for use in

Python 2, which were resolved in Python 3.1. However, protocols 0–2 cannot be exchanged between Python 3.0 and 3.1. For cross-version compatibility, use protocol 3 when sharing data between Python 3, and upgrade to at least Python 3.1 to exchange data with Python 2.

Getting out of a pickle: dill

Not everything can be successfully pickled.

- Lambda functions, generators, defaultdicts etc.
- Attempts to do so will raise a **PicklingError**, e.g.

```
import multiprocessing as mp

#works - converting each int to a str
if __name__ == "__main__":
    with mp.Pool(4) as mp_pool:
        print(mp_pool.map(str, range(1,21)))

#won't work - we can't Pickle a lambda!
if __name__ == "__main__":
    with mp.Pool(4) as mp_pool:
        print(mp_pool.map(lambda x: pow(x,3), range(1,21)))

_pickle.PicklingError: Can't pickle <function <lambda> at 0x02FABBFB8>:
attribute lookup <lambda> on __main__ failed
```

Fix this using a Dill-enabled multiprocessing framework

```
import pathos.multiprocessing as pmp

if __name__ == "__main__":
    with pmp.Pool(4) as pmp_pool:
        print(pmp_pool.map(lambda x: pow(x,3), range(1,21)))
```

QA

In this example we are trying to apply multiprocessing techniques to a simple lambda function (one which cubes numbers in a given range).

Built-in functions used in such multiprocessing pools generally don't cause a problem – their data being communicated between processes – via pickle – perfectly fine.

However, as soon as a lambda function is introduced, problems occur.

The solution is to use the dill module instead.

The Pathos framework (<https://pypi.org/project/pathos/>) provides high-level interface for launching parallel computations. It utilises dill as part of its inter-process communication.

Sample output:

```
Mapping str function (no problem)      ['1', '2', '3',
'4', '5', '6', '7', '8', '9', '10', '11', '12', '13', '14',
'15', '16', '17', '18', '19', '20']
```

Mapping cubing lambda using Pathos [1, 8, 27, 64,
125, 216, 343, 512, 729, 1000, 1331, 1728, 2197, 2744, 3375,
4096, 4913, 5832, 6859, 8000]

Note.

You will need to install pathos in order to get this example working – at time of writing it is not part of Python's standard library.

Python provides **database drivers** for most popular relational database systems, and many are bundled with standard or third-party distributions. These drivers adhere to a common interface defined by the Python Database API Specification v2.0 (DB-API 2.0).

To interact with a database:

1. Establish a connection using a connection object.
2. Use this connection to create a cursor object, which handles queries and transaction control (commit and rollback).

The **cursor** is the primary tool for executing SQL statements and getting results.

Some Common Cursor Methods

- `arraysize` Number of rows fetched by `fetchmany()`
- `close()` Close the cursor
- `description` Metadata about the result set or table
- `execute()` Execute a single SQL statement (supports placeholders)
- `fetchall()` Retrieve all remaining rows
- `fetchone()` Retrieve the next
- `rowcount` Number of rows affected by the last operation

Python is shipped with **SQLite**, a lightweight, single-file relational database

that requires no server setup and is ideal for small to medium-sized applications.

Example — SQLite from Python

Use the `sqlite3` module.

- Bundled with the Python release.
- Example using `cursor()`, `execute()`, `fetchall()` and `close()` methods:

```
import sqlite3

db = sqlite3.connect('whisky')

cur = db.cursor()
cur.execute('SELECT BRANDS.BNAME, REGION.RNAME \
            FROM BRANDS,REGION \
            WHERE REGION.REGION_ID = BRANDS.REGION_ID \
            ORDER BY BRANDS.BNAME;')

for row in cur.fetchall():
    print("{0[0]:<30s} {0[1]:<30s}".format(row))

db.close()
```

QA

310

This example demonstrates how to use Python's **sqlite3** module to connect to a relational database and execute a simple **SQL query**. The `connect()` function establishes a connection to a database file named **whisky** - if the file does not already exist, SQLite automatically creates it. From this connection, a **cursor object** is obtained using `db.cursor()`. The cursor is the main interface for sending SQL commands and retrieving results.

The `execute()` method runs an SQL `SELECT` statement that retrieves brand names and their corresponding regions from two tables, **BRANDS** and **REGION**. The `WHERE` clause ensures that the tables are joined using a common key (`REGION_ID`), while the `ORDER BY` clause sorts the results alphabetically by brand name. Once the query executes, the `fetchall()` method retrieves all matching rows as a list of tuples. The `for` loop iterates through each row, and the `print()` statement uses formatted string output to neatly display the brand and region names in two left-aligned columns, each 30 characters wide.

Finally, the `db.close()` call closes the database connection, ensuring that all resources are released properly. In modern Python code, it's often recommended to use a **context manager** (with

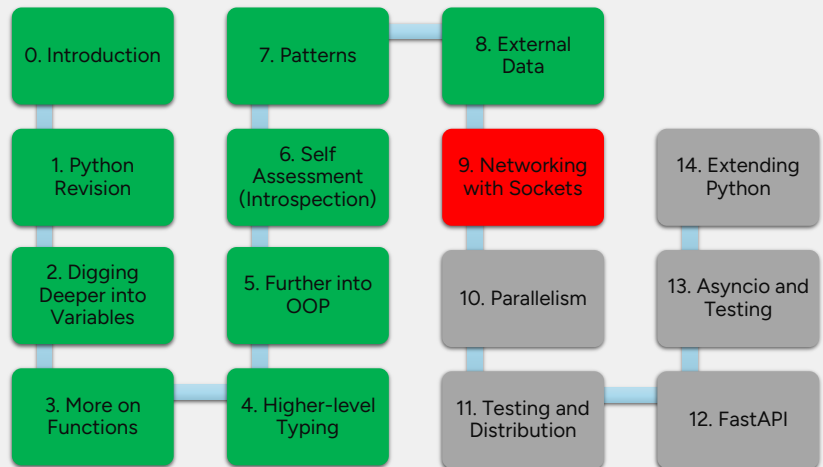
`sqlite3.connect('whisky') as db:)` to handle connection closure automatically, even if an error occurs.

Intermediate Python

Networking with Sockets



Module roadmap



QA

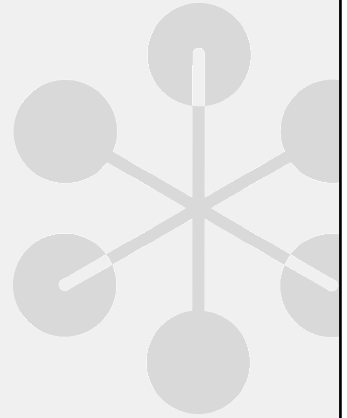
Networking with sockets

Learning outcomes:

- Build Python applications that can communicate securely over a network

Prerequisites:

Delegates are required to have knowledge and skills relating to basic Python programming, including data structures, functions, loops and flow control.

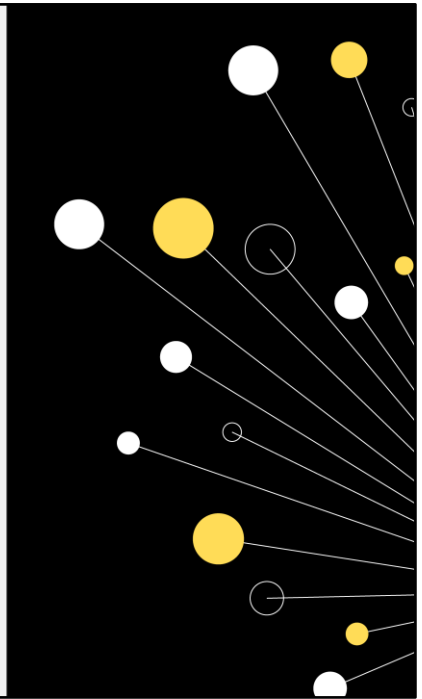


Learning Objectives

By the end of this session, you should be able to:

- Define sockets and explain their role in network communication
- Use the Python socket module to create and manage sockets
- Distinguish between socket types, families, and internet domain sockets
- Implement connection-oriented sockets using context managers
- Apply the socketserver framework to build server-side applications
- Configure SSL for secure socket communication
- Compare connectionless sockets with connection-oriented alternatives
- Differentiate between blocking and non-blocking socket operations

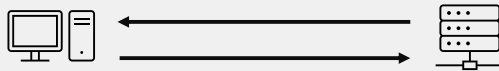
QA



What is a socket?

A socket is a two-way communication endpoint that enables data exchange between two processes.

- It provides a foundation for distributed computing.
- Sockets follow the client–server communication model.
- They facilitate communication between applications running on different operating systems.
- They support inter-process communication (IPC).
- Every socket is associated with a specific process and socket type.
- Socket communication can be either connection-oriented or connectionless.
- Sockets belong to address families, which define:
 - How sockets are named and addressed.
 - The protocols used for transmitting data.



QA

315

Sockets are communication endpoints built on the client-server architecture of interprocess communication. They function as named, bidirectional channels that allow processes to exchange data locally or across a network. This communication can happen through either reliable virtual circuit connections or connectionless datagram streams.

Establishing this communication requires an asymmetrical setup where the client and server play distinct roles. A server application typically sits dormant, listening at a specific, well-known address. It stays inactive until a client initiates a connection request to that address. Once requested, the server wakes up to process the client's demands and perform the necessary actions.

By themselves, sockets cannot fully define a communication link. They must function inside a specific communication domain, which is also known as an address or protocol family. This domain determines how the socket is named and dictates the specific protocols used to transfer the data. As a rule, sockets can only exchange information with other sockets sharing the same domain. Additionally, every socket is assigned a specific

type that defines the exact manner in which data moves across the active link.

Python socket module

Included as part of Python's standard library

- Provides a low-level networking API
- Rooted in the Berkeley Software Distribution (BSD) 4.3 specification, dating back to 1983
- Broadly supported across most modern operating systems, including Unix-based systems, Microsoft Windows, and Apple macOS
- Highly portable across platforms
- Compatible with both IPv4 and IPv6, though certain methods are exclusive to IPv4
- For those familiar with sockets in C:
 - The underlying principles and many of the functions remain consistent
 - Python's socket interface is considerably more user-friendly
 - A number of Python-specific methods are provided to simplify and abstract common operations
- Python additionally offers built-in support for the Secure Sockets Layer (SSL) via the `ssl` module

QA

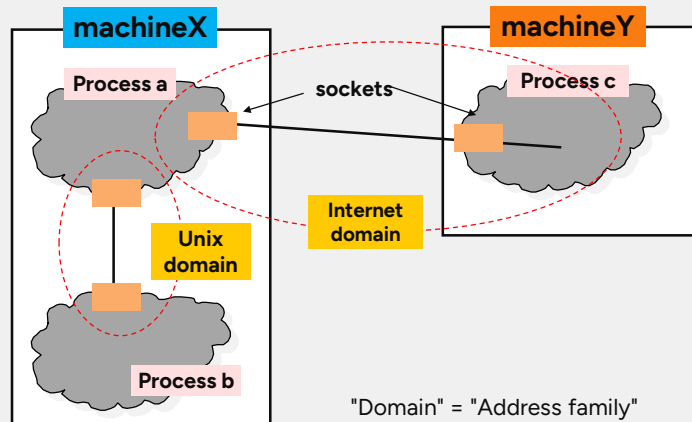
316

The socket module provides a network programming interface based on the 4.3 Berkeley Software Distribution (BSD) sockets model, combined with several Python-specific extensions. It offers a single API that abstracts the underlying network infrastructure, giving developers a streamlined programming interface. While this API closely mirrors the traditional C socket programming interface, it is highly abstracted and significantly easier to implement.

The original BSD socket API was limited to IPv4 and lacked native support for IPv6. Although modern operating systems—including Windows—fully support IPv6, the newer protocol requires different data structures and parameters. Modern Python applications achieve network protocol-independence seamlessly by using `socket.getaddrinfo()` to automatically handle whatever IP version the host or domain provides

Communication domains

- Internet domain protocol
 - TCP/IP, UDP/IP
- Used for inter-machine and intra-machine communication
- Domain AF_UNIX on UNIX/Linux



QA

317

Most UNIX implementations that feature socket support provide two standard address families or communication domains. The first is the UNIX domain, represented by the AF_UNIX address family, which facilitates communication between processes running on the same local machine by using local file paths as socket names. The second is the Internet domain, represented by the AF_INET address family, which connects processes running on completely different machines across a network using the TCP/IP protocol suite and globally unique network addresses. While developers have created other domains over the years to support systems like the now obsolete Xerox Network Systems (XNS) protocols, Windows Sockets historically implemented only a single communication domain: the Internet domain, with the option to expand to more families in later specifications. As an example of this connectivity, the diagram shows a particular setup, where process "a" on machineX has established links through sockets with process "b" on machineX and "c" on machineY. Both of these use the Internet domain.

Socket 'names' on internet domain

A socket name is composed of the following elements:

- An Internet Protocol (IP) address
- A port number — a 16-bit value used by both TCP and UDP
- The communication protocol

IPv4 uses a 32-bit IP address, represented in the familiar four-octet dotted decimal notation

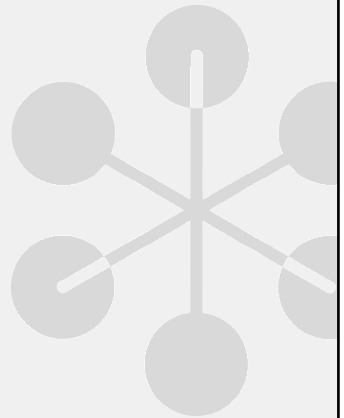
- e.g. 172.16.27.1

IPv6 uses a 128-bit IP address, structured as eight groups of 16-bit values

- e.g. FEDC:AB19:12FE:0234:98EF:1178:8891:CAFF

Hostnames serve as human-readable abstractions of underlying IP addresses

Well-known services act as named abstractions of specific port number and protocol combinations



QA

318

Globally unique socket names in the Internet domain rely on three core components.

For IPv4, the first component is a 32-bit IP address that identifies a specific network interface, which is the hardware network card connecting a machine to a network. While most hosts contain only a single network interface, this address ensures the device can be located. The second component is a 16-bit port number assigned by the host to represent a specific application. The final component is an implicit 16-bit protocol number that identifies whether the connection uses UDP for datagram sockets or TCP for stream sockets. Together, these elements form the local name that is bound to a socket after its initial creation.

IPv6 upgrades this infrastructure by using 128-bit addresses, which are traditionally divided into eight 16-bit segments. Because long 128-bit strings are difficult to manage, the protocol allows developers to omit leading zeros within each segment and compress consecutive sections of zeros into a single double-colon (: :). Furthermore, unlike IPv4 addresses which identify an entire host machine, an IPv6 address specifically references an individual network interface, natively accounting for multi-homed machines that contain multiple network cards.

Across both IP versions, port numbers are allocated directly by the host.

However, many standard TCP/IP services, such as FTP, rely on standardized "well-known" ports that remain identical on every host. Because these specific port numbers are globally reserved, they are unavailable for general application use. On most operating systems, port numbers below the IPPORT_RESERVED threshold of 1024 are strictly locked and reserved exclusively for privileged system processes.

Socket types and address families

Socket types define the manner in which data is transmitted through the socket

- The following standard socket types are supported within the Internet domain:
 - SOCK_DGRAM — Datagram-based communication; unreliable transmission using the User Datagram Protocol (UDP)
 - SOCK_STREAM — Virtual circuit-based communication; reliable, in-order packet delivery using the Transmission Control Protocol (TCP)
- Both socket types support duplex communication
- The most commonly used socket families include:
 - AF_INET — The default Python socket family, supporting IPv4
 - AF_INET6 — Provides support for IPv6 addressing
 - AF_UNIX — Used for local inter-process communication on UNIX-based systems
 - AF_BLUETOOTH — Enables communication over Bluetooth connections
 - AF_VSOCK — Facilitates communication between virtual machines and their hosts

Like a person dropping several stamped, individually addressed postcards into a mailbox.



Like two people talking on a landline telephone with a continuous wire connecting them.



The TCP/IP suite delivers both connection-oriented and connectionless transport services. Transmission Control Protocol (TCP) operates as a reliable, connection-oriented service designed for sending large, ordered sequences of data. In contrast, User Datagram Protocol (UDP) acts as a connectionless, best-effort service optimized for transferring smaller, individual data units.

Every socket is defined by a specific type that dictates how data moves across its active link. Sockets generally communicate exclusively with other endpoints matching their own type, though cross-type interaction remains possible if supported by the underlying protocol layers.

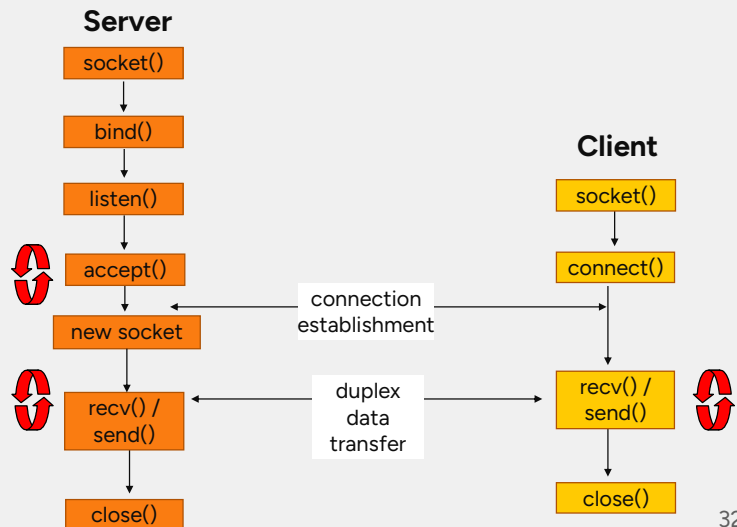
Within the Internet domain, applications rely heavily on two primary socket configurations:

SOCK_DGRAM sockets manage connectionless datagram communication powered by UDP. Data transfers as distinct, standalone packets without any delivery confirmations, meaning messages can be dropped, duplicated, or received completely out of order. Crucially, datagram sockets preserve strict record boundaries within the data stream, closely mirroring the hardware-level behaviour of traditional packet-switched networks like Ethernet.

SOCK_STREAM sockets implement reliable, full-duplex virtual circuits

powered by TCP. Data flows continuously in perfect sequence without duplication or native record boundaries. This type demands a formal connection handshake before any data can be exchanged, and the operating system automatically alerts the participating processes if that connection drops.

Using connection oriented sockets



QA

320

A server application typically monitors a well-known port or address, remaining idle until a client initiates a connection request. Once a request arrives, the server process awakens and begins servicing the client by carrying out whatever operations are required.

The process begins with the server creating a socket using the `socket()` method, at which point the socket type and communications domain are defined. In Python, a socket is handled as an object, much like an open file. The socket is then bound to a local name using `bind()`, where, in the Internet domain, the name represents a combination of a unique network address and a host-relative port number.

Once bound, the server calls `listen()` to indicate that the socket is ready to accept incoming connections. This method also specifies the maximum size of the queue that will hold pending connection requests. When the server is ready to handle a connection, it calls `accept()`. If no requests are currently queued, this call causes the process to block until one arrives. If requests are present, a connection is established.

The `accept()` method returns a new socket object that shares the same characteristics as the original listening socket. This new socket is then used to handle the actual data exchange with the client via the `recv()` and `send()` methods. This design allows the server to delegate data transfer to

a subprocess or thread, freeing the original socket to continue listening for incoming requests from other clients.
See chapter appendix for equivalent connectionless flow

Simple socket based connection – server

my_simple_server.py

```
# Server code
from socket import socket, AF_INET, SOCK_STREAM

my_socket = socket(AF_INET, SOCK_STREAM, 0)
ipv4_tuple = '127.0.0.1', 8000
my_socket.bind(ipv4_tuple)
my_socket.listen(1)

print(f"Waiting for connection on port {ipv4_tuple[1]}")
new_socket_object, r_address = my_socket.accept()

print(f"Connection made from: {r_address}")
received_data = new_socket_object.recv(4096, 0)

decoded_received_data = received_data.decode("utf-8")
print(f"Data received: '{decoded_received_data}'")

my_socket.close()
print("Connection closed")
```

QA

321

A pair of simple Python 3 scripts demonstrate a basic client-server messaging system operating over a network.

The server is designed to await incoming TCP connections on a designated port. Setting up the listening socket involves four API calls, introduced in the following sequence: `socket()`, `bind()`, `listen()`, and `accept()`.

Once the connection is established, the client sends its message as a raw byte sequence before closing the socket, utilising the `accept()`, `recv()`, and `close()` methods throughout this process.

The server can then present the received message to the user by converting the incoming bytes into readable text. This conversion is performed using the `decode()` method of the `str` class, with "utf-8" specified as the target encoding. UTF-8, or the 8-bit Unicode Transformation Format, is a variable-width character encoding capable of representing over one million Unicode characters through sequences of between one and four 8-bit units.

Although both scripts can be run on a single machine for initial testing purposes, a more meaningful evaluation involves deploying the client and server on two separate network hosts. It is worth considering what changes to the code would be required to enable communication across two distinct machines.

Simple socket based connection – client

```
# Client terminal:  
Sending message: b'What hath God wrought!'  
message successfully sent  
Connection closed
```

```
# Server terminal:  
Waiting for connection on port 8000  
Connection made from: ('127.0.0.1', 52268)  
Data received: 'What hath God wrought!'  
Connection closed
```

my_simple_client.py

```
# Client code  
from socket import socket, AF_INET, SOCK_STREAM  
  
my_socket = socket(AF_INET, SOCK_STREAM, 0)  
ipv4_tuple = '127.0.0.1', 8000  
  
my_socket.connect(ipv4_tuple)  
  
msg = b"What hath God wrought!"  
print(f"Sending message: {msg}")  
  
my_socket.send(msg, 0)  
print("message successfully sent")  
  
my_socket.close()  
print("Connection closed")
```

QA

322

The client-side code is notably more straightforward in comparison to the server.

In essence, the client only needs to make use of three methods: `connect()`, `send()`, and `close()`.

One important consideration is that any string data must be converted into a byte sequence prior to transmission. While the `encode()` method could be used for this purpose, the scripts instead make use of the "b" string prefix as a more concise alternative.

socket API – allocating a socket

Creates an endpoint

- No connection or name binding made at this stage

```
from socket import socket, AF_INET, SOCK_STREAM
skt = socket(family, type, protocol)
```

returned
socket object

e.g. AF_INET

SOCK_STREAM
SOCK_DGRAM

protocol number,
depends on address
family, usually 0

QA

323

The `socket()` method initializes a network endpoint by allocating a descriptor based on a specified address family, data type, and protocol. A successful call returns this descriptor, while any failure triggers a `socket.error` exception. To set up the socket, you must define its operating domain and type using constants from the `socket` module. For TCP/IP networking, the domain is strictly restricted to `AF_INET`, which limits the valid socket types to either `SOCK_STREAM` or `SOCK_DGRAM`.

Because a chosen domain and socket type combination automatically implies a specific network protocol—such as TCP for an Internet stream socket—you can generally pass `0` for the protocol argument. Finally, while socket handles can function synchronously, you can use the `setsockopt()` function to modify how `select` and `accept` behave for individual threads.

socket API – Associating a 'name' with a socket

Associates a socket with a specific endpoint address

- The name varies depending on the domain
- For servers, the special machine name INADDR_ANY can be represented as an empty string in Python

```
from socket import socket, AF_INET, SOCK_STREAM  
  
skt.bind(address)
```

For IPv4 Internet Domain:
tuple (machine, port_number)

For IPv6 Internet Domain:
tuple (machine, port_number,
flowinfo, scopeid)

Optional

QA

324

When a socket is created using the `socket()` function, it is associated with a specific address family and is represented by a socket object. The `bind()` function is used to link a local endpoint address, or "name," to the socket, with the format of this name depending on the socket's domain. The local name for binding is typically specified using a generic tuple. If a client application does not require a specific IP address, or if a server wants to listen on any available network interface IP address, a special Internet address called `INADDR_ANY` can be used. This approach simplifies programming for systems with multiple network interfaces.

If the client does not care about the port number to which it is bound, it can specify a port value of 0, prompting the system to assign an unused local port automatically. When the port is set to 0, the assigned port will be between 1024 and 5000, as ports below 1024 are reserved for privileged services such as TELNET.

In such scenarios, the application can use `getsockname()` after calling `bind()` to determine the address and port assigned. However, in the Internet domain, `getsockname()` may not provide the Internet address until the socket is connected, because multiple Internet addresses may be valid if the client and server have several network interfaces.

By default, a socket cannot bind to a local address that is already in use.

Since every connection is uniquely identified by the combination of local and remote addresses, it is possible to have multiple sockets bound to the same local address as long as their remote addresses differ. This capability, known as local-address reuse, can be enabled through a socket option.

Server listening for connections

Gets a bound, connection-oriented socket ready to accept incoming connections and is often referred to as a "passive" socket.

```
from socket import socket, AF_INET, SOCK_STREAM  
  
skt.listen(backlog)
```

maximum number of pending connection requests are allowed before refusing new connections.

Historically: 5

Note: this is optional for recent versions of Python.

QA

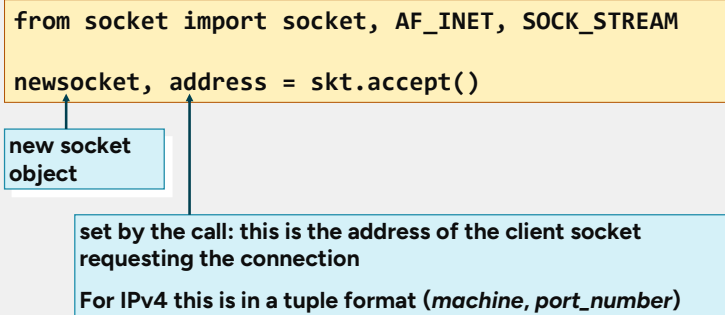
325

The `listen()` function is used to prepare a connection-oriented socket (of type `SOCK_STREAM`) to accept incoming connection requests. Essentially, this call creates a queue where connection requests are held. The maximum size of this queue is defined here as a parameter. If connection requests arrive when the queue is full, they will be rejected, resulting in an `ECONNREFUSED` error during the client's `connect()` attempt. This function is commonly used by servers that can handle multiple connections at the same time. Using the telephone system analogy for connection-based sockets, `listen()` functions like a call-waiting system. Callers are placed in a queue and put on hold up to a certain limit while waiting for a connection. Any additional callers beyond this limit will receive a busy signal and be refused a connection. Currently, the queue backlog is silently limited to a maximum of 5. Any illegal values (less than 1 or greater than 5) are automatically adjusted to the nearest valid value.

Server accepting a connection

The server side establishes the connection between the client and server using a connection-oriented socket.

- This process typically blocks until a connection request is received
- Note: non-blocking sockets are also an option



QA

326

How the accept() Method Works

Once a server configures its socket characteristics, it enters a listening state to await incoming connection requests. This is handled by the `accept()` method, which pulls the first pending request from the listening socket's queue. If the queue already contains waiting requests, `accept()` processes the first one immediately without waiting.

Socket Management After a Connection

When a connection request is successfully received and established, `accept()` behaves as follows:

- **Creates a New Socket:** It returns a brand-new socket object with the identical properties of the original listening socket.
- **Dedicated Communication:** This new socket descriptor is used exclusively for data exchange (`send()` and `recv()`) between the server and that specific client. It cannot be used to accept additional connections.
- **Keeps the Original Socket Active:** The original listening socket remains open, uninterrupted, and ready to welcome subsequent incoming requests.

Multi-Connection Architecture

This design allows a server process to operate in two modes simultaneously: listening for new connection requests on one socket while exchanging data across separate sockets. Because of this separation, developers can easily scale the server to handle multiple concurrent connections using threads or subprocesses.

Client Identification

Upon a successful connection, `accept()` also retrieves the client socket's name (address). The exact format of this identifier depends entirely on the address family being used.

Note: For details on implementing this behaviour in a non-blocking mode, please refer to the chapter appendix.

Client requesting a connection

The client end establishes the connection between client and server via a connection-oriented socket

- It typically blocks until the connection is made

```
from socket import socket, AF_INET, SOCK_STREAM
skt.connect((machine, port_number))
error = skt.connect_ex((machine, port_number))
```

On an error occurring:
`s.connect` raises the exception
`s.connect_ex` returns an error code

address of socket to connect
to
(IPv4 shown)

QA

327

The primary purpose of the `connect()` method is to initiate the client-side rendezvous procedure, which establishes a communication link between processes over a connection-oriented socket. For this link to form successfully, the server must have already called, or be prepared to call, `accept()`. When the client executes `connect()`, the system requires both the client's own socket and the specific name of the target server socket to complete the connection.

While the client socket must be bound to a local name for this process, the system will automatically step in if it is currently unbound. In this scenario, the system assigns unique local values and officially marks the socket as bound. In an Internet domain context, this means the system automatically selects an IP address from the available network interfaces and pairs it with any unused, valid port. Because the primary goal is simply reaching the server, the client process typically does not need to manage or care about its own local association details.

By default, if the socket has not been explicitly configured as non-blocking, a call to `connect()` will pause the executing thread. It will remain blocked until the connection is either successfully established or rejected by the remote host, which can happen due to a full server backlog or a protocol-specific timeout.

Additionally, `connect()` is not strictly limited to connection-oriented streams; it can also be utilized with connectionless protocols. In those cases, it serves to bind a specific socket so that it exclusively directs all of its outgoing datagrams to a single, designated destination socket. Further details regarding the specific behavioural differences between blocking and non-blocking sockets can be found in the chapter appendix.

Sending data

Note: bytes objects are passed

Connection-oriented sockets

- *Not guaranteed* to send all data

```
from socket import socket, AF_INET, SOCK_STREAM
skt.send(bytesbuffer, flags)
```

- Sends *all* data

```
skt.sendall(bytesbuffer, flags)
```

Connectionless sockets

```
from socket import socket, AF_INET, SOCK_STREAM
skt.sendto(bytesbuffer, flags, address)
```

These usually block until buffer space available

QA

328

The `send()` and `sendto()` functions are the primary mechanisms used to transmit outgoing data through a socket. Both functions include a `flags` parameter that allows developers to specify protocol-specific behaviours, though this argument is normally set to zero for standard operations. Additionally, both routines return the total number of bytes successfully transmitted. It is important to note that this returned value may sometimes be less than the total number of bytes requested, and a successful function completion only indicates that the data was sent from the local system, not that it was successfully delivered to the recipient. While they share similarities, the two functions are tailored for different socket types. The `send()` function is typically utilized to write data over connection-oriented sockets, though it can also function on connectionless sockets if a previous `connect()` call has already locked in a destination address. Conversely, `sendto()` is specifically designed for connectionless sockets, requiring the destination's name to be explicitly bundled with every individual data packet. When using `sendto()`, the system automatically injects the sending socket's name into the outgoing message. If a developer calls `sendto()` on a connection-oriented socket, the system simply ignores the extra address parameter, making the operation functionally identical to a standard `send()` call.

Both functions return the total number of bytes successfully transmitted. However, you should be aware that the amount of data actually sent might be less than you originally requested. Furthermore, a successful return from `send()` or `sendto()` only means the data left the local system; it does not guarantee that the message was actually delivered to the recipient.

Receiving data

Note: bytes objects are returned

- *buffsize* is usually 4096

Connection-oriented sockets

```
from socket import socket, AF_INET, SOCK_STREAM
data = skt.recv(buffsize, flags)
```

Connectionless sockets

```
from socket import socket, AF_INET, SOCK_STREAM
(bytes, address) = skt.recvfrom(buffsize, flags)
(nbytes, address) = skt.recvfrom_into(buffsize, flags)
```

Usually block until data is available

QA

329

The `recv()` and `recvfrom()` functions are utilized to accept incoming data through a socket. Both routines feature a `flags` parameter for managing protocol-specific details, which is typically set to zero for standard operations. However, developers can use specific flags like `MSG_PEEK` to inspect the incoming data waiting in the socket without actually consuming or removing it.

The `recv()` function is generally used to read data from connection-oriented sockets, though it can also be applied to connectionless sockets if a prior `connect()` call was established to lock in the expected packet source. On the other hand, `recvfrom()` is particularly useful for connectionless setups because it additionally captures and returns the identity or name of the sender. Regardless of the function used, when dealing with datagram sockets, the system extracts data from the very first datagram in the queue, filling the provided buffer up to its maximum capacity.

Unless the socket is nonblocking, the `recv()` call waits for data to arrive if no incoming data is available at the socket.

Closing sockets

There are several available methods:

close()

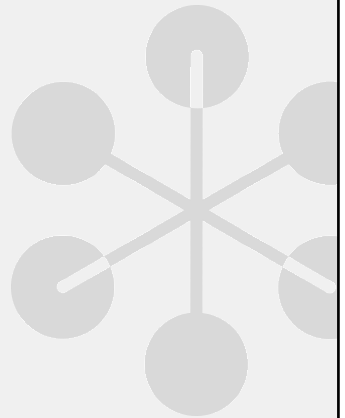
Frees resources but may not immediately close the connection.

shutdown(how)

Prevents further sending or receiving based on the specified parameter, without releasing resources.

detach()

Introduced in Python 3.2, this method detaches the connection without closing the file descriptor, enabling the socket to be reused.



The `shutdown()` function accepts several flags to control its behaviour: `SHUT_RD` prevents any further data reads, `SHUT_WR` stops any more data writes, and `SHUT_RDWR` disallows both reads and writes. There are important distinctions between `shutdown()` and `close()`. A file descriptor can be duplicated, and `close()` only disconnects the socket when the last descriptor is closed. In contrast, `shutdown()` immediately deactivates the socket. Additionally, using the `How` flags, `shutdown()` can be applied in just one direction, either for reading or writing.

Context manager usage

my_revised_server.py

```
# Revised Server code
from socket import socket, AF_INET, SOCK_STREAM

ipv4_tuple = '127.0.0.1', 8000

with socket(AF_INET, SOCK_STREAM, 0) as my_socket:

    my_socket.bind(ipv4_tuple)
    my_socket.listen(1)

    print(f"Waiting for connection on port {ipv4_tuple[1]}")
    new_socket_object, r_address = my_socket.accept()

    with new_socket_object:
        print(f"Connection made from: {r_address}")
        while True:
            received_data = new_socket_object.recv(4096, 0)
            if not received_data:
                break
            decoded_received_data = received_data.decode("utf-8")
            print(f"Data received: '{decoded_received_data}'")
            print(decoded_received_data)

    print("Connection closed")
```

QA

331

The `socket.socket()` function creates a socket object that supports the context manager protocol, allowing it to be used within a `with` statement. This eliminates the need to explicitly call `s.close()`. The slides shows a revised version of the server script that utilises the context manager syntax, with additional modifications to demonstrate looping through received data until the client stops sending more.

Python socket errors and exceptions

Most socket methods raise exceptions when an error occurs.

- Some methods return error codes that require checking before proceeding.

socket.error - Common socket errors,

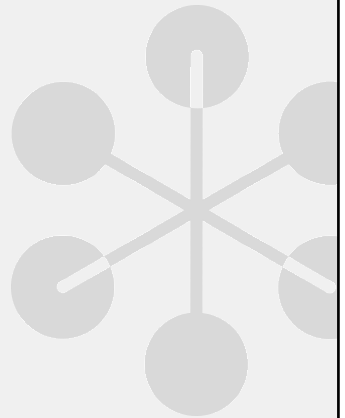
- typically including a message string and sometimes a platform-specific error number.

socket.herror - Host address-related errors

- encountered with functions like `gethostbyname()` and `gethostbyaddr()`.

socket.gaierror - Address information errors

- raised by functions such as `getaddrinfo()` and `getnameinfo()`.



QA

332

Socket APIs in C libraries typically return -1 when an error occurs. In contrast, Python usually follows its convention by raising an exception (though not always*). The most common exception encountered is `socket.error`, which may contain an error number. These error numbers are often specific to the platform; for instance, 10054 corresponds to the Windows error `WSAECONNRESET`, one of the most frequent errors. For detailed information about exceptions, refer to the description of each method in the online documentation.

* Probably the most important exception to Python's "raise on error" pattern is `recv()` returning `b''` on connection close:

```
data = sock.recv(4096)
if data == b'':
    print("Peer closed connection")
```

No exception is raised because an orderly shutdown is not considered an error.

Network information database functions

Database extraction and conversion routines that are used to construct socket names, include:

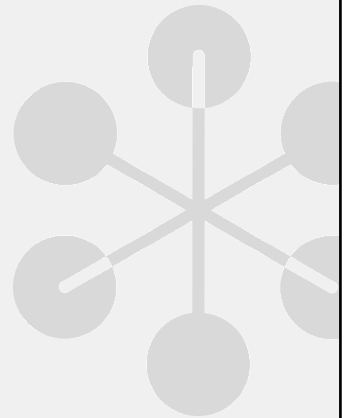
- `gethostname()`, `gethostbyname()`, `gethostbyaddr()`, `getservbyname()`, `getprotobyname()`, and `getprotobyndnumber()`

These functions may sometimes block due to Domain Name Server (DNS) activity.

Some of the older methods only support IPv4.

Additionally, other useful routines include:

- `inet_addr`, `inet_ntoa`, `getaddrinfo`, and `getnameinfo`



QA

333

The information that forms a socket name can be represented using user-friendly textual names, which are typically stored in local database files. Several routines exist to translate between these textual names and their underlying forms. Network interaction may be necessary during this process, such as when using DNS, which can cause blocking. These functions are generally divided into three categories.:

The **host name resolution functions** include:

- `gethostname()`, which returns the name of the host machine.
- `gethostbyaddr()`, which retrieves the name(s) and address corresponding to a network address.
- `gethostbyname()`, which obtains the IPv4 address associated with a host name.
 - By combining `gethostbyname()` with `gethostname()`, you can find the current IP address, for example:

```
localIP = gethostbyname(gethostname())
```

- For IPv6, `gethostbyname_ex()` serves a similar purpose to `gethostbyname()`

The **service name resolution functions** include:

- `getservbyname()`, which returns the port number corresponding to a service name.

- `getservbyport()`, which retrieves the service name(s) and port number associated with a given port.

The **protocol name resolution functions** include:

- `getprotobyname()`, which translate protocol names (e.g., "icmp", "tcp") into the numeric protocol identifiers used by the OS networking stack.

Socket server framework

The `socketserver` module simplifies the creation of network servers

- included in Python's standard library.
- It provides four different server classes (all of which inherit from the `BaseServer` class):
 - `TCPServer`
 - `UDPServer`
 - `UnixStreamServer` (for Unix domain sockets only)
 - `UnixDatagramServer` (also for Unix domain sockets only)
- To build a server, use one of the above classes and create a custom handler class to manage client requests.
- Additional methods are available, such as:
- `verify_request`, `process_request`, and `finish_request`.
- The module supports both thread-based and process-based workers through the `ForkingMixIn` and `ThreadingMixIn` classes.

QA

334

All four classes handle requests synchronously, meaning each request must be completely processed before the next one begins. This approach can be inefficient when requests require significant processing time, such as compiling results to return. Therefore, using a thread- or process-based solution is preferable, as these support asynchronous handling of requests.

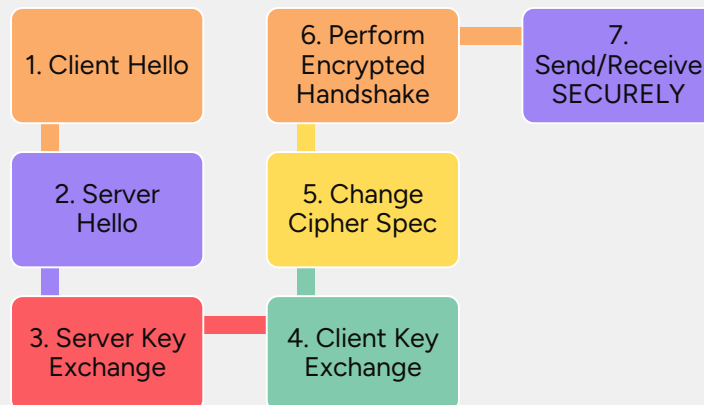
See the chapter appendix for a `socketserver` example.

SSL – quick recap

- **Data Confidentiality:** Achieved through encryption to keep network data private.
- **Core Technologies:** Evolution from SSL (e.g., v3.0) to the modern TLS standard (e.g., TLS v1.3).
- **The Handshake Process:** Relies on Private Keys for data encryption and Certificates to verify client/server authenticity.
- **Python's ssl Module:** A standard library component that provides built-in encryption and peer authentication for sockets.
- **Versatile Implementation:** Supports both client and server architectures by "wrapping" standard sockets into secure SSL sockets.
- **Under the Hood:** Built on the OpenSSL library, meaning behaviours can vary slightly based on the underlying OS socket APIs.
- **Modern Best Practice:** Requires using an SSL Context (standard since Python 3.2) for secure configuration.

Please Note: exact details can vary depending on variances in the underlying OS Socket APIs.

SSL/TLS – handshake



QA

336

Establishing a secure, encrypted connection between a client and a server is achieved through a multi-step handshake protocol.

The SSL/TLS handshake process works as follows:

1. Client Hello

The process begins with the client, which sends a message specifying the SSL/TLS versions and cipher suites it supports, listed in order of preference.

2. Server Hello

The server reviews the client's options and selects the most suitable (usually the strongest) version and cipher suite that both sides support, then informs the client.

3. Server Key Exchange

Once both parties agree on the protocol and cipher suite, the server provides the necessary credentials (if required by the chosen suite), including:

1. **Certificate:** Verified by the client to confirm the server's identity.
2. **Public key and signature:** Typically included in the certificate. The public key is used for encryption, while the server keeps the private key secure for decryption.
3. *Optionally*, the server may request a certificate from the client as well, enabling mutual authentication (mutual TLS).

4. Client Key Exchange

The client responds with key exchange information, using either RSA or Diffie-Hellman methods.

5. Change Cipher Spec

Both client and server send a message indicating that future communication will use the agreed encryption settings and keys.

6. Encrypted Handshake

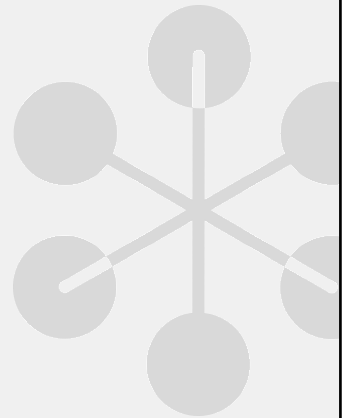
Each side sends encrypted messages to verify that the handshake process and key exchange were successful.

7. Secure Communication

With the handshake complete, a secure session is established, allowing both the client and server to exchange data safely.

Creating an SSL certificate for testing

- **Testing Requirements:** Creating both server and client certificates is essential for fully testing a Python SSL connection.
- **The Tools:** Utilize the `openssl` command-line utility, which is readily available across Linux, Windows, and other major operating systems
- **Windows Installation:** Windows users can obtain the binary from the Win32OpenSSL distribution site ([Win32/Win64 OpenSSL Installer for Windows - Shining Light Productions](#))
- **Certificate Authority:** While production environments require certificates from a trusted Certificate Authority (CA), self-signed certificates are perfectly adequate for development and testing.
- **Mutual TLS (mTLS):** Generating both client and server certificates establishes a two-way trust, enabling secure mutual authentication.



QA

337

Note: You can generate self-signed keys by using Python's cryptography module.

Creating server certificate and private key

Creating the Server's certificate using OpenSSL tool:

```
openssl req -new -newkey rsa:2048 -days 365 -nodes -x509 -keyout
server.key -out server.crt

Generating a RSA private key
.....+++++
.+++++
writing new private key to 'server.key'
-----

You are about to be asked to enter information that will be incorporated
into your certificate request.  What you are about to enter is what is
called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.

-----
Country Name (2 letter code) [AU]:
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:QA
Organizational Unit Name (eg, section) []:QA
Common Name (e.g. server FQDN or YOUR name) []:qa-example.com
Email Address []:mark.fishpool@qa.com
```

QA

338

- **Generating the Server Certificate:** Use the interactive OpenSSL interface, or execute a non-interactive command directly from your terminal using `openssl req ....`
- **Critical Step:** Make sure to note the **Common Name (CN)** being used—in this case, "qa-example.com".
- **File Management:** Keep track of where the resulting certificate file is saved on your filesystem, as you will need to reference this path in your Python code.
- **Next Phase:** Once the server certificate is ready, the next step is to generate the client certificate.
- You can download and install `openssl` from here:
<https://slproweb.com/products/Win32openssl.html>

Running command will generate the following files in the folder the command is run in:

server.crt

server.key

Creating client certificate and private key

Creating the Client's certificate using OpenSSL tool:

```
openssl req -new -newkey rsa:2048 -days 365 -nodes -x509 -keyout
client.key -out client.crt

Generating a RSA private key
.....+++++
.....+++++
writing new private key to 'client.key'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a
DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:.
Organizational Unit Name (eg, section) []:.
Common Name (e.g. server FQDN or YOUR name) []:.
Email Address []:.
```

QA

339

Generating the Client Certificate: Use the interactive OpenSSL interface, or execute a non-interactive command directly from your terminal using `openssl req ....`

File Management: Keep track of the resulting certificate's filesystem location, as you will need to reference this path when writing the client-side Python code.

Running command will generate the following files in the folder the command is run in:

client.crt
client.key

SSL client

```
from socket import socket, AF_INET, SOCK_STREAM, SHUT_RDWR
import ssl
import time

ipv4_tuple = '127.0.0.1', 8000
# Server details and certificate
server_sni_hostname = 'qa-example.com'
server_cert = r'C:\certs\server.crt'
# Client certificate and key
client_cert = r'C:\certs\client.crt'
client_key = r'C:\certs\client.key'

# Create SSL context
my_context = ssl.create_default_context(ssl.Purpose.SERVER_AUTH, cafile=server_cert)
my_context.load_cert_chain(certfile=client_cert, keyfile=client_key)
# Create normal socket
my_socket = socket(AF_INET, SOCK_STREAM, 0)
# Wrap socket in SSL
ssl_connection = my_context.wrap_socket(my_socket, server_side=False, server_hostname=server_sni_hostname)

# Connect and transmit
ssl_connection.connect(ipv4_tuple)
msg = b"What hath God wrought!"
print(f"Sending message: {msg}")
ssl_connection.send(msg, 0)
print("message successfully sent")
time.sleep(0.1) # critical to cure race condition
ssl_connection.shutdown(SHUT_RDWR)
ssl_connection.close()
print("Connection closed")
```

SSL server (1)

```
from socket import socket, AF_INET, SOCK_STREAM
import ssl

ipv4_tuple = '127.0.0.1', 8000

# Server details, certificate and key
server_hostname = 'qa-example.com'
server_key = r'C:\certs\server.key'
server_cert = r'C:\certs\server.crt'

# Client certificate
client_cert = r'C:\certs\client.crt'

# Create SSL context
my_context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
my_context.verify_mode = ssl.CERT_REQUIRED
my_context.load_cert_chain(certfile=server_cert, keyfile=server_key)
my_context.load_verify_locations(cafile=client_cert)
my_context.options |= ssl.OP_NO_TLSv1 | ssl.OP_NO_TLSv1_1
my_context.set_ciphers('EECDH+AESGCM:EDH+AESGCM:AES256+EECDH:AES256+EDH')

# Create normal socket
my_socket = socket(AF_INET, SOCK_STREAM, 0)
my_socket.bind(ipv4_tuple)
my_socket.listen(1)
```

The server (typically) requires more setup than the client!

Note the use of available private keys (.key) and public certificates (.crt) files.

QA

341

Configuring the SSL Context

The `create_default_context()` function generates a new `SSLContext` object pre-configured with secure default settings tailored to a specific purpose, such as client or server authentication.

Customizing Handshake Behaviour and Protocol Versions

Developers can modify various context settings to alter the TLS/SSL handshake behaviour:

- **Protocol Version Control (The Legacy Way):** Historically, developers modified the context's bitwise options (`OP_ALL`) and used OR operations to disable specific protocols (e.g., blocking TLSv1 and TLSv1.1 connections). While you will frequently encounter this in legacy codebases, **this approach is deprecated**.
- **The Modern Way:** A much cleaner approach is to use the `SSLContext.minimum_version` and `SSLContext.maximum_version` properties. Note that this requires the Python `ssl` module to be compiled with OpenSSL 1.1.0g or later.
- **Protocol Selection:** Utilizing `ssl.PROTOCOL_TLS` is generally the easiest option. Despite its somewhat exclusive-sounding name, it automatically negotiates the highest mutually supported protocol version between the client and the server.

Enforcing Mutual Authentication

To secure the connection further, the text highlights two critical components:

- **Mandatory Validation:** Setting the `verify_mode` property to `CERT_REQUIRED`

enforces strict, mandatory TLS client certificate authentication. Under this configuration, the server demands a certificate from the client, which must present a valid, trusted one to proceed.

- **Loading Trusted Certificates:** The `load_verify_locations()` method features three optional parameters (defaulting to None). Passing a file path to the `cafile` parameter allows you to point to a `.pem` file containing concatenated Certificate Authority (CA) certificates. This file is what the system uses to actually validate the peer's certificate when `verify_mode` is active.

Cipher Suite Selection

The `set_ciphers()` method restricts the available cryptographic ciphers for the context. It accepts a formatted string adhering strictly to the OpenSSL cipher list syntax.

Documentation Note: The exhaustive (and quite extensive) list of these cipher strings can be found in the official OpenSSL documentation (: <https://www.openssl.org/docs/man1.1.1/man1/ciphers.html>). While the standard reference manual covers OpenSSL 1.1.1, the page typically maintains links to the most up-to-date manual pages for newer versions.

SSL server (2)

server

```
while True:
    print(f"Waiting for connection on port {ipv4_tuple[1]}")
    new_socket_object, r_address = my_socket.accept()
    print(f"Connection made from: {r_address}")
    ssl_connection = None
    try:
        ssl_connection = my_context.wrap_socket(new_socket_object, server_side=True)
        received_data = ssl_connection.recv(4096, 0)
        if not received_data:
            print("Client closed connection")
            continue
        decoded_received_data = received_data.decode("utf-8")
        print(f"Data received: '{decoded_received_data}'")
    except (ssl.SSLError, ConnectionAbortedError, ConnectionResetError) as e:
        print(f"Connection error: {e}")
    finally:
        if ssl_connection:
            try:
                ssl_connection.shutdown(socket.SHUT_RDWR)
            except Exception:
                pass
            ssl_connection.close()
            new_socket_object.close()
        print("Connection closed")
```

This part of the server's code is more familiar!
Note the wrapping of the socket within a try...except block; many different exceptions can occur.

QA

342

Note on Error Handling: While various exceptions can occur, catching an `SSLError` essentially surfaces an error generated directly by the underlying OpenSSL implementation. This typically points to a specific encryption or authentication failure, with the error message "payload" copied directly from OpenSSL.

See also...

Standard library modules with network-related support:

- `asynchat` Asynchronous socket command/response handler
- `asyncore` Asynchronous socket handler (don't block while handling requests)
- `http.cookies` HTTP state management
- `select` Waiting for I/O completion
- `socket` Low-level networking interface
- `socketserver` A framework for network servers
- `SimpleXMLRPCServer` XML-RPC server built on top of `socketserver`
- `ssl` A TLS/SSL wrapper for socket objects
- `urllib` URL handling modules `urllib.request`, `urllib.error`, `urllib.parse`, `urllib.robotparser`
- `urlparse` Parse URLs into components

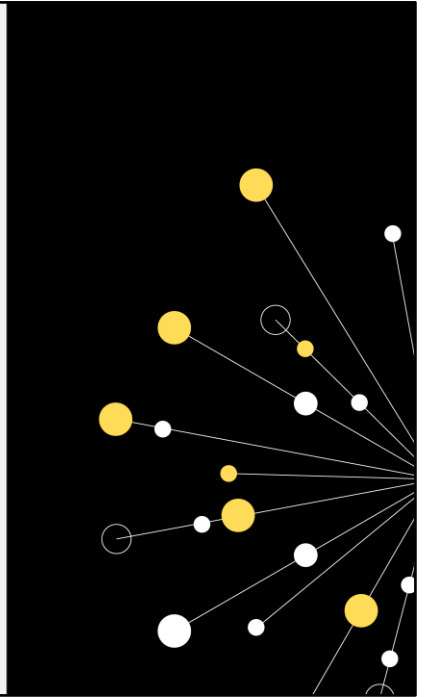
Exercise EG09: Networking with Sockets

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA

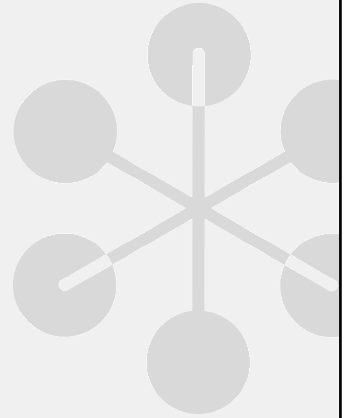


Suggested time: 40-60 mins.

Activity includes differentiated tasks.

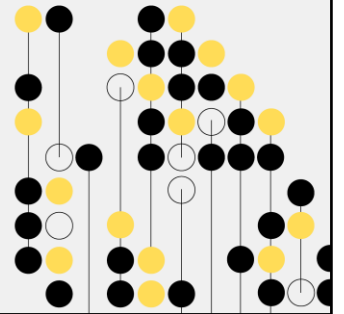
Summary

- Sockets are an industry standard for Inter-Networking
- Various protocols supported
- Easy to communicate with different operating systems
- Socket exist within a communications domain
- Protocols
- Naming conventions
- Sockets provide a duplex communication channel
- Connection-oriented
- Connectionless
- The socketserver module makes server creation much simpler
- SSL best practice (Python ^3.2) is to create SSLContext instances



Any questions?

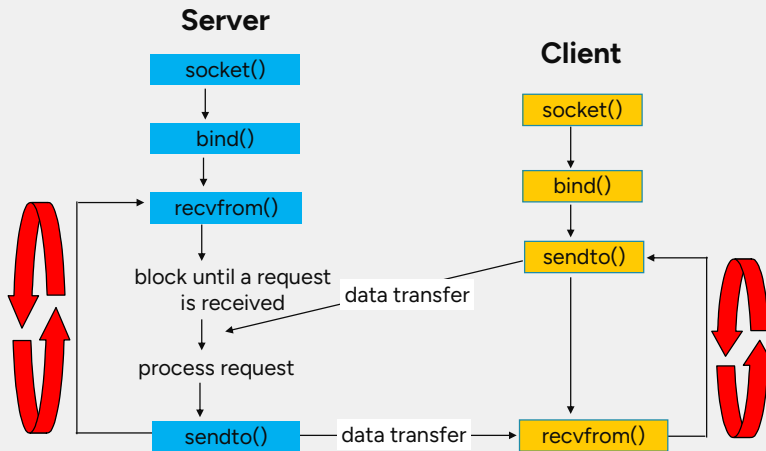
QA



Appendices



Using connectionless sockets



QA

348

While connection-based services are the norm, some services are based on the use of datagram sockets.

Datagram sockets lead to a slightly different pattern of interaction between the client and server processes.

The server must still create a socket and bind it to a local name, as described for stream sockets. However, no firm connection is made in this case; the server waits for a request by calling the `recvfrom()` method. This normally blocks the process until a request comes in from a client. The request will contain the socket name of the client, so that after the request has been processed, a reply can be sent to the client using the `sendto()` call, specifying this name.

The client process creates a socket through which to communicate with the server. It must also create a socket for itself and bind a local name to that socket. Again, this bind can be done sensibly by the system, as described earlier. The client socket must be bound so that the server process has a name to which it can send the reply.

The request is made of the server using the `sendto()` call, and the reply received using `recvfrom()`. Since there is no connection between the processes, there is no need to 'shut down' the link.

Byte ordering

- **Different processors have different byte ordering**
 - May be different to network order (big endian)
- **IP addresses or port numbers passed to or from a socket API are in network order**
- **Sometimes translation will need to be done from host order to network order**
 - `htons()` and `ntohs()`
 - `htonl()` and `ntohl()`
- **Python abstracts these differences for socket methods**
 - But you might have to translate application data

QA

349

In a distributed environment, assumptions cannot be made about the type of remote computer with which communication will take place. Different processors have different byte ordering, thus care must be taken to ensure correct orientation.

When four parts of a dotted IP address a.b.c.d are specified, each part is interpreted as a byte of data and assigned, from left to right, to the four bytes of an Internet address. Note that when an Internet address is viewed as a 32-bit integer quantity on the Intel architecture, the bytes referred to above appear as "d.c.b.a". That is, the bytes on an Intel processor are ordered from right to left.

IP addresses or port numbers passed to or from a socket API are in network order (Big-Endian), however the Python socket module abstracts this so it is not normally visible.

Conversion routines are supplied:

`ntohl()/ntohs()` - convert long/short from network order to host order

`htonl()/htons()` - convert long/short from host order to network order

Other useful routines are:

`inet_addr()` - convert IP address from dotted-string format to network ordered 32-bit value

`inet_ntoa()` - convert IP address from network-ordered 32-bit value to dotted-string format

Blocking vs. non-blocking

- **By default socket calls will block**
 - For example, `recv()` waits for available data
- `s.setblocking(0)` can mark a socket as non-blocking
- **A non-blocking socket will raise an `error` exception if:**
 - A `recv` gets no data
 - A `send` has no reader
- **Timeout mode is similar - `s.settimeout(seconds)`**
 - Replaces `select()` in the C API
 - Raises a `timeout` exception if triggered
 - Timeout can also affect an `s.connect()`

QA

350

By default, socket calls will block, if necessary, until the requested operation is complete or fails. This is obvious in the case of, say, `recv()` where one thread waits on another to send data. It is also well understood that a server `accept()` or a client `connect()` may block before a connection is established. It is not quite so obvious that the database functions will also block, especially if network interaction is required to resolve names, as is the case if the DNS service is used.

This blocking behaviour is often unacceptable for some applications that could be getting on with other things instead of being suspended for an arbitrarily long period pending a network event. To support such applications, sockets can be set to non-blocking or timeout mode. In this mode, any operation on a socket that would cause a call to block for an extended period of time, returns with an error exception. This indicates that the requested operation is pending, and the socket can be inspected at a later date to test whether the operation has completed.

Of course, in a pre-emptive multitasking environment it is possible to just start another thread that will call a socket API that is likely to block. However, there are no API extensions that cope with multi-threading, so

care must be taken to synchronise access to a socket between threads. Failure to do so may lead to unpredictable results, much as it would with, say file I/O. For example, with two 'simultaneous' calls to `send()`, there is no guarantee of the order the data will be sent in.

Socket server framework example (1)

```
import socketserver
import threading

class MyRequestHandler(socketserver.BaseRequestHandler):
    def handle(self):
        data = self.request.recv(1024)
        echo_msg = data.decode("utf-8")
        print(f"Received: {echo_msg}")

class MyServer(socketserver.TCPServer):
    def __init__(self, server_address, handler_class=MyRequestHandler):
        print("Server initialised")
        socketserver.TCPServer.__init__(self, server_address, handler_class)

    def serve_forever(self):
        print("Listening for requests...")
        while True:
            self.handle_request()

    def handle_request(self):
        return socketserver.TCPServer.handle_request(self)

    def server_close(self):
        print("Server shutdown")
        return socketserver.TCPServer.server_close(self)
```

QA

351

In order to create a server you need to complete 4 steps:

1. Create a new class which can handle requests – do this by extending the `BaseRequestHandler` class and overriding its `handle()` method.
 - `setup()` and `finish()` methods are also available (pre- and post-handle)
2. Create a new class which extends one of the four available server classes (in this example, we've extended the `TCPServer`); we've coded four key methods here.
3. Instantiate your custom Server class, it will need:
 - server's address (note. the Operating System kernel will allocate an available port automatically if 0 is supplied as the 2nd argument of the tuple – see next slide)
 - your customer request handler class
4. You can then use `handle_request()` method for one request, or use `serve_forever()` for many.

As you can see, this worked example also uses a context manager.

Note: The `serve_forever()` method will continue to handle requests until an explicit `shutdown()` request is received and this has to be in a different thread otherwise it will deadlock.

Socket server framework example (2)

```
if __name__ == '__main__':  
    import socket  
    import threading  
  
    ip_address = ('localhost', 8000)  
    with MyServer(ip_address, MyRequestHandler) as my_server:  
        ip, port = my_server.server_address  
        print(f"IP: {ip} is listening on PORT: {port}")  
        my_server.serve_forever()
```

```
python serversocketexample.py
```

```
Server initialised  
IP: 127.0.0.1 is listening on PORT: 8000  
Listening for requests...  
Received: What hath God wrought!
```

QA

352

This version was tested with the earlier socket-based client code, adjusting its socket to the allocated by the socketserver's TCPServer initialisation (in this example, 14337).

Other Server methods are available for overriding, e.g.

- `verify_request` - return True/False whether a request should be processed, e.g. based on a filtered IP range
- `process_request` - potentially create a new thread to process the request

SSL TEST – client and server

client

```
python ssl_client.py
Sending message: b'What hath God wrought!'
message successfully sent
Connection closed
```

server

```
python ssl_server.py
Server initialised
IP: 127.0.0.1 is listening on PORT: 8000
Listening for requests...
Received: What hath God wrought!
Waiting for connection on port 1066
```

QA

353

However, should we try to set the target server hostname incorrectly in the client, e.g. qa2-example.com, the following run-time error will result as a result of being unable to verify the server's identity successfully:

```
Traceback (most recent call last):
  File "ssl_client.py", line 26, in <module>
    ssl_connection.connect(ipv4_tuple)
  File "C:\Users\Mark\AppData\Local\Programs\Python\Python38-32\lib\ssl.py", line 1342, in connect
    self._real_connect(addr, False)
  File "C:\Users\Mark\AppData\Local\Programs\Python\Python38-32\lib\ssl.py", line 1333, in _real_connect
    self.do_handshake()
  File "C:\Users\Mark\AppData\Local\Programs\Python\Python38-32\lib\ssl.py", line 1309, in do_handshake
    self._sslobj.do_handshake()
ssl.SSLCertVerificationError: [SSL:
CERTIFICATE_VERIFY_FAILED] certificate verify failed:
Hostname mismatch, certificate is not valid for 'qa2-
example.com'. (_ssl.c:1108)
```

Client/Server TLSv1.3 handshake:

[illegible]**QA**

354

These packet capture extracts from Wireshark using Npcap Lookback Adapter to demonstrate the TLS/SSL handshake and data transfer between our test client/server setup, verifying that the connection is indeed secured.

Other helper modules

- Classic mechanism to handle multiple sockets in one thread
 - Returns a list of sockets that are readable, writeable or in error
 - Alternative mechanism to non-blocking sockets
 - See also the `asyncore` module
-
- Message oriented API
 - Includes authentication support

QA

355

Before threads were popular, particularly on UNIX, **select** was commonly used from C to achieve multiplexing. That is, we will process those sockets which have outstanding requests on them, while waiting for others. In Python, `select` is a separate library module.

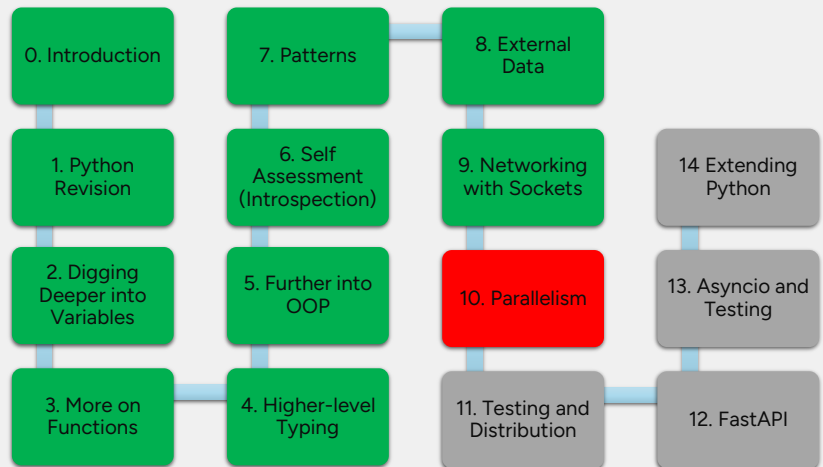
See also, in PyPi: `iptools`, `lpy`, `netaddr`

Intermediate Python

Parallelism



Module roadmap



QA

Parallelism

Learning outcomes:

- Apply various Python techniques to solve problems using parallel processing techniques

Prerequisites:

- Delegates are required to have knowledge and skills relating to basic Python programming, including data structures, functions, loops and flow control.

QA

358

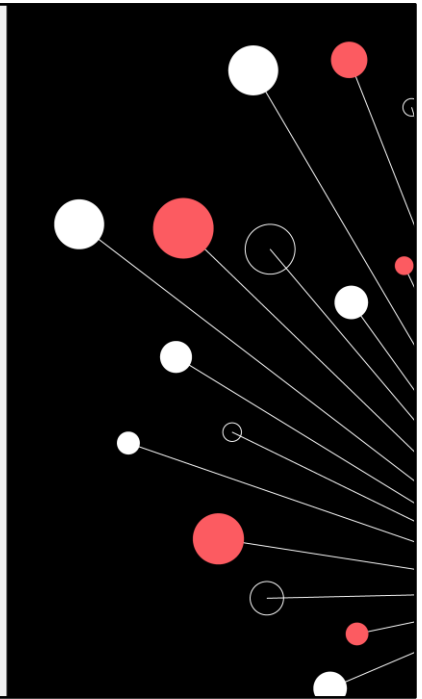
Appendices include details on simple database access, XML and reading binary files created in other programming languages (e.g. C)

Learning Objectives

By the end of this session, you should be able to:

- Define core concurrency and parallelism concepts, including processes and threads
- Explain the Fork/Join model and its role in parallel computation
- Identify and use key Python concurrency modules (e.g., multiprocessing, threading)
- Create and manage processes, including returning values and sharing data
- Compare shared-memory and message-passing approaches (Queue, Pipe, Manager)
- Determine when to use threads versus processes in different scenarios
- Describe common challenges in threading, including synchronization issues
- Use high-level concurrency abstractions such as Concurrent Futures and Pools

QA



General definitions

Parallelism

- Able to run multiple operations at the same time

Multiprocessing

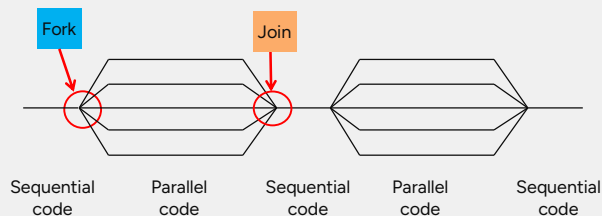
- A way to achieve parallelism by running code on multiple CPUs or multiple CPU cores

Concurrency

- Manage multiple tasks at the same time by interleaving their execution

The Fork/Join model

- Only the primary thread creates new threads, minimizing the synchronization required
- Parallelization can be added incrementally, one code block at a time
- Parallel parts of the program can be isolated for testing and benchmarking
- The alternative is the "General Threading Model," which is more flexible for complex tasks.
 - Any thread can create another thread.
 - Makes the model harder to analyse.



QA

361

The Fork/Join model is unrelated to the UNIX fork API call; rather, it refers to a straightforward approach where there is a single point to fork tasks and a single point where all tasks join back together. Despite its simplicity, this model is robust and highly effective. It is important to recognize that some parts of an application must remain sequential, which is unavoidable and highlights the fact that no matter how many CPU cores are available, some tasks—such as initialization and summation—can only run on one core. The Fork/Join model makes these serialized sections clear, offering opportunities for optimization or reduction.

In contrast, the General Threading Model can be more chaotic, both in terms of structure and behaviour. This model allows worker threads to create additional threads, which complete independently. While this flexibility is powerful, it often makes debugging and optimization challenging. Although the Fork/Join model is not necessarily easy to debug, it is generally less difficult to manage compared to the General Threading Model.

Processes vs. threads

Process	Thread
• A running instance of a program • A container that holds threads of execution • Identified by a unique Process ID (PID) at runtime	• A unit of execution <i>within</i> a process • Every thread in the same process shares: • Core information: PID, UID*, GID, Env. block, FD table, etc. • Off-stack (global) data • Threads have their own: • Program counter • Thread ID (TID) • Registers • Thread Specific Memory

QA

362

Most programmers are familiar with the concept of a process, but threads may be less familiar. Even if you have encountered threads in languages such as C# or Java, Python's approach to multithreading has some unique characteristics.

One of the main advantages of threads is that they share the same memory space, allowing data to be accessed and exchanged efficiently between them. However, this is also their primary drawback. Because multiple threads can access and modify the same data, it can be difficult to predict when a shared resource might change.

As a result, multithreaded programming requires careful synchronisation to ensure that shared data remains consistent and to prevent race conditions. While synchronisation mechanisms help manage access to shared resources, they introduce additional complexity and performance overhead.

*Security: Windows supports thread impersonation, which allows a thread to temporarily operate under a different Security Identifier (SID) before reverting to its original security context.

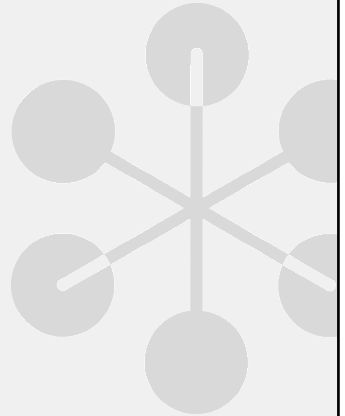
Key Python modules

Python provides two main libraries for concurrent execution:

- **threading**
 - a high-level interface for creating and managing threads.
- **multiprocessing**
 - an API for creating and managing processes with a design similar to the threading module.

The choice between these libraries depends on the type of workload being performed.

We will begin by exploring multiprocessing, as it is often the preferred approach when using cPython. The reasons for this will become clear later.



We will look at both libraries, compare their relative merits before exploring another library that, for good reason, is probably even more popular.

Multiprocessing example – starting processes

```
from multiprocessing import Process
import time
import os
import random

def race_func(*args):
    print(args[0])
    sleep_duration = random.random() * 5
    print(f"{args[0]} sleeping for {round(sleep_duration, 2)} secs")
    time.sleep(sleep_duration)
    print(f"{args[0]} is awake!")
    print(f"parent process id: {os.getppid()}")
    print(f"process id: {os.getpid()}")
    time.sleep(2)

if __name__ == "__main__":
    race_func("exec from main")
    my_proc1 = Process(target=race_func, args=("1st func",))
    my_proc2 = Process(target=race_func, args=("2nd func",))
    my_proc1.start()
    my_proc2.start()
    my_proc1.join()
    my_proc2.join()
    print("Finished")
```

```
C:\Python\Python38-32>python procdemo1.py
exec from main
exec from main sleeping for 4.44 secs
exec from main is awake!
parent process id: 9584
process id: 2952
2nd func
2nd func sleeping for 0.34 secs
1st func
1st func sleeping for 2.13 secs
2nd func is awake!
parent process id: 2952
process id: 11432
1st func is awake!
parent process id: 2952
process id: 7968
Finished
```

QA

364

This code has been tested directly from the command prompt (process 9584 in this example).

Within main, two processes are created. Each process requires a target, which is a reference to a function, and optionally a tuple of arguments to pass to that function. It is crucial not to accidentally call the function with arguments when setting the target, as this would assign the target to the result of the function call rather than the function itself.

Observe the relationships among the processes, their process IDs (PIDs), and parent process IDs (PPIDs).

Processes are started using the `start()` method, and the `join()` method is used to wait for the processes to finish before the main code block continues. This flow is essentially the opposite of forking.

Here's a teaser: In Microsoft Windows, all Python code that uses the multiprocessing module must be guarded by the `if __name__ == "__main__"` statement. Do you know why?

Multiprocessing example – multiple processes

```
from multiprocessing import Process
import time
import os
import random

def race_func(*args):
    print(args[0])
    sleep_duration = random.random() * 5
    print(f"{args[0]} sleeping for {round(sleep_duration, 2)} secs")
    time.sleep(sleep_duration)
    print(f"{args[0]} is awake!")
    print(f"parent process id: {os.getppid()}")
    print(f"process id: {os.getpid()}")
    time.sleep(2)

if __name__ == "__main__":
    my_processes = []
    race_func("exec from main")
    for i in range(2):
        temp_proc = Process(target=race_func, args=(f"process {i}",))
        temp_proc.start()
        my_processes.append(temp_proc)
    for each_process in my_processes:
        each_process.join()
    print("Finished")
```

```
C:\Python\Python38-32>python procdemo2.py
exec from main
exec from main sleeping for 2.55 secs
exec from main is awake!
parent process id: 9584
process id: 1372
process 0
process 0 sleeping for 3.25 secs
process 1
process 1 sleeping for 0.74 secs
process 1 is awake!
parent process id: 1372
process id: 4800
process 0 is awake!
parent process id: 1372
process id: 16008
Finished
```

QA

365

This is a similar example, but it uses loops to create the processes, append them to a list of Process objects and then another loop to join them.

Returning values from processes

Returning values from parallel processes requires alternative approaches, as the standard method of storing a function's return value is unavailable. Options include:

- **Using shared data** – pass a data collection as an argument for the function to append to, or reference a global one (not recommended)
- **Alternatives:**
 - Shared Memory Map
 - Manager object
 - Queue object
 - Pipe object / Pipe list
 - Pool object

QA

366

We will look at each approach in turn by working through a simple processing example.

Returning values from process – using shared memory map

```
from multiprocessing import Process, Array

def apply_interest(account_index, balances):
    # Simulate a banking operation:
    # Apply 5% interest to the account balance at the given index.
    current_balance = balances[account_index]
    updated_balance = int(current_balance * 1.05) # apply 5% interest
    balances[account_index] = updated_balance

if __name__ == "__main__":
    # Initial account balances (shared memory)
    initial_balances = [1000, 1500, 2000, ... 4000, 4500, 5000, 5500]
    balances = Array('i', initial_balances) # shared integer array
    processes = []

    for i in range(len(balances)):
        p = Process(target=apply_interest, args=(i, balances))
        processes.append(p)
        p.start()

    for p in processes:
        p.join()

    print(f"Updated balances: {list(balances)}")

Updated balances: [1050, 1575, 2100, ... 4200, 4725, 5250, 5775]
```

367

This example illustrates how to use a shared memory map with the Array class.

Behind the scenes, each process accesses a synchronized wrapper around a sharedctypes.c_long_Array_10 object. The first argument in the constructor, 'i', is the typecode for a signed integer, while the second argument is a list of balances. Each process updates the shared memory map by applying a 5% interest and storing the new balance at the appropriate index within the array.

For display purposes, this shared object is finally converted into a list.

It is important to note multiprocessing.Array objects only support primitive types so creating and passing instances of classes (such as BankAccount objects) is not supported.

Note: that the Array constructor includes a named-parameter-only argument called 'lock'. By default, this is set to True, which creates a new lock object to synchronize access to the data. If set to False, the memory map may not be safe to use across multiple processes.

To find out more see the following:

Shared ctype objects

<https://docs.python.org/3.8/library/multiprocessing.html#shared-ctypes-objects>

ctypes foreign function library

<https://docs.python.org/3.8/library/ctypes.html#module-ctypes>

Returning values from process – using a manager

```
from multiprocessing import Process, Manager

def apply_interest(account_index, results_dict, initial_balances):
    #Apply 5% interest and store result in a shared dictionary.
    current_balance = initial_balances[account_index]
    updated_balance = int(current_balance * 1.05)
    results_dict[account_index] = updated_balance

if __name__ == "__main__":
    initial_balances = [1000, 1500, 2000, ...4000, 4500, 5000, 5500]

    with Manager() as manager:
        results = manager.dict() # shared dictionary
        processes = []

        for i in range(len(initial_balances)):
            p = Process(target=apply_interest,
                        args=(i, results, initial_balances))
            processes.append(p)
            p.start()

        for p in processes:
            p.join()

    print(f"Raw dictionary (likely unordered):\n{dict(results)}")
```

Raw dictionary (likely unordered):
{6: 4200, 5: 3675, 9: 5775, 3: 2625, 2: 2100, 1: 1575, 4: 3150, 0: 1050, 8: 5250, 7: 4725}

68

In this example, a Manager object is used to control a server process that holds Python objects like lists and dictionaries, allowing other processes to manipulate these objects via proxies. Notably, this capability can extend to processes running on different machines over a network, with data serialized using Pickle—though this serialization can introduce bottlenecks or limitations.

The Manager's `dict()` method creates a shared dictionary object and returns a proxy to it. This proxy is passed as an argument to the target function, where it is accessed to store the `updated_balance` of each value using its corresponding key.

The dictionary ultimately contains all the results. Due to the asynchronous way it is populated, the dictionary is usually unsorted, but this does not pose an issue since each key correctly corresponds to its value.

As usual, references to each process are stored in a list and joined to signal the completion of processing and the continuation of the main logic.

Queue objects

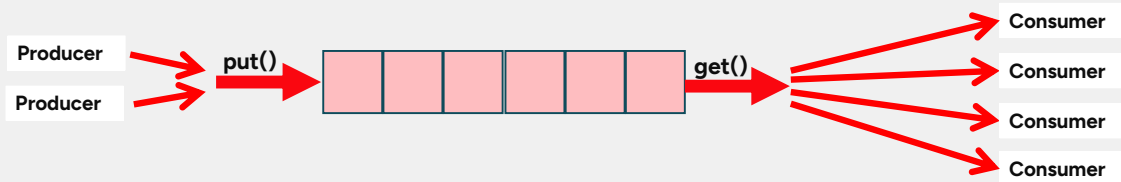
Producer writes to the queue with `Queue.put()`

- Will block (by default) if the queue is full

Consumer reads from the queue with `Queue.get()`

- Will block (by default) if the queue is empty

Queues have their own `join()`



QA

369

The Queue/queue module supplies a serialised communication mechanism between processes. All the locking (for synchronization) is done for us using abstracted shared semaphores - we do not need to worry about atomicity and other nasty details - every operation is atomic. These FIFO structures support multiple producers and consumers and are based on the generic `queue.Queue` class within the standard library. Producers add to the queue using the `put()` method and a consumer removes from the queue using the `get()` method. Queues are ideal for the producer-consumer module, where one set of processes adds data items into the queue and another set removes and processes them. One of the good things about queues is that it does not matter if the data items take different lengths of time to process, each "thread" gets the next item from the queue when it has nothing else to do.

Notes: There is an optional timeout on `put()` and `get()`, e.g.

```
put(obj[, block[, timeout]])
```

```
get([block[, timeout]])
```

`ValueError` is raised if the queue is closed and either a `get()` or a `put()` is attempted

Killing a process programmatically (via `os.kill()` or the `Process.terminate()`) is likely to corrupt a queue.

On the next slide we'll take a look at an adapted version of the interest adding code that uses the multiprocessing queue.

Returning values from process – using a queue

```
from multiprocessing import Process, Queue

def apply_interest(account_index, result_queue, initial_balances):
    # Apply 5% interest and send result back via queue.
    current_balance = initial_balances[account_index]
    updated_balance = int(current_balance * 1.05)
    result_queue.put((account_index, updated_balance))

if __name__ == "__main__":
    initial_balances = [1000, 1500, 2000, ...4000, 4500, 5000, 5500]
    result_queue = Queue()
    processes = []
    for i in range(len(initial_balances)):
        p = Process(target=apply_interest,
                    args=(i, result_queue, initial_balances))
        processes.append(p)
        p.start()

    results = {}
    for _ in range(len(initial_balances)):
        key, value = result_queue.get()
        results[key] = value

    for p in processes:
        p.join()

    print(f"Raw dictionary (likely unordered): {results}")
```

Raw dictionary (likely unordered):
{0: 1050, 2: 2100, 1: 1575, 9: 5775, 7: 4725,
5: 3675, 6: 4200, 3: 2625, 4: 3150, 8: 5250}

QA

370

Note the use of:

```
for _ in range(len(initial_balances)):
    key, value = result_queue.get()
```

Which ensures:

- The code retrieves **exactly one result per process**
- No deadlocks or missed data

With a Queue processes send messages back to the parent instead of mutating shared state. This is often preferred in real systems because it avoids race conditions and locking complexity.

Returning values from process – using a pipe

```
from multiprocessing import Process, Pipe

def apply_interest(account_index, conn, initial_balances):
    # Apply 5% interest and send result back via pipe.
    current_balance = initial_balances[account_index]
    updated_balance = int(current_balance * 1.05)
    conn.send((account_index, updated_balance))
    conn.close()

if __name__ == "__main__":
    initial_balances = [1000, 1500, 2000, ...4000, 4500, 5000, 5500]
    processes = []
    parent_conns = []

    for i in range(len(initial_balances)):
        parent_conn, child_conn = Pipe()
        p = Process(target=apply_interest, args=(i, child_conn, initial_balances))
        processes.append(p)
        parent_conns.append(parent_conn)
        p.start()

    results = {}
    for conn in parent_conns:
        key, value = conn.recv()
        results[key] = value
        conn.close()

    for p in processes:
        p.join()
    print(f"Raw dictionary (Ordered!): {results}")
```

```
Raw dictionary (Ordered!): {0: 1050, 1: 1575, 2: 2100, 3: 2625, 4: 3150, 5: 3675, 6: 4200, 7: 4725, 8: 5250, 9: 5775}
```

QA

371

A Python pipe consists of a pair of Connection objects representing the "send" and "receive" ends. On creation, its constructor accepts a Boolean argument:

- Pipe(False) — unidirectional: the 1st connection receives only, the 2nd sends only
- Pipe(True) — bidirectional (default)

Two key methods:

- send() — sends an object to the other end of the connection
- recv() — returns the sent object; blocks until something is available to receive

Constraints:

- Objects must be pickleable
- Size limit of ~32 MiB (OS-dependent)

Why one pipe per process? Data can become corrupted if two processes attempt to read or write from the same end of a pipe simultaneously. However, there is no issue with different processes accessing different ends of the same pipe at the same time.

Queues vs Pipes:

- Queues abstract synchronisation between processes, making them both process- and thread-safe — at the cost of performance

- Since Queue is built on top of Pipe, it carries an extra performance overhead

When to use each:

- Use a **Queue** when multiple processes need to communicate
- Use a **Pipe** when performance is a priority

Returning values from process – using a pool

```
from multiprocessing import Pool

def apply_interest(args):
    # Apply 5% interest and return result.
    account_index, initial_balances = args
    current_balance = initial_balances[account_index]
    updated_balance = int(current_balance * 1.05)
    return (account_index, updated_balance)

if __name__ == "__main__":
    initial_balances = [1000, 1500, 2000, 2500, 3000,
                        3500, 4000, 4500, 5000, 5500]

    tasks = [(i, initial_balances) for i in range(len(initial_balances))]

    with Pool(processes=4) as pool:
        # Map runs apply_interest across all tasks
        results_list = pool.map(apply_interest, tasks)

        # Convert list of tuples into dictionary
        results = dict(results_list)

    print(f"Raw dictionary (Ordered!): {results}")
```

Raw dictionary (Ordered!): {0: 1050, 1: 1575, 2: 2100, 3: 2625, 4: 3150, 5: 3675, 6: 4200, 7: 4725, 8: 5250, 9: 5775}

QA

72

The Pool object provides a simple way to execute a function across multiple values simultaneously, using a pool of worker processes to handle distribution.

Key advantages:

- Clean and easy to understand
- Using a **Context Manager** minimises housekeeping
- Overall, an elegant and practical technique
- Results come back ordered. Use of the `map()` function guarantees the output order matches input order even though processing is parallel
 - You can use Pool's `imap_unordered()` method if an unordered result is wanted

For more see:

<https://docs.python.org/3.8/library/multiprocessing.html#using-a-pool-of-workers>

When to use threads

You can use threads when true parallel execution is not required.

- Often runs effectively on a single CPU core.
- Appropriate when the operating system can manage task switching through pre-emptive multitasking.
- Useful when one or more tasks spend time waiting for events or resources.
 - Helps maintain application responsiveness for users.
 - Well suited to I/O-bound operations, such as:
 - Printing documents
 - Reading from or writing to storage devices
 - Communicating over a network
- Best used when potential race conditions are understood and can be controlled through proper synchronisation.

Potential drawbacks of threads:

- Synchronisation mechanisms can introduce performance overhead.
- The Global Interpreter Lock (GIL) can limit parallel execution in CPython.
- Poorly designed synchronisation can lead to subtle and difficult-to-debug errors.

QA

373

The Global Interpreter Lock (GIL) is a mechanism in CPython that allows only one thread to execute Python bytecode at a time, even on a multi-core processor.

It exists to simplify memory management, make CPython's internal data structures thread-safer and reduce the complexity of the interpreter. For I/O-bound tasks (networking, file access, waiting for user input), threads work well because a thread can release the GIL while waiting but for CPU-bound tasks (heavy calculations), threads often do not improve performance because only one thread can execute Python code at a time. Despite the limitations imposed by the GIL, threads remain useful in Python, particularly for I/O-bound applications. For example, one thread can keep a graphical user interface responsive while another performs background tasks.

More generally, threads are effective whenever a task spends time waiting for external events or resources. During I/O operations—such as user interaction, file access, or network communication—the GIL is released, allowing other threads to run, even when the underlying operating system uses non-blocking I/O.

Simple threads

Python threading usually uses the threading module

- Call `threading.Thread(function)`

```
import random
from threading import Thread
import time

def myfunc(*args):
    time.sleep(random.random() * 5)
    print(f"From thread {args[0]}")

th1 = Thread(target=myfunc, args='1')
th2 = Thread(target=myfunc, args='2')
th1.start()
th2.start()
print("From main")
th1.join()
th2.join()
```

```
From main
From thread 2
From thread 1
```

?

- Or derive your own class derived from `threading.Thread`

QA

374

The Python **threading** module is a high-level interface based on the thread module. If you have used threads procedurally, for example Win32 threads or pthreads from C/C++, then you will be familiar with the procedural interface.

`thread.join()` does not return the value returned by the target function - you have to use some other mechanism to pass data back from a thread. Alternatively, you can derive your own class from the threading base class:

```
import random
import threading
import time

class MyThread(threading.Thread):
    def run(self):
        time.sleep(random.random() * 5)
        print("From thread", self.name)
        time.sleep(5)

th1 = MyThread()
th2 = MyThread()
th1.start()
th2.start()
print("From main")
```

```
th1.join()  
th2.join()
```

Python threads support various locking mechanisms: Condition, Event, Semaphore, Locks (and RLock), and Thread local data.

Threading object synchronization

There are several mechanisms available for coordinating threads

- **Condition variables** - for signalling between threads
- **Barrier objects** - used to pause execution until all participating threads reach a common point
- **Events** - for simple signalling between threads
- **Thread-local storage** - allows variables to appear global while actually being unique per thread
- **Locks** - similar to mutexes, with ownership semantics for ensuring safe access
- **Semaphores** - counting locks that limit concurrent access (e.g., allow up to three threads at once)
- **Timers** - comparable to waitable timers in Win32 and interval timers in UNIX systems

QA

375

The threading module provides a range of objects for thread synchronisation:

- **Condition Variables** — derived from the POSIX pthreads library; include a Lock to protect a predicate. Generally more complex to use than Events
- **Events** — simple to use; threads wait on an event until another thread releases them. Support a timeout parameter.
- **Thread Local Storage** — allows a global variable to be shared between functions within a thread, while remaining unique per thread. Primarily a workaround for converting single-threaded programs to multi-threaded ones — avoid global variables and this won't be needed.
- **Locks** — the basic locking mechanism; a thread either holds a lock or waits for it.
- **Semaphores** — unlike Locks, not "owned" by any thread. A limit is set on how many threads can hold a semaphore at once; additional threads wait until one is released
- **Timers** — useful for triggering functions at specific times or at set intervals

Synchronizing access to shared resources in a thread

One key issue with threading is concurrent access to shared resources such as standard output (stdout), files, and shared variables

- different threads can interrupt each other's access, causing data corruption.
- A fix is to apply a synchronisation technique, such as a Lock, to control access.

```
import random
from threading import Thread, Lock
import time

lock_stdout = Lock()

def myfunc(*args):
    lock_stdout.acquire()
    time.sleep(random.random() * 5)
    print(f'From thread {args[0]}')
    lock_stdout.release()

th1 = Thread(target=myfunc, args='1')
th2 = Thread(target=myfunc, args='2')
th1.start()
th2.start()
lock_stdout.acquire()
print("From main")
lock_stdout.release()
th1.join()
th2.join()
```

```
From thread 1
From thread 2
From main
```

QA

376

This demonstrates the use of Threading.Lock to create a Critical Section (a term from Windows referring to code that can only execute in one thread at a time). While a lock is held, no other thread can access the code it protects.

In this example, the lock_stdout object guards the stdout buffer, being acquired and released around each print() call.

The trouble with threads – the GIL

Threads are inherently difficult to work with:

- Sharing variables requires the use of locking mechanisms
- Subtle timing differences make debugging challenging

The Python Global Interpreter Lock (GIL):

- A MUTually EXclusive (mutex) lock that forces the interpreter to only run one thread at a time
- Stops two threads located in the same process from running lines of Python code simultaneously
- Threads are locked for approximately 100 bytecode instructions

Why does the GIL exist?

- Simplifies and protects the interpreter from race conditions, particularly around variable reference counts
- CPython's memory management is not thread-safe

Performance implications:

- Whilst improvements to the GIL strategy in Python 3.2 reduced the overhead it is still a performance bottleneck for multi-threaded code — especially CPU-bound workloads rather than I/O-bound ones

QA

377

Guido van Rossum once remarked:

“Unfortunately, for most developers, writing correct multithreaded programs is simply too difficult... even in Python.”

In CPython, when executing pure Python code, the interpreter periodically releases the Global Interpreter Lock (GIL)—typically after every hundred bytecode instructions. However, this behaviour depends on how the code is compiled into bytecode. For example, a complex mathematical operation might consist of a single bytecode instruction, meaning the GIL will *not* be released until that operation completes.

There is an important exception: C extension modules. These modules—whether built-in or third-party—can be designed to explicitly release the GIL while performing their work, allowing other threads to run concurrently. Not all Python implementations rely on the GIL and others sometimes allow for more flexibility in threading. Some notable examples include:

Jython – <https://www.jython.org/>

IronPython – <https://ironpython.net/>

Cython – <https://cython.org/>

It's also worth noting that Python is not alone in facing this challenge. For instance, Ruby has a similar mechanism called the Global VM Lock (GVL). Despite numerous attempts and discussions about removing or replacing

the GIL, no alternative has yet proven to offer a consistently better solution.

Further reading:

"Reworking the GIL" by Antoine Pitrou

<https://mail.python.org/pipermail/python-dev/2009-October/093321.html>

Providing abstraction: `concurrent.futures`

`concurrent.futures` provides a high-level interface for concurrent execution using a pool of thread or process workers:

- Introduced in Python 3.2 (see PEP 3148)
- Simple APIs reduce the need for manual thread/process management
- The API is consistent whether using threads or processes

Key components:

- Executor - an Abstract Base Class with two concrete implementations:
 - `ProcessPoolExecutor` — uses multiprocessing
 - `ThreadPoolExecutor` — uses multithreading
- `futures` — manages the results produced by workers

How it works:

- Tasks (function calls) are submitted to a pool of threads or processes
- `wait()` blocks until completion and returns the function's result
- `map()` provides a similar alternative

Considerations:

- Carries some overhead, but worthwhile for large workloads

QA

378

This module does not replace the existing `multiprocessing` or `threading` modules; instead, it provides a higher-level abstraction over them. In this context, the term "futures" refers not to upcoming Python releases, but to tasks that will execute asynchronously at a later time.

The primary classes are `ProcessPoolExecutor` and `ThreadPoolExecutor`, which act as lightweight wrappers around `multiprocessing` and `multithreading`, respectively. However, it's important to note that the same limitations and caveats of the lower-level modules still apply.

There are several ways to execute asynchronous tasks, with the most common being the `submit()` method. This allows you to pass in a callable along with its arguments. Another option is `map()`, which behaves similarly to Python's built-in `map()` function.

Once tasks are submitted, they begin executing immediately—there is no need to explicitly start them as you would with lower-level APIs. Instead of using `join()`, you can use `wait()`, which not only blocks until completion but also provides access to the callable's result. Additionally, the `as_completed()` iterator allows you to process results as tasks finish, as demonstrated in the next example.

It's worth noting that benchmarks suggest the processes created by

`concurrent.futures` are generally heavier than those spawned directly with `multiprocessing`. As a result, this module is most beneficial when your workload is well-suited to its higher-level abstraction and execution model.

concurrent.futures: using map and ThreadPoolExecutor

```
import threading
from concurrent.futures import ThreadPoolExecutor

def apply_interest(args):
    # Apply 5% interest and return result.
    account_index, initial_balances = args
    current_balance = initial_balances[account_index]
    updated_balance = int(current_balance * 1.05)
    print(f'executing {threading.current_thread().name}')
    return (account_index, updated_balance)

if __name__ == "__main__":
    initial_balances = [1000, 1500, 2000, ... 4000, 4500, 5000, 5500]
    tasks = [(i, initial_balances) for i in range(len(initial_balances))]

    with ThreadPoolExecutor(max_workers=4) as executor:
        # map applies the function to each item in tasks
        results_list = list(executor.map(apply_interest, tasks))

    results = dict(results_list)
    print(f"Raw dictionary (Ordered!): {results}")
```

```
executing ThreadPoolExecutor-0_0
executing ThreadPoolExecutor-0_0
executing ThreadPoolExecutor-0_0
executing ThreadPoolExecutor-0_0
executing ThreadPoolExecutor-0_1
executing ThreadPoolExecutor-0_1
executing ThreadPoolExecutor-0_1
executing ThreadPoolExecutor-0_0
executing ThreadPoolExecutor-0_2
executing ThreadPoolExecutor-0_1
Raw dictionary (Ordered!): {0: 1050, 1:
1575, 2: 2100, 3: 2625, 4: 3150, 5: 3675,
6: 4200, 7: 4725, 8: 5250, 9: 5775}
```

QA

379

The `map()` method applies a given function — here `apply_interest()` — to a set of inputs, producing a collection of results.

Key behaviour of `map()`:

- Regardless of concurrent execution order, the returned results always follow the original input sequence
- Adding a time delay inside the threaded function will not affect output order, only execution time

Working with the result:

- `map()` returns a generator object that acts as an iterator
- Simply wrap it in `list()` to obtain the final results

A note on performance:

- Due to the GIL and thread-related overhead, there may be little advantage to using concurrent multithreading for this type of processing
- It is worth adding timings to compare this approach against a linear iteration — `time.perf_counter()` from Python's standard library is a suitable tool for this

concurrent.futures: using map and ProcessPoolExecutor

```
import threading
from concurrent.futures import ProcessPoolExecutor
from os import getpid

def apply_interest(args):
    # Apply 5% interest and return result.
    account_index, initial_balances = args
    current_balance = initial_balances[account_index]
    updated_balance = int(current_balance * 1.05)
    print(f'executing PID{getpid()}')
    return (account_index, updated_balance)

if __name__ == "__main__":
    initial_balances = [1000, 1500, 2000, ... 4000, 4500, 5000, 5500]
    tasks = [(i, initial_balances) for i in range(len(initial_balances))]

    with ProcessPoolExecutor(max_workers=4) as executor:
        # map applies the function to each item in tasks
        results_list = list(executor.map(apply_interest, tasks))

    results = dict(results_list)
    print(f"Raw dictionary (Ordered!): {results}")
```

```
executing PID5724
executing PID10768
executing PID5724
executing PID10768
executing PID10768
executing PID5724
executing PID10768
executing PID10768
executing PID5724
Raw dictionary (Ordered!): {0: 1050, 1: 1575,
2: 2100, 3: 2625, 4: 3150, 5: 3675, 6: 4200,
7: 4725, 8: 5250, 9: 5775}
```

QA

380

Switching from a thread-based pool to a process-based one requires minimal changes — simply replace `ThreadPoolExecutor` with `ProcessPoolExecutor`. The only other addition is the standard multiprocessing entry-point guard required on Windows. Examining the output will show the PIDs assigned to each worker process.

concurrent.futures and scheduling

While `map()` is commonly used, you can also schedule individual tasks by submitting a callable object directly.

- This is done using the `submit()` method.
- Calling `submit()` returns a `Future` object.

About Futures:

- A `Future` represents the execution of a task.
- Developers typically don't create `Future` objects manually.
- Instead, they are created and managed by concurrency frameworks such as:
 - `concurrent.futures`
 - `Asyncio`

Working with Futures:

- Once a task completes, its state transitions from running to finished.
- You can check whether it has completed using the `done()` method, which returns a `Boolean`.
- To retrieve the result of the task, call the `result()` method on the `Future`.

QA

381

The next slide shows a short example of using `submit()` with a callable. We will revisit Futures in a later chapter when we look into `asyncio`.

concurrent.futures: scheduling

Here's a working example:

```
from concurrent.futures import ThreadPoolExecutor
import time, random

# Simulate applying interest to a bank account
def apply_interest(account_id, bal):
    time.sleep(random.uniform(0.1, 0.5)) # simulate processing delay
    new_bal = bal * 1.05
    return f"Account {account_id}: £{bal:.2f} -> £{new_bal:.2f}"

if __name__ == "__main__":
    accounts = {101: 1000, 102: 1500, 103: 2000, 104: 2500, }

    with ThreadPoolExecutor(max_workers=3) as executor:
        futures = []
        # Submit tasks individually
        for acc_id, bal in accounts.items():
            future = executor.submit(apply_interest, acc_id, bal)
            futures.append(future)
        # Retrieve results
        for future in futures:
            result = future.result() # blocks until task complete
            print(result)
```

```
Account 101: £1000.00 -> £1050.00
Account 102: £1500.00 -> £1575.00
Account 103: £2000.00 -> £2100.00
Account 104: £2500.00 -> £2625.00
```

QA

382

The above code demonstrates:

- `submit()` schedules each banking task (interest calculation)
- Each call returns a Future object
- `result()` retrieves the completed result (waiting if necessary)

The pattern is ideal when you want fine-grained control over individual tasks rather than using `map()`.

concurrent .futures: as_completed

The
as_completed()
method returns
an iterator over
Future instances

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time, random

# Simulate a bank transaction (e.g., fraud check or balance update)
def process_transaction(account_id, amount):
    processing_time = random.uniform(0.1, 0.6)
    time.sleep(processing_time) # simulate variable delay
    new_bal = 1000 + amount # pretend starting balance is 1000
    return f"Acc {account_id}: proc: £{amount:+.2f}, new bal: £{new_bal:.2f}"

if __name__ == "__main__":
    transactions = [(201, 200), (202, -150), (203, 500), (204, -50), (205, 300),]
    with ThreadPoolExecutor(max_workers=3) as executor:
        # Submit all tasks and keep track of futures
        future_to_account = {
            executor.submit(process_transaction, acc_id, amount): acc_id
            for acc_id, amount in transactions
        }
        # Process results as they complete (order may vary)
        for future in as_completed(future_to_account):
            result = future.result()
            print(result)
```

```
Acc 201: proc: £+200.00, new bal: £1200.00
Acc 202: proc: £-150.00, new bal: £850.00
Acc 203: proc: £+500.00, new bal: £1500.00
Acc 205: proc: £+300.00, new bal: £1300.00
Acc 204: proc: £-50.00, new bal: £950.00
```

QA

383

If result order matters, use `map()`. If order is unimportant, `as_completed()` is a suitable alternative, processing each result as soon as it becomes available.

Note that the output from this approach will deliberately vary, depending on the processing time of each Future and the number of `max_workers` assigned.

concurrent.futures: add_done_callback() - 1

A callback function can be associated with a Future using the `add_done_callback()` method.

- The callback is automatically executed when the Future completes.

Callbacks are triggered whether the Future completes successfully, fails with an exception, or is cancelled.

- Within the callback, you may need to determine the Future's outcome:
 - Completed successfully – check with `done()` and retrieve the result.
 - Completed with an exception – check with `done()` and `exception()`.
 - Cancelled – check with `cancelled()`.

The next example demonstrates how callbacks can be used with Futures.

concurrent.futures: add_done_callback() - 2

```
from concurrent.futures import ThreadPoolExecutor
import time, random

# Simulate processing a bank transfer
def process_transfer(account_id, amount):
    time.sleep(random.uniform(0.2, 0.6)) # simulate delay
    new_balance = 1000 + amount # assume initial balance
    return (account_id, amount, new_balance)

# Callback function that runs when the task is done
def notify_completion(future):
    account_id, amount, new_balance = future.result()
    print(f"[Callback] Acc {account_id}: "
          f"Proc: £{amount:+.2f}, new bal: £{new_balance:.2f}")

if __name__ == "__main__":
    transfers = [(301, 200), (302, -100), (303, 350), (304, -50),]
    with ThreadPoolExecutor(max_workers=3) as executor:
        futures = []
        # Submit tasks and attach callbacks
        for acc_id, amount in transfers:
            future = executor.submit(process_transfer, acc_id, amount)
            future.add_done_callback(notify_completion)
            futures.append(future)
        # Optional: wait for all tasks to complete
        for f in futures:
            f.result() # ensures main thread waits
```

Registering a Callback against a Future

```
[Callback] Acc 302: Proc: £-100.00, new bal: £900.00
[Callback] Acc 301: Proc: £+200.00, new bal: £1200.00
[Callback] Acc 303: Proc: £+350.00, new bal: £1350.00
[Callback] Acc 304: Proc: £-50.00, new bal: £950.00
```

QA

385

In this example the function `notify_completion()` is registered as a callback against the completion of the Future using the `add_done_callback()` method.

As noted, although this will be executed as a callback, it doesn't guarantee a result. For example, it's possible the Future raised an exception or was cancelled.

Fortunately, all of these potential states can be checked with a range of methods via simple if statements:

Callback function that runs when the task is done

```
def notify_completion(future):
    if future.cancelled():
        print(f"Cancelled: {future.arg}")
    elif future.done():
        if err := future.exception():
            print(f"Error: {future.arg} {err}")
        else:
            account_id, amount, new_balance = future.result()
            print(f"[Callback] Acc {account_id}: "
                  f"Proc: £{amount:+.2f}, new bal:
```

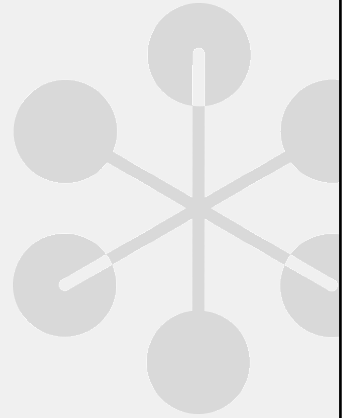
```
£{new_balance:.2f}")
```

In order to get parameters passed to the future we simply dynamically add an attribute to the Future dynamically (arg); see PEP232 for this feature. This can then be accessed within the callback function.

See also...

Standard library modules

- `concurrent.futures` Launching parallel tasks (Python 3.2 or later)
- `multiprocessing` Process-based parallelism
- `sched` Event scheduler
- `signal` Set handlers for asynchronous events
- `subprocess` Subprocess management
- `threading` Thread-based parallelism
- `queue` A synchronized queue class (Python 3)
- `asyncio` Foundation for multiple asynchronous frameworks



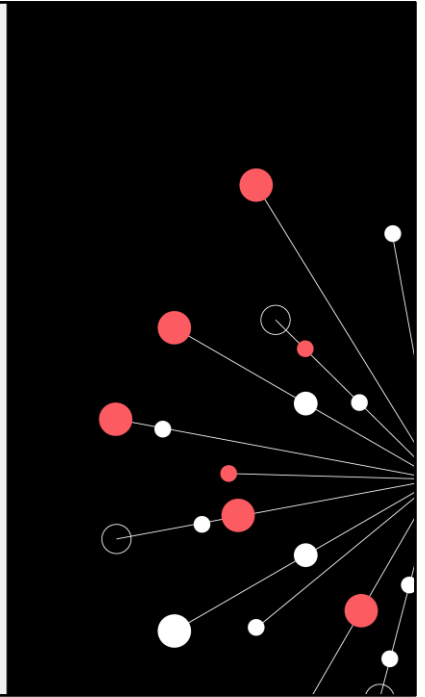
Exercise EG10: Parallelism

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA



Suggested time: 30-45 mins.

Activity includes differentiated tasks.

Summary

Programming for parallelism requires a different mindset

- Tasks must often be broken down in new and appropriate ways
- You need to decide whether threads or processes are more suitable

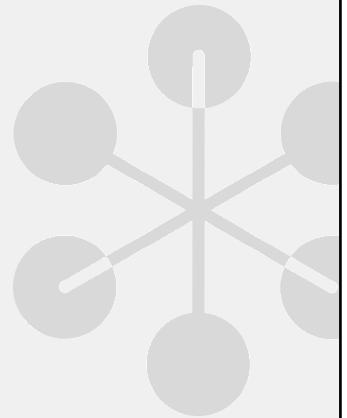
As a general rule:

- Use threads for I/O-bound tasks
- Use processes for CPU-bound tasks
- Threads can sometimes introduce more complexity than benefits
 - This is partly due to the Global Interpreter Lock (GIL)
 - Synchronization issues can also arise

For simpler parallelism, consider alternative approaches such as:

- The multiprocessing module
- Queues
- `concurrent.futures`

Remember, standard synchronization mechanisms are available if needed



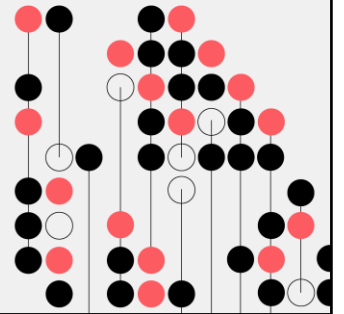
QA

388

We'll revisit the concept of working with asynchronous processes in a later session when we investigate `asyncio`.

Any questions?

QA



Appendices

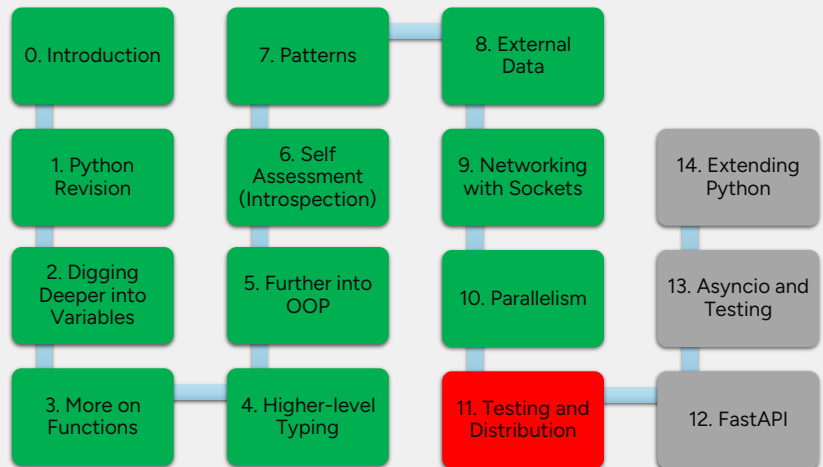


Intermediate Python

Testing and Distribution



Module roadmap



QA

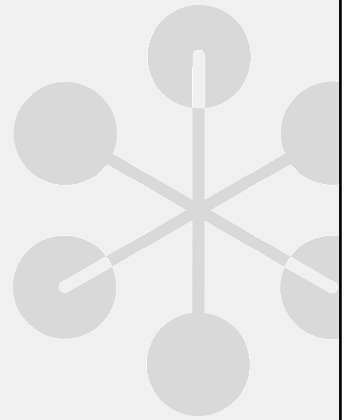
Testing and Distribution

Learning outcomes:

- Write tests and find the test coverages
- Differentiate available distribution tools and consider good practice

Prerequisites:

Delegates are required to have knowledge and skills relating to basic Python programming, including data structures, functions, loops and flow control.

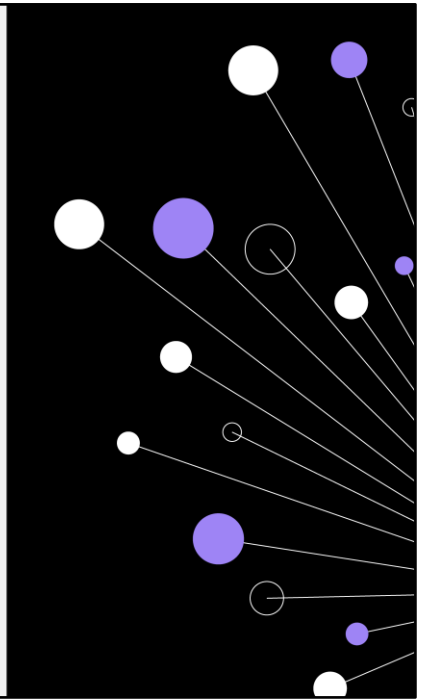


Learning objectives

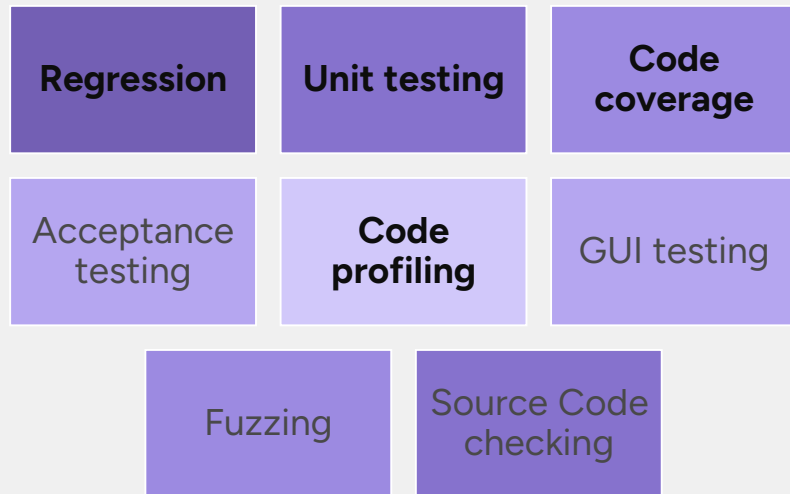
By the end of this session, you should be able to:

- Identify different types of testing available in Python and when to use them
- Describe profiling and tracing tools (profiler, trace) for analysing code behaviour
- Explain regression testing and its role in maintaining code quality
- Develop unit tests using the unittest module, including test structure and assertions
- Capture and test standard output using StringIO
- Apply trace alongside unittest for enhanced testing insight
- Compare testing frameworks such as unittest, pytest, and doctest
- Create and configure simple tests using doctest, including its API and options
- Integrate doctest with unittest and understand test distribution practices

QA



Different categories of Python testing tools



QA

395

A number of different types of testing tools are available for Python. However, we will focus on four of them:

- Code profiling
- Code coverage
- Regression testing
- Unit testing

Python testing – what tools are available?

Standard tools

- Python profiler
- trace module

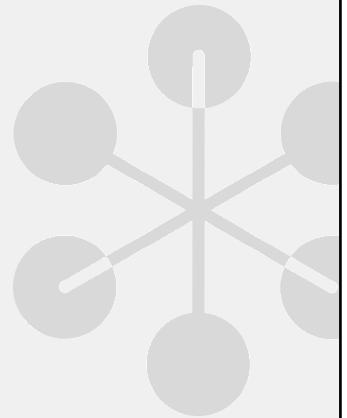
Standard library

- doctest
- unittest
- test

PyPI and elsewhere

- pytest
- nose
- testify

Or write your own! <http://wiki.python.org/moin/PythonTestingToolsTaxonomy>



QA

396

Python's standard library offers several testing and profiling tools:

- **Profiler:** identifies where time is being spent in your code; shows which functions are called and how long each takes.
- **trace module:** provides line-level execution detail, showing which lines run and which do not. Primarily designed to measure test effectiveness, commonly known as **Coverage**.
- **doctest module:** allows tests to be embedded directly in module documentation, with expected outputs that are automatically verified
- **unittest:** a comprehensive testing framework based on Java's JUnit.
- **Third-party options:** also available — nose is a popular alternative downloadable from PyPI, and as with most things in Python, writing a custom solution is always an option.

cProfile – The Python profiler

The cProfile module: a built-in tool that measures where a program spends its time by collecting detailed statistics on function calls and execution performance.

- Profile a specific function via a script:

```
import banking_utilities
import cProfile
cProfile.run('bank_utils.calc_daily_interest(accounts, days)', 'stats.prof')
```

↑
Save statistics to this file (optional)
Default: display statistics to stdout

- Or run a whole script from the command-line:

```
C:\QA>python -m cProfile mymodule.py
```

Analyse the output file using pstats shell:

```
C:\QA>python -m pstats stats.prof
Welcome to the profile statistics browser.
% helpcallers
```

QA

397

cProfile is the standard profiling module, that comes bundled with Python. A pure Python alternative called **profile** also exists but is slower despite offering identical functionality.

What cProfile tells you:

- How many times each function or method was called
- How long was spent executing each function
- Functions are identified by line number and filename

How it can be used:

- Run against an entire script or targeted at specific sections of code which makes it particularly useful during unit testing
- Output goes to stdout by default, but can be saved in binary format to a file for later analysis

Analysing results with pstats:

- Can be run interactively as a shell or programmatically
- The programmatic interface is best suited to analysing multiple statistics files in complex environments (see the Python documentation: <https://docs.python.org/3/library/profile.html>)

Using trace module to determine how good your tests are (1)

- Part of the standard library
- Equivalent to Coverage tools in other languages
- Available via both command-line and programmatic interfaces
- Can optionally produce *.cover statistics files
- Highly configurable — refer to the standard library documentation for full options
- Example of summary (-s) output:

```
python -m trace --count --summary -C . --ignore-dir=C:\Python313 run_tests.py
lines  cov%  module  (path)
   18  100%  bank_utils  (C:\QA\QAINTPYFS\11_Testing_and_Distribution\bank_utils.py)
    5  100%  run_tests  (run_tests.py)
   45  100%  unit_tests(C:\QA\11_Testing_and_Distribution\unit_tests.py)
```

- Trace can also be run programmatically

The trace module is primarily used to measure code coverage during testing, helping you determine which parts of a module are exercised by your test suite.

Several options are available. In the example shown on the slide:

-c generates coverage listing files (*.cover). These are plain text files and can be viewed directly without additional processing. When used from the command line, the output files are written to the same directory as the module being analysed. When used programmatically (count=1), the files are written to the current working directory.

-m highlights lines of code that were not executed during the test run.

-s produces a summary of the coverage results.

Another useful option is -t (--trace), which displays each line of code as it is executed. This can generate a large volume of output and is generally too detailed for routine debugging. However, it can be useful when a complete execution trace is required, for example when combined with the script command on UNIX/Linux systems.

Do not confuse this with -T (--trackcalls), which analyses function call relationships by showing which functions invoke other functions. This can be particularly valuable when maintaining or understanding a large, unfamiliar codebase.

Alternative coverage tools are also available, including **coverage.py** and **figleaf**, both of which can be installed from the Python package ecosystem.

As with any coverage or profiling tool, the quality of the results depends on the quality of the tests. Coverage statistics are only meaningful if the tests accurately represent the typical workload and usage patterns of the software being analysed.

Using trace module to determine how good your tests are (2)

A .cover file example....

Function defined once

Function called 4 times

This line was never tested!

Statement executed 12 times

```
1: def calc_daily_interest(accounts, days):
4:     results = {}
4:     if days == 0:
        days = 1
9:     for acc_id, data in accounts.items():
5:         balance = data["opening_balance"]
5:         rate = data["interest_rate"]
5:         transactions_by_day = data["transactions_b
12:     for day in range(days):
        # Apply that day's transactions
10:         for txn in transactions_by_day[day]:
3:             balance += txn["amount"]
        # Inefficient: recompute cumulative
7:         cumulative = 0
16:         for past_day in range(day):
13:             for txn in transactions_by_day[past_day]:
4:                 cumulative += txn["amount"]
7:         interest = (balance + cumulative) * rate
7:         balance += interest
5:         results[acc_id] = balance
4:     return results
```

QA

399

The **trace** module produces output files with the suffix '.cover'. These are ordinary text files that can be viewed using a text editor.

Python code can be written to produce the output files programmatically:

```
tracer = trace.Trace(
    ignoredirs=[sys.prefix, sys.exec_prefix],
    trace=0,
    count=1,
    timing=True)
```

```
sys.path.append('C:/QA/Python/testing/interp')
```

```
# This is where test code is executed in this case
tracer.run('import mytests')
```

```
r = tracer.results()
r.write_results(show_missing=True,
                summary=True, coverdir=".")
```

In this case, `mytests.py` was a full python program. This means that importing it will run all the tests. We will show another example, with the `unittest` module, later.

Regression testing

Execute all Python programs within a specified directory.

- Capture and store their output as a baseline ("master") set of results.
- Re-run the programs and compare the new output against the baseline to identify any changes.

Hints:

- Once the baseline output has been created, consider marking the files as read-only to prevent accidental modification.
- Use `filecmp.cmp()` for straightforward file comparisons.
- Use `difflib` when more detailed analysis of differences is required.

```
import difflib

with open("bank.master") as f:
    master = f.readlines()

with open("bank.out") as f:
    current = f.readlines()

if master == current:
    print("PASS: No differences found")
else:
    print("FAIL: Differences detected\n")

    diff = difflib.unified_diff(
        master,
        current,
        fromfile="bank.master",
        tofile="bank.out"
    )

    print("".join(diff))
```

QA

400

After fixing that problem you spent ages chasing, what do you do (aside from celebrate)? You run your regression tests, of course.

There are several approaches, you can use any of the testing regimes in this chapter, but often you will want to write your own. The general strategy is to run them with the base code and redirect all the output to "master" files. Having checked that output manually, you now have a set of master files that can be used to compare test runs.

That's all well and good, but what if the output goes to `stdout`? Yes, you can use shell redirection, but here's a trick you can use in your program:

```
stdout = sys.stdout
sys.stdout = open(_modulename + '.out', 'w')

# Run your tests

sys.stdout.close()
sys.stdout = stdout    # restore stdout
```

You probably want to do a similar thing to `stderr`.

It is surprisingly easy to mess this up and overwrite the masters by accident. So as a basic precaution, remove write access from your master files. There are several ways to compare the test output with your masters. The simplest is `filecmp.cmp`, but more complex comparisons can be done with `difflib`, both in the standard library. It is possible to be a bit more creative with output of the regression test by generating an html table:

```
# Alternative Output
Hdiff = difflib.HtmlDiff().make_table(master, current)
open("differences.html", "w").writelines(Hdiff)
```

Of course, don't forget to add the test that found your original error to your suite of regression tests.

Python unit tests – unittest module

Part of the standard library

- Python version of the Java testing framework (JUnit,)

Tests are built by creating a subclass of unittest

- Can be run from within IDE or the command-line

A Test Suite is an optional collection of Test Cases

```
class MyUnitTests (unittest.TestCase):
```

Test framework class containing calls to unittest



modules, functions, to be tested; the code "units"

QA

401

unittest (sometimes called PyUnit) is Python's built-in testing framework, based on Java's JUnit. It is fully featured — the key components and architecture are summarised below.

Core concepts:

- **Test Case:** the fundamental unit of testing, representing a single test scenario. Must be written by the tester; unittest provides a framework and helper methods to simplify this
- **Test Suite:** an optional collection of Test Cases, useful for grouping and running related tests together

Note: When running unittest, all warnings are enabled by default (-Wd).

Anatomy of a unit test

Your test class can be named anything, but must inherit from `unittest.TestCase`

Fixture methods (optional) handle initialisation and cleanup:

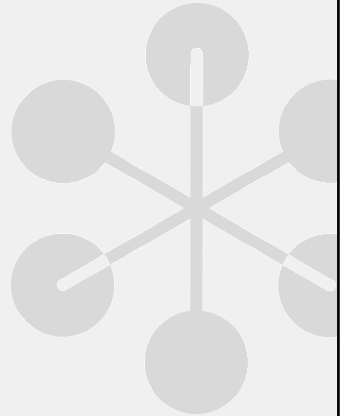
- `setUp` and `tearDown` — both take only `self` as a parameter

Test methods (Test Cases) can be named anything, provided they are prefixed with `test`

- Take only `self` as a parameter
- Pass or fail based on assertion methods
- All test cases collectively form a Test Suite

A Test Runner is used to execute the tests

See next slide for an example...



QA

402

To create a unit test in Python, define a test class that inherits from `unittest.TestCase`. The class can be given any valid name, but individual test methods must begin with the prefix `test_` so that the testing framework can identify and execute them. Each test determines success or failure by using one of the assertion methods provided by `unittest`. Optional lifecycle methods include `setUp()`, which is executed before each test, and `tearDown()`, which runs after each test and is often used for cleanup activities.

Tests are typically executed by calling `unittest.main()`, which acts as the test runner. More detailed output can be obtained by running the test program with the `-v` (verbose) command-line option.

Python also supports test discovery, allowing tests to be located and executed automatically across multiple directories, although this feature is not covered here.

A **Test Suite** is represented by an instance of `unittest.TestSuite` and acts as a collection of test cases. Tests can be added individually using `addTest()` or in groups using `addTests()`. Calling `run()` on the suite executes all tests it contains.

Example of some simple tests

```
def deposit(balance, amount):
    if amount <= 0:
        raise ValueError("amount must be positive")
    return balance + amount
```

Test
methods
with
assertions

```
import unittest
from banking_logic import deposit, withdraw

class TestBankTransactions(unittest.TestCase):

    def test_deposit_increases_balance(self):
        result = deposit(100, 50)
        self.assertEqual(result, 150)

    def test_deposit_negative_amount_raises_error(self):
        with self.assertRaises(ValueError):
            deposit(100, -10)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Test runner

```
python -m unittest -v banking_logic_tests.py
test_deposit_increases_balance (banking_logic_tests.TestBankTransactions.test_deposit_increases_balance) ... ok
test_deposit_negative_amount_raises_error (banking_logic_tests.TestBankTransactions.test_deposit_negative_amount_raises_error) ... ok
```

Output...

QA

403

In this example, the Test Cases are `test_deposit_increases_balance()` and `test_deposit_negative_amount_raises_error()` - the only part of the name that is fixed is the 'test' prefix.

Unittest support a couple of special methods called `setUp()` and `tearDown()`. The methods are optional and are invoked just before and after each test, so, for example, if you have five tests then `setUp()` and `tearDown()` will each be called five times.

Assertion methods available

Note: All assert methods have an optional `msg=` parameter which can be helpful when debugging

Assert Method	Explanation
<code>assertEqual(a,b)</code>	<code>a == b</code>
<code>assertNotEqual(a,b)</code>	<code>a != b</code>
<code>assertTrue(x)</code>	<code>bool(x)</code> is True
<code>assertFalse(x)</code>	<code>bool(x)</code> is False
<code>assertIs(a, b)</code>	<code>a</code> is <code>b</code>
<code>assertIsNot(a, b)</code>	<code>a</code> is not <code>b</code>
<code>assertIsNone(x)</code>	<code>x</code> is None
<code>assertIsNotNone(x)</code>	<code>x</code> is not None
<code>assertIn(a, b)</code>	<code>a</code> in <code>b</code>
<code>assertNotIn(a, b)</code>	<code>a</code> not in <code>b</code>
<code>assertIsInstance(a, b)</code>	<code>isinstance(a, b)</code>
<code>assertNotIsInstance(a, b)</code>	<code>not isinstance(a, b)</code>
<code>assertRaises(e, c, ...)</code>	<code>e</code> is raised when <code>c</code>
<code>assertWarns(w, c, ...)</code>	<code>w</code> is raised when <code>c</code>

QA

404

These assertion methods are all members of `unittest.TestCase`. The original set was small, but in most cases the core assertions are sufficient.

Additional type-specific assertions are also available:

`assertMultiLineEqual`
`assertSequenceEqual`
`assertListEqual`
`assertTupleEqual`
`assertSetEqual`
`assertDictEqual`

Also available, but omitted from the table, are `assertRaisesRegex` and `assertWarnsRegex`, both of which accept a regular expression as their second parameter.

A note on `assertRaises`: Trapping a string exception with `assertRaises` can be problematic without a context manager. A simple workaround is to wrap the callable in a lambda:

```
self.assertRaises(IndexError, lambda: myfunc())
```

Ignoring some tests

Decorators can be used to skip tests

- Classes and methods can be skipped
- `@unittest.expectedFailure()`
- `@unittest.skip("string comment")`
- `@unittest.skipif(condition, "string comment")`
- `@unittest.skipUnless(condition, "string comment")`

```
import unittest
import sys

class TestPlatformSpecific(unittest.TestCase):

    @unittest.skipUnless(sys.platform.startswith("win"), "Requires Windows")
    def test_windows_only_feature(self):
        self.assertTrue(True)
```

Note: The "string comment" is only displayed in verbose mode.

QA

405

Sometimes we expect tests to fail, or we might wish to skip tests, particularly when we are building our test cases. Having control over which tests run is very useful, particularly since we can defer the decision until runtime (`skipif()` and `skipUnless()`).

A common test to carry out here is the Python version, testing `sys.version_info` for example.

Testing stdout using StringIO

```
import unittest
from io import StringIO
import sys
from bank_utils import print_account_summary

class TestAccountSummary(unittest.TestCase):
    def setUp(self):
        self._stdout = sys.stdout
        self.captured_output = StringIO()
        sys.stdout = self.captured_output

    def test_print_account_summary(self):
        print_account_summary("ACC1001", 12.5)

        output = self.captured_output.getvalue().strip()
        expected = self.assertEqual(output, "ACC1001 has a balance of £12.50")

    def tearDown(self):
        sys.stdout = self._stdout

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

QA

06

A common challenge in testing is handling output to stdout. The solution is to use a StringIO file handle:

```
import from io
```

Using trace with unittest

The command-line interface will only trace the test itself, showing how much of the test program is executed

To trace the module under test, use the programmatic API:

- Create a trace object
- Discover tests.
- Create a test runner.
- Execute the test suite.
- Create a CoverageResults object and write the results to files

Each tested module will produce its own .cover file

```
import unittest, trace, sys, os

def run_tests_with_trace():
    tracer = trace.Trace(count=True, trace=False,
                        ignoredirs=[sys.prefix, sys.exec_prefix])
    suite = unittest.defaultTestLoader.discover(
        ".", pattern="test_bank_utils.py")
    runner = unittest.TextTestRunner(verbosity=2)
    def run():
        return runner.run(suite)
    result = tracer.runfunc(run)
    results = tracer.results()
    results.write_results(
        show_missing=True, summary=True, coverdir=".")
    return result

if __name__ == "__main__":
    run_tests_with_trace()
```

Create
tracer

Discover
tests

Run tests inside
the tracer

QA

407

Using the trace module from the command line is not particularly useful when working with a unittest program, because it typically only reports how many tests were executed—information that is already known from the test framework itself.

Instead, trace can be invoked programmatically, as shown in the example on the slide. In this approach, it replaces the usual call to `unittest.main()` within the `__main__` block with a call to a function that contains the relevant code.

Once tests have been executed, the results can be accessed and written out to files for further analysis. These outputs can also be processed programmatically by reading and interpreting the generated .cover files.

pytest module

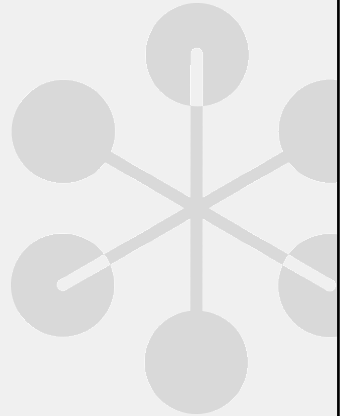
pytest is a modern Python testing framework

- Compatible with both unittest and doctest, so existing tests do not need to be rewritten

Test functions should be prefixed with test

Test files should be prefixed with test_ or suffixed with _test

- Custom naming rules can be defined in `setup.py`
- Markers can be used to group and run named tests selectively



QA

408

Pytest is a modern testing framework for Python.

It reduces the amount of boilerplate code required—for example, tests no longer need to inherit from `TestCase`.

It also provides greater flexibility in how fixtures are defined and controlled, including when they are executed.

While other frameworks such as `Testify`, `Trial`, and `nose2` exist, they have largely been replaced in popularity and usage by `pytest`.

Running a pytest

Run the pytest file from the command line

```
pytest -v test_bank_utils.py
===== test session starts =====
platform win32 -- Python 3.13.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\QA\Testing_and_Distribution\17_using_trace_with_unittest
plugins: anyio-4.9.0, asyncio-1.3.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None,
asyncio_default_test_loop_scope=function
collected 4 items

test_bank_utils.py::TestCalculateFees::test_combined_fees PASSED [ 25%]
test_bank_utils.py::TestCalculateFees::test_no_fees PASSED [ 50%]
test_bank_utils.py::TestCalculateFees::test_overdraft_fee PASSED [ 75%]
test_bank_utils.py::TestCalculateFees::test_withdrawal_fees PASSED [100%]

===== 4 passed in 0.04s =====
```

QA

409

To write a pytest test case we first create a testing file. The methods used for each test case should have names prefixed '**test**'. These methods indicate success or failure by calling the standard python assert function.

Note to check for exceptions in a test we use a context manger:

```
with self.assertRaises(ZeroDivisionError):
    1/0
```

We run the pytest case test from the command line, often with the `-v` flag for more verbosity on which tests failed and why.

Note you can typically run pytest directly from with an IDE by right clicking in the code editor window and selecting the "Run Python" (or equivalent) option

You can see in the above text box an example being run in a file called "test_bank_utils.py" containing three tests named:

```
test_combined_fees
test_no_fees
test_overdraft_fee
```

test_withdrawal_fees

All of which have passed

pytest options

There are a number of useful command line options that can be used with pytest

- Add **verbosity** to the output of the test (-v)

- Run an isolated test (:::)

```
user@comp:~/Dev$ pytest -v test_file.py::test_test1
```

- Run whose name contains a substring (-k)

```
user@comp:~/Dev$ pytest -v test_file.py -k check
```

- Run tests that are **marked** with a specific marker tag (-m)

```
user@comp:~/Dev$ pytest -v test_file.py -m "not win"
```

Get pytest **help** (-h)

QA

410

Running a subset of tests is often useful, as we may have tests for different systems or configurations, or might just want to test a certain area of functionality.

The `::` command allows for single test to be ran

The `-k` allows for test that contain a certain substring to run. You can contain logic in the string following the `-k` option, for example "check and not final" will run all tests that contain the word check except those that contain the word final. The string must be an expression that python can evaluate.

The `-m` allows for tests marked using decorators to run, more on this shortly

Example of basic pytest file

```
import pytest
from banking_logic import deposit, withdraw

def setup_function():
    """Executed before each test."""
    pass

def test_deposit_increases_balance():
    result = deposit(100, 50)
    assert result == 150

def test_deposit_negative_amount_raises_error():
    with pytest.raises(ValueError):
        deposit(100, -10)

def teardown_function():
    """Executed after each test."""
    pass
```

QA

See the chapter appendix for full worked example

411

This is the same example that was used to demonstrate `unittest` except note the tests do **not** need to be declared within a class so there's no `TestCase`, no need for any boilerplate logic, no use of the `self` keyword.

Note how less boiler-plate code has been used, and use of the python `assert` keyword is now used to do all the checking in the test functions.

Setup teardown methods

Fixtures can be created using the following setup and teardown methods:

Setup_function()

Called before any test defined as a function

Setup_method()

Called before any test defined as a method

Setup_class()

called when a class containing tests is created

Setup_module()

called when the module containing tests is loaded

Each of these has a corresponding optional teardown counterpart.

QA

412

Note: unittest also supports setup and teardown at the method, class, and module levels.

pytest additionally supports session-level fixtures. While most cases use a single fixture, multiple fixtures are supported.

Fixtures are a fundamental part of testing, they provide a controlled environment or object that tests can depend on.

The challenge is how can different fixtures be applied to different functions?

In older frameworks, the solution was to separate tests into classes grouped by fixture — adding boilerplate code and possibly forcing logically related tests apart as a consequence. pytest addresses this more elegantly.

How pytest fixes unittest's fixture problem

In unittest, fixtures are tied to classes:

- If another test needs a different setup, you often:
 - Create another class
 - Duplicate or split related tests
 - Add boilerplate

```
class TestWithDatabase(unittest.TestCase):
    def setUp(self):
        self.db = connect_db()
    def test_something(self):
        ...
```

In pytest, fixtures are standalone functions

- Tests opt-in by declaring what they need:
- The fixture is:
 - Injected automatically
 - Only used where needed

```
@pytest.fixture
def db():
    return "database connection"

@pytest.fixture
def api_client():
    return "API client"

def test_db_operation(db):
    assert db == "database connection"

def test_api_call(api_client):
    assert api_client == "API client"
```

Mocking variables

Mocking involves temporarily overriding a variable in production code for the duration of a test, replacing real behaviour with a predictable "fake" response

Commonly used to:

- Alleviate processing bottlenecks
- Work around unavailable components or services
- Avoid unpredictable network access
- Isolate tests from external dependencies

In pytest, this is achieved using the monkeypatch fixture

Pytest's monkeypatch fixture includes a number of useful helper methods:

```
monkeypatch.chdir(path)
monkeypatch.delattr(obj, name, raising=True)
monkeypatch.delenv(name, raising=True)
monkeypatch.delitem(obj, name, raising=True)
monkeypatch.setattr(obj, name, value, raising=True)
monkeypatch.setenv(name, value, prepend=False)
monkeypatch.setitem(mapping, name, value)
monkeypatch.syspath_prepend(path)
```

We'll use one of these to illustrate their effects in the next slide.

pytest mocking

bank_utils.py

```
# Simulated external dependency
def send_payment_to_bank_api(id, amt): 
    # Imagine this calls a real banking API
    return {"status": "SUCCESS"}

def process_payment(id, amt):
    if amount <= 0:
        raise ValueError("Amount must be positive")

    response = send_payment_to_bank_api(id, amt)

    if response["status"] != "SUCCESS":
        raise RuntimeError("Payment failed")

    return "Payment processed"
```

```
import bank_utils
from bank_utils import process_payment

def test_process_payment_success(monkeypatch):
    # Mock implementation
    def mock_send_payment(account_id, amount):
        return {"status": "SUCCESS"}

    # Replace the real function
     monkeypatch.setattr(
        bank_utils,
        "send_payment_to_bank_api",
        mock_send_payment
    )

    result = process_payment("ACC1001", 100)

    assert result == "Payment processed"
```

test_bank_utils.py

QA

415

As you can see, the use of the `setattr` helper method safely (but temporarily) patches the default extension applied in the `send_payment_to_bank_api` function.

This of course works because a module is an object and as such, it is possible to dynamically change the value of its attributes at run-time.

The genuine `send_payment_to_bank_api` function as well as incurring an indeterminate result could take an arbitrary processing time to respond. The mocking of the method, for the test function alone, allows it bypass such indeterminate actions and isolate the processing of the function we want to test, focusing on the key question: Does it correctly validate the amount and invoke the `send_payment_to_bank_api` function?

fixture decorator

Create a function decorated with `pytest.fixture`

- Pass it as an argument to the function

```
import pytest
import bank_utils
from bank_utils import process_payment
```

Now not a direct test method

```
@pytest.fixture(scope="module")
```

```
def mock_successful_payment(monkeypatch):
    def mock_send_payment(account_id, amount):
        return {"status": "SUCCESS"}
```

```
    monkeypatch.setattr(
        bank_utils,
        "send_payment_to_bank_api",
        mock_send_payment
    )
```

Tests get a fresh object each time with "function" scope

```
def test_process_payment_success(mock_successful_payment):
    result = process_payment("ACC1001", 100)
    assert result == "Payment processed"
```

QA

416

This method of creating fixtures both reduces the amount of boiler-plate code needed and allows for dynamic fixture assignment to functions in the same file. If fixture scope is set to "module" then setup is called once per module. If set to "session" or "function", then it is only called once per test session, or before every test function, respectively. Set to "class" it will be called before each test-class method.

The setup function yields rather than returns so that you could add cleanup code after the yield that would be executed after the test has run. If there is no cleanup code a simple return will suffice.

In the above example the same generator would be given to each test function that request it. However, if the scope were to be set to "function", every test function would be getting a different instance of the generator. We can create the teardown function for a fixture decorator by nesting a function, for instance:

```
@pytest.fixture(scope="class")
def create_resource(request):
    print("creating resource..")

    def teardown_resource():
```

```
print("deleting resource..")  
request.addfinalizer(teardown_resource)
```

Also note how we have given this class scope with the named parameter

Parameterised Fixtures

Fixtures can be parameterised
leading to less code duplication

QA

```
import ...

@pytest.fixture
def mock_payment(monkeypatch):
    def _mock(status):
        def fake(account_id, amount):
            return {"status": status}
        monkeypatch.setattr(
            bank_utils,
            "send_payment_to_bank_api",
            fake
        )
    return _mock

def test_process_payment_success(mock_payment):
    mock_payment("SUCCESS")
    result = process_payment("ACC1001", 100)
    assert result == "Payment processed"

def test_process_payment_failure(mock_payment):
    mock_payment("FAILED")
    with pytest.raises(RuntimeError):
        process_payment("ACC1001", 100)
```

417

Decorate test with markers

Tests can be decorated with markers to allow multiple test suites in the same file

Run only fast tests:
`pytest -m fast`

Run only error tests:
`pytest -m error`

Exclude slow tests:
`pytest -m "not slow"`

```
import pytest
from bank_utils import transfer_funds

@pytest.mark.fast
def test_small_transfer():
    assert transfer_funds(1000, 100) == 900

@pytest.mark.slow
def test_large_transfer():
    # Simulate something "heavier" conceptually
    assert transfer_funds(1000000, 500000) == 500000

@pytest.mark.error
def test_transfer_insufficient_funds():
    with pytest.raises(ValueError):
        transfer_funds(100, 200)

@pytest.mark.error
def test_transfer_negative_amount():
    with pytest.raises(ValueError):
        transfer_funds(100, -10)
```

QA

418

This test file can be run as follows:

```
pytest test_bank_utils.py -m fast
```

We can also invert the markers as follows:

```
pytest test_bank_utils.py -m "not slow"
```

This will run all the tests except for `test_large_transfer()`. We can create many markers and decorate tests with multiple, to create different test suites, etc.

pytest will report a warning with unregistered mark names (in the above example they would be 'fast', 'slow' and 'error' although execution will run as expected

The markers that currently exist for our test suite can be listed with the command:

```
pytest --markers
```

More markers can be added to suppress the warning by changing the `pytest.ini` file as follows:

```
[pytest]
```

```
markers =
```

```
    fast: quick-running tests
```

```
    slow: longer-running tests
```

```
    error: tests that verify error conditions
```

Parameterised tests

Tests can be run as a batch using the `parametrize` fixture

- Specify input arguments in a list of tuples
- IDs can be added for readability

```
import pytest
from bank import BankAccount
```

```
@pytest.mark.parametrize(
    "initial_balance, withdraw_amount, expected_balance",
    [
        (100.0, 20.0, 80.0),
        (50.0, 50.0, 0.0),
    ],
    ids=["normal withdrawal", "full withdrawal"]
)
```

```
def test_withdraw_success(initial_balance, withdraw_amount, expected_balance):
    account = BankAccount(initial_balance)
    new_balance = account.withdraw(withdraw_amount)
```

```
test_withdraw_success[normal withdrawal] PASSED
test_withdraw_success[full withdrawal] PASSED
```

QA

419

Tests can be decorated with the `parametrize` fixture to run multiple tests in a batch. The `parametrize` fixture takes two arguments: a string listing the function's parameters separated by commas, and a list of tuples where each tuple contains the values to be passed to those parameters.

In the example above, three arguments, "initial_balance", "withdraw_amount" and "expected_balance," are passed to the function. The first argument to the decorator is therefore the string "initial_balance, withdraw_amount, expected_balance". The second argument is a list of tuples, each containing an initial_balance, withdraw_amount and the expected response from the call to the `account.withdraw` method. Each tuple represents a separate test, so the number of tuples in the list determines how many tests will be executed.

IDs can be optionally added to the decorator data to help produce clearer test output:

```
test_withdraw_success[normal withdrawal] PASSED
```

```
test_withdraw_success[full withdrawal] PASSED
```

Without them the test result messages would display the tuple used for each scenario:

```
test_withdraw_success[100.0-20.0-80.0] PASSED
```

test_withdraw_success[50.0-50.0-0.0] PASSED

Parameterised tests – Handling exceptions

Tests can be run as a batch using the `parametrize` fixture

- Specify input arguments in a list of tuples

```
import pytest
from bank import BankAccount, InsufficientFundsError

@pytest.mark.parametrize(
    "initial_balance, withdraw_amount, expected_exception",
    [
        (100.0, 0.0, ValueError),           # zero withdrawal
        (100.0, -10.0, ValueError),         # negative withdrawal
        (100.0, 200.0, InsufficientFundsError), # overdraft
    ]
)
def test_withdraw_failures(initial_balance, withdraw_amount, expected_exception):
    account = BankAccount(initial_balance)

    with pytest.raises(expected_exception):
        account.withdraw(withdraw_amount)
```

QA

420

The same `parameterize` decorator can be used to specify values that will cause the code under test to trigger an exception.

skip test and stop after failures

@pytest.mark.skip(reason=None):

- Use the decorator to exclude a test from execution, optionally providing a reason for the skip. For example:
`@pytest.mark.skip(reason="Loan interest calculation not implemented yet")`

@pytest.mark.skipif(condition):

- Use `@pytest.mark.skipif()` to conditionally skip a test when a specified condition evaluates to True. The condition is evaluated at runtime, and if it is satisfied, the test is not executed. For example:
`@pytest.mark.skipif('sys.platform == "win32"')`

@pytest.mark.xfail(condition, reason=None, run=True, raises=None, strict=False)

- Marks a test that is expected to fail. This is useful for tests that should fail, and passing would be an error
- Note, By default, pytest reports:
 - XFAIL for an expected failure
 - XPASS for an unexpected pass (but does not treat the test as a failure)

QA

421

There are other mark decorators we can use with pytest, for example the `skip` and `skipif` mark allow the skipping on tests for a reason or is a condition is true.

We can also mark tests with `xfail` to say they should fail (and a pass is incorrect). If a test with `xfail` fails, it will not be reported as an error with a trace back.

Distribution tools

Python package distribution relies on several key tools and services.

- **pip** is the standard package manager for installing and managing Python packages.
 - pip downloads and installs packages from the Python Package Index (PyPI).
 - PyPI (pypi.org) is a searchable online repository containing thousands of third-party Python packages.

setuptools is used to define project metadata and build distributable packages from source code.

- The `bdist_wheel` extension for setuptools creates wheel distributions.
 - A **wheel** (.whl) is the standard binary packaging format for Python distributions.

twine is the preferred tool for uploading packages to PyPI.

- twine is favoured because it uses secure HTTPS-based authentication when publishing packages.
- It supports multiple distribution formats, including wheel packages.

Distribution – key files

Properly packaging a Python project is essential, especially if you plan to publish or distribute it.

- It's recommended to follow a consistent and standard project structure
- Important files typically include:
 - `__init__.py` – an initialization file that marks a directory as a Python package (it may be empty)
 - `setup.py` – a script used by `setuptools` to build and install the package
 - `LICENSE` – defines how the package can be used and shared
 - `README.md` – a Markdown file that documents the purpose and usage of the project
- Alongside these, your project will of course include the main source code files.

Perhaps the most important distribution-specific file is `pyproject.toml` – we'll examine this next.

```
project/
|  — src/
|      — mypackage/
|          — __init__.py
|          — core.py
|  — tests/
|  — pyproject.toml
|  — README.md
```

QA

423

When packaging a Python project, the typical goal is to produce distribution packages – compressed archives that can be uploaded to the Python Package Index (PyPI) and subsequently downloaded and installed by other developers via `pip`.

A note on `__init__.py`: This file marks a directory as a Python package. It may be empty — its presence alone is often sufficient.

Further reading:

Markdown syntax: <https://guides.github.com/features/mastering-markdown/>

Open-source licence types: <https://choosealicense.com/>

Distribution – pyproject.toml

`pyproject.toml` is a script which provides information to twine about a package.

Here's an example `pyproject.toml` file:

```
[build-system]
requires = ["setuptools", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "my_bank_package"
version = "0.1.0"
description = "A simple banking package"
readme = "README.md"
authors = [
    { name = "Your Name", email = "your.email@example.com" }
]
requires-python = ">=3.8"
dependencies = ["requests>=2.28.0"]

[project.optional-dependencies]
dev = ["pytest", "flake8"]

[project.urls]
Homepage = "https://github.com/user/my_bank_package"

[tool.setuptools.packages.find]
where = ["."]
```

QA

424

`pyproject.toml` typically includes the following:

- **Build system configuration** — defines the build backend (e.g. `setuptools`) and required build dependencies
- **Package metadata** — name, version, description, authors, and other core project details
- **Python version requirements** — specified using `requires-python` (e.g. `>=3.8`)
- **Dependencies** — lists required packages needed for the project to run
- **Optional dependencies** — extra groups (e.g. `dev`, `test`) that can be installed separately
- **Source URL(s)** — links to the project homepage, repository, or documentation (often GitHub)
- **Package discovery settings** — instructions for finding source packages (replacing `find_packages()` in `setup.py`)
- **PyPI classifiers** — metadata used to categorise the package on PyPI (see <https://pypi.org/classifiers/>)
- **Long description** — typically loaded from a `README.md` file using the `readme` field
- **Version naming** — follows the scheme defined in PEP 440:

<https://www.python.org/dev/peps/pep-0440/>

- **Tool configuration** — sections for development tools (e.g. pytest, black, flake8) under [tool.*]

Distribution – a 6-step guide

1. Prepare project structure

- Organise code into proper package layout



2. Define project metadata

In `pyproject.toml`, include:

- Project name
- Version
- Dependencies
- Build system



3. Create a build environment

Install build tools:

```
python -m pip install build
```



4. Run tests

Ensure code quality before release:
`pytest`



5. Build the distribution

Generate installable artefacts:

```
python -m build
```

This produces:

- **Wheel** (`.whl`) → installable binary package
- **Source distribution** (`.tar.gz`) → source code package
- Output goes into `dist` folder



6. Upload the package

Use **twine** to publish:

- Test upload (recommended first)
`twine upload --repository testpypi dist/*`
- Production upload (PyPI)
`twine upload dist/*`

QA

425

twine is used for securely uploading Python packages to PyPI.

Test PyPI (`test.pypi.org`) is a separate package index specifically designed for testing. It allows developers to upload and experiment with packages without affecting the real PyPI repository. Before using it, you must create an account:

Register at: <https://test.pypi.org/account/register/>

Then verify your email address to enable uploads.

When uploading to Test PyPI, authentication is required:

The username is always `__token__`

The password is an API token generated from:

<https://test.pypi.org/manage/account/#api-tokens>

Once a package has been successfully uploaded, it can be installed on another system using `pip`, for example:

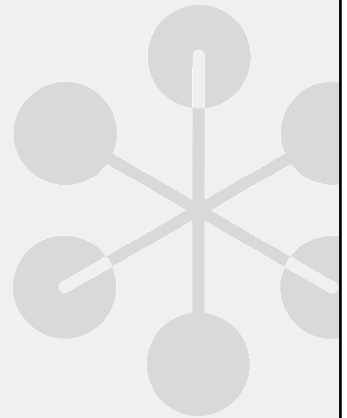
```
python3 -m pip install --index-url  
https://test.pypi.org/simple/ --no-deps demo-pkg-  
YOURUSERNAME!
```

This installs the package from Test PyPI rather than the official PyPI index, allowing safe testing without impacting production packages.

See also...

Standard library modules

- `cProfile` Deterministic profiling (C version)
- `doctest` Test interactive Python examples
- `io.StringIO` An in-memory stream for text I/O (Python 3)
- `profile` Deterministic profiling (Python version)
- `pstats` Analysis of profiler data
- `resource` Resource usage information
- `StringIO` Read and write strings as files (Python 2)
- `test` Regression tests for Python (internal)
- `timeit` Measure execution time of small code snippets
- `Unittest`, `pytest` Popular unit testing frameworks



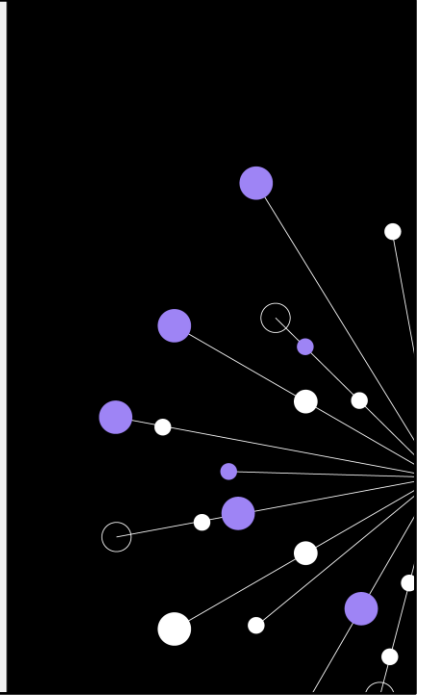
Exercise EG11: Testing and Distribution

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA



Suggested time: 30-45 mins.

Activity includes differentiated tasks.

Summary

Standard library testing tools:

- **cProfile** — performance profiler
- **trace** — test coverage analysis
- **doctest** — docstring-based testing
- **unittest** — JUnit-inspired testing framework
- **pytest** — simpler, more flexible, and more Pythonic

`trace` and `doctest` can be integrated with both `unittest` and `pytest`, and custom tools can always be written — potentially building on the existing ones.

Third-party tools are also available via PyPI, several of which are based on `unittest` and `pytest`.

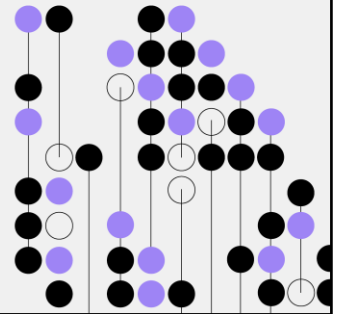
Distributing a Python project:

- `pyproject` — packaging and configuration file
- `twine` — handles uploading distributions to PyPI

Always follow best-practice guidelines when preparing a distribution

Any questions?

QA



Appendices



Simple testing – doctest (1)

Run a module as a main program

```
if __name__ == "__main__":
    import doctest
    doctest.testmod()
```

Testing is built into Python

- Searches for docstrings containing interactive sessions/REPL

```
"""
    This is a sample module which
    does various date operations.

    >>> today = date(2026,6,5)
    >>> print(today)
    2026-06-05
"""
```

Run with the -v option

```
python Simple_testing_doc_test.py -v
Trying:
    today = date(2026,6,5)
Expecting nothing
ok
Trying:
    print(today)
Expecting:
    2026-06-05
ok
1 item passed all tests:
  2 tests in __main__
2 tests in 1 item.
2 passed.
Test passed.
```

QA

431

There are problems with writing your own test harness, discussed on the previous slide. Over the passage of time the test harness gets lost or does not get delivered with the module. Modifications done to the module are often omitted from the test harness, and it gets out of step.

A better strategy is to embed the tests inside the module - that way there is no extra test harness file to go missing, and the tests can be seen by anyone altering the code. Running the module as a main program will run the tests if we use the standard **doctest** module.

The **doctest** module searches for docstrings and, in its simplest form, will just check these. If an interactive session is found within a docstring then it will be seen as a test and run it. It looks for the >>> prompt as a command with output which follows.

Normally tests are separated by a blank-line. If the test output itself contains a blank-line then it should be marked as:
<BLANKLINE>

There are other markers and options available, in particular ELLIPSIS which

indicates unpredictable output (see next slide).

Simple testing – doctest (2)

Options to qualify output

- Specified as a comment
- Can be OR'ed together and passed to doctest functions

```
DONT_ACCEPT_TRUE_FOR_1
DONT_ACCEPT_BLANKLINE
NORMALIZE_WHITESPACE
ELLIPSIS
IGNORE_EXCEPTION_DETAIL
SKIP
COMPARISON_FLAGS
REPORT_UDIFF
REPORT_CDIFF
REPORT_NDIFF
REPORT_ONLY_FIRST_FAILURE
REPORTING_FLAGS
```

Reporting flags

```
>>> start_timer()
>>> end_timer()      #doctest: +ELLIPSIS
End time      : ... seconds
```

QA

432

The doctest options can be passed to the API routines shown later, or placed as a comment in your test code, as shown in the example on the slide.

Check the doctest documentation for details, but mostly they are intuitive. One of the most useful is ELLIPSIS (shown) which accepts that part of the output will be indeterminate. This is useful where the output could depend on timing, or might change, like a username, IP address, or date. The part of the output which might vary is marked by three dots ... (an ellipsis, but not the single Unicode character).

You can even create your own flags using `doctest.optionflag()`.

Simple testing – doctest (3)

All have optional arguments, including options

The most common doctest **API functions** are:

- **testmod**
 - Specifies the name of the module to be tested

```
import doctest
doctest.testmod(mytimer)
```

- **testfile(filename)**
 - Specifies the name of a file containing tests
 - Treats *filename* as one huge docstring
 - Can includes comments, not just tests

```
import doctest
doctest.testmod("mytests.txt")
```

QA

433

Sometimes it might not be convenient to put the tests in your test harness, so you can run a specific module using `testmod()`.

It could be that your tests are so extensive that they obscure the documentation. In this case the tests can be kept in a separate file and invoked using `testfile()`.

`testmod()` and `testfile()` both return a tuple of (*number of errors*, *number of tests*).

There is also an advanced API, but you probably don't want to look at that until you have used the basic doctest in anger and considered alternatives.

Integrating doctest and unittest

Create a function called `load_tests`

- To enable doctest tests to be used as unittest tests

`doctest.DocFileSuite`

- Uses tests from a file (like `doctest.testfile`)
- Returns a `unittest.TestSuite` object

`doctest.DocTestSuite`

- Uses tests from a module (like `doctest.testmod`)
- Returns a `unittest.TestSuite` object
- `TestSuite.run()` will run all the tests in the suite

QA

434

The two testing modules in the Python standard Library are not necessarily incompatible. The **doctest** module has the methods shown to generate `unittest` objects from doctests.

Reporting flags can be set with `doctest.set_unittest_reportflags()`.

pytest – worked example (1)

A worked example using a custom class and error:

We'll now use pytest to create a suitable unit test...

```
import re

class NotStringError(TypeError):
    pass

class Palindrome(str):
    """
    Class to process potential palindrome strings.
    """

    def __init__(self, string: str):
        if not isinstance(string, str):
            raise NotStringError("You must initialise with a string.")
        self.__text = string

    def is_palindrome(self) -> bool:
        self.__clean_palindrome()
        return self.__text == self.__text[::-1]

    def __clean_palindrome(self) -> None:
        self.__text = re.sub("[^a-z]", "", self.__text,
            flags=re.I).lower()
```

pytest – worked example (2)

```
import pytest
from palindromeclass import Palindrome, NotStringError

@pytest.fixture
def simple_palindrome():
    """Returns a valid simple palindrome object"""
    return Palindrome("racecar")

def test_simple_valid_palindrome(simple_palindrome):
    assert simple_palindrome.is_palindrome() is True

def test_return_type_valid(simple_palindrome):
    assert isinstance(simple_palindrome.is_palindrome(), bool) is True

def test_simple_invalid_raises_exception_on_notstring():
    with pytest.raises(NotStringError):
        simplePal = Palindrome(10)

def test_simple_valid_wrong_case_palindrome():
    awkwardPal = Palindrome("RaCEcAr")
    assert awkwardPal.is_palindrome() is True

def test_complex_palindrome_clean():
    awkwardPal = Palindrome("R*a$1ce21122121car1221%$£!")
    assert awkwardPal.is_palindrome() is True
```

QA

436

As discussed, fixtures separate test data boilerplate from specific test logic (refactoring).

They are used to replace duplicated setup code, e.g. creating test objects, however each test will have a unique instance of the object.

In this example, the `simple_palindrome` method returns an instance of the `Palindrome` class which can then be passed to different tests.

Note.

Always add a docstring to the fixture as it will show with `pytest --fixtures`

pytest – worked example (3)

```
@pytest.mark.parametrize("string, outcome", [
    ("racecar", True),
    ("raeccar", False),
    ("banana", False),
    ("race1car", True),
    ("a nut for a jar of tuna", True),
    ("A man, a plan, a canal, Panama!", True)
])
def test_batch_palindromes(string, outcome):
    simplePal = Palindrome(string)
    assert simplePal.is_palindrome() == outcome
```


pytest – worked example (4)

```
python -m pytest test_palindrome.py -v
===== test session starts =====
collecting ... collected 11 items

test_palindrome.py::test_simple_valid_palindrome PASSED [ 9%]
test_palindrome.py::test_return_type_valid PASSED [ 18%]
test_palindrome.py::test_simple_invalid_raises_exception_on_notstring PASSED [ 27%]
test_palindrome.py::test_simple_valid_wrong_case_palindrome PASSED [ 36%]
test_palindrome.py::test_complex_palindrome_clean PASSED [ 45%]
test_palindrome.py::test_batch_palindromes[racecar-True] PASSED [ 54%]
test_palindrome.py::test_batch_palindromes[raeccar-False] PASSED [ 63%]
test_palindrome.py::test_batch_palindromes[banana-False] PASSED [ 72%]
test_palindrome.py::test_batch_palindromes[race1car-True] PASSED [ 81%]
test_palindrome.py::test_batch_palindromes[a nut for a jar of tuna-True] PASSED [ 90%]
test_palindrome.py::test_batch_palindromes[A man, a plan, a canal, Panama!-True] PASSED [100%]

===== 11 passed in 0.04s =====
```



All tests appear to have completed successfully.

438

We'll revisit pytest a later chapter to explore some of the more complex options available.

virtualenv – why is this needed?

Python has the ability to download, store and resolve packages

- Installation usually via `pip` or `easy_install`
- Two types of packages:
 - **System packages** – part of the standard Python Library
 - **Site packages** – third party libraries

Packages can be stored in different locations

- **System packages** are stored in a separate subdirectory in the parent directory specified by `sys.prefix`
- **Site packages** are stored in separately named folders pointed to by `site.getsitepackages`

What's the problem?

- Each project you build has its own *specific* package and module *requirements*
 - Package versions *vary*, e.g. v1.1 and v1.6 could be *very* different
 - Projects *may only work with certain versions* of a package
- Python *can't distinguish* between different versions in the site-packages folder because...
 - They are *only identified by name*

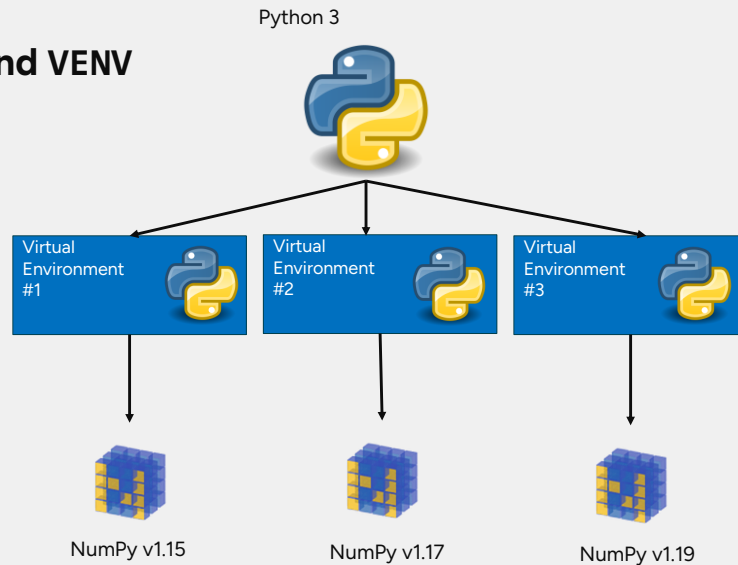
QA

439

It's due to this problem, managing specific versions of site packages, that Python developers use **virtual environments**.

This can be achieved using a number of different tools, with **virtualenv** and **venv** being the most commonly used.

virtualenv and VENV



Virtual environments allow each to install and use different versions of the same package.

QA

440

Python 3 uses **venv** and it should already have been installed as part of the standard library.

A general (Linux) approach can be to:

1. Create a new virtual environment folder by running the venv module for a new environment:

```
Python -m venv myenv1
```

2. This creates a new folder called "myenv1" with a full hierarchy below it including bin, include and lib folders plus shortcuts to different Python tools

3. To use a new environment's packages in isolation, you need to activate it first.

Do this by typing:

```
source myenv1/bin/activate
```

4. Your prompt should now reflect the named environment, i.e. myenv1
5. (You can always switch back to "global" mode by using "deactivate")

While you are in "global" mode, packages and modules installed in an activate virtual environment cannot be used.

The opposite is equally true. Or indeed between different virtual environment – each is essentially separate.

This provides a degree of segregation in terms of the installation and use of

different packages and modules for different projects – all controlled via virtual environments.

Of course, there are additional concerns and some Operating System differences to be wary of – but the best way to learn virtual environments is to experiment with them.

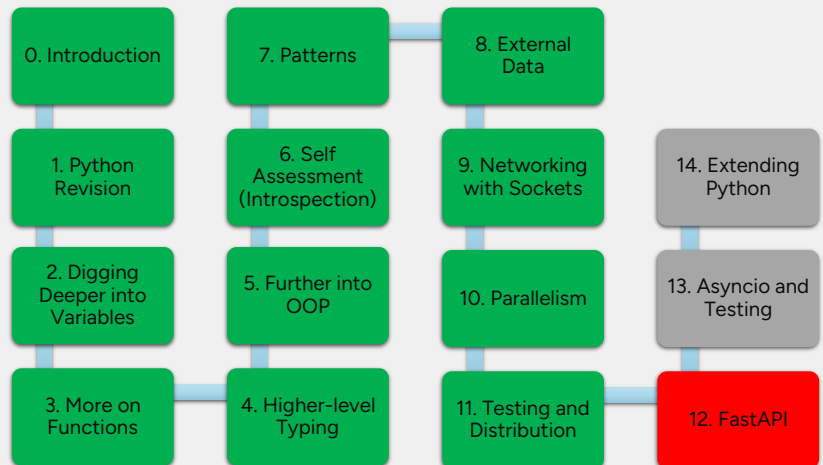
Note. in a typical Windows environment try to activate a virtual environment using `\myenv1\Scripts\activate`

Intermediate Python

FastAPI Fundamentals



Module roadmap



QA

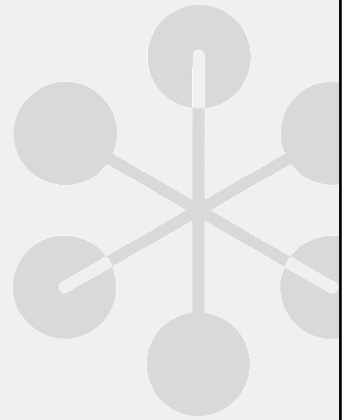
FastAPI Fundamentals

Learning outcomes:

- Explore how to develop scalable, production-ready FastAPI applications.

Prerequisites:

- Delegates are required to have knowledge and skills relating to basic Python programming, including data structures, functions, loops and flow control.

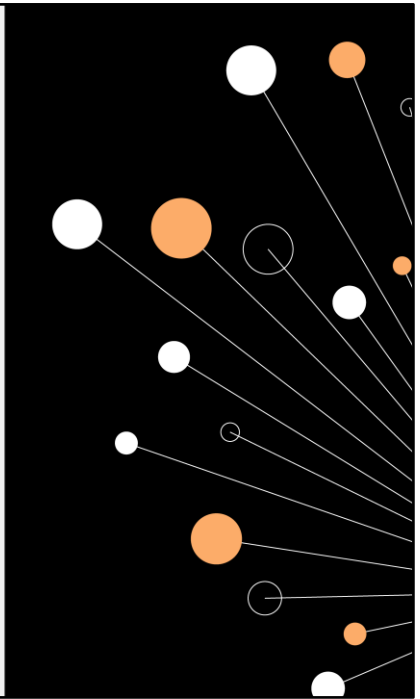


Learning objectives

By the end of this session, you should be able to:

- Scaffold a production-ready FastAPI application
- Structure projects for scalability and maintainability
- Implement robust GET and POST endpoints
- Understand why and how to use Templates
- Apply validation and dependency injection
- Serve multiple response types (JSON, XML, CSV)
- Follow API design best practices

QA



FastAPI Hands-on Session: From Setup to Production

FASTAPI Fundamentals



What is FastAPI?

Modern, high-performance Python web framework

- Built on:
 - Starlette (web layer)
 - Pydantic (validation & data models)

Key features:

- Type-driven validation
- Automatic OpenAPI documentation
- Async-first design

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{id}")
def read_item(id: int):
    return {"id": id}
```

QA

446

FastAPI is a modern Python web framework specifically designed for building APIs that are fast, reliable, and easy to maintain. It leverages Python type hints, meaning the types you specify directly shape your API's behaviour.

FastAPI is built on two main components: Starlette, which manages the web layer including routing, asynchronous requests, and middleware; and Pydantic, which handles data validation and parsing to ensure incoming data conforms to expected types and formats.

One major benefit of FastAPI is its use of type hints to automatically manage validation, serialization, and documentation. This reduces boilerplate code while providing strong data integrity guarantees.

For the example on the slide, FastAPI will automatically validate that the `id` is an integer and return an error if it is not.

FastAPI also auto-generates API documentation using OpenAPI, accessible through `/docs` (interactive Swagger UI) or `/redoc`.

Designed for high performance, FastAPI uses asynchronous programming (`async/await`) and the ASGI standard, making it well-suited for scalable, production-ready systems.

In summary, FastAPI offers a powerful combination of ease of use, robust validation, and high performance, making it an excellent choice for modern API development.

FastAPI Fundamentals - endpoints

- APIs are built using **path operations (endpoints)**
- Request → Validation → Processing → Response
- Python type hints power:
 - Input validation
 - Output serialization
 - API documentation

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{id}")
def get_item(id: int, q: str | None = None):
    return {"id": id, "query": q}
```

QA

447

FastAPI is structured around the concept of path operations, commonly referred to as endpoints, each of which defines how the API responds to a particular HTTP request.

Every incoming request follows a well-defined lifecycle: it is received, validated, processed by your handler function, and a response is returned to the client.

A central principle of FastAPI is that Python type hints do more than just annotate code — they actively drive behaviour. Rather than writing separate validation or serialisation logic, you declare types directly in your function signature and FastAPI takes care of the rest. In practice, this means FastAPI will automatically validate that `id` is an integer, treat `q` as an optional query parameter, and generate documentation for both inputs without any additional effort on your part.

This approach significantly reduces boilerplate and promotes consistency throughout your API. Serialisation is handled automatically as well, with Python objects converted to JSON responses without any extra configuration.

In short, FastAPI unifies routing, validation, and serialisation into a single, elegant model — all driven by type hints.

HTTP Basics Refresher

Common Methods

- GET → Retrieve data
- POST → Create resources
- PUT/PATCH → Update
- DELETE → Remove

Standard Status Codes

- 200 → OK
- 201 → Created
- 400 → Bad Request
- 404 → Not Found

```
@app.get("/items")
def get_items():
    return []

@app.post("/items")
def create_item():
    return {"status": "created"}

@app.post("/items", status_code=201)
def create_item():
    return {"message": "Item created"}
```

QA

448

APIs communicate through HTTP, making it important to understand the key HTTP methods and status codes when developing applications with FastAPI.

Each HTTP method serves a distinct purpose:

- **GET** retrieves data without making any changes.
- **POST** creates new resources.
- **PUT** and **PATCH** update existing resources.
- **DELETE** removes resources.

FastAPI provides a straightforward way to associate these HTTP methods with Python functions through decorators.

In addition to HTTP methods, status codes communicate the outcome of a request:

- **200 OK** – The request was successful.
- **201 Created** – A new resource was created successfully.
- **400 Bad Request** – The request contains invalid data or input.
- **404 Not Found** – The requested resource could not be found.

FastAPI also allows developers to define response status codes explicitly. Using appropriate HTTP methods and status codes helps create APIs that are predictable, consistent, and easy to integrate with. This is a fundamental aspect of designing robust, production-ready APIs.

Environment Setup and Running your App

1. Create and activate a virtual environment:

```
python -m venv venv
source venv/bin/activate # macOS/Linux
venv\Scripts\activate   # Windows
```

Run these from within
the folder your code is
located

2. Install Dependencies:

```
pip install fastapi uvicorn jinja2
```

3. Start the FastAPI server

```
uvicorn app.main:app --reload
```

The name of your
python module

```
PS C:\QAL\FASTAPI> uvicorn echo_items_id:app --reload

INFO:      Will watch for changes in these directories: ['C:\\QAL\\QAINTPYFS\\FASTAPI']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [2360] using StatReload
...
INFO:      127.0.0.1:57685 - "GET / HTTP/1.1" 404 Not Found
INFO:      127.0.0.1:57685 - "GET /favicon.ico HTTP/1.1" 404 Not Found
```

QA

449

Before building a FastAPI application, it is important to create an isolated Python environment. This ensures that project dependencies do not conflict with other projects on your system.

A **virtual environment (venv)** allows you to manage dependencies locally within your project. The slide shows you the two entries you need to make into a terminal window in order to create and activate a virtual environment (step 1). Once activated, install the required dependencies (`fastapi`, `uvicorn` and for later use `jinja2` – step 2). Using a virtual environment ensures that your project is portable and reproducible, which is essential in team environments and production systems.

Once your environment is set up, you can run your FastAPI application using an ASGI server such as Uvicorn (Step 3). This command consists of:

- `app.main` → the Python module
- `pathapp` → the FastAPI instance
- `--reload` → automatically restarts the server when code changes

When the server starts your API will be available at:

`http://127.0.0.1:8000`. In order to test an end-point you will need to add data to the end of the url that maps to the route signature declared in the `app.get` decorators e.g.:

```
@app.get("/items/{id}") => http://127.0.0.1:8000/items/123
```

Launching when Developing the App

Instead of using Uvicorn directly, create a small launcher script.

```
# Code debuggable in PyCharm and VSC
from fastapi import FastAPI
import uvicorn

app = FastAPI()

@app.get("/items/{id}")
def read_item(id: int):
    return {"id": id}

if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=8000)
```

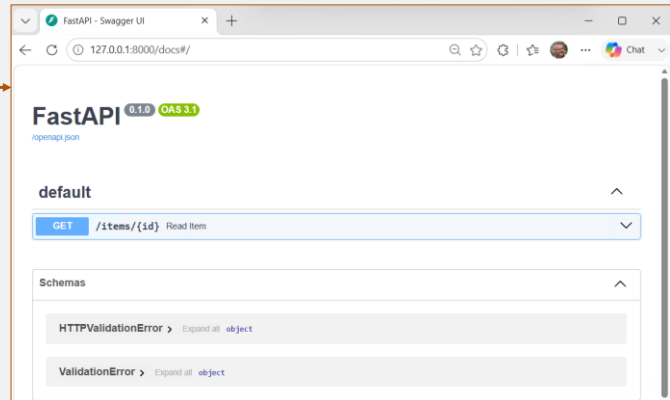
Interactive API Documentation

FastAPI automatically generates:

- Swagger UI → /docs
- ReDoc → /redoc

Benefits:

- Explore endpoints
- Test APIs interactively
- View request/response schemas



QA

451

FastAPI automatically creates interactive API documentation using the OpenAPI standard, one of its most valuable features since it eliminates the need for manual documentation.

When your application is running, you can access two built-in documentation interfaces:

Swagger UI → `http://127.0.0.1:8000/docs`

ReDoc → `http://127.0.0.1:8000/redoc`

Swagger UI is interactive, allowing you to:

- Explore available endpoints
- Provide input parameters
- Execute requests directly from the browser
- View responses in real time

ReDoc offers a cleaner, read-only view, ideal for reviewing the API structure.

These documentation pages are generated automatically from your code. For example, an endpoint like:

```
@app.get("/items/{id}")
def get_item(id: int):
    return {"id": id}
```

Will automatically appear in /docs with details such as:

- Parameter type (`int`)
- Validation rules
- Example responses

This close integration ensures your documentation remains accurate and up to date, which is crucial for collaboration and production APIs.

Production-Ready Architecture

FASTAPI Fundamentals



Recommended Project Structure

Why Project Structure Matters

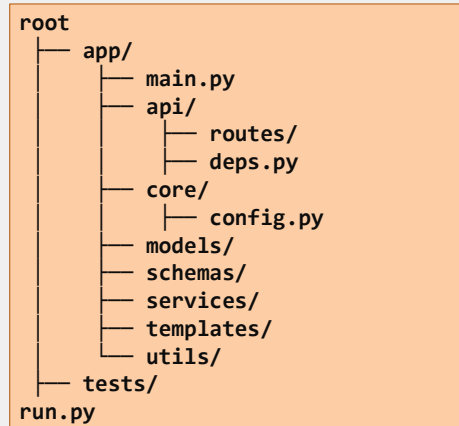
- Improves maintainability and readability
- Enables scalable development
- Separates concerns (API, logic, data, config)
- Supports team collaboration

Key Design Principles

- Keep endpoints thin (delegate logic to services)
- Separate input/output models (schemas)
- Group related routes together
- Centralise configuration

Anti-Pattern to Avoid

- Putting all logic in `main.py`
- Mixing validation, business logic, and routing



QA

A well-structured project layout is essential when building production-ready FastAPI applications. While small demos may place everything in a single file, real-world applications should be modular and organised by responsibility.

The goal of a good structure is to improve:

- Maintainability
- Readability
- Scalability as the application grows

A typical FastAPI project is organised into separate layers:

- **main.py** → Application entry point where FastAPI is created and routers are registered
- **api/routes** → Defines endpoints grouped by feature (e.g. items, users)
- **schemas** → Pydantic models for request and response validation
- **models** → Database models (if using an ORM)
- **services** → Business logic, kept separate from API layer
- **core/config.py** → Application configuration and settings
- **templates/static** → Used for rendering frontend content

This separation ensures that each part of the application has a clear responsibility, which makes the code easier to test and extend.

For example, instead of putting everything in one file you would be better defining schemas, services, and routes in separate modules. This approach aligns with best practices used in larger production systems and teams.

Application Entry Point

Role of main.py

- Application entry point
- Creates FastAPI instance
- Registers routers
- Configures middleware and startup logic

Key Concepts

- FastAPI() → creates the app
- include_router() → attaches modular routes
- prefix → groups endpoints under a path
- tags → improves API documentation

Best Practices

- Keep main.py minimal
- Do not place business logic here
- Use routers for feature separation
- Scale by adding more routers, not more code in main

```
from fastapi import FastAPI
from api.routes import items
app = FastAPI()
app.include_router(items.router,
                    prefix="/items", tags=["Items"])
```

app/main.py

Located in
same folder as
the app folder

```
import uvicorn

if __name__ == "__main__":
    uvicorn.run("app.main:app", host="127.0.0.1", port=8000)
```

run.py

QA

454

The main.py file acts as the **entry point** of your FastAPI application. It is responsible for creating the FastAPI app instance and wiring together the different parts of your application.

In a production-ready setup, main.py should remain **small and focused**, primarily handling application setup rather than business logic.

The slide hosts a typical example. Here, the FastAPI() instance creates the application, and include_router() connects your route modules to the app.

The prefix argument ensures that all routes in the router are grouped under a common path (e.g. /items). The tags argument helps organise endpoints in the API documentation.

This approach allows you to **split your API into multiple routers**, each responsible for a specific feature area, without cluttering the main file. For example, each feature (items, users, orders) can have its own router, making the application easier to scale and maintain.

Keeping main.py minimal ensures that your application remains clean, readable, and easy to extend as new features are added.

The use of the run.py file allows for the code to be launched from within an IDE and be debugged in the conventional way. The "app.main:app" clause

tells uvicorn where to find your FastAPI application instance.

Configuration Management

What is Configuration Management?

- Separating settings from application code
- Supporting multiple environments (dev, test, prod)
- Avoiding hardcoded values

Common Configuration Types

- Database connection strings
- API keys / secrets
- Feature flags
- Logging levels

app/core/config.py

```
from pydantic import BaseSettings

class Settings(BaseSettings):
    app_name: str = "FastAPI App"
    debug: bool = True

settings = Settings()
```

How FastAPI Handles It

- Use BaseSettings (Pydantic)
- Load from environment variables
- Support .env files

Best Practices

- Never hardcode secrets
- Use environment-specific configs
- Centralise config access
- Inject config using dependencies (Depends)

app/api/routes/accounts.py

```
from fastapi import Depends
from app.api.deps import get_db

@router.get("/")
def get_items(db=Depends(get_db)):
    return {"db": db}
```

QA

455

Configuration management is the practice of **separating application settings from code**, so they can vary between environments (development, testing, production) without changing the codebase. Instead of hardcoding values like database URLs, API keys, or environment flags, these are stored externally and loaded into the application at runtime. FastAPI commonly uses **Pydantic's BaseSettings** to manage configuration in a structured and type-safe way (see example on slide).

Pydantic will automatically:

- Validate configuration values
- Convert types (e.g. strings → booleans)
- Load values from environment variables

For example, you can override values via environment variables:

```
export APP_NAME="My Production API"
```

You can also use a .env file for local development:

```
APP_NAME=My Local App
DEBUG=True
```

This approach ensures your application behaves differently depending on where it runs, without changing code. Configuration is typically accessed

throughout the application using dependency injection (see example on slide). This keeps configuration **centralised, reusable, and testable**.

Routers (Modular APIs) - 1

What are Routers?

- Modular components for grouping endpoints
- Built using APIRouter
- Help organise large applications

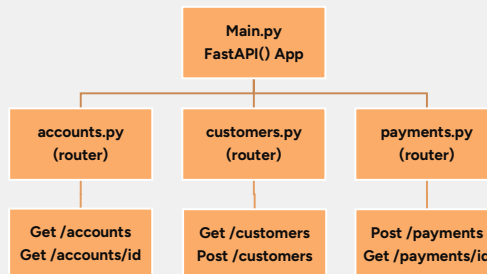
Feature-Based Routing

- Group endpoints by domain:
 - accounts.py
 - customers.py
 - payments.py
- Keeps related logic together
- Improves readability and scalability

app/api/routes/accounts.py

```
from fastapi import APIRouter
router = APIRouter(prefix="/items", tags=["Items"])

@router.get("/")
def list_items():
    return []
```



QA

456

Routers in FastAPI are used to organise endpoints into logical groups, making your application easier to manage as it grows. Instead of defining all endpoints in a single file, you split them into smaller, feature-based modules.

A router is created using APIRouter, which works similarly to the main FastAPI app but is designed for modular use. For example, you can define routes for "items" in a dedicated file (see code on slide).

This groups all item-related endpoints under the /items path and labels them in the API documentation.

Routers (Modular APIs) - 2

Including Routers in the App

- prefix → URL grouping
- tags → API documentation grouping

Benefits

- Cleaner code structure
- Easier to maintain and extend
- Better team collaboration
- Scales to large applications

app/main.py

```
from fastapi import FastAPI
from app.api.routes import items

app = FastAPI()
app.include_router(items.router, prefix="/items", tags=["Items"])
```

QA

457

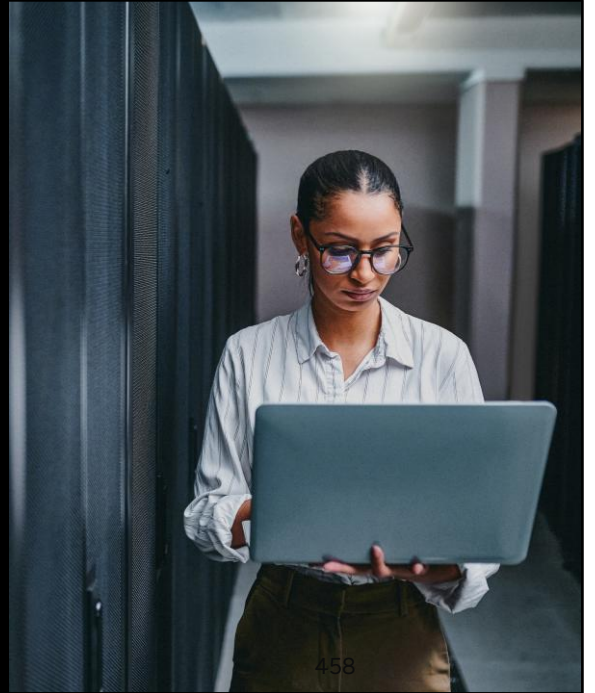
Routers promote a feature-based structure, where each domain (e.g. items, users, orders) has its own file or module. This improves readability, maintainability, and team collaboration.

To make these routes available in your application, you must include them in the main app (see code on slide)

FastAPI then combines all routers into a single application. This approach keeps your codebase clean and scalable, especially in larger systems where many endpoints are required.

GET Endpoints

FASTAPI Fundamentals



458

GET endpoints and Query Parameters

A **GET** endpoint is a URL in a web API that responds to the HTTP **GET** method. GET is used **to retrieve data** without changing anything on the server.

What are Query Parameters?

- Passed in URL after ?
- Used for filtering and searching
- Optional by default

Benefits

- Flexible data retrieval
- No need for multiple endpoints
- Automatically validated

GET /accounts?accounts_id=3

```
@router.get("/")
def get_account_details(account_id: str):
    if account_id:
        return {
            "account_id": account_id,
            "customer_id": 2,
            "balance": 500.0
        }
    else:
        return {}
```

```
{
  "account_id": "3",
  "customer_id": 2,
  "balance": 500
}
```

QA

459

Query parameters let clients filter or customize results without altering the endpoint path. They appear in the URL after a ? and are typically used for searching, filtering, or pagination. In FastAPI, they're defined as function arguments that aren't part of the path.

```
@router.get("/")
def get_account_details(account_id: str | None = None):
    return {"account_id": account_id}
```

Here, `account_id` is an optional query parameter. If supplied, FastAPI captures its value; otherwise, it falls back to `None`. A request such as:

GET /accounts?account_id=3

Returns:

```
{
  "account_id": "3",
  "customer_id": 2,
  "balance": 500
}
```

FastAPI handles this automatically by:

- Parsing query parameters from the URL
- Validating their types

- Documenting them in /docs

You can also define multiple query parameters at once:

```
def get_customer_account_details(account_id: str |  
    None = None, customer_id: str | None = None):
```

Query parameters are key to building flexible APIs that give clients control over how data is fetched.

Path Parameters

What are Path Parameters?

- Part of the URL path
- Used to identify resources
- Required by default

Key Advantage

- Strong typing and automatic validation

```
GET /accounts/1
```

```
@router.get("/{account_id}")
def get_account_details(account_id: int):
    return {
        "account_id": account_id,
        "balance": 500.0
    }
```

```
{
  "account_id": 3,
  "balance": 500
}
```

QA

460

Path parameters serve to pinpoint specific resources within an API by being directly included in the URL path. They are commonly used to retrieve, update, or delete individual entities. In FastAPI, these parameters are specified within the route path using curly braces {}. For example:

```
@router.get("/{id}")
def get_account_details(account_id: int):
    return {"id": account_id}
```

Here, `id` is a mandatory parameter that uniquely identifies an item. A request such as:

```
GET /items/5
```

would return:

```
{"id": 5}
```

FastAPI automatically:

- Extracts the value from the URL
- Converts it to the specified type (`int`)
- Validates the input

If an invalid value is given, for example:

```
GET /accounts/abc
```

FastAPI responds with a 422 validation error.

Path parameters are essential in resource-based API design, ensuring each resource has a unique URL.

Validation & Type Safety

What is Validation?

- Ensures input data is correct
- Prevents invalid requests

Built-in Features

- Type enforcement
- Automatic error responses (422)
- Constraint validation

Benefits

- Fewer bugs
- Cleaner logic
- Safer APIs

```
@router.get("/customers/{customer_id}/accounts/{account_id}")
def get_customer_account_details(customer_id: int, account_id: int):
    return get_account(customer_id, account_id)
```

GET /accounts/customers/{customer_id}/accounts/{account_id} Get Customer Account Details

Parameters

Name	Description
customer_id * required	
integer (path)	abc
account_id * required	
integer (path)	xyz

Please correct the following validation errors and try again.

- For 'customer_id': Value must be an integer.
- For 'account_id': Value must be an integer.

Execute

QA

461

One of FastAPI's standout features is its automatic validation powered by Python type hints. By simply specifying types in your function parameters or data models, FastAPI guarantees that incoming data conforms to the expected format. For example:

```
@router.get("/{account_id}")
def get_iaccount_details(account_id: int):
    return {"id": account_id}
```

If a client sends a request like:

GET /items/abc

FastAPI will reject it and respond with a 422 Unprocessable Entity status. Validation is also enforced on query parameters and request bodies, ensuring all inputs are thoroughly checked before reaching your core business logic. For more advanced validation, you can leverage Pydantic models (best placed in the schemas folder):

```
from pydantic import BaseModel, Field
class Account(BaseModel):
    account_id: int
    customer_id: int
    balance: float = Field(gt=0)
```

This approach guarantees:

- Required fields must be included
- Values meet specified constraints (e.g. balance must be > zero)

Validation improves API reliability and prevents invalid data from being processed.

Error Handling

What is Error Handling?

- Managing failure scenarios
- Returning meaningful responses

Common Status Codes:

- 400 → Bad Request
- 404 → Not Found
- 500 → Server Error

Best Practice

- Always handle expected failures
- Return clear messages

```
from fastapi import HTTPException
# Pretend this is data from a database
accounts = {
    1001: {"account_name": "Current Account", "balance": 1250.50},
    1002: {"account_name": "Savings Account", "balance": 5000.00},
}

@router.get("/{account_id}")
def get_account(account_id: int):
    account = accounts.get(account_id)

    if account is None:
        raise HTTPException(
            status_code=404,
            detail=f"Account {account_id} not found"
        )

    return account
```

Request URL	
http://127.0.0.1:8000/accounts/7	
Server response	
Code	Details
404 Undocumented	Error: Not Found
Response body	
{ "detail": "Account 7 not found" }	

QA

462

FastAPI handles errors through HTTP exceptions, enabling you to send meaningful, descriptive responses back to clients. Rather than returning incorrect or empty data, you should explicitly account for failure scenarios. For example:

```
from fastapi import HTTPException
@router.get("/{id}")
def get_item(id: int):
    if id != 1:
        raise HTTPException(status_code=404,
                             detail="Item not found")
    return {"id": id}
```

If the item doesn't exist, FastAPI responds with:

```
{
  "detail": "Item not found"
}
```

accompanied by a 404 Not Found status code.

Leveraging exceptions guarantees:

- **Consistent** error responses
- **Clear communication** with API consumers
- **Easier debugging** and monitoring

FastAPI also incorporates these errors into its OpenAPI documentation, making

them visible directly in /docs.

Solid error handling is a cornerstone of any robust, production-ready API.

Templates

FASTAPI Fundamentals



463

Templates - Fundamentals

What are Templates?

- Typically, HTML files with dynamic placeholders
- Rendered by the backend
- Allow mixing of static + dynamic content

What is Jinja2?

- Python templating engine
- Uses `{{ }}` for variables
- Supports loops, conditions, and filters

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{{title}}</title>
</head>
<body>

</body>
</html>
```

`app/templates/index.html`

Why Use Templates?

- Build simple UIs without a frontend framework
- Render server-side HTML
- Combine APIs with web pages

Key Requirement

- Always pass request into the template context

```
from fastapi.templating import Jinja2Templates
...
templates = Jinja2Templates(directory="app/templates")
...
@router.get("/home")
def render_home(request: Request):
    return templates.TemplateResponse(
        request,
        name="index.html",
        context={
            "title": "Home"
        }
    )
```

`app/api/routes/accounts.py`

464

While FastAPI is best known for building APIs, it's also capable of rendering HTML pages through templates — handy when you want to return dynamic content like dashboards or basic web interfaces.

Templates let you blend static HTML with data your application generates on the fly. Rather than hardcoding values directly into your HTML, you pass data from a FastAPI route into the template at request time.

To handle templating, FastAPI relies on Jinja2, a widely-used Python templating engine. Getting started involves creating a templates directory (e.g., `app/templates`) and setting it up accordingly.

When rendering a template, two things are required: the Request object and a context dictionary. The Request object is mandatory because Jinja2 depends on it internally. Within the HTML file itself, Jinja2 syntax lets you drop in dynamic values wherever needed.

This setup makes it possible to build pages whose content changes dynamically based on your application's data. It's a common approach when pairing FastAPI with a frontend, or when building simple server-rendered apps without a separate frontend framework.

FastAPI Templates for Internal (HTML) Admin Pages

- FastAPI can serve HTML pages as well as JSON (and XML) APIs.
- Jinja2 templates let you create lightweight, server-rendered admin dashboards, operational tools, and reporting interfaces without the need for a separate front-end framework.

```
from fastapi.templating import Jinja2Templates

router = APIRouter(prefix="/admin", tags=["Admin"])
templates = Jinja2Templates(directory="app/templates")

@router.get("/admin/customers")
async def customer_dashboard(request: Request):
    customers = [
        {"id": "C1234", "name": "Sadia Liaqat", "risk": "Low"},
        {"id": "C5678", "name": "Kamran Saleem", "risk": "High"},
    ]
    return templates.TemplateResponse(
        request=request,
        name="admin/customers.html",
        context={"customers": customers}
    )
```

QA

465

- FastAPI is commonly used for APIs but can also return HTML.
- Templates are ideal for internal administrative interfaces.
- The same FastAPI application can expose both REST APIs and server-rendered pages.
- This approach works well when a full React/Angular front end would be excessive.

Templates benefits include:

- Simple operational dashboards
- Fast development for internal users
- Reuse of existing FastAPI authentication and business logic
- No separate Single Page Deployment (SPA) deployment pipeline
- Can consume the same backend services as your APIs

Compliance Customer Review Page

Customer Risk Rating

Customer	Risk Level
Sadia Liaqat	Low
Kamran Saleem	High

```
<!DOCTYPE html>
<html>
<head>
  <title>Customer Review</title>
</head>
<body>

<h2>Customer Risk Rating</h2>

<table border="1">
  <tr>
    <th>Customer</th>
    <th>Risk Level</th>
  </tr>

  {% for customer in customers %}
  <tr>
    <td>{{ customer.name }}</td>
    <td>{{ customer.risk }}</td>
  </tr>
  {% endfor %}
</table>

</body>
</html>
```

QA

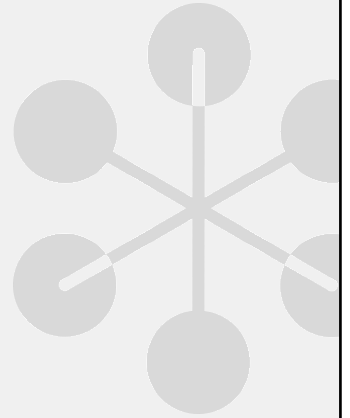
466

- The page is rendered on the server and sent as HTML.
- Jinja2 placeholders (`{{ }}`) display data and loops (`{% for %}`) generate rows dynamically.
- In financial services, these pages are commonly used for investigations, compliance review, and operational support rather than customer-facing experiences.

Using Templates to generate non-HTML Documents

Templates are useful whenever you're producing structured text with placeholders. Examples include:

- Email bodies (HTML or plain text)
- Generated configuration files
- SQL scripts
- Kubernetes manifests
- Markdown reports
- XML documents



Using Templates – A worked example (1)

Templates can be any file type

- HTML
- TXT
- XML
- MD

Injected data is marked with jinja placeholders

- `{{ field_name }}`
- `{% for result in results %}`
- `{% if result.status_code != 200 %}`
- `{% else %}`
- `{% endif %}`
- `{% endfor %}`

`app/templates/emails/overdraft_warning.txt`

Subject: Overdraft Warning

Dear `{{ customer_name }}`,

Your account `{{ account_id }}` is about to go into overdraft.

Current balance: £`{{ balance }}`

Please ensure sufficient funds are available to avoid fees.

Kind regards,
Your Bank

QA

468

Note: the template file will usually contain embedded jinja instructions that represent simple output values (parameter placeholders) surrounded in double sets of curly brackets: `{{ customer_name }}` and logic instructions that can be loops, conditionals, etc. marked by double percent symbols inside a single set of curly brackets: `{% for result in results %}`. This code is expecting to receive a collection called `results` that it will iterate around and is equivalent to Python's `for result in results`.

A fuller html-based example:

```
<body>
<h1>Batch Transaction Results</h1>
{% for result in results %}
<div class="account">
  <h3>Account: {{ result.account_number }}</h3>
  {% if result.status_code != 200 %}
    <p class="error">{{ result.error }}</p>
  {% else %}
    <p>Transactions: {{ result.count }}</p>
    <ul>
      {% for tx in result.transactions[:3] %}
        <li>{{ tx }}</li>
      {% endfor %}
    </ul>
  </div>
{% endfor %}
```

```
        </ul>
    {% endif %}
</div>
{% endfor %}
</body>
```

Using Templates – A worked example (2)

Create generation logic as a service

Even though it's plain text, we still use Jinja2

- It works for any text, not just HTML
- Can generate any structured text, including:
 - email bodies
 - Reports
 - logs

app/services/notification_service.py

```
from fastapi.templating import Jinja2Templates

templates = Jinja2Templates(directory="app/templates")

def generate_overdraft_email(
    customer_name: str,
    account_id: int,
    balance: float) -> str:
    template = templates.get_template("emails/overdraft_warning.txt")

    return template.render(
        customer_name=customer_name,
        account_id=account_id,
        balance=balance
    )
```

Using Templates – A worked example (3)

app/api/routes/accounts.py

Plain Text Benefits

- Works with all email clients
- Simpler and more reliable
- Easier to test and debug
- Preferred for system notifications

```
from fastapi import APIRouter
from app.services.notification_service import generate_overdraft_email

@router.get("/{account_id}/overdraft-email")
def overdraft_email(account_id: int):
    email_content = generate_overdraft_email(
        customer_name="Ernst Blofeld",
        account_id=account_id,
        balance=-51.47
    )

    return {"email_preview": email_content}
```

```
{
  "email_preview": "Subject: Overdraft Warning\n\nDear Ernst Blofeld,\n\nYour account 12 is about to go into overdraft.\n\nCurrent balance: £-51.47\n\nPlease ensure sufficient funds are available to avoid fees.\n\nKind regards,\nYour Bank"
}
```

QA

470

POST Endpoints + Safe Processing

FASTAPI Fundamentals



Request Models (Input Validation)

What are Request Models?

- Define structure of incoming data
- Use Pydantic (BaseModel)
- Automatically validated

app/api/routes/accounts.py

```
@router.post("/")
def create_account(account: AccountCreate):
    return account
```

The schemas layer is responsible for:

- Defining input models (request bodies)
- Defining output models (responses)
- Enforcing validation rules

app/schemas/account_schema.py

```
from pydantic import BaseModel, Field

class AccountCreate(BaseModel):
    customer_id: int
    balance: float = Field(gt=0)
```

Ensures balance is greater than 0

QA

472

When building POST endpoints, data is typically sent in the request body. FastAPI uses Pydantic models to define the structure and validation rules for this incoming data.

A request model ensures that:

- Required fields are present
- Data types are correct
- Constraints (e.g. positive numbers) are enforced

For example:

```
from pydantic import BaseModel, Field
class AccountCreate(BaseModel):
    customer_id:
    int balance: float = Field(gt=0)
```

In this model:

- customer_id must be an integer
- balance must be greater than zero

Using this in a route:

```
@router.post("/accounts")
def create_account(account: AccountCreate):
    return account
```

FastAPI will:

- Parse JSON input automatically
- Validate the data
- Return a 422 error if validation fails

This approach removes the need for manual validation and ensures consistent, reliable input handling.

Response Models (Output Control)

app/api/routes/accounts.py

What are Response Models?

- Define output structure
- Control what data is returned
- Improve API consistency

Why Separate Models?

- Prevent over-posting
- Hide sensitive fields
- Improve clarity

```
@router.post("/", response_model=AccountOut, status_code=201)
def create_account(account: AccountCreate):
    # model_dump() converts a Pydantic model into a
    # standard Python dictionary by:
    # - Taking all fields from account
    # - Adding a new field: "account_id": 1
    return {"**account.model_dump(), "account_id": 1}
```

app/schemas/account_schema.py

```
from pydantic import BaseModel, Field

class AccountCreate(BaseModel):
    customer_id: int
    balance: float = Field(gt=0)

class AccountOut(AccountCreate):
    account_id: int
```

QA

473

Response models define the structure of the data sent back to clients, ensuring that only the intended fields are included and that responses remain consistent.

A best practice is to separate input and output models. For instance, when creating an account, the client should not supply an `account_id` because it is generated by the system. Example response model using Pydantic:

```
from pydantic import BaseModel
class AccountOut(BaseModel):
    account_id: int
    customer_id: int
    balance: float
```

Using a response model in an endpoint:

```
@router.post("/accounts", response_model=AccountOut)
def create_account(account: AccountCreate):
    return {
        "account_id": 1,
        "customer_id": account.customer_id,
        "balance": account.balance
    }
```

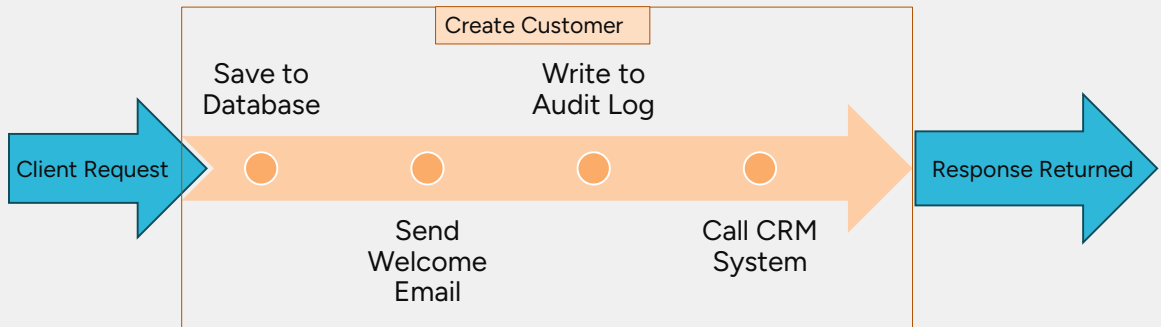
FastAPI will:

- Validate the response data
- Remove any unexpected fields
- Automatically document the response in the /docs interface

This approach keeps your API predictable and secure, especially when handling sensitive information.

Managing Background Tasks to Avoid Blocking - 1

The user waits for every operation to complete



QA

474

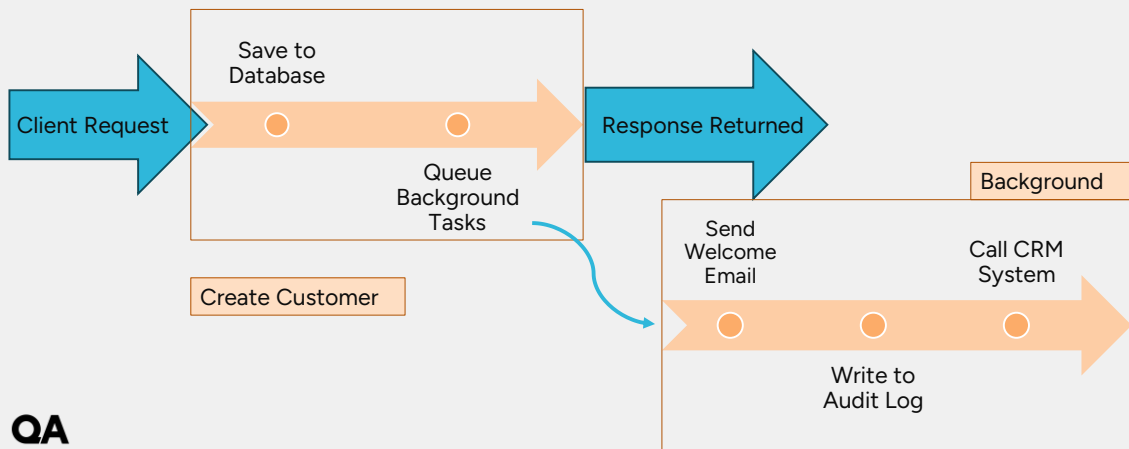
When a new investment account is opened:

- Create customer account
- Send onboarding email
- Write compliance audit record
- Notify CRM system
- Return success response

The customer experiences a slow response because the processes are executed sequentially. Each process must finish before the next one can begin, and the user interface may become unresponsive until all processing is complete.

Managing Background Tasks to Avoid Blocking -2

Background tasks allow an API to return a response immediately while non-essential work continues in the background.



QA

475

When a new investment account is opened:

- Create customer account
- Trigger Background tasks
- Return success response
- The customer receives a faster response because the background tasks (Send onboarding email, Write compliance audit record, Notify CRM system) run after the API returns.

BackgroundTasks - Example

BackgroundTasks allow for:

- Faster API responses
- Improved user experience
- Separation of core logic and side effects
- Simple to implement for lightweight background work

Best Practice:

- Use BackgroundTasks for small, non-critical operations.

```
from fastapi import BackgroundTasks
from datetime import datetime
import time

def record_audit_event(account_id: str):
    time.sleep(5) # sleep for 5 seconds
    print(f"Audit record created for {account_id}, "
          f"time: {(datetime.now())}")

@router.post("/accounts_with_background_tasks")
async def create_account(
    background_tasks: BackgroundTasks):
    account_id = "ACC12345"

    background_tasks.add_task(
        record_audit_event,
        account_id)

    return {
        "message": "Account created",
        "account_id": account_id,
        "time": datetime.now()
    }
```

QA

476

A FastAPI background task lets you kick off work triggered by a request, without making that work part of the response itself. The API should prioritize completing the customer's main action first, before anything else runs. Common examples include sending emails and/or logging audit records. While background tasks help keep things responsive, they still run inside the same process as the FastAPI application. For enterprise-grade systems, message queues tend to be the preferred choice, since they offer better reliability and scalability.

Providing API-Key Dependency (X-API-Key)

- API keys provide simple service-to-service security, while secrets and environment variables keep sensitive values out of source code.

app/api/deps.py

```
from dotenv import load_dotenv
import os

load_dotenv()

def get_api_key() -> str:
    return os.getenv("API_KEY")
```

app/.env

```
API_KEY=YqqZ7hI0mWw115TQx4Q4f5I6xM...Y
```

```
from fastapi import Depends, Header, HTTPException
from app.api.deps import get_api_key
from fastapi.security import APIKeyHeader
```

```
api_key_header = APIKeyHeader(
    name="X-API-Key",
    auto_error=False
)

def validate_api_key(
    x_api_key: str = Header(alias="X-API-Key"),
    api_key: str = Depends(get_api_key)
):
    if x_api_key != api_key:
        raise HTTPException(
            status_code=401,
            detail="Invalid API Key"
        )

    @router.get("/api/accounts/{account_id}",
        dependencies=[Depends(validate_api_key)]
    )
    async def get_account(account_id: str):
        return {"account_id": account_id}
```

app/api/routes/accounts.py

477

QA

The version shown demonstrates:

- Environment variables
- Dependency injection
- Custom headers
- Route protection

Note:

- The dependency executes before the endpoint logic.
- Authentication is applied consistently across multiple routes.
- The API key should never be hard-coded in source code.
- Secrets should be stored in environment variables, Azure Key Vault, or another enterprise secret-management solution (hence the use of the `.env` file. Note to access the file you will need to: `pipinstall python-dotenv`). When the code goes into production you should replace the use of the `.env` file and store the secret directly in the host environment. If you do this you won't need to change the application code.
- Make sure to add `.env` to your `.gitignore` file. Your secrets should never be committed to source control.
- API keys are commonly used for internal integrations, batch jobs, and

service-to-service communication. They do not identify individual users. You should consider using JSON Web Tokens if this is what required (see later).

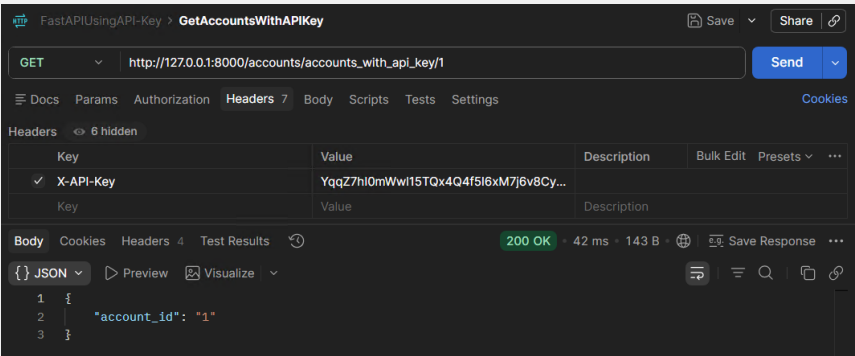
An API key can be generated using:

```
import secrets
print(secrets.token_urlsafe(32))
```

Testing X-API-Key from Postman

X-API-Keys can be tested by using a tool such as Postman.

- A graphical client for testing APIs that saves you from writing code every time you want to call an endpoint.



Idempotency, Validation Errors and Safe Defaults

An idempotent operation can be performed multiple times with the same result. Good APIs combine idempotency, validation, and safe defaults to prevent accidental data corruption

- **HTTP PUT** - Typically idempotent. Sending the same request 1 time or 100 times leaves the resource in the same state
 - Retrying a PUT is typically safe `PUT /accounts/123 { "email": "customer@bank.com " }`
- **HTTP POST** - Typically not idempotent. If the request is submitted twice, two payments may be created
 - Retrying a POST payment therefore not safe `POST /payments { "amount": 100.00 }`

Validation Errors

- Reject invalid requests before changing data e.g. Missing account number, negative payment amount...
 - Consider returning 422 Unprocessable Entity

Safe Defaults

- Prefer designs that minimise unintended consequences e.g. require confirmation before closing an account

QA

479

Safe Defaults:

Prefer designs that minimise unintended consequences:

- Require confirmation before closing an account
- Default transaction limits to restrictive values
- Validate input before persisting data
- Return errors instead of guessing missing values

Note:

- Network failures often cause clients to retry requests.
- Idempotent operations make retries safer.
- Validation should occur before business logic executes.
- In financial services, duplicate payments and duplicate trades are common examples of risks that idempotency helps prevent.
- Safe defaults reduce the impact of human error and unexpected client behaviour.

Key Takeaways:

PUT updates a known state and should be idempotent.

POST usually creates new resources and may not be idempotent.

Strong validation and safe defaults protect systems from bad data, retries

and user mistakes.

Dependency Injection

FASTAPI Fundamentals



Why Use Dependency Injection?

What is Dependency Injection?

- Providing shared resources to endpoints
- Keeps code modular and reusable
- Reduces duplication

Why Use Dependency Injection?

- Reuse common logic
- Improve testability
- Keep endpoints clean

Common Use Cases

- Database sessions
- Config settings
- Authentication

You do not need
to call `get_db()`
manually

app/api/routes/accounts.py

```
@router.get("/{id}")
def get_account(id: int):
    # repeated in every route
    db = "DB connection"
    return {"id": id, "db": db}
```



app/api/routes/accounts.py

```
from fastapi import Depends
from app.api.deps import get_db

@router.get("/{id}")
def get_account(id: int, db=Depends(get_db)):
    return {"id": id, "db": db}
```



app/api/deps.py

```
def get_db():
    return create_db_connection()
```

QA

481

Dependency Injection (DI) in FastAPI is a pattern for supplying shared functionality to your routes without tightly coupling code together.

Rather than creating resources — such as database connections or configuration objects — inside every route, you define them once and reuse them throughout your application.

This approach helps to:

- Reduce duplication
- Keep routes clean and focused on their core purpose
- Improve maintainability over time

A dependency is simply a function that provides something your route needs. For example:

```
def get_db():
    return create_db_connection()
```

Instead of calling this function manually in each route, FastAPI lets you inject it automatically via `Depends`. This way, your route stays focused purely on handling the request, leaving the setup of dependencies to FastAPI itself.

DI is particularly valuable for managing:

- Database connections
- Configuration settings
- Authentication

- Shared utilities

It's a fundamental building block for crafting scalable, production-ready APIs.

Practical Dependency Example (config)

```
from pydantic_settings import BaseSettings
```

```
class Settings(BaseSettings):  
    app_name: str = "Banking API"
```

```
settings = Settings()
```

```
app/core/config.py
```

```
from app.core.config import settings
```

```
def get_settings():  
    return settings
```

```
...
```

```
from fastapi import APIRouter, Depends  
from app.api.deps import get_settings
```

```
router = APIRouter(prefix="/info",  
tags=["Info"])
```

```
@router.get("/info")  
def get_info(settings = Depends(get_settings)):  
    return {"app_name": settings.app_name}
```

```
app/api/deps.py
```

```
app/api/routes/info.py
```

Why This Matters

- Centralised configuration
- Easier testing
- Cleaner architecture

QA

482

In real-world applications, dependencies are commonly used to provide configuration or shared services throughout the app.

For example, you might define a configuration object with Pydantic:

```
from app.core.config import settings  
def get_settings():  
    return settings
```

This dependency can then be injected into a route:

```
from fastapi import Depends  
@router.get("/info")  
def get_info(settings = Depends(get_settings)):  
    return {"app_name": settings.app_name}
```

In this setup:

- Configuration is defined once
- It is injected wherever needed
- Routes stay simple and focused

This method ensures:

- Centralized configuration management
- Easier handling of changes
- Simplified testing by allowing dependencies to be mocked

Dependency injection helps your application scale while maintaining clear

separation of concerns.

Multiple Response Types

FASTAPI Fundamentals



483

Response Classes Overview

Response Classes

- `JSONResponse` (default)
- `Response` (custom formats)
- `StreamingResponse` (files/data streams)
- `FileResponse` (static files)

When to Use

- JSON → most APIs
- XML → legacy systems
- CSV → exports/reports

Key Benefit

- Full control over responses

QA

484

FastAPI offers a range of response classes to accommodate various use cases. Although JSON is the default response format, you have full control over how responses are delivered.

Key response classes include:

- **`JSONResponse`** → The default format for most APIs
- **`Response`** → A generic response for custom content like XML
- **`StreamingResponse`** → Ideal for large files or data streams generated on the fly
- **`FileResponse`** → Used to serve static files

Each response class lets you customize:

- Content type
- Headers
- Response behaviour

For example:

```
from fastapi.responses import JSONResponse
return JSONResponse(
    content={"message": "ok"},
    status_code=200
)
```

Choosing the right response type ensures:

- Smooth client integration
- Efficient handling of data
- Compatibility with external systems

Mastering response classes helps you create APIs that are both flexible and adaptable to diverse needs.

JSON Response (Default)

JSON in FastAPI

- Default response format
- Automatically serialised
- No manual conversion needed

Benefits

- Easy to use
- Widely supported
- Built-in validation support

app/api/routes/accounts.py

```
@router.post("/{customer_id}/{balance}",
             response_model=AccountOut, status_code=201)
def create_account(account: AccountCreate):
    return {**account.model_dump(), "account_id": 1}
```

app/schemas/account_schema.py

```
from pydantic import BaseModel, Field

class AccountCreate(BaseModel):
    customer_id: int
    balance: float = Field(gt=0)

class AccountOut(AccountCreate):
    account_id: int
```

JSON

```
{
  "customer_id": 30,
  "balance": 33.33,
  "account_id": 1
}
```

POST /accounts/{customer_id}/{balance} Create Account

Parameters

No parameters

Request body **required**

Edit Value | Schema

```
{
  "customer_id": 30,
  "balance": 33.33
}
```

485

By default, FastAPI returns data as JSON responses — the most widely adopted format for APIs.

When a route returns a Python dictionary, FastAPI automatically:

- Converts it to JSON
- Sets the correct content type (application/json)
- Serialises complex objects into JSON-compatible formats

For example:

```
@router.get("/accounts/{id}")
def get_account(id: int):
    return {"account_id": id, "balance": 500.0}
```

A request to:

GET /accounts/1

Returns:

```
{
  "account_id": 1,
  "balance": 500.0
}
```

No additional configuration is needed — JSON handling is fully automatic.

If response models are defined, FastAPI also uses them to validate and shape the JSON output, ensuring your API returns consistent, well-structured responses

every time.

JSON is particularly well-suited for:

- Web applications
- Mobile apps
- Microservices communication

XML Response

Custom Response Types

- Use Response class
- Set media_type

app/api/routes/accounts.py

```
from fastapi.responses import Response
@router.get("/{id}/xml")
def get_account_xml(id: int):
    xml_data = f"""
    <account>
        <account_id>{id}</account_id>
        <balance>500.0</balance>
    </account>
    """
    return Response(content=xml_data,
                    media_type="application/xml")
```

http://127.0.0.1:8000/accounts/50/xml

XML

```
<account>
  <account_id>50</account_id>
  <balance>500.0</balance>
</account>
```

QA

486

Although JSON is the default response format, some systems—especially legacy or enterprise integrations—require XML responses.

FastAPI supports returning custom formats by using response classes like Response. For example:

```
from fastapi.responses import Response
@router.get("/accounts/{id}/xml")
def get_account_xml(id: int):
    xml_data = f"""
    <account>
        <account_id>{id}</account_id>
        <balance>500.0</balance>
    </account>
    """
    return Response(content=xml_data, media_type="application/xml")
```

In this example:

- A string containing XML is created
- Response is used to return it
- The media type is explicitly set

This tells the client how to interpret the response.

XML responses are useful when integrating with:

- Legacy banking systems
- SOAP-based services
- External partners requiring XML

Streaming CSV Download

File Responses

- Use StreamingResponse
- Supports large datasets

Key Feature

- Streams data efficiently
- Enables file download

```
from fastapi.responses import StreamingResponse
import io

def generate_csv():
    buffer = io.StringIO()
    buffer.write("account_id,balance\n1,500.0\n2,234.67\n")
    buffer.seek(0)
    return buffer

@router.get("/export")
def export_accounts():
    return StreamingResponse(
        generate_csv(),
        media_type="text/csv",
        headers={"Content-Disposition":
            "attachment; filename=accounts.csv"}
    )
```

QA

487

FastAPI also supports returning files and streamed responses, making it well-suited for exporting data such as reports or transaction histories.

For large datasets, streaming is preferable to loading the entire file into memory at once. The StreamingResponse class lets you generate and send data progressively as it's produced.

In the example on the slide:

- A CSV file is constructed in memory
- It is returned as a downloadable file
- The Content-Disposition header triggers an automatic download prompt

This approach is commonly used for:

- Reports
- Data exports
- Financial statements

Production Considerations

FASTAPI Fundamentals



Running in Production

Use:

```
gunicorn -k uvicorn.workers.UvicornWorker app.main:app
```

- Enable logging
- Run behind reverse proxy (e.g. Nginx)

Why Not Use `uvicorn --reload`?

- only designed for development usage
- Not optimised for performance

Production Setup

- Reverse proxy (Nginx)
- Logging & monitoring
- Multiple workers

QA

489

The development server (`uvicorn --reload`) is great for local testing but is not intended for production use.

In production, FastAPI apps are commonly deployed with Gunicorn using Uvicorn workers, which improves performance and process management. For example:

```
gunicorn -k uvicorn.workers.UvicornWorker app.main:app
```

This configuration enables:

- Multiple worker processes
- Efficient handling of concurrent requests
- Enhanced reliability

Additionally, deployment of production code often includes:

- A reverse proxy (e.g., Nginx)
- Load balancing
- HTTPS termination

It's also important to configure logging for error tracking and application monitoring.

Key considerations for production environments are:

- Performance
- Security

- Stability

Middleware & Extensions

What is Middleware?

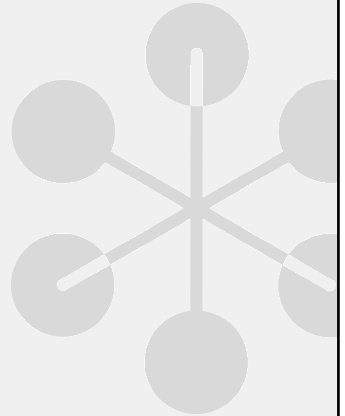
- Runs before/after every request
- Applies global behaviour

Common Use Cases

- CORS handling
- Authentication
- Request logging
- Error monitoring

Benefit

- Centralised control of request handling



QA

490

Middleware in FastAPI enables requests and responses to be processed globally before they reach route handlers which gives us the opportunity to implement cross-cutting concerns such as logging, authentication, and security.

For example, Cross-Origin Resource Sharing (CORS) can be enabled as follows:

```
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Middleware executes for every request and can:

- Inspect incoming requests
- Modify responses
- Add or update headers

Common use cases include:

- Authentication and authorization
- Request logging
- Error handling

By centralising shared functionality, middleware promotes consistent behaviour

across the API and reduces duplicated code within individual routes.

API Best Practices

FASTAPI Fundamentals



Good vs Bad API Design

Good API Design

- /accounts (plural, noun-based)
- Uses correct HTTP methods
- Structured, consistent responses



```
@router.get("/accounts")
def list_accounts():
    return [{"account_id": 1, "balance": 500.0}]
```

Bad API Design

- /getAccounts
- Inconsistent naming
- No validation or status codes



```
@router.get("/getAccounts")
def getAccounts():
    return "data"
```

Key PrincipleDesign

- APIs for clarity and consistency

QA

492

Effective API design is essential for building services that are easy to use, maintain, and extend. Well-designed APIs are consistent, intuitive, and predictable, making them straightforward for developers to integrate with. Common characteristics of poorly designed APIs include:

- Inconsistent naming conventions
- Misuse of HTTP methods and standards
- Ambiguous or inconsistent response formats

For example, the following endpoint has several API design issues:

```
@app.get("/getAccounts")
def getAccounts():
    return "data"
```

Problems here include a non-RESTful endpoint name, an unstructured response, and no validation or data models.

A more RESTful approach would be:

```
@router.get("/accounts")
def list_accounts():
    return [{"account_id": 1, "balance": 500.0}]
```

Good API design principles include:

- Using nouns to represent resources (for example, /accounts)

- Applying HTTP methods appropriately (GET, POST, PUT, DELETE)
- Returning structured JSON responses
- Implementing validation and meaningful HTTP status codes

Following these conventions reduces ambiguity and makes APIs far easier for consumers to adopt..

Key Best Practices

Core Best Practices

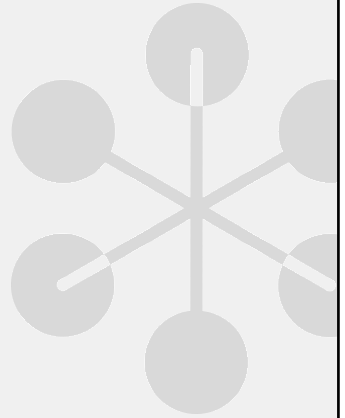
- Use correct HTTP methods
- Validate all inputs
- Return meaningful status codes
- Separate schemas (In vs Out)
- Use JSON Web Tokens for client authentication (see appendix)

Clean Architecture

- Structure projects cleanly
- Thin routes
- Logic in services
- Use dependency injection

Benefits

- Easier maintenance
- Improved scalability
- Better team collaboration



QA

493

Building production-ready APIs comes down to a handful of proven best practices.

- Always use the correct HTTP methods for each operation to maintain RESTful consistency and improve usability.
- Validate all inputs using Pydantic models to prevent invalid data from reaching your application logic
- Keep your input and output schemas separate to avoid exposing internal or sensitive fields.
- Keep routes thin by moving business logic into dedicated service layers — this makes your code easier to maintain and test.
- Use dependency injection to manage shared resources such as database connections or configuration
- Maintain a clear project structure by separating routers, schemas, services, and configuration.

Following these practices produces APIs that are reliable, maintainable, and scalable.

API Versioning and Pagination

- **API Versioning**

- As APIs evolve, new functionality and data structures may be introduced. Good API design helps maintain backwards compatibility and supports growing datasets.
- API Versioning allows multiple API versions to coexist.

- Example routes:

`/api/v1/accounts`
`/api/v2/accounts`

FastAPI Example

```
@app.get("/api/v1/accounts")
```

- **Pagination**

- Pagination prevents APIs from returning large result sets in a single response.

`GET /api/v1/accounts?limit=20&offset=40`

returns

20 records
Starting from record 41

QA

494

API Versioning Benefits:

- Existing clients continue to work
- New features can be introduced safely
- Breaking changes can be managed
- Easier API lifecycle management

Pagination Benefits:

- Faster responses
- Reduced memory usage
- Better user experience
- Improved scalability

Best Practice: Apply pagination to any endpoint that can return large collections of data.

API versioning protects consumers from breaking changes, while pagination enhances performance and scalability. Together, they help produce APIs that are maintainable and ready for production.

Using Appropriate HTTP Status Codes

- HTTP status codes provide a standard way for APIs to communicate the result of a request.

Common Status Codes

Code	Meaning	Example
200 OK	Request succeeded	Retrieve account details
201 Created	Resource created	Create new account
204 No Content	Success with no response body	Delete account
400 Bad Request	Invalid request data	Validation error
401 Unauthorized	Authentication required/invalid JWT	Invalid login token
403 Forbidden	Authenticated but not permitted	Insufficient permissions
404 Not Found	Resource does not exist	Unknown account ID
500 Internal Server Error	Unexpected server failure	Unhandled exception

QA

495

The status code alone should tell clients everything they need to know about how a request turned out. When HTTP status codes are used consistently, APIs become easier to work with, debug, and integrate into other systems.

Here's an example that demonstrates how to raise an HTTPException and specify an appropriate code:

```
raise HTTPException(  
    status_code=404,  
    detail="Account not found"  
)
```

Observability

Observability helps developers understand application behaviour, diagnose issues quickly, and maintain reliable services in production.

Structured Logging	Request IDs (Correlation IDs)	Centralised Error Handling	Monitoring and Metrics
• Log important application and business events • Include timestamps, log levels, endpoint names, and user context • Avoid logging sensitive data such as passwords or tokens	• Generate a unique ID for every request • Include the ID in all logs and API responses • Enables end-to-end tracing across services and databases	• Implement global exception handlers • Return consistent, user-friendly error responses • Log detailed diagnostics and stack traces internally	• Track request volume, latency, error rates, and application health • Use dashboards and alerts to detect issues proactively

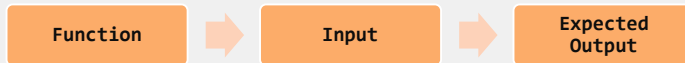
QA

496

Key Takeaway: Effective observability combines logging, request correlation, metrics, and error handling to make FastAPI applications easier to support, troubleshoot, and scale.

Functional Testing Fast API with pytest and httpx

Unit tests focus on individual functions and classes in isolation.



Functional tests validate API behaviour by exercising actual HTTP endpoints.



Why Use httpx?

- httpx allows pytest tests to make HTTP requests against a FastAPI application without deploying it.

```
import pytest
from httpx import AsyncClient, ASGITransport

@pytest.mark.asyncio
async def test_get_account():
    response = await client.get("accounts/me")
    assert response.status_code == 200

    assert response.json()["account_id"] == "ACC-001"
```

QA

497

The Key Difference from Unit Testing is the focus moves from testing implementation details to validating externally observable API behaviour.

In unit tests, we called functions directly. With httpx, we're interacting with the application exactly as a consumer would, sending HTTP requests and validating responses. This gives confidence that routing, validation, dependency injection, and business logic all work together correctly.

Things to Consider When Functional Testing APIs

Authentication

Many endpoints require authentication.

- Generate test JWTs
- Mock authentication services where appropriate
- Test both authorised and unauthorised scenarios

Data Management

Tests often require application state.

- Create known test data
- Avoid dependency on production or shared environments
- Reset databases between tests where possible

Dependency Overrides

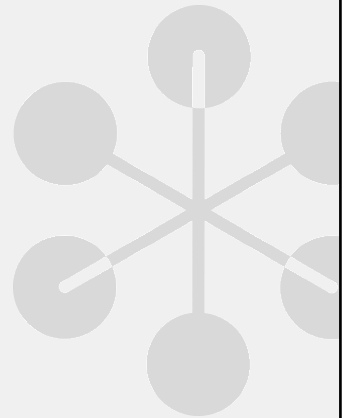
FastAPI makes it easy to replace dependencies during testing.

Examples: Database connections, External APIs, Authentication providers

Async Testing

FastAPI endpoints are commonly asynchronous.

Use `AsyncClient` from `httpx`



QA

498

What to Verify

- Response codes (200, 404, 401, 500)
- Response payloads
- Validation behaviour
- Error responses
- Security controls

Test the API contract, not the internal implementation. Functional tests should remain stable even if the underlying code is refactored. A common mistake is treating functional tests like large unit tests. The goal is not to verify which methods were called internally. Instead, verify that a request produces the correct response, regardless of how the application is implemented behind the endpoint. This results in more resilient tests and greater confidence in the deployed API.

Behavioural Testing with Selenium and Cucumber*

Behavioural tests validate complete business scenarios from a user's perspective.

Role of Selenium:

Selenium automates browser interactions:

- Clicking buttons
- Entering data
- Navigating pages
- Validating displayed content

Role of Behave/Cucumber

Cucumber describes behaviour using business-readable language.

Scenario: Successful login

Given a registered customer

When they enter valid credentials

Then they should see their account dashboard

*In the Python ecosystem you would typically use Behave (the most popular Cucumber-compatible framework) rather than the original Cucumber implementation.

QA

499

Benefits:

Business-focused tests

Improved collaboration between developers and stakeholders

Validation of end-to-end user journeys

Documentation of expected system behaviour

Key Difference: Behavioural tests validate the system as a user experiences it, rather than testing individual functions, classes, or APIs. Unlike unit or API tests, behavioural tests exercise the complete application stack including UI, API, business logic, and data access. The focus is on user outcomes rather than implementation details.

Behavioural Testing - Special Considerations and Best Practices

Synchronisation and Timing

Web applications are asynchronous.

Common issues:

- Pages loading slowly
- AJAX requests
- Dynamic content

Use explicit waits rather than fixed delays.

```
WebDriverWait(driver, 10)
```

Test Maintenance

Behavioural tests are slower and more expensive to maintain than unit tests.

Focus on:

- Critical business journeys
- High-value workflows
- **NOT** Testing every UI detail

Test Data Management

Scenarios require predictable data.

Best practice:

- Create known test users
 - Isolate test data
 - Reset environments where possible
- Selector Stability

Avoid fragile selectors such as:

```
<button class="btn-12345">
```

Prefer:

```
<button data-test="login-button">
```

QA

500

Best Practice: Use Selenium and Behave/Cucumber sparingly for critical end-to-end business scenarios, while relying on unit and API tests for the majority of test coverage. Note if your FastAPI app has no HTML based admin pages then Selenium will have little or no use (because it relies on recording browser interactions and then "playing" them back. However, behave/cucumber can absolutely be used because it works at a higher level.

A common anti-pattern is building large numbers of UI tests to cover every feature. Behavioural tests provide excellent confidence but are slower, more brittle, and more expensive to maintain. Most organisations use a small number of end-to-end Selenium/Cucumber tests to validate key user journeys while keeping the majority of tests at the unit and API layers.

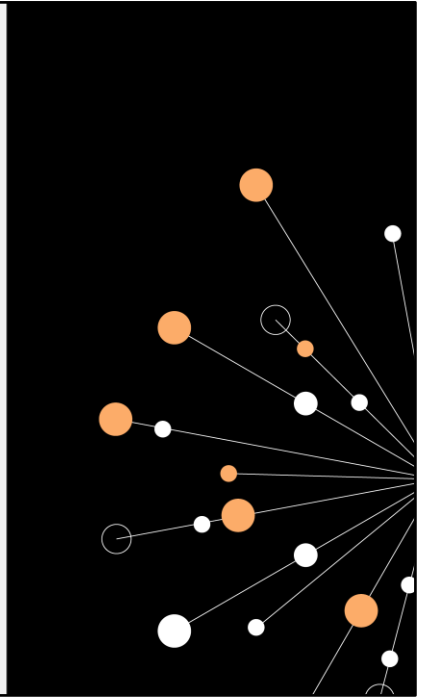
Exercise EG12: FASTAPI

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA

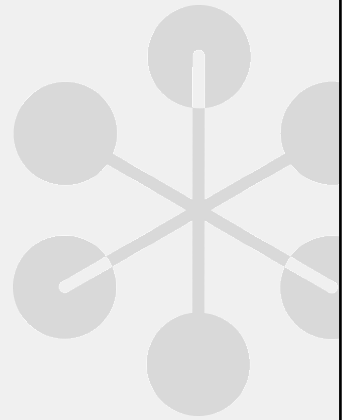


Suggested time: 30-45 mins.

Activity includes differentiated tasks.

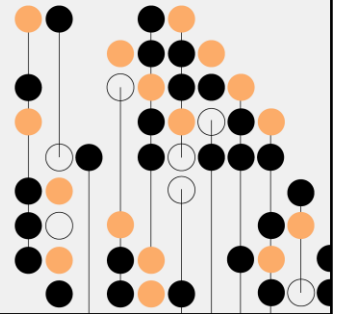
Summary

- FastAPI enables fast, safe API development
- Type hints provide validation + docs
- Modular structure scales well
- Dependency Injection improves maintainability
- Supports multiple response formats



Any questions?

QA



Appendices



Using JSON Web Tokens

JWTs are a compact, secure way to transmit identity and authorization information between systems.

How JWTs Work:

- User logs in with credentials
- Application validates the user
- Server generates a JWT
- JWT is sent to the client
- Client includes the JWT in future requests
- API validates the JWT and grants access

Benefits:

- Stateless authentication
- No server-side session storage required
- Fast and scalable
- Works well with APIs and microservices
- Can carry user roles and permissions

QA

505

JWTs are commonly used in modern web applications and APIs because they allow authentication information to travel with each request. The API only needs to validate the token signature rather than maintain a server-side session.

Using JWTs in FastAPI

Validation Process

- Extract token from the Authorization header
- Verify the signature
- Check expiration time
- Extract user claims
- Allow or deny access

Benefits for Financial Systems

- Secure account access
- Role-based permissions
- Audit-friendly identity information
- Scalable across multiple services

```
from fastapi.security import HTTPBearer

security = HTTPBearer()

@router.get(
    "/accounts/me",
    response_model=AccountResponse,
)
def get_account(
    user=Depends(get_current_user),
):
    return {
        "account_id": user["account_id"],
        "balance": user["balance"],
        "currency": "GBP",
    }
```

A FastAPI application typically issues a JWT during login and validates it on every protected endpoint. The user's identity and permissions are extracted from the token claims.

OAuth2 and JSON

It's common to hear OAuth2 and JWT mentioned together, but they serve different purposes.

- OAuth2 = Authorization Framework
- JWT = Token Format
- FastAPI = API Framework

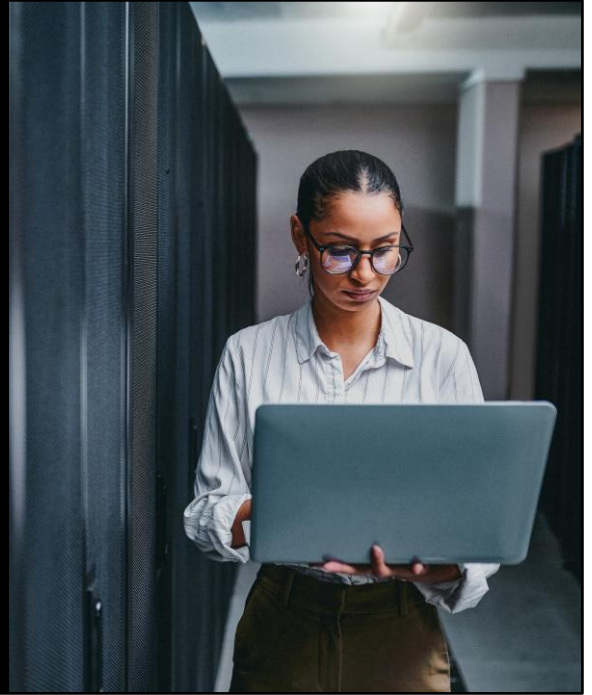
A FastAPI application often does not create the JWT itself in production. Instead:

- OAuth2 Identity Provider authenticates the user
- Identity Provider issues the JWT
- FastAPI validates the JWT and authorizes access

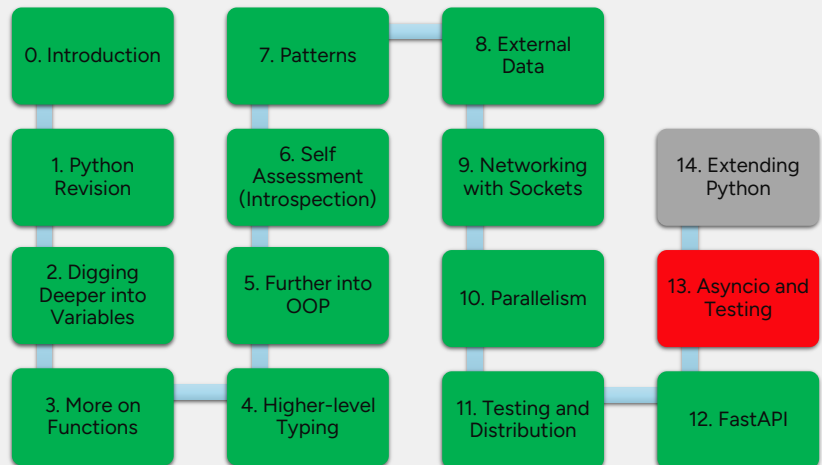
For small applications, FastAPI can both generate and validate JWTs. In enterprise environments, systems such as Microsoft Entra ID, Auth0, or Okta typically handle OAuth2 authentication and token issuance, while FastAPI focuses on validating tokens and enforcing authorization rules.

Intermediate Python

ASYNCIO and Testing



Module roadmap



QA

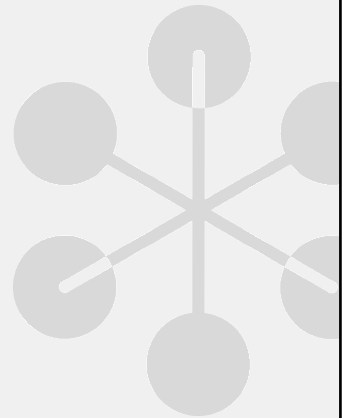
Introducing asyncio

Learning outcomes:

- Explore the use of the `asyncio` library in building Python applications with concurrency.

Prerequisites:

- Delegates are required to have knowledge and skills relating to basic Python programming, including data structures, functions, loops and flow control.

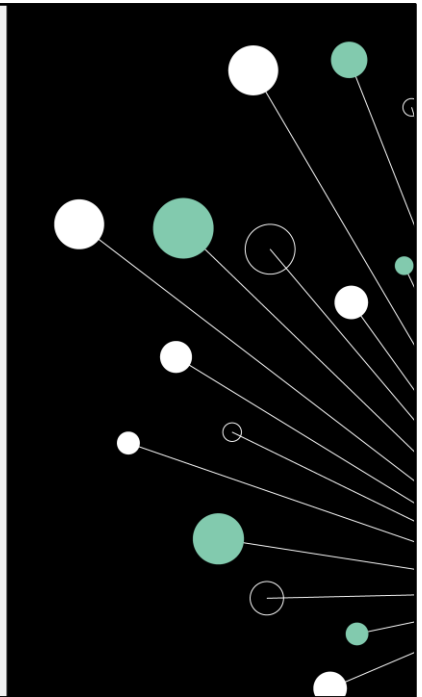


Learning objectives

By the end of this session, you should be able to:

- Understand core async concepts: coroutines, `async / await`, and event loops
- Differentiate between subroutines and coroutines
- Create and manage tasks using `asyncio.create_task()` and `TaskGroup`
- Handle errors and ensure consistency in concurrent async workflows
- Identify coroutine functions and understand their behaviour
- Build simple TCP clients and servers using asyncio streams
- Use async libraries (`aiohttp`, `aiofiles`) for real-world I/O
- Recognise common challenges when testing async code
- Write and run async tests using `pytest-asyncio`
- Use fixtures (including `unused_tcp_port`) for reliable async testing

QA



Recap: General definitions

Parallelism	Multiprocessing	Concurrency
• The ability to run multiple operations, simultaneously	• How parallelism can be achieved, via: - multiple CPUs or - multiple CPU cores	• The ability of a program to make progress on multiple independent tasks at the same time with possibly overlapping execution

The story so far...

Python provides several approaches for achieving parallelism and concurrency.

- **Multiprocessing**

- Spawns a separate process for each concurrent task.
- Incurs relatively high overhead due to process creation and management.
- Sharing data between processes is more complex, though supported through various mechanisms.
- Well suited to CPU-bound workloads, where tasks perform intensive computation.

- **Threading**

- Creates a separate thread for each concurrent task.
- The Global Interpreter Lock (GIL) prevents true parallel execution of Python bytecode in a single process.
- Requires careful synchronization to avoid issues such as race conditions and deadlocks.
- Well suited to I/O-bound workloads, where tasks spend time waiting for external resources.

Python's concurrency ecosystem includes the `multiprocessing`, `threading`, and `concurrent.futures` modules

QA

513

If genuine parallel concurrency is needed, there are other languages better suited for this purpose. However, for basic IO concurrency, Python remains a strong option.

Multiprocessing, despite running tasks in parallel, incurs significant system overhead because creating new processes is much more costly in terms of OS calls than creating new threads. Although threading is constrained by the Global Interpreter Lock (GIL), it is still effective for simple IO concurrency. Nevertheless, the challenges associated with multi-threaded programs can make debugging a lot more difficult.

What is asyncio? - an overview

- **Asynchronous I/O** – Enables non-blocking input/output operations
- **Multi-language support** – Not limited to Python; also available in Go, C#, Scala, and more
- **Python compatibility** – Packaged for Python 3.4 and above, with ongoing development through PEP 492
- **Single-threaded, single-process execution** – Eliminates common threading complications
 - **Synchronous-style syntax** – Write clean, readable code that feels like synchronous programming
 - **Cooperative multitasking** – Functions can continue running during lengthy wait periods that would otherwise block execution
- **Built-in concurrency support** – Handles concurrent I/O operations using straightforward Python syntax
 - **Intelligent task scheduling** – Automatically manages and selects tasks for execution
- **Rich third-party ecosystem** – Broad library support, including HTTP requests and other external integrations

QA

514

Asyncio simplifies concurrent programming by allowing developers to build applications in a familiar synchronous style, while automatically handling concurrency behind the scenes.

By default, it operates on a single thread, freeing developers from the complexities of race conditions, deadlocks, mutexes, and similar pitfalls. For situations where activity needs to be throttled — such as preventing an excessive number of simultaneous requests to a single web address — semaphores can be used to cap the level of concurrency.

Consider a thread that encounters a blocking I/O operation, such as downloading a file from a remote server. Ordinarily, this would result in wasted CPU cycles while the program sits idle waiting for the operation to finish. Asyncio eliminates this inefficiency by allowing other code within the same thread to execute in the meantime. Best of all, this comes without any need for low-level thread management — making it a straightforward and efficient solution for the vast majority of Python developers.

Subroutine vs. Coroutine

Coroutines are special functions that can pause execution (await) and resume later, enabling asynchronous, non-blocking code. They are a key concept used by `asyncio`

Coroutines and subroutines compared:

Subroutine

- Common function
- Invoked by listing its name and specifying any parameters in parentheses
- Runs until finished
- Does not preserve its local state between calls.

Coroutine

- Specialised function
- Calling it traditionally won't run it
- Runs until suspended
- Maintains state
- Resumable
- Can yield to other coroutines
- coroutines were originally implemented using generators, but since Python 3.5 they are a distinct language feature with their own syntax and type.

QA Whilst there are some parallels, don't confuse these "native" coroutines with generator-based coroutines. 515

Subroutines are conventional functions that are invoked, run to completion, and then return a result to the caller.

Coroutines, by contrast, can suspend their execution, transfer control to other coroutines, and later resume from where they left off while preserving their internal state. Multiple instances of the same coroutine can be active concurrently within a single thread.

It is important not to confuse these with generator-based coroutines.

Although they share some underlying concepts, the coroutines used by `asyncio` are more accurately referred to as **native coroutines**.

For simplicity we will generally refer to native coroutines simply as **coroutines** throughout this chapter.

asyncio: the async reserved word

A native coroutine is declared with the `async` reserved word:

```
async def do_something(myarg):  
    pass
```

Things to remember:

- It's a coroutine even *if there is no `await` expression*
- Coroutines return a coroutine object
- You cannot use `yield` here (*but see notes...*)

QA

516

A coroutine *function* is a function defined with "`async def`".

In earlier versions of Python (pre 3.10) you could do the following:
Wrap an existing generator-based coroutine using a decorator in the `asyncio` module:

```
@asyncio.coroutine # NOT supported for versions of Python >= 3.10  
def do_something(myarg):  
    # Perform some calculations here..!  
    result = yield calculated_var
```

The decorator (if available) wraps generator-based coroutine as an awaitable proxy object.

Be wary of generator-based coroutines decorated like this – support for them was removed in Python v3.10

asyncio: the await reserved word

await causes the coroutine's execution until the awaited operation completes and produces a result:

```
async def do_something(myarg):  
    data = await something_awaitable(args)
```

Awaitables include:

- A coroutine object created by an async function
- Any object implementing an `__await__()` method that returns an iterator

Note: `await` can only be used inside functions declared with `async def`

QA

517

A coroutine object is the result of calling a native coroutine function—similar to how a generator-based coroutine does not execute immediately but instead returns a generator object.

The next slide contains a simple example you can try for yourself...

A simple coroutine:

Creating a native coroutine and running it:

- **async** – defines a function as a coroutine function
- **await** – gives up ("yields") control to the event loop, allowing other coroutines to run

```
import asyncio

# coroutine definition
async def transfer_funds(from_account, to_account, amount):
    print(f"Starting transfer of £{amount}...")
    # simulate network/database delay
    await asyncio.sleep(2)
    print(f"Debiting £{amount} from {from_account}")
    await asyncio.sleep(1)
    print(f"Crediting £{amount} to {to_account}")
    return f"Transfer of £{amount} complete"

async def main():
    result = await transfer_funds("Alice", "Bob", 100)
    print(result)

asyncio.run(main())
```

```
Starting transfer of £100...
Debiting £100 from Alice
Crediting £100 to Bob
Transfer of £100 complete
```



See notes below for technical details

QA

518

Let's examine the usage of those `async` and `await` keywords using a simple Python script. In this example the `asyncio.run` function is called, passing into it the coroutine function `main`. The `main()` coroutine awaits the completion of the `transfer_funds()` coroutine. However, during the time that `transfer_funds()` is "sleeping", it yields control (cooperatively) back to the event loop, which if it had other tasks to run, could process these while it waits for this coroutine to complete. In reality we only have the one task here, but there could have been more.

Let's quickly recap those new keywords:

Notes

- You can only `await` *inside* a coroutine function, and `async` functions should be called by the `asyncio` library or passed to an event loop (more on event loops later)
- A coroutine method has to be run in an event loop.
- Python has several high-level functions that manage what is known as an "event loop" for us. The one used on the slide (`asyncio.run()`) simply executes the passed-in coroutine ("`transfer_funds`"); it always creates a new event loop (the "running" loop) and closes it when finished.
- The behaviour of `asyncio.sleep()` is more subtle than it first appears. Rather than pausing execution in the traditional sense, it returns a

coroutine object that resolves after a specified number of seconds. Unlike `time.sleep()`, it does not block during this period — instead, it suspends the current task and yields control to the event loop, allowing other asynchronous tasks within the same thread to run in the meantime.

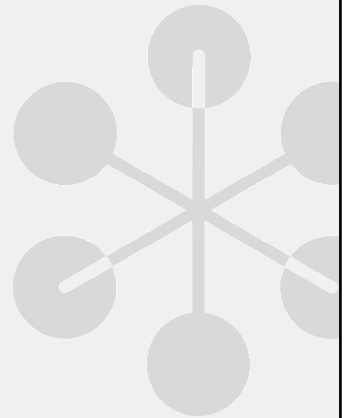
Awaitables

The following are awaitable (objects that can be used with `await`)

- Coroutine
- Future object
- Object defining `__await__` that returns an iterator

New dunder methods (some look familiar...):

- `__aenter__` and `__aexit__`
 - For using as a context manager
- `__aiter__` and `__anext__`
 - For asynchronous iterators
- `__await__`
 - For custom awaitables



QA

519

An "awaitable" object is either a coroutine object, a future object, or an object defining the `__await__` special method (which should return an iterator).

The `__enter__` and `__exit__` dunder functions have an asynchronous versions.

The `__iter__` and `__next__` functions also have an asynchronous versions (for async loops and comprehensions).

Tasks

Tasks are used to run coroutines in an Event Loop

Inherits from the `Future` class (and majority of its API)

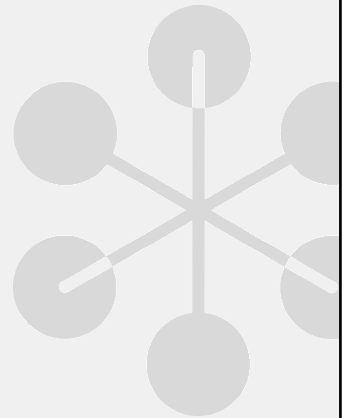
There are three basic ways to create a task:

- High-level technique: `asyncio.create_task()`
- Low-level techniques: `loop.create_task()`
`asyncio.ensure_future()`

Basic syntax:

```
asyncio.create_task(coroutine(), name=name)
```

Note: The `name` parameter is supported Python ^3.8



QA

520

Prior to Python 3.7 developers used to use the `asyncio.ensure_future()` method. However, assuming you are using a more recent version of Python you are encouraged to use `asyncio.create_task()` instead.

Creating tasks

Let's create three tasks and await them:

```
Tasks created, doing other work...
Fetching balance for A100...
Fetching balance for B200...
Fetching balance for C300...

Results:
{'account': 'A100', 'balance': 100},
{'account': 'B200', 'balance': 100},
{'account': 'C300', 'balance': 100}
```

```
import asyncio, random

async def fetch_balance(account_id):
    print(f"Fetching balance for {account_id}...")
    await asyncio.sleep(random.random()) # simulate network delay
    return {"account": account_id, "balance": 100}

async def main():
    # Create tasks (start immediately)
    task1 = asyncio.create_task(fetch_balance("A100"))
    task2 = asyncio.create_task(fetch_balance("B200"))
    task3 = asyncio.create_task(fetch_balance("C300"))

    print("Tasks created, doing other work...")

    # Await results later
    result1 = await task1
    result2 = await task2
    result3 = await task3

    print(f"\nResults: {result1}, {result2}, {result3}")

asyncio.run(main())
```

QA

521

In this instance we are creating two coroutines and creating *unnamed* tasks from them.

An "event loop" will manage the running of them automatically for us. It is imperative to use `asyncio.sleep()` instead of `time.sleep()` to run concurrently. Note if we did use the standard library's more familiar `time.sleep()` here an error would occur as we can only await something that is... awaitable (`time.sleep()` is a blocking call).

As you can see, the three coroutines run concurrently!

So, essentially when the first version of `fetch_balance()` hits the `asyncio.sleep()` method it essentially passes control back to the event loop and that can then continue processing another task (in this case the second and then third calls of `fetch_balance ()`).

TaskGroup

- TaskGroup replaces manual task creation
- Results must be collected manually
- Note: ordering is **not** preserved (see notes)

```
Starting tasks with TaskGroup...
Fetching balance for A100...
Fetching balance for B200...
Fetching balance for C300...
```

```
Results:
{'account': 'B200', 'balance': 100}
{'account': 'C300', 'balance': 100}
{'account': 'A100', 'balance': 100}
```

```
import asyncio
import random

async def fetch_balance(account_id):
    print(f"Fetching balance for {account_id}...")
    await asyncio.sleep(random.random()) # simulate network delay
    return {"account": account_id, "balance": 100}

async def main():
    print("Starting tasks with TaskGroup...")

    # Store results externally
    results = []

    async def run_and_store(account_id):
        result = await fetch_balance(account_id)
        results.append(result)

    async with asyncio.TaskGroup() as tg:
        tg.create_task(run_and_store("A100"))
        tg.create_task(run_and_store("B200"))
        tg.create_task(run_and_store("C300"))

    # All tasks complete here
    print("\nResults:")
    for result in results:
        print(result)

asyncio.run(main())
```

522

If ordering needs to be preserved:

```
async def main():
    account_ids = ["A100", "B200", "C300"]
    results = [None] * len(account_ids)

    async def run_and_store(index, account_id):
        results[index] = await fetch_balance(account_id)

    async with asyncio.TaskGroup() as tg:
        for i, acc in enumerate(account_ids):
            tg.create_task(run_and_store(i, acc))

    print("\nResults:")
    for result in results:
        print(result)
```

Handling TaskGroup exceptions - 1

```
import asyncio
import random

class BankAccount:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    async def withdraw(self, amount):
        await asyncio.sleep(random.uniform(0.1, 0.5))
        if amount > self.balance:
            raise ValueError(
                f"{self.name}: Insufficient funds"
            )
        self.balance -= amount

    async def deposit(self, amount):
        await asyncio.sleep(random.uniform(0.1, 0.5))
        self.balance += amount

    async def transfer(self, from_acc, to_acc, amount):
        await from_acc.withdraw(amount)
        await to_acc.deposit(amount)
        print(f"{amount} transferred from {from_acc.name} to {to_acc.name}")
```

QA

523

1.The example code on the slide shows code for a BankAccount class. Note how the Withdraw function raises a ValueError if the account doesn't have enough money to satisfy the request.

Handling TaskGroup exceptions – 2

Except*

- Handles multiple exceptions at once
- exceptions is a list of all failures
- All remaining tasks are cancelled
 - they do not complete normally!

```
import asyncio
from bankaccount import BankAccount, transfer

async def main():
    acc_a = BankAccount("Alice", 100)
    acc_b = BankAccount("Bob", 200)
    acc_c = BankAccount("Charlie", 50)

    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(transfer(acc_a, acc_b, 50)) # may succeed or be cancelled!
            tg.create_task(transfer(acc_b, acc_c, 100)) # may succeed or be cancelled!
            tg.create_task(transfer(acc_c, acc_a, 200)) # will fail
            tg.create_task(transfer(acc_a, acc_b, 50)) # may succeed or be cancelled!

    except* ValueError as eg:
        print("\nOne or more transfers failed:")
        for exc in eg.exceptions:
            print(exc)

    print("\nFinal balances:")
    print(f"Alice: {acc_a.balance}")
    print(f"Bob: {acc_b.balance}")
    print(f"Charlie: {acc_c.balance}")

asyncio.run(main())
```

```
One or more withdrawals failed:
Charlie: Insufficient funds

Final balances:
Alice: 0 (or 0 or 100 or 100!)
Bob: 200 (or 100 or 200 or 100!)
Charlie: 50 (or 50 or 50 or 50!)
```

QA

524

When any task in a TaskGroup triggers an unhandled exception:

- 1.The TaskGroup signals cancellation to all other currently running tasks within the group
- 2.It then waits for all those tasks to finish cancelling before propagating the exception upward
- 3.The exception is then raised in the context of the `async with` block as an `ExceptionGroup` (introduced in Python 3.11)

Key things to note:

The `except*` syntax is used specifically for handling `ExceptionGroup` — it's distinct from a regular `except`

If **multiple** tasks raise exceptions, they are all collected and wrapped together into a single `ExceptionGroup`, so none are silently swallowed. This is intentional by design — it follows a **fail-fast** philosophy, avoiding a situation where tasks continue running in an uncertain or potentially broken state.

So, in the above code the inconsistency in the results is down to the random sleep durations — some transfers may partially complete (withdraw done, deposit not yet) before cancellation kicks in. To get

consistent results you need to make each transfer atomic, so either the full operation completes or none of it does.

Achieving TaskGroup consistency – `asyncio.lock`

The cleanest way to get deterministic results is to use `asyncio.lock` on each account from within the transfer function

```
One or more transfers failed:  
Charlie: Insufficient funds
```

```
Final balances:  
Alice: 0  
Bob: 150  
Charlie: 50
```

```
import asyncio, random

class BankAccount:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
        self.lock = asyncio.Lock()

    async def withdraw(self, amount):
        await asyncio.sleep(random.uniform(0.1, 0.5))
        if amount > self.balance:
            raise ValueError(f"{self.name}: Insufficient funds")
        self.balance -= amount

    async def deposit(self, amount):
        await asyncio.sleep(random.uniform(0.1, 0.5))
        self.balance += amount

    async def transfer(from_acc, to_acc, amount):
        # Always acquire locks in a consistent order to prevent deadlocks
        first, second = sorted([from_acc, to_acc], key=lambda a: a.name)
        async with first.lock:
            async with second.lock:
                await from_acc.withdraw(amount)
                await to_acc.deposit(amount)
                print(f"{amount} transferred from {from_acc.name} to {to_acc.name}")
```

QA

525

Key changes and why:

- **`asyncio.Lock` per account within the `transfer()` function** — ensures that a transfer holds exclusive access to both accounts for the entire duration of the operation, preventing partial updates from being visible to other tasks
- **Consistent lock ordering** — locks are always acquired in alphabetical order by account name. This is critical; without it you risk a **deadlock** where two transfers are each waiting for a lock the other holds
- **Atomicity** — because the withdraw and deposit now happen inside the lock, if the withdrawal raises an exception the deposit never occurs, leaving balances in a clean state

The final balances will now be consistent on every run regardless of the random sleep timings.

Introspection

Sometimes we may want to know *which* tasks are running when using asyncio.

Two key functions:

- `asyncio.current_task()` Returns the current running task.
- `asyncio.all_tasks` Returns a **set** of live tasks run by the Event Loop

Example `current_task()` output (using `__str__`):

```
<Task pending name='Task-2' coro=<print_and_wait() running at asyncio4.py:8>  
cb=[gather.<locals>._done_callback() at  
C:\Users\User\AppData\Local\Programs\Python\Python38-  
32\lib\asyncio\tasks.py:751]>
```

As you can see, we can immediately identify:

- Task has an identifier ("Task-2")
- Its coroutine ("print_and_wait()")
- Current execution line and source file
- The callback

asyncio streams

asyncio Streams are high-level async/await compliant primitives

- designed to work with network connections

Streams allow send and receive of data over networks

- without using callbacks
- without using protocols or transports

Two key classes exist:

- **StreamReader** methods include read, readline etc...
- **StreamWriter** methods include write, writeline, close etc...

Typical stream methods include:

- **open_connection** establishes a network connection and returns reader and writer object
- **start_server** starts a socket-based server

Let's explore a simple example!

QA

527

asyncio – simple TCP server

- a bank server that accepts requests
- a client requesting a balance (next slide)
- communication over TCP streams

StreamReader

- Reads incoming data from the connection
- Async, non-blocking

StreamWriter

- Sends data to the client
- `drain()` ensures buffer is flushed

```
import asyncio

ACCOUNTS = {"A100": 500, "B200": 300, "C300": 900,}

async def handle_client(reader: asyncio.StreamReader, writer: asyncio.StreamWriter):
    data = await reader.readline()
    account_id = data.decode().strip()

    print(f"Request for account: {account_id}")
    balance = ACCOUNTS.get(account_id, None)
    if balance is None:
        response = "ERROR: Account not found\n"
    else:
        response = f"BALANCE:{balance}\n"

    writer.write(response.encode())
    await writer.drain()

    writer.close()
    await writer.wait_closed()

async def start_server():
    server = await asyncio.start_server(handle_client, "127.0.0.1", 4444)

    print("Bank server running on port 4444...")
    async with server:
        await server.serve_forever()

# Run server:
asyncio.run(start_server())
```

QA

528

For internal testing purposes, we could use a Telnet client to "poke" our simple server but alternatively, we could create a python client...

asyncio – simple TCP client

StreamReader

- Reads incoming data from the connection
- Async, non-blocking

StreamWriter

- Sends data to the client
- `drain()` ensures buffer is flushed

Starting tasks with TaskGroup...

Requesting balance for A100
Requesting balance for B200
Requesting balance for X999

Results:

Server response: BALANCE:500
Server response: BALANCE:300
Server response: ERROR: Account not found

```
import asyncio

async def check_balance(account_id):
    reader, writer = await asyncio.open_connection("127.0.0.1", 4444)
    print(f"Requesting balance for {account_id}")
    writer.write(f"{account_id}\n".encode())
    await writer.drain()

    response = await reader.readline()

    writer.close()
    await writer.wait_closed()
    return f"Server response: {response.decode().strip()}"

async def main():
    print("Starting tasks with TaskGroup...")
    results = []

    async def run_and_store(account_id):
        result = await check_balance(account_id)
        results.append(result)

    async with asyncio.TaskGroup() as tg:
        tg.create_task(run_and_store("A100"))
        tg.create_task(run_and_store("B200"))
        tg.create_task(run_and_store("X999")) # invalid

    print("\nResults:")
    for result in results:
        print(result)

asyncio.run(main())
```

QA

529

Key concepts:

- Streams are **low-level networking primitives**
- You manually define your **protocol** (e.g. "BALANCE:...")
- Full control over I/O
- StreamReaders and StreamWriters work well with asyncio concurrency

This pattern is useful for:

- custom financial protocols
- real-time transaction systems
- building lightweight services without HTTP
- internal service communication

aiohhttp library - server

Client/Server asynchronous HTTP library for python

- Homepage:
<https://docs.aiohttp.org>
- Install via PyPI:
pip install aiohttp
- Supports HTTP + JSON
payloads

```
from aiohttp import web

ACCOUNTS = {"A100": 500, "B200": 300, "C300": 900,}

async def get_balance(request):
    data = await request.json()
    account_id = data.get("account_id")

    print(f"Request for account {account_id}")

    balance = ACCOUNTS.get(account_id)

    if balance is None:
        return web.json_response(
            {"status": "error", "message": "Account not found"},
            status=404
        )

    return web.json_response(
        {"status": "success", "account": account_id, "balance": balance}
    )

app = web.Application()
app.router.add_post("/balance", get_balance)

# Run server:
web.run_app(app, host="127.0.0.1", port=8080)
```

simple web
server example

QA

530

The aiohttp library allows us to interact with http traffic asynchronously and supports all the http verbs.

Interestingly it also supports Flask-like decorators for HTTP verb GET/VERB-based routes, so could be used to create a very simple API.

Note.

This useful library requires at least Python 3.5.2 and requires the following packages:

attrs, async-timeout, chardet, multidict, idna, yarl

aiohttp library - client

Client/Server asynchronous HTTP library for python. It:

- sends multiple concurrent HTTP POST requests
- uses aiohttp for async networking
- uses TaskGroup for structured concurrency
- collects and prints results

```
import asyncio, aiohttp

async def check_balance(session, account_id):
    url = "http://127.0.0.1:8080/balance"
    print(f"Requesting balance for {account_id}")

    async with session.post(url, json={"account_id": account_id}) as resp:
        data = await resp.json()
        return f"Server response: {data}"

async def main():
    print("Starting requests with TaskGroup...")
    results = []

    async def run_and_store(session, account_id):
        result = await check_balance(session, account_id)
        results.append(result)

    async with aiohttp.ClientSession() as session:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(run_and_store(session, "A100"))
            tg.create_task(run_and_store(session, "B200"))
            tg.create_task(run_and_store(session, "X999")) # invalid

    print("\nResults:")
    for result in results:
        print(result)

asyncio.run(main())
```

simple web
client example

QA

531

The `aiohttp.ClientSession()` represents a and as such it manages:

- TCP connections
- connection reuse (keep-alive)
- resource cleanup

Without a session `await aiohttp.post(...)` is inefficient. With a session `session.post(...)` is efficient because the connections are pooled, which in a real system would reduce latency, improve scalability and avoid connection exhaustion.

`async with session.post(url, json={"account_id": account_id}) as resp:`

- Sends a POST request Automatically:
 - serializes JSON payload
 - sets headers (Content-Type: application/json)
- Returns an asynchronous response object

The `async with` structure ensures the response body is properly consumed and the connection is released back to the pool.

`data = await resp.json()`

Reads the response body asynchronously and parses JSON into a Python dict in a non-blocking manner.

aiofiles library - writing

Read and write files asynchronously

- Homepage:
<https://pypi.org/project/aiofiles/>
- Install via PyPI:
pip install aiofiles

```
import asyncio, aiofiles, json

# Save account data asynchronously
async def save_accounts(accounts, filename):
    async with aiofiles.open(filename, "w") as f:
        data = json.dumps(accounts)
        await f.write(data)
    print("Accounts saved")

async def main():
    filename = "accounts.json"

    accounts = {
        "A100": {"name": "Alice", "balance": 500},
        "B200": {"name": "Bob", "balance": 300},
    }

    await save_accounts(accounts, filename)

asyncio.run(main())
```

QA

532

aiofiles allows for asynchronous interaction with files.

Note the call to add the "main" coroutine to the event loop is not shown.

```
if __name__ == '__main__':
    asyncio.run(main())
```


aiofiles library - reading

Read and write files asynchronously

```
import asyncio, aiofiles, json

# Load account data asynchronously
async def load_accounts(filename):
    async with aiofiles.open(filename, "r") as f:
        data = await f.read()
        accounts = json.loads(data)
        print("Accounts loaded")
    return accounts

async def main():
    filename = "accounts.json"

    loaded_accounts = await load_accounts(filename)

    print("\nAccount data:")
    for acc_id, details in loaded_accounts.items():
        print(f"{acc_id}: {details}")

asyncio.run(main())
```

Testing problems... and a solution

pytest, as we have seen, is a popular option for testing Python functionality.

However, applications using asyncio use coroutines

- This concurrency makes them difficult to test!
- Async def functions (native coroutines) are not supported by pytest

Solution: Use a library specifically designed for testing asyncio with pytest.

- **pytest-asyncio**
- An Apache2 licensed library
- Python ^3.5
- Homepage: <https://pypi.org/project/pytest-asyncio/>
- Install from PyPI: **pip install pytest-asyncio**
- Adds asyncio friendly pytest Fixtures and Markers
 - acts as a plug-in, essentially

QA • Requires an entry in **pytest.ini** – see notes

534

Note:

pytest-asyncio also collects and installs the following packages: wcwidth, packaging, colorama, atomicwrites, py, pluggy, more-itertools, pytest, pytest-asyncio

To use the pytest_asyncio plugin a *pytest.ini* file must exist in your test directory path that has the following entry:

```
[pytest]
asyncio_mode=auto
...
```

Alternatively, call *pytest* with the **--asyncio-mode=auto** option.

Demonstrating the problem...

By using `await`, these test functions must become coroutines

they require the `async` keyword prepended to the declaration:

```
async def test_all_valid():
    outcomes = await check_postcodes()
    assert all(outcomes)

async def test_some_valid():
    outcomes = await check_postcodes()
    assert any(outcomes)
```

Unfortunately, `async def` functions (native coroutines) aren't supported by pytest "out of the box"

```
python -m pytest -v asyncpostcodes2.py
===== test session starts =====
platform win32 -- Python 3.8.1, pytest-5.4.2, py-1.8.1, pluggy-0.13.1 --
C:\Users\User\AppData\Local\Programs\Python\Python38-32\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\User\AppData\Local\Programs\Python\Python38-32
plugins: asyncio-0.12.0
collected 2 items

asyncpostcodes2.py::test_all_valid SKIPPED [ 50%]
asyncpostcodes2.py::test_some_valid SKIPPED [100%]
```

QA

We need to *either* refactor the code (see notes) or use `pytest-asyncio`

535

Apart from `pytest-asyncio`, other testing plug-ins are available for the `async` framework, these include:

- `pytest-trio`
- `pytest-tornasync`
- `pytest-twisted`

As with all projects, there may be varying spells between updates.

We could restructure the code slightly to avoid the problem:

```
def test_all_valid():
    outcomes = asyncio.run(check_postcodes())
    assert all(outcomes)

def test_some_valid():
    outcomes = asyncio.run(check_postcodes())
    assert any(outcomes)
```

However, it's unwieldy to make such adaptations for all circumstances and your code can start to look convoluted. No, the plug-in is surely the route

to go.

Using pytest- asyncio

Once installed testing
just works!

However:

- you need a
pytest.ini file

- Must contain:

```
[pytest]
    asyncio_mode = auto
```

Or:

Each test must be
decorated with:

```
@pytest.mark.asyncio
```

```
import pytest
from bankaccount import BankAccount, transfer

# Basic deposit test
async def test_deposit():
    acc = BankAccount("Alice", 100)
    await acc.deposit(50)
    assert acc.balance == 150

# Basic withdraw test
async def test_withdraw():
    acc = BankAccount("Alice", 100)
    await acc.withdraw(40)
    assert acc.balance == 60

# Insufficient funds
async def test_withdraw_insufficient_funds():
    acc = BankAccount("Alice", 50)
    with pytest.raises(ValueError):
        await acc.withdraw(100)
    # Balance should remain unchanged
    assert acc.balance == 50

# Successful transfer
async def test_successful_transfer():
    acc_a = BankAccount("Alice", 100)
    acc_b = BankAccount("Bob", 200)
    await transfer(acc_a, acc_b, 50)
    assert acc_a.balance == 50
    assert acc_b.balance == 250

# Failed transfer should not credit target account
async def test_failed_transfer_insufficient_funds():
    acc_a = BankAccount("Alice", 100)
    acc_b = BankAccount("Bob", 200)
    with pytest.raises(ValueError):
        await transfer(acc_a, acc_b, 150)
    # Ensure balances unchanged
    assert acc_a.balance == 100
    assert acc_b.balance == 200

# TaskGroup behaviour: one failure cancels others
async def test_taskgroup_transfer_failure():
    import asyncio

    acc_a = BankAccount("Alice", 100)
    acc_b = BankAccount("Bob", 200)
    acc_c = BankAccount("Charlie", 50)

    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(transfer(acc_a, acc_b, 50))
            tg.create_task(transfer(acc_b, acc_c, 100))
            tg.create_task(transfer(acc_c, acc_a, 200))

    except* ValueError as eg:
        # Ensure at least one failure occurred
        assert any("Insufficient funds" in str(e)
                   for e in eg.exceptions)

    # Ensure no invalid credit happened
    assert acc_c.balance <= 150 # safe upper bound
```

QA

536

As you can see, if you have a `pytest.ini` file with the appropriate entry testing your `asyncio` code is as simple and straightforward as testing any other regular code. The alternative is to use the `@pytest.mark.asyncio` decorator on all of your test methods e.g.

```
# Basic deposit test
@pytest.mark.asyncio
async def test_deposit():
    acc = BankAccount("Alice", 100)
    await acc.deposit(50)
    assert acc.balance == 150
```

```
# Basic withdraw test
@pytest.mark.asyncio
async def test_withdraw():
    acc = BankAccount("Alice", 100)
    await acc.withdraw(40)
    assert acc.balance == 60
```

asyncio_pytest fixtures

There are no significant
differences between
Fixtures and Asynchronous
Fixtures

- except you are
decorating coroutines
rather than normal
functions

Simply use
`@pytest.fixture` as
normal, e.g.

```
@pytest.fixture
def sample_accounts():
    return (
        BankAccount("Alice", 100),
        BankAccount("Bob", 200),
        BankAccount("Charlie", 50),
    )

# TaskGroup behaviour: one failure cancels others
async def test_taskgroup_transfer_failure(sample_accounts):
    import asyncio

    acc_a, acc_b, acc_c = sample_accounts

    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(transfer(acc_a, acc_b, 50))
            tg.create_task(transfer(acc_b, acc_c, 100))
            tg.create_task(transfer(acc_c, acc_a, 200))

    except* ValueError as eg:
        # Ensure at least one failure occurred
        assert any("Insufficient funds" in str(e)
                   for e in eg.exceptions)

    # Ensure no invalid credit happened
    assert acc_c.balance <= 150 # safe upper bound
```

QA

537

To use fixtures in the "conventional" way it's easiest to use the
`pytest_asyncio` plugin by adding a `pytest.ini` file to your test directory path
that has the following entry:

```
[pytest]
asyncio_mode=auto
...
```

And decorate your fixture functions with `@pytest.fixture`.

You could also directly import `pytest_asyncio` and use
`@pytest_asyncio.fixture`, instead.

Testing http requests (1) – the server logic

app is now returned as its own Application instance so pytest-aihttp test environment can specify the port number

```
from aiohttp import web

ACCOUNTS_TEMPLATE = {"A100": 500, "B200": 300, "C300": 900,}

def create_app():
    accounts = ACCOUNTS_TEMPLATE

    async def get_balance(request):
        data = await request.json()
        account_id = data.get("account_id")

        balance = accounts.get(account_id)

        if balance is None:
            return web.json_response(
                {"status": "error", "message": "Account not found"},
                status=404
            )

        return web.json_response(
            {"status": "success", "account": account_id, "balance": balance}
        )

    app = web.Application()
    app.router.add_post("/balance", get_balance)

    return app

if __name__ == "__main__":
    web.run_app(create_app(), host="127.0.0.1", port=8080)
```

updated web server example

QA

538

The highlighted logic (the `create_app()` function and the code that returns the app object) are configured in the way they are to ensure the live environment exposes the server on port 8080 but the test environment can specify its own port.

Testing http requests (2) – the tests

use aiohttp's test utilities so you:

- don't need a real running server
- get fast, isolated tests
- can test HTTP behaviour properly
- Need to pip install:
 - pytest-aiohttp
 - aiohttp
 - pytest
 - pytest-asyncio

```
import pytest
from aiohttp import web
from bankserver import create_app

# aiohttp test client fixture
@pytest.fixture
async def client(aiohttp_client):
    app = create_app() # NEW instance per test
    return await aiohttp_client(app)

# Test: valid account
async def test_get_balance_success(client):
    resp = await client.post("/balance", json={"account_id": "A100"})

    assert resp.status == 200
    data = await resp.json()

    assert data["status"] == "success"
    assert data["account"] == "A100"
    assert data["balance"] == 500

# Test: invalid account
async def test_get_balance_not_found(client):
    resp = await client.post("/balance", json={"account_id": "X999"})

    assert resp.status == 404
    data = await resp.json()

    assert data["status"] == "error"
    assert data["message"] == "Account not found"
```

Tests

QA

539

By simply passing `aiohttp_client` as a parameter to the client function the runtime will spin up the app in a test server and :

```
@pytest.fixture
async def client(aiohttp_client):
    app = create_app() # NEW instance per test
    return await aiohttp_client(app)
```

Consequently, there's no need to invoke `web.run_app()` and launch a real web server.

When making HTTP requests:

```
resp = await client.post("/balance", json={...})
```

The code runs fully asynchronously and behaves like a real HTTP client

asyncio: unused_tcp_port fixture example

As the name suggests, this locates and yields an unused TCP on the localhost interface, i.e. the loopback

Useful for binding ports for temporary test servers

An `unused_tcp_port_factory` fixture also exists

- Locates and yields a *different* TCP port each time it's called

```
import asyncio, pytest
from bankserver import start_server

async def test_valid_account(unused_tcp_port):
    port = unused_tcp_port
    server = await start_server(port=port)
    async with server:
        # Start serving in background
        server_task = asyncio.create_task(server.serve_forever())
        try:
            reader, writer = await asyncio.open_connection("127.0.0.1", port)
            writer.write(b"A100\n")
            await writer.drain()
            response = await reader.readline()
            assert response.decode().strip() == "BALANCE:500"
            writer.close()
            await writer.wait_closed()
        finally:
            server_task.cancel()
```

QA

540

Here's an example of using the `unused_tcp_port` fixture; it's quite handy for testing purposes.

The logic in the `bankserver` module that starts up the server is:

```
# Make server configurable and return it
async def start_server(host="127.0.0.1", port=4444):
    server = await asyncio.start_server(handle_client, host, port)
    return server

# Only run when executed directly
if __name__ == "__main__":
    async def main():
        server = await start_server()
        print("Bank server running on port 4444...")

        async with server:
            await server.serve_forever()

    asyncio.run(main())
```

And finally!

Static Type checks using mypy are also useful...

will identify silly errors that can creep into large codebases

Coroutines are Typed just like a normal function:

```
import asyncio

async def timed_warning(msg: str="Resume in...", counter: int=5) -> str:
    while counter > 0:
        print(f"{msg} {counter} seconds")
        await asyncio.sleep(1)
        counter -= 1
    return "Completed!"

if __name__ == "__main__":
    print("Action #1")
    result: str = asyncio.run(timed_warning("Go in..", 3))
    print(result)
    print("Action #2")
```

```
python async_timed_msg.py
Action #1
Go in.. 3 seconds
Go in.. 2 seconds
Go in.. 1 seconds
Completed!
Action #2
```



Running a static type check:

QA

```
mypy async_timed_msg.py
Success: no issues found in 1 source file
```



541

Fortunately (although you may have initially suspected the worst), static typing for coroutines happily follows the *same* rules as normal functions.

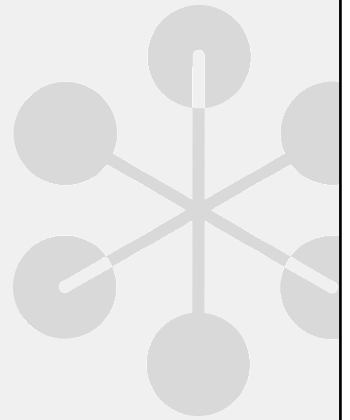
In other words, type the parameters as you usually would and the coroutines return type is essentially the same as the type of value you would expect to get back from awaiting the coroutine's completion in the event loop which launched it.

Other aio libraries

Other asyncio specialised libraries include:

- aiohttp: HTTP client/server
- aiohttp-security security for aiohttp
- aiohttp-session session management for aiohttp
- aiofiles: file management
- aiopg postgres database
- aiomysql mysql database support

Others are available, see <https://github.com/aio-libs>



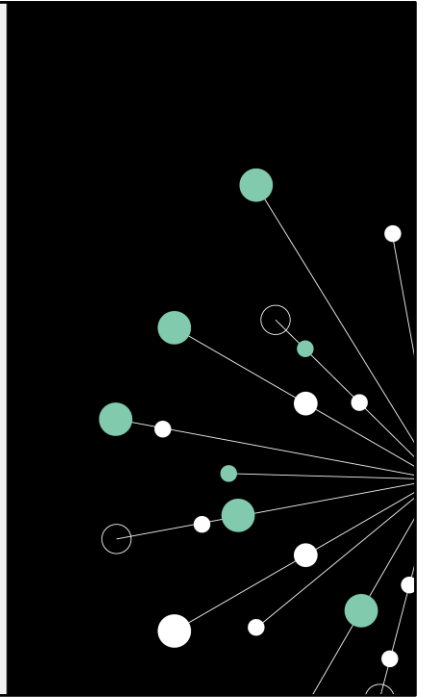
Exercise EG13: Asyncio

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA

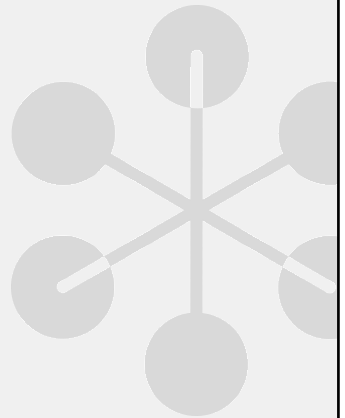


Suggested time: 30-45 mins.

Activity includes differentiated tasks.

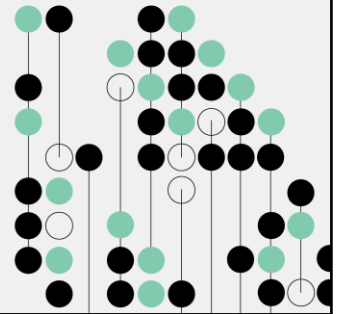
Summary

- `asyncio` is a popular Python library
 - Allows developer to write concurrent code
- `asyncio` revolves around an "event loop"
 - Manages the cooperative multitasking in a single thread
- Uses `async/await` syntax
 - Define native coroutines with `async`
 - `await` is a form of `yield`; suspending execution
- `Tasks` and `TaskGroup`
- Use `pytest-asyncio` for testing
- Often used to underpin many asynchronous Python frameworks
 - Networking
 - Servers
 - Database
- QA** • Distributed Queue processing



Any questions?

QA



Appendices



Detection of coroutines

Two techniques for inspecting the nature of functions

- to determine programmatically if they are **native coroutines**

```
import asyncio, inspect

#coroutine
async def say_hello1(name, delay):
    await asyncio.sleep(delay)
    print("Hello,", name)

def say_hello2(name, delay):
    # Error: won't be awaited but rest of code will still run
    asyncio.sleep(delay)
    print("Hello,", name)

async def main():
    res = await say_hello1("Ted 1", 3)
    res = say_hello2("Ted 2", 2)

    print(f"Is coroutine? {inspect.iscoroutinefunction(say_hello1)}")
    print(f"Is coroutine? {inspect.iscoroutinefunction(say_hello2)}")

asyncio.run(main())
```

```
Hello, Ted 1
Hello, Ted 2
Is coroutine? True
Is coroutine? False
C:\QAL\14_detection_of_coroutines.py:10:
RuntimeWarning: coroutine 'sleep' was never awaited
asyncio.sleep(delay) # Error: won't be awaited but rest of code will still run
```

547

The inset shows the output from the program.

Note the behaviour difference (this was tested on Python 3.8): Although the method exists in both the `asyncio` and `inspect` modules, their yielded results are different when examining a decorated legacy generated-based coroutine (i.e. `say_hello2()`).

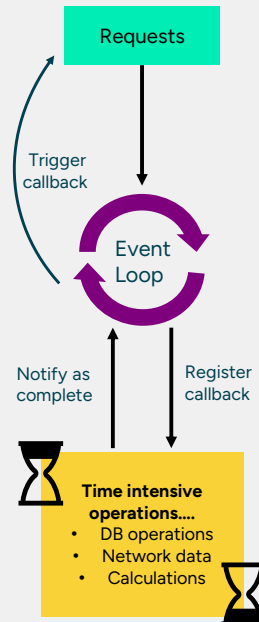
Something to be wary of!

Note. another method called `iscoroutine()` exists (for both `inspect` and `asyncio`) and can be used to detect whether an object is a coroutine object (or not); a similar difference in opinion occurs between the two modules.

Event loops

asyncio relies on the Event Loop

- Manages the concurrency for us
 - A "master scheduler"
 - Manages the cooperative multitasking
 - Performs I/O operations
 - Runs callbacks
 - Runs asynchronous tasks
- Automatically created by high level functions
- Can be accessed and manually created at a low level
- Useful for framework and library writes
- Sometimes required by 3rd party libraries as a parameter



QA

548

The event loop is at the centre of every application that uses asyncio. Since async methods (coroutines) can only be called by *other* async methods, it creates a bit of a catch-22 situation; the solution is to surrender control of the current processing thread to an event loop; the event loop will then invoke the first asynchronous method.

Note that application developers should generally use the higher level asyncio functions mentioned previously and should rarely need to reference the loop object.

`loop = asyncio.get_event_loop()` - gets the current event loop. If no event loop has been set, creates one.
`loop.run_until_complete()` - runs until all asyncio tasks are complete
`loop.run_forever()` - runs until `loop.stop()` is explicitly called

asyncio.gather

Gather is a useful high-level method to run multiple coroutines and collect their results into a single result

- **Tasks run concurrently**
 - doesn't block one task on another
 - improves performance for I/O-bound work
- **Order is preserved even though tasks run in parallel**
- **if one task fails, gather:**
 - raises the exception
 - cancels the others

Note: from Python 3.11+ TaskGroup is recommended over gather()

```
import asyncio

async def get_balance(name):
    await asyncio.sleep(1) # simulate network/database delay

    return f"{name}: £100"

async def main():
    results = await asyncio.gather(
        get_balance("Alice"),
        get_balance("Bob"),
        get_balance("Charlie"),
    )
    print(results)

asyncio.run(main())
```

```
['Alice: £100', 'Bob: £100', 'Charlie: £100']
```

QA

549

This example shows how multiple coroutines can be tasked to the event loop using the high level gather method. The output of the above script would be:

```
['Alice: £100', 'Bob: £100', 'Charlie: £100']
```

Futures

A Future represents an eventual result of an asynchronous operation

- Can have callback functions registered to run *after* the future is "done"

```
import asyncio

def callback(future):
    print(f"future done! : {future.result()}")

async def register_callbacks(future_obj):
    future_obj.add_done_callback(callback)

async def main():
    my_future = asyncio.Future()
    await register_callbacks(my_future)
    my_future.set_result('future result!!!')

asyncio.run(main())
```

```
future done! : future result!!!
```

QA

550

Another low level part of the `asyncio` module is the `Future` object, which represents the result of work that has not been completed yet. The event loop will check the `Future`'s state to see if it has completed that it is done, allowing for the waiting of something to finish before something else is done within the application.

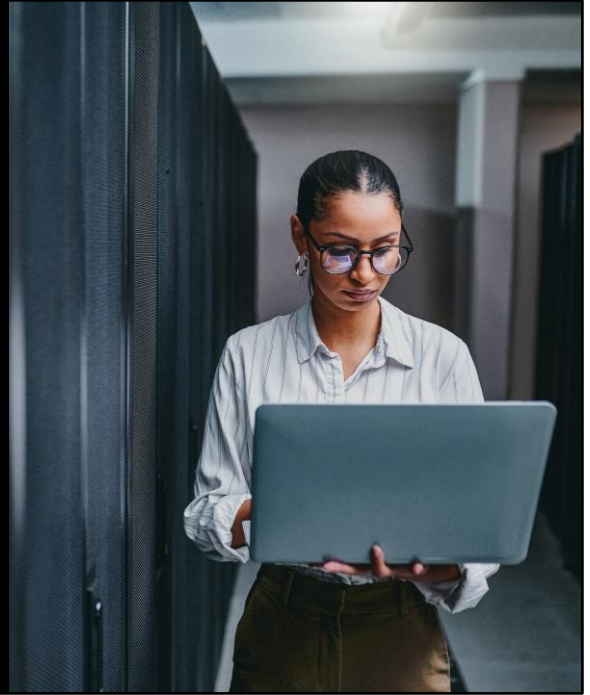
A `Future` behaves just like a coroutine, so just like waiting for a coroutine, we can also wait for the future to be marked as done.

The callback function should expect one argument, the `Future` object. To pass additional arguments use `functools.partial()` to create a wrapper. The result of the `Future` object once complete, can be accessed using the `result` method.

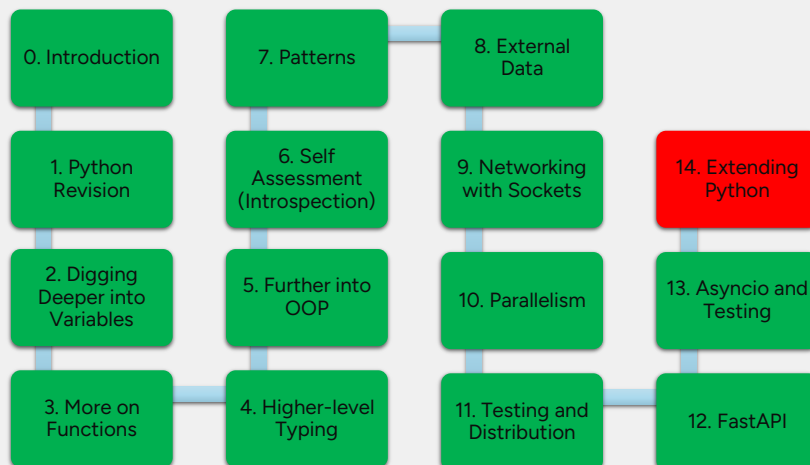
However, "Futures are the low-level building blocks of `asyncio`, but in modern Python you almost never use them directly — `Task` and `TaskGroup` handle that for you."

Intermediate Python

Extending Python



Module roadmap



QA

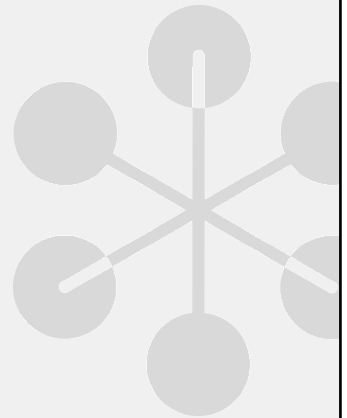
Extending Python – first steps

Learning outcomes:

Extend Python using other programming languages such as C to create more efficient solutions

Prerequisites:

Delegates are required to have knowledge and skills relating to basic Python programming, including data structures, functions, loops and flow control.

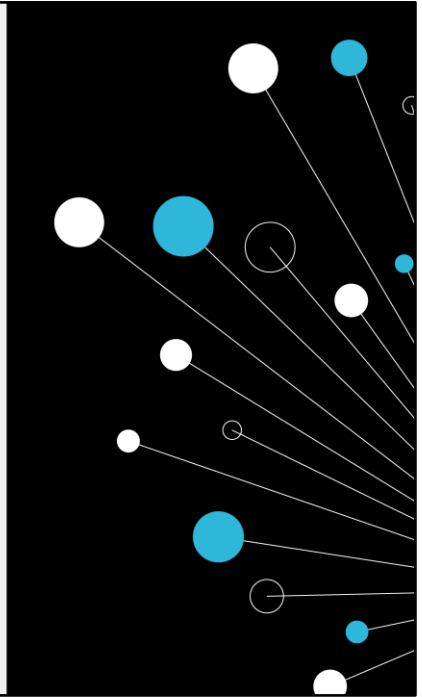


Learning Objectives

By the end of this session, you should be able to:

- Explain why and when to extend Python with C to improve performance or access low-level functionality
- Compare key approaches for integrating C with Python (CPython API, CFFI, etc.)
- Describe the core concepts of calling C from Python, including data exchange and execution flow
- Implement a minimal C extension and CFFI functions integrate them with Python
- Apply parameter passing and return value handling between Python and C, including complex types
- Use the Python C API core patterns and common functions effectively
- Manage errors and exceptions safely across the Python–C boundary
- Apply correct memory management practices, including reference counting, ownership rules, and allocation
- Understand and manage thread state and the Global Interpreter Lock (GIL)

QA



Why extend Python with C?

There are three basic ways to make your Python code faster:

1. Improve its efficiency
2. Convert your code to parallelism (where appropriate)
3. Rewrite slow Python routines in a faster language
4. Can give system-level access

C is a classic example of a *faster* language (many others exist)

Why use C?

- C is statically typed
- C is compiled to native binary instructions
- C can be tightly optimised during compilation
- C syntax is familiar to anyone who has programmed in a "C family" language, e.g., C++, Java, C#, Perl, PHP, Objective-C, JavaScript and ... Python (to a lesser extent)

QA

555

If you can't improve the efficiency of your Python code or make full use of parallelism (CPython's GIL may be the culprit here), then writing modules in a faster language may be the solution.

C is often the easiest language to integrate.

But why use C?

- It's a statically typed language unlike Python; variables cannot change type so there's no need to test to see what a variable is before its used.
- It's compiled to native binary instructions (not compiled to bytecode and then interpreted like Python using a virtual machine) that executes on the CPU
- Due to its compilation, its code can be fully optimised to the most efficient machine code instructions, saving precious clock cycles at execution time

Note. other languages such as C++ etc can be used.

Options for Extending Python with C

- Write C using the Python C API
- Compile into Python modules
- Highest performance
- Most complex to develop

C Extensions



- Call C libraries directly from Python
- No compilation required
- Quick and simple
- Less safe and lower performance

ctypes (built-in)



- C Foreign Function Interface
- Cleaner interface to C libraries
- Optional compilation for speed
- Easier + safer than ctypes
- Good balance of power and usability

CFFI (modern alternative)



QA

556

Rule of thumb:

- Need speed → **C extensions**
- Need simplicity → **ctypes / CFFI**

Extensions are the low-level, powerful foundation of Python interoperability whilst CFFI is the modern, safer alternative for most use case.

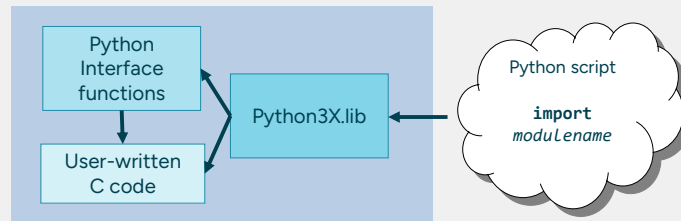
Feature	C Extensions	CFFI
Complexity	High	Medium
Performance	5 stars	4 stars
Memory management	Manual	Automatic
Learning curve	Steep	Moderate

Python C API

The Python C API is well documented in the Python manuals

- Extending and Embedding tutorial
- Python/C API reference guide
- Chapter in *Programming Python* 4th Edition

General principle is:



Compiled as a `.dll` or `.so` file named `module_name.pyd`

QA

557

The principle behind embedding your own C in Python is quite simple. If you have existing C code you must add the interface functions, which are described later. These are called directly from Python when the user script does an **import**. If the existing C is a 'main' program it has to be converted to a dynamically linked library (Windows) or shared object file (UNIX/Linux). The file extension used is **.pyd**, and the file itself should be in a well-known location, usually in the **site-packages** directory.

Calls normally using the C run-time library (RTL) might have to be converted to Python RTL calls, in particular those using standard streams like **printf**. When writing new code you should use the Python API as much as possible.

Variables can be passed as arguments in both directions, and there are API calls to create Python objects, and to convert between Python and C in both directions.

What is a Python C Extension?

A Python C Extension is:

- A compiled module written in C
- Loaded into Python and used like a normal module

```
import mymodule  
Mymodule.fast_function()
```

What it contains:

- C functions exposed to Python
- Module definition and initialization

Result:

- Write performance-critical code in C
- Call it directly from Python

QA

558

A C extension is just a compiled C module that Python can import which, from Python's perspective, it looks like any other module. However, underneath the function is actually written in C and executed natively. The C code must include the functions, the code that exposes them to Python and any necessary initialisation logic. Giving you the speed of C with the usability of Python.

Key Concepts

PyObject*

- Represents all Python objects in C
- Universal interface between Python and C

Reference Counting

- Tracks object usage
- Prevents memory leaks and crashes

Python C API

- Functions/macros to interact with Python
- Used for:
 - handling data
 - raising exceptions
 - building return values

Key idea:

- You're working inside Python's runtime — but using C

QA

559

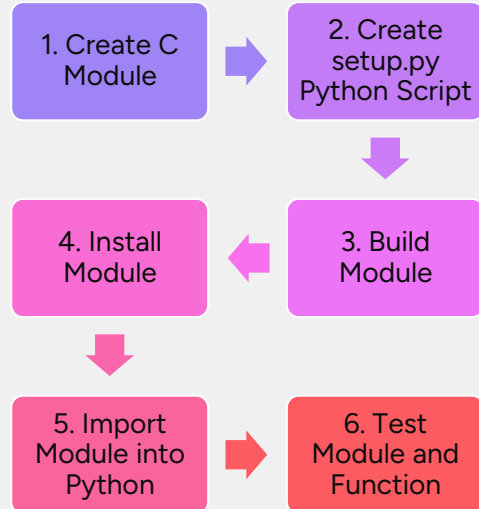
The three ideas on the slide are the foundation of everything that follows:

- **PyObject*:** Everything in Python is an object — and in C, it's represented as PyObject* which can be thought of as a universal wrapper for all Python data. It is rare to work with raw C types directly it is usually done through PyObject.
- **Reference Counting:** Python uses reference counting to manage memory
 - each object tracks how many references point to it
 - when it reaches zero → it gets cleaned up
 - Getting this wrong in C may well cause crashes or memory leaks
 - There are simple patterns that can be used to handle this safely
- **Python C API:** This is a set of functions that lets C talk to Python which can be used to:
 - convert data
 - call Python functions
 - raise exceptions

You are not leaving Python, you're working *inside* its runtime using C

Creating a new module (1)

Creating a new module is a six-step process:



QA

560

It is worth noting that any step can go wrong, so patience is required.

Common issues include:

- Typographical or syntax errors in the C module -which will be flagged during the build process (3)
- Insufficient user privileges to install the module, e.g. in Linux this step is often performed with "sudo"
- Incorrect return types in the C module
- Semantic errors in the C module which are discovered during testing (6)

Notes.

The build and install process can be attempted multiple times until issues in the C module and Python setup.py script are resolved.

A successful build and install does not mean the module (and its function(s)) will work as expected – as with all code, test thoroughly, e.g. use unit tests to prove these functions, just as you would "normal" Python functions.

The following process is performed and demonstrated in a Linux

distribution, but the general flow is similar for Microsoft Windows.

Before you start ensure you have the following:

1. Python.h
2. Working GCC (Linux, can be tested at the shell prompt) or MSVC

1. Creating the .c module

A Windows DLL or UNIX .so file is created as a .pyd file

In the C source code:

- a) `#include <Python.h>`
- b) Describe the C functions with their Python names
- c) Describe the module
- d) Create a module initialization function

The simplest way to build is using distutils

- Includes the correct directories and libraries

In the Python code just import the module as normal

QA

561

In detail, the first thing to do is to include **Python.h**, which also means that the Python include directory should be in the included file path (usually `-I` option). Note the capital 'P' to `Python.h`, particularly on Linux!

Within the C code we have to setup structures that describe the module and its functions (next few slides) and create a module initialisation function, which is of a fixed name and is non-static.

Building the extension can be done using conventional Makefiles or using Visual Studio. We shall cover these later, but they are fiddly. The simplest is do a standard python build of a `setup.py` file:

```
python setup.py build
```

Provide a Minimal C Function

Steps a & b: define a callable C function

- Code resides in a file with .c extension
- Add `#include <Python.h>`
- Functions return `PyObject*`
- Must follow Python C API conventions
- `Py_RETURN_NONE` returns Python None

```
#include <Python.h>

static PyObject* hello_world(PyObject* self,
PyObject* args) {
    printf("Hello from C!\n");
    Py_RETURN_NONE;
}
```

myminimalmodule.c

QA

562

In Python 3, the module is imported through a **PyInit_modulename** function, which initialises the Python module, referencing the **Module Definition**. Most of the names shown are only by convention, but this function *must* be of that format.

The **Module Definition** in turn references the **Method Table**.

The **Method Registration Table (MRT)** defines each C entry-point (function) that is to be called by Python and is a global array of **PyMethodDef** structs.

Each entry point is of type **PyCFunction**, which will return a pointer to a **PyObject**, and take two pointers to **PyObject**s as arguments. The Method Table is referenced in the **PyModuleDef** struct.

Module Definition

Step c: Map Python names → C functions

- Expose the Function

```
static PyMethodDef MyModuleMethods[] = {  
    {"hello_world", hello_world, METH_NOARGS, "Prints greeting"},  
    {NULL, NULL, 0, NULL}  
};
```

- Define Module Structure

```
static struct PyModuleDef myminimalmodule = {  
    PyModuleDef_HEAD_INIT,  
    "myminimalmodule",  
    NULL,  
    -1,  
    MyModuleMethods  
};
```

Method Registration Table

myminimalmodule.c

QA

563

In Python 3, the module is imported through a **PyInit_modulename** function (see next slide), which initialises the Python module, referencing the **Module Definition** (`myminimalmodule` in the example). Most of the names shown are only by convention, but this function *must* be of that format.

The **Module Definition** in turn references the **Method Table** (`MyModuleMethods` in the example).

The **Method Registration Table (MRT)** defines each C entry-point (function) that is to be called by Python, and is a global array of **PyMethodDef** structs.

Each entry point is of type **PyCFunction**, which will return a pointer to a **PyObject**, and take two pointers to **PyObject**s as arguments. The Method Table is referenced in the **PyModuleDef** struct.

Module Initialization

Step d: Initialise

- Module Initialisation

myminimalmodule.c

```
PyMODINIT_FUNC PyInit_myminimalmodule(void) {  
    return PyModule_Create(&myminimalmodule);  
}
```

The finished module

Note: The module initialization function is the only one which is not static

**Method
Registration
Table (MRT)**

Module Initializer

```
#include <Python.h>

static PyObject* hello_world(PyObject* self, PyObject* args) {
    printf("Hello from C!\n");
    Py_RETURN_NONE;
}

static PyMethodDef MyModuleMethods[] = {
    {"hello_world", hello_world, METH_NOARGS, "Prints greeting"},
    {NULL, NULL, 0, NULL}
};

static struct PyModuleDef myminimalmodule = {
    PyModuleDef_HEAD_INIT,
    "myminimalmodule",
    NULL,
    -1,
    MyModuleMethods
};

PyMODINIT_FUNC PyInit_myminimalmodule(void) {
    return PyModule_Create(&myminimalmodule);
}
```

**Module
Definition**

QA

565

This is a simple module called "myminimalmodule". On this slide we can see the important **PyModuleDef** struct which describes the module. Among other things, it gives the module name and the name of the array of **PyMethodDef** structs.

The members are: a header, the name of the module, the module documentation (NULL in the example), a flag indicating if state is kept in global variables (-1), and the Method Table.

The array has one element for each 'method' (C function), gives the name to be exported, a pointer to the function itself (so it must be prototyped), and describes the arguments passed (more on that later).

the Module initialization function, called **PyInit_modulename** - this is a fixed prefix and is looked for specifically by Python when an **import** is done. It consists of a Python API call to create a module, passing the address of the **PyModuleDef** struct (in this case **myminimalmodule**).

In this module there is only one C function, called "hello_world". It takes no parameters from Python and returns the Python None object.

2. Create setup.py Python Script

Create a Python setup.py file

- It is used in the build and install parts of the process
- It forms a key part of a project
- contains project metadata
- Requires additional information to specify our C source file (uses the ext_modules field)
- Build with setuptools
- Compiles into .pyd/.so file

```
from setuptools import setup, Extension

setup(
    name="myminimalmodule",
    ext_modules=[Extension("myminimalmodule", ["myminimalmodule.c"])]
)
```

Setup.py

QA

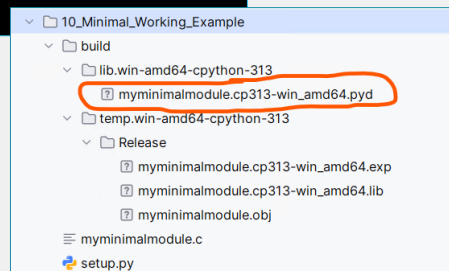
566

Save this file as "setup.py" in the *same folder* as your "mymnimalmodule.c" C source file.

3. Build the module

```
PS C:\QAL\10_Minimal_Working_Example> python setup.py build

running build
running build_ext
building 'myminimalmodule' extension
creating build\temp.win-amd64-cpython-313\Release
"C:\Program Files (x86)\Microsoft Visual
Studio\18\BuildTools\VC\Tools\MSVC\14.51.36231\bin\HostX86\x6
4\cl.exe"...
```



QA

567

The next step is to build your module. This will invoke MSVC (windows) or GCC (Linux) and attempt to compile your file.

Issue the command:

python setup.py build

Depending on the speed of your host the time taken can vary but it's generally instantaneous. Do not attempt to abort the process.

If there are any errors in either your `myminimalmodule.c` source code or the `setup.py` Python script it is likely this process will fail. Reading the error output should guide you towards the culprit; debug, save and retry the process.

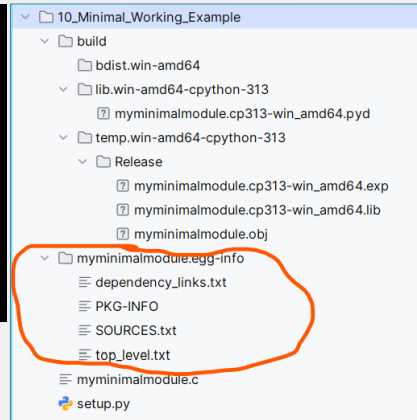
The output should be ".so" (shared object) files, in Linux or a .pyd files in Windows.

4. Install the module using: `pip install .`

```
PS C:\QAL\10_Minimal_Working_Example> pip install .
```

```
Processing .\.  
Installing build dependencies ... done  
Getting requirements to build wheel ... done  
Preparing metadata (pyproject.toml) ... done  
Building wheels for collected packages:  
myminimalmodule  
  Building wheel for myminimalmodule (pyproject.toml)  
  ... done  
Created wheel for...
```

- Builds your project
 - Uses `setup.py` or `pyproject.toml`
 - Compiles any C extensions (e.g. `.c` → `.pyd` / `.so`)
- Creates a distributable package (wheel or similar)
- Installs it into your Python environment (typically for Windows into: `<python>/Lib/site-packages/`)



QA

568

The next step is to install your built module.

As noted, this will attempt to install files into certain parts of the operating system. If you are a Linux user and do not have appropriate permissions for these folders use either `sudo` (better) or switch to root (obviously not ideal).

Then, issue the command:

```
pip install .
```

Again, it's possible this process could still fail.

If it does, go back and check your `myminimalmodule.c` and `setup.py` files again. It is likely you have a semantic ("meaning") error rather than a syntactic one.

The outcome from this part of the process is that setup has moved your `.so` (shared object) file to the Python dist-packages directory and created egg-info for your module.

Note. directory name swill vary depending on implementation and

operating system.

Steps 5 and 6: Import the module into a Python file and use it!

```
import myminimalmodule

print(myminimalmodule.hello_world.__doc__) # doc string
print(myminimalmodule.hello_world) # type
print(callable(myminimalmodule.hello_world))

myminimalmodule.hello_world()
```

```
C:\Python313\python.exe "C:\QAL\20_Using
_the_module\myminimalmodule_client.py"
Prints greeting
<built-in function hello_world>
True
Hello from C!
Process finished with exit code 0
```

Note: It is possible that the import process can fail – this is typically due to module initialiser errors; check the .c source file, debug and rebuild.

QA

569

After successfully building and installing, it's now time to import your module!

Start a new Python project and add a .py file. Import your module and add code that calls the modules `hello_world()` function.

The Python/C API formats

In order to pass values to and from C functions, Python data must be converted to C data types.

- Python objects (typically function arguments) can be converted to C variable types
- Python objects can be built from C variable types (typically for return value)

Format specifiers are used to describe the conversion

- Used in building Python variables, and parsing arguments
- There are many specifiers, see the Python documentation
- Examples include:

i/I	signed/unsigned int
d/D	signed/unsigned double
s	C-style string (char *)
z	C-style string or None
y	bytes object
o	Generic Python object (oh)
o!	Typed Python object (oh!)

QA

570

When packing and unpacking Python objects between Python and CD we often have to supply format specifiers. Each specifier refers to a single variable of that type, either to copy from, typically in `Py_BuildValue()`, or to, typically in `PyArg_ParseTuple()`.

See Parsing arguments and building values in the Python help text.

Parameter Passing Basics

- Python passes arguments via a tuple (args)
- Use `PyArg_ParseTuple()` to extract values

```
static PyObject* square(PyObject* self, PyObject* args) {
    int x;

    if (!PyArg_ParseTuple(args, "i", &x)) {
        return NULL;
    }

    int result = x * x;
    return PyLong_FromLong(result);
}

static PyMethodDef MyMathsModuleMethods[] = {
    {"square", square, METH_VARARGS, "Squares a number"},
    {NULL, NULL, 0, NULL}
};
```

QA

571

Specify the calling convention in the Method Table

METH_NOARGS

- No parameters – fast

METH_VARARGS

- Variable positional parameters
- Arguments are passed as a Python tuple and are unpacked

Use **PyArg_ParseTuple** to convert the tuple to C variables

The calling convention is specified in the `PyMethodDef` struct for each callable C function. `METH_VARARGS` is the most common calling convention.

`PyArg_UnpackTuple` is an alternative to `PyArg_ParseTuple`, but is more complex.

In this example, a single integer argument is being unpacked:

`i` (format specifier for an integer), which will be stored at the address of `iCount` variable.

If these cannot be successfully parsed a `NULL` is returned which should cause Python to "complain" about the arguments used in the Python-side

function call.

Multiple Parameters

```
static PyObject* power(PyObject* self, PyObject* args) {
    int base, exponent;
    if (!PyArg_ParseTuple(args, "ii", &base, &exponent)) {
        return NULL;
    }
    long long result = 1;
    for (int i = 0; i < exponent; i++) {
        result *= base;
    }
    return PyLong_FromLong(result);
}
```

QA

572

In this example, two arguments are being unpacked (both of which are integers):

`i` (format specifier for an integer), which will be stored at the address of `iCount` variable.

Return Values

Must be a pointer to a Python object

- Often created using `Py_BuildValue`

```
PyObject *tup = Py_BuildValue("(iis)",
                               iCount, iLen, szName);
```

- Or from specific **API calls**

```
PyObject *RetnList = PyList_New(0);
PyList_Append(RetnList, tup);
Py_DECREF(tup);
```

- Notice the **reference count decrement** when adding to a list

There are some helper macros and functions

- | | |
|--------------------------------|--|
| • <code>Py_RETURN_FALSE</code> | • <code>PyLong_FromLong()</code> → int |
| • <code>Py_RETURN_TRUE</code> | • <code>PyFloat_FromDouble()</code> → float |
| • <code>Py_RETURN_NONE</code> | • <code>PyUnicode_FromString()</code> → string |
- QA `PyBool_FromLong()`

573

Whatever we return, it must be a Python object. This could be `None`, in which case you probably want to use the `Py_RETURN_NONE` macro. Or it could be a more complex object such as a container.

`Py_BuildValue` will create a tuple if the specifiers are enclosed inside `(...)`, a list inside `[...]` and a dictionary inside `{...}`, the items being in key-value order.

Python container objects can be created in C by using a `Pytype_New` function, for example: `PyList_New`, `PyTuple_New`, `PySet_New`, `PyDictNew`. Once created, all the usual container methods are available in the API. The `_New` functions take a single parameter or the number of items passed. Zero is a sensible default if of indeterminate length.

Scalar values can also be converted, for example: `PyFloat_FromString`, `PyLong_FromSize_t`, `PyBytes_FromString`.

`Py_DECREF(obj)` tells Python you no longer need a reference to that object.

Passing and returning complex types

Create a pointer to a generic Python object

- Use `O` (`oh`) as the format specifier
- `O!` checks for a Python object to be of the specified type (`PyTuple_Type`)
- Example passing two Python lists into C

```
PyObject *join(PyObject *self, PyObject *args)
{
    PyObject * tuple1;
    PyObject * tuple2;

    // "O!" means: "Expect an object of a specific type"
    if (!PyArg_ParseTuple(args, "O!O!", &PyTuple_Type, &tuple1,
                          &PyTuple_Type, &tuple2))
        return NULL;

    // Ensure both arguments are actually Python tuples
    if (!PyTuple_Check(tuple1) || !PyTuple_Check(tuple2)) {
        PyErr_SetString(PyExc_TypeError, "Both arguments must be tuples.");
        return NULL;
    }

    // Concatenate them together and assign to a new variable
    PyObject* new_tuple = PySequence_Concat(tuple1, tuple2);
    return new_tuple;
}
```

QA

574

Python types can be defined by such as `PyBool_Type`, `PyBytes_Type`, `PyTuple_Type`, `PyList_Type`, `PyDict_Type`, and so on (it is even possible to create your own types).

With `O!`, Python checks that the object passed is of the correct type, and if not raises a `TypeError` exception.

Error handling and exceptions

Python exception objects are part of the API

- PyExc_EnvironmentError, PyExc_TypeError, PyExc_ValueError, PyExc_RuntimeError

Set an exception, then return NULL

- PyErr_SetString
- PyErr_SetFromErrno
- PyErr_SetFromErrnoWithFilename
- PyErr_SetObject

```
struct stat st;
if (stat(path, &st) != 0 || !S_ISDIR(st.st_mode)) {
    char errbuf[1024];
    snprintf(errbuf, sizeof(errbuf), "%s is not a directory", path);
    PyErr_SetString(PyExc_NotADirectoryError, errbuf);
    return NULL; Must return NULL
}
```

QA

75

For Python to detect and raise an exception, the underlying C function must signal failure by returning NULL — or -1 in the case of functions that return a signed integer.

On Windows, errors broadly fall into two categories. The first comes from the C runtime library, which uses `errno` and `strerror()` to report and describe errors. The second comes from the Win32 API, which has its own separate mechanism using `GetLastError()` and `FormatMessage()`. While the `Errno` exception classes can be used with `errno` on Windows, they are not suitable for Win32 API errors. For those, you should instead use one of:

- `PyErr_SetFromWindowsErr()`
- `PyErr_SetExcFromWindowsErr()`
- `PyErr_SetFromWindowsErrWithFilename()`
- `PyErr_SetExcFromWindowsErrWithFilename()`

Why Memory Management Matters

Memory in C Extensions

- Python uses automatic memory management
- In C extensions, you must manage object lifetimes

What can go wrong?

- Memory leaks (objects never freed)
- Crashes (freeing too early)
- Unpredictable behaviour

Python won't protect you when using C

QA

576

In standard Python development, memory management is handled entirely by the interpreter — allocating and freeing memory is simply not something you need to think about.

Writing C extensions is a different matter entirely. You are now operating at a lower level, and while Python continues to use reference counting under the hood, the responsibility for interacting with it correctly falls on you.

Getting this wrong can have serious consequences:

- **Memory leaks** — objects that are never freed will accumulate and consume memory over time
- **Crashes and segmentation faults** — objects freed too early can lead to undefined behaviour and hard-to-diagnose failures
- **Subtle, hard-to-reproduce bugs** — memory errors often don't manifest consistently, making them particularly tricky to track down

This represents one of the most significant shifts when moving from Python to C extensions: what was once handled automatically now demands careful, manual attention.

Reference Counting Basics

What is Reference Counting?

- Each Python object has a reference count
- Tracks how many "owners" exist

Key idea

- Count = 0 → object is deleted

```
PyObject *obj = PyLong_FromLong(10); // ref count = 1
Py_DECREF(obj); // ref count = 0 → object freed
```

QA

577

Every Python object keeps track of how many references point to it. This is called its **reference count**.

- When a new reference is created → count increases
- When a reference is removed → count decreases
- When the count reaches zero → the object is destroyed

In C extensions, you interact with this system using:

- `Py_INCREF(obj)` → increase reference count
- `Py_DECREF(obj)` → decrease reference count

Ownership Rules

Who owns a reference?

- You own a reference if you:
 - Create a new object
 - Call functions returning a new reference

Common "new reference" functions

- `PyLong_FromLong()`
- `Py_BuildValue()`
- `PyTuple_New()`

Rule:

"If you own it → you must eventually `Py_DECREF` it"

QA

578

The key concept in managing references is ownership.

If your code owns a reference, it is responsible for eventually releasing it.

You generally own a reference when:

- You create a new object (e.g., `PyLong_FromLong`)
- A function returns a "new reference"

You usually do not own a reference when:

- You receive a "borrowed reference" (which is common in many API calls)

A useful guideline is:

If a function creates or returns a new object, you must call `Py_DECREF` on it later.

Misunderstanding ownership is one of the leading causes of memory bugs in C extensions.

Common Mistakes

Forgetting Py_DECREF

```
PyObject *obj = PyLong_FromLong(10);  
// no Py_DECREF → memory Leak
```

Double Py_DECREF

```
PyObject *obj = PyLong_FromLong(10);  
Py_DECREF(obj);  
Py_DECREF(obj); // crash
```

Returning borrowed references

- Returning objects you don't own
- Leads to undefined behaviour

Memory bugs are often subtle and difficult to debug

QA

579

There are several common errors when working with reference counting:

- Forgetting to decrement reference count causes memory to accumulate over time.
- Decrementing too many times may free memory that is still in use resulting in a crash
- Returning an object you don't own (i.e. a borrowed reference) can lead to undefined behaviour because Python expects a valid reference.

Memory bugs are often:

- hard to trace
- intermittent
- worse under load

Carefully designed code making use of consistent patterns help avoid these problems.

Reference count – Py_INCREF

In C:

- When you build a python value then add it to a container, you should decrement the reference count
- If **you** return a built-in value, increment its reference count

```
Py_INCREF(Py_None);  
return Py_None;
```

```
Py_RETURN_NONE  
does this for you!
```

- *Borrowed references*
 - **Do not decrement the reference count!**

In Python, check reference counts with:

```
txt = Simple.MyFunc()  
print("Ref count:", sys.getrefcount(txt))
```

- Be aware that some values are shared!

QA

580

When placing an owned item into a container, you should decrement its reference count afterwards — provided you no longer need to reference that item directly. This is a common source of confusion with functions like `PyList_Append()`, which actually *increments* the reference count of the item being added rather than decrementing it. Failing to manually decrement the count in such cases will result in a memory leak.

A convenient way to inspect reference counts is via `sys.getrefcount()`, though there are a couple of important caveats to keep in mind. First, the act of calling `sys.getrefcount()` itself temporarily increments the reference count, meaning the value it returns will always be one higher than you might expect. Second, certain objects are shared widely across the interpreter and various modules — `None`, for instance, may have hundreds of references at any given time, as will commonly used values such as zero and one, which Python interns for efficiency. In these cases, rather than relying on the absolute reference count, it is more reliable to record the count *before* the operation you are testing and compare it to the count *after*, looking for the relative change rather than a specific number.

Bank Account Object Inspector – Correct use of Py_INCREF

```
static PyObject* process_account(PyObject* self, PyObject* args) {
    PyObject *account;

    // Parse input (expects any Python object)
    if (!PyArg_ParseTuple(args, "O", &account)) {
        return NULL;
    }

    // ♦ Show we received a borrowed reference
    printf("Received account object\n");

    // Simulate some work...

    // ♦ Return the original object
    Py_INCREF(account); // must increment before returning

    return account;
}
```

```
import bankmodule
import sys

# Simulated bank account object
account = {"name": "Alice", "balance": 1000}

# Pass to C extension
result = bankmodule.process_account(account)

# Check returned object identity
print("Same object returned:", result is account)

del account
# Will work and code will terminate cleanly
print(result["name"])
```

```
Received account object
Same object returned:
True
Alice
```

Bank Account Object Inspector – Incorrect use of Py_INCREF

```
static PyObject* process_account(PyObject* self, PyObject* args) {
    PyObject *account;

    // Parse input (expects any Python object)
    if (!PyArg_ParseTuple(args, "O", &account)) {
        return NULL;
    }

    // • Show we received a borrowed reference
    printf("Received account object\n");

    // Simulate some work...

    // • Return the original object
    // Py_INCREF(account); // must increment before returning

    return account;
}
```

```
import bankmodule
import sys

# Simulated bank account object
account = {"name": "Alice", "balance": 1000}

# Pass to C extension
result = bankmodule.process_account(account)

# Check returned object identity
print("Same object returned:", result is account)

del account
# Will work and code will terminate cleanly
print(result["name"])
```

```
Received account object
Same object returned: True
Alice
Process finished with exit code -1073741819 (0xC0000005)
```

QA

582

Excluding the increment (Py_INCREF) just before the return will result in the python client apparently running OK but terminating with a message like the following:

Process finished with exit code -1073741819 (0xC0000005)

which means that the Python script crashed due to a Windows STATUS_ACCESS_VIOLATION (Segmentation Fault). This is a low-level critical crash. It occurs when a program tries to read or write to a restricted or invalid memory location. Because pure Python code manages its own memory, it almost never throws this error. Seeing this error indicates a problem with native compiled code interacting with Python.

Allocating Memory

- Avoid using `malloc`, `calloc`, or `realloc` for Python objects — the interpreter has no awareness of memory allocated this way, as Python uses its own internal heap management system rather than the standard C heap
- Always use the Python heap for Python objects, via the following functions:
 - `PyMem_Malloc()` and `PyMem_Realloc()` — allocate or reallocate memory on the Python heap, returning a void * pointer on success or NULL on failure; note that allocated memory is not initialised
 - `PyMem_Free()` — releases memory previously allocated through the Python heap
- On allocation failure, return `PyErr_NoMemory()` to properly signal an out-of-memory error back to the interpreter

QA

583

Python maintains its own memory management system and heap implementation, separate from that of the underlying operating system. For your own C variables and structs, using the process default heap is perfectly acceptable — on Windows in particular, multiple heaps are supported — however, it must not be used to store Python objects. Fortunately, Python's memory management APIs are straightforward to work with, mapping closely to their C runtime counterparts with very little additional complexity to contend with.

Strings & Buffers

- "s" → converts Python string → C string
- Strings are read-only in C
- Encoding handled automatically (UTF-8)

```
const char *text;

if (!PyArg_ParseTuple(args, "s", &text)) {
    return NULL;
}
```

QA

584

Strings are one of the most common data types passed between Python and C extensions. In Python, strings are high-level objects, but in C, they are typically handled as null-terminated character arrays (char*).

- Python converts the string into a C-compatible UTF-8 string
- The pointer you receive is **read-only**
- You should **not attempt to modify it directly**

If you need to modify string data, you must:

- create a new Python string, or
- copy the data into your own buffer

Typical use cases:

- file paths
- user input
- text content (e.g. writing files, logging transactions)

It's important to remember that:

You are reading Python-managed memory — not allocating your own.

Lists and Tuples

- `PyList_Size(list)` gets the total number of items currently inside a Python list object (equivalent to calling `len(list)` in Python)
- `Py_ssize_t` is a signed integer type used by Python to safely handle container lengths and indexing, even for very large lists
- `PyList_GetItem()` fetches the specific element at index `i` from the list (equivalent to writing `item = list[i]` in Python)
 - Returns a borrowed reference so does not increase the reference count of the item
 - Do **NOT** `Py_DECREF` it

```
PyObject *list;

if (!PyArg_ParseTuple(args, "O!", &PyList_Type, &list)) {
    return NULL;
}

Py_ssize_t size = PyList_Size(list);
PyObject *item = PyList_GetItem(list, *);
```

QA

585

Lists and tuples allow Python programs to pass structured data into C extensions.

The "O!" format specifier ensures:
the argument is of a **specific Python type** (e.g. list or tuple)

Processing Transactions

Pattern: extract → iterate → convert → return

- Parse input
 - Ensures the function only accepts a list.
- Get collection size
 - Used to iterate through the data.
- Loop through elements
 - Returns a borrowed reference
 - Safe to use but not to modify or free
- Convert to C type
 - Converts Python float → C double
- Return result
 - Creates a new Python object
 - Ownership is transferred to the caller

```
static PyObject* sum_balances(PyObject* self, PyObject* args) {
    PyObject *list;
    double total = 0;

    if (!PyArg_ParseTuple(args, "O!", &PyList_Type, &list)) {
        return NULL;
    }

    Py_ssize_t size = PyList_Size(list);

    for (Py_ssize_t i = 0; i < size; i++) {
        PyObject *item = PyList_GetItem(list, i); // borrowed
        total += PyFloat_AsDouble(item);
    }

    return PyFloat_FromDouble(total);
}
```

QA

586

This example shows how a C extension can process a collection of numerical values — similar to summing balances or transactions in a banking application.

The coding pattern demonstrates how C extensions:

- accept structured Python data
- process it efficiently
- return results in Python-compatible form

The same pattern can be used in:

- financial calculations
- batch transaction processing
- analytics and reporting

Python C API: Core Pattern

Standard Workflow

- Most C extension functions follow this pattern:
Parse → Process → Return
- Which mostly becomes:
 - Convert Python → C types
 - Perform logic
 - Convert back to Python

PyArg_ParseTuple(...)



Do work in C



Return PyObject*

By mastering this pattern most extensions will become straightforward

Common API functions

Frequently Used Functions

- Parsing arguments

```
PyArg_ParseTuple(...)
```

- Returning values

```
PyLong_FromLong()  
PyFloat_FromDouble()  
PyUnicode_FromString()  
Py_BuildValue()
```

- Error handling

```
PyErr_SetString(...)  
PyErr_SetFromErrno(...)
```

Learning a small set of functions solves most use cases

Thread state and the Global Interpreter Lock

Considerations when running within a *Python* thread

- **We obtain the GIL in order to call API functions**
 - Obtained by standard library modules, like `threading`

But we should not always keep the GIL

- If we are doing system calls, I/O, or long running operations that do not involve Python objects

We should release the GIL if we don't need it

- Allows other Python threads to run
- Improves responsiveness
- `Py_BEGIN_ALLOW_THREADS` (macro)
 - Saves the current thread state and releases the GIL
- `Py_END_ALLOW_THREADS` (macro)
 - Restores the current thread and acquire the GIL

QA

589

You might recall that an issue with using threads in Python is the Global Interpreter Lock, the GIL.

When we are running code within Python threads, we should be aware that we could be multi-threaded. The GIL will be obtained for us by the `threading` module, but we don't necessarily need it. In Python, the Python interpreter can yield the GIL to another thread when it is engaged in wait operations, but when we are in C it cannot know when that is appropriate. It is therefore up to the C code to allow other threads to proceed with the GIL.

These are macros, so they should be called within the same C-code block, otherwise difficult to debug compilation errors might result.

Note: Do NOT access Python objects while GIL is released

cffi

- **cffi (C Foreign Function Interface)**

- Call C code directly from Python
- No Python C API required
- No manual reference counting

Benefits

- Simpler than C extensions
- Safer (less risk of crashes)
- Works with existing C libraries

“cffi lets Python talk to C without deep integration”

QA

590

CFFI (C Foreign Function Interface) is a library that allows Python to call C code **without needing to write full C extensions using the Python C API**.

Unlike the approach we've seen previously:

- You do **not** need to work with PyObject*
- You do **not** manage reference counts (Py_INCREF / Py_DECREF)
- You do **not** write module definitions or initialization functions

Instead, you:

1. Declare the C function signature (cdef)
2. Provide or load the implementation
3. Call it like a normal Python function

This makes CFFI:

- easier to learn
- safer to use
- faster to develop with

Key Idea:

CFFI removes a lot of boilerplate and complexity, while still allowing Python

to execute C code efficiently.

Banking Scenario

Process a Withdrawal

```
#include <stdio.h>
// Note, code is specified here to show it as pure C
// The cffi code does not use or reference this in any way.
double withdraw(double balance, double amount) {
    if (amount > balance) {
        return -1.0; // signal error
    }
    return balance - amount;
}
```

QA

591

This example models a simple banking operation: a withdrawal.

We separate responsibilities:

Python: manages the account (dictionary) and handles validation and errors

C: performs a fast calculation (balance update)

Why this is useful

In real systems:

- financial calculations may need to be fast and efficient
- logic may already exist in C libraries
- integration is easier with tools like CFFI

The above example uses a simple rule:

- returns a special value (-1.0) to signal an error

In real systems, you might:

- return error codes
- use `errno`
- or design more complex error handling

Key idea:

Python handles business logic and orchestration, while C focuses on performance-critical computation.

Python + cffi Integration

`cdef()` → declare function signature
`verify()` → compile and link C code

Note:

- All the code shown here lives in a .py file
- No Python.h
- No PyObject*
- No build step

```
from cffi import FFI

ffi = FFI()

ffi.cdef("""
    double withdraw(double balance, double amount);
""")

C = ffi.verify("""
    double withdraw(double balance, double amount) {
        if (amount > balance) {
            return -1.0;
        }
        return balance - amount;
    }
""")
```

QA

592

`cdef()` tells Python:

- what the function looks like
- what types it expects

`Verify()`:

- compiles the C code
- links it into Python
- returns a callable object

Key idea

CFFI creates a bridge between Python and C using simple declarations and inline compilation.

Using the Function in Python

- Clean separation:
 - Python → orchestration
 - C → computation
- No manual memory management
- No reference counting
- Minimal boilerplate

```
from cffi_module import C

account = {"name": "Alice", "balance": 1000.0}

def process_withdrawal(account, amount):
    result = C.withdraw(account["balance"], amount)

    if result < 0:
        raise ValueError("Insufficient funds")

    account["balance"] = result
    return result

# Unit tests
import pytest
def test_withdrawal():
    account["balance"] = 1000.00
    process_withdrawal(account, 200.0)
    assert account["balance"] == 800.00

def test_withdraw_too_much_raises_error():
    account["balance"] = 1000.00
    with pytest.raises(ValueError):
        process_withdrawal(account, 1000.01)
```

QA

593

Once the C function is loaded, it behaves like a normal Python function:
So:

- C performs the calculation
- Python handles business rules and errors

This shows a clean separation of concerns:

- C → efficient computation
- Python → control flow and error handling

Key idea:

CFFI lets you integrate C performance into Python workflows with minimal complexity.

Topic

Complexity
Memory mgmt.
Setup
Safety

C API

High
Manual
Compile + build
Risky

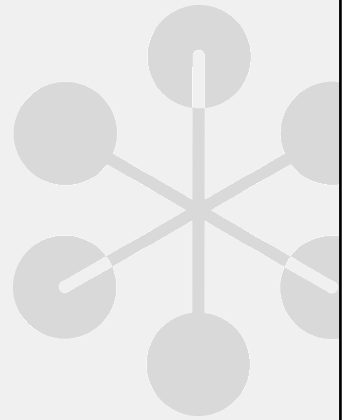
CFFI

Low
Automatic
Inline
Safer

See also...

Standard library modules

- `gc` Garbage Collector interface
- `os` Miscellaneous operating system interfaces
- `platform` Access to underlying platform's identifying data
- `resource` Resource usage information
- `sysconfig` Access to Python's configuration



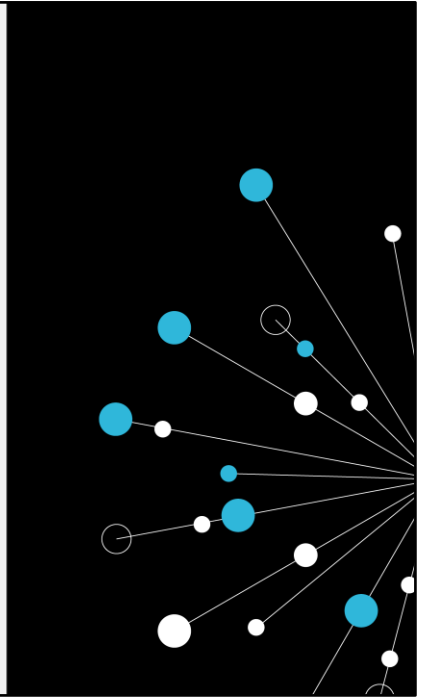
Exercise EG14: Extending Python: first steps

Activity instructions:

Working solo or in a programming pair, solve the set problem idiomatically and with best coding practice.

Compare and contrast your solution with your peers.

QA



Suggested time: 30-40 mins.

Activity includes differentiated tasks.

Summary

Rationale & Approaches

- Why extend Python with C (performance, system-level access)

Core Integration Concepts

- How Python and C interact (data exchange, execution flow)
- Creating minimal C extensions and CFFI bindings

Working with Data

- Parameter passing and return values, handling complex types, strings, and buffers

Robustness & Reliability

- Error handling and exception propagation

Memory Management Essentials

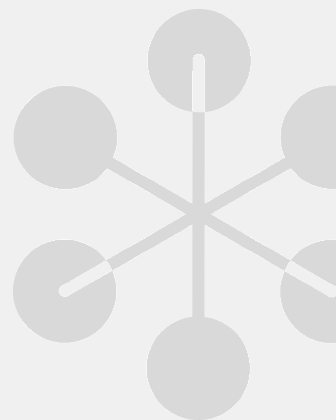
- Reference counting and ownership rules, safe allocation and deallocation of memory

Python C API in Practice

- Core design patterns and common API functions

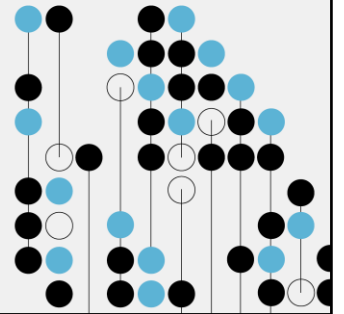
Concurrency Considerations

- Thread state management and the Global Interpreter Lock (GIL)



Any questions?

QA



Appendices



Alternative approaches – ctypes

Enables run-time dynamic linking to foreign libraries

- .dll on Windows, .so on UNIX/Linux
 - Main interface is through cdll
- Windows interface includes:
 - windll - stdcall calling convention interface
 - oledll - for HRESULT return codes (and stdcall)

```
from ctypes import *

msvcrt = cdll.msvcrt
text = b"Hollow World!\n"
msvcrt.printf(b"%s", text)

mydll = cdll.LoadLibrary("C:\\NQQ\\Win32Dlls\\DllModule7")
mydll.DllFunc7()
```

C RTL call
Use libc.so on UNIX/Linux

Windows _cdecl DLL call

QA

599

The **ctypes** library module is a very easy way to interface Python with C/C++ written shared libraries.

There is a slight performance penalty over a pure C interface in that each call: every variable and value has to have a Python wrapper built around it, however that is more than compensated by its ease of use.

You do have to know quite a lot about C to be able to use this module, not to mention the shared library you are calling.

The example on the next slide calls the Win32 API **CreateProcess**. Notice a few things:

- **CreateProcess** itself is not called, that is because it is actually a macro to **CreateProcessA** or **CreateProcessW** depending on the character set used for the call (A - ASCII, W - Wide characters). We have elected to use the ASCII interface here, which means we need to use Python byte-strings (we would not have to supply the b" " decoration in Python 2).
- We use **windll** rather than **cdll** because most Win32 APIs use the Windows **__stdcall** calling convention (the exceptions are those in the C runtime library, such as **_beginthreadex**).

We are using `WinError` to find the error message, if any. This calls (by default) `GetLastError()` and `FormatMessage()`.

`ctypes` is an extensive module with many features - see the documentation for further details.

Win32 ctypes example

Most base APIs are in kernel32.dll

Many Microsoft specific types are already defined

```
from ctypes import *
from ctypes.wintypes import *
kernel = windll.kernel32

Startup = STARTUPINFO(0)
ProcInfo = PROCESS_INFORMATION(0)
retn = kernel.CreateProcessA(None, b"myprog.exe",
                             None, None,
                             False, 0, None, None,
                             byref(Startup),
                             byref(ProcInfo))

print(WinError())
```

Define
Structures
here (see
notes)

QA

600

For an explanation of the example, see the notes for the previous slide.

Alternative approaches – SWIG

Patch required for Python 3

Steps:

- Write a SWIG declarations file

```
%module Win32Specifics
%{
#include "Win32Specifics.h"
%}

typedef unsigned long mode_t;

extern DIR * opendir(const char *name);
extern struct dirent *readdir(DIR *dir);
extern int  closedir (DIR *dir);
```

Normal header file

External Interfaces

- Run swig with -python option
 - Build the dll with a leading _
- QA Copy dll and generated py file to site_packages

601

SWIG (and SIP, which follows) are particularly suitable for C or C++ libraries which already exist.

There is a special SWIG for Windows, called swigwin. It contains a pre-compiled swig.exe, which must be run from its installed directory.

The first step is to patch swigwin-2.0.0/Lib/python/pyrun.swg, assuming you are using that version. The patch is described at: <http://paste.pocoo.org/show/253494/>

When carefully done, it takes about 10 minutes to apply. How will I know if I need the patch? You will get an unresolved external reference to `PyCObject_Import()`, which has been deprecated for some time and was removed at Python 3.2.

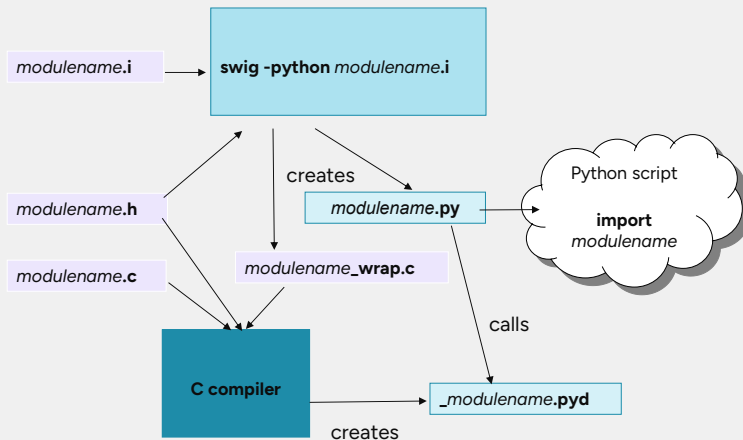
Assuming you already have a C file which works, you now need to generate the declarations (interface) file.

The SWIG declarations file contains the module name, and a description of the prototypes, usually from a conventional C header file. In addition, you

must declare each prototype to be available in Python with **extern**.

SWIG architecture

Files and software involved



QA

602

Once you have created the interface file you can run `swig`.

```
swig -python interface-file
```

should generate a wrapper C file and a python file. On Windows, create a Visual Studio `.dll` project which includes the Python header and `libs` directories, and `python32.lib`. The `.dll` name should have a leading underscore `_`. Build the project in Release (not Debug) and copy the `dll` and python files to the Python `Lib\site_packages` directory.

Details of modifying a Visual Studio project for Python will be discussed in a later chapter.

The generated `.py` and `.pyd` files should be copied to the `site_packages` file.

Thankfully **`distutils.extension`** also supports SWIG interface `(.i)` files.

Alternative approaches – SIP

Similar to SWIG, but Python specific

- A Sip is a small Swig
- See PyPi (Python 3 packages)
- Written specifically to maintain PyQt
- Optimised for Python

Interface is specified in a .sip file

- Different format to SWIG, but similar

Arguably integrates with C++ better than SWIG

Code generation is done with a distutils extension

- Generates a .pyd file

QA

603

SIP is maintained by the same company who maintains the PyQt package for Nokia's **qt** GUI builder, and it is used for that purpose so is actively tested with new releases.

Since an extension to `distutils` is used, there is rather less effort required to generate the code and manage the compiler with SIP.

Other alternative approaches

Cython was derived from Pyrex

- A Python-like language which generates C
- Allows Python and C to be mixed
- Produces a Python extension
- Available for Python 2 and 3
- Fast, possibly faster than CPython

Boost.Python is specifically for C++

- Like SWIG and SIP, it requires an interface wrapper
 - Similar to IDL
- Possibly more work than SWIG or SIP for large libraries

Others include CXX (Python C++ API) and weave

Nuitka is not a C interface, but recompiles Python into C API calls

- Produces very fast code